Algorithms for the Study of RNA and Protein Structure

Alexander David Stivala

Submitted in total fulfilment of the requirements
of the degree of Doctor of Philosophy

October 2010

Department of Computer Science and Software Engineering
The University of Melbourne
Produced on archival quality paper

Abstract

The growth in the number of known sequences and structures of RNA and protein molecules has led to the need to solve many computationally demanding problems in the analysis of RNA and protein structure. This thesis describes algorithms for structural comparison of RNA and protein molecules. In the case of proteins, it also describes a technique for automatically generating two-dimensional diagrammatic representations for visual comparison.

A general technique for parallelizing dynamic programs in a straightforward way, by means of a shared lock-free hash table implementation and randomization of subproblem ordering is described. This generic approach is applied to several well-known dynamic programs, as well as a dynamic program for structural alignment of RNA molecules by aligning their base pairing probability matrices.

Two algorithms for protein structure and substructure searching are described. These algorithms are also capable of finding non-sequential matches, that is, matches between structures where the sequential order of secondary structure elements is not preserved. The first algorithm is based on the relaxation of an earlier quadratic integer problem (QIP) formulation to a quadratic program (QP).

The second algorithm uses the same formulation but approximates it using simulated annealing. It is shown that this results in significant increases in speed. This algorithm is also capable of greater accuracy when assessed as a fold recognition method. A parallel implementation of this algorithm on modern graphics processing unit (GPU) hardware is also described. This parallel implementation results in a further significant speedup, and, to the best of our knowledge, is the first use of a GPU for the protein structural search problem.

Finally, a system to automatically generate two-dimensional representations of protein structure is described. Such diagrams are particularly useful in analysing complex protein folds. A method for using these diagrams as an interface to the protein substructure search methods is also described.

Declaration

This is to certify that

(i) the thesis comprises only my original work towards the PhD except where indicated in the Preface,

(ii) due acknowledgement has been made in the text to all other material used,

(iii) the thesis is less than 100,000 words in length, exclusive of tables, maps, bibliographies and appendices.

Alexander David Stivala

- Chapter 4 is substantially previously published as:

  A. Stivala, P. Stuckey, M. Garcia de la Banda, M. Hermenegildo, A. Wirth, "Lock-free Parallel Dynamic Programming" *J. Parallel Distrib. Comput.* 2010, **70**:839–848

- Chapter 5 is substantially previously published as:

  A. Stivala, A. Wirth, P. Stuckey, "Tableau-based protein substructure search using quadratic programming" *BMC Bioinformatics* 2009, **10**:153

- Chapter 6 is substantially previously published as:

  A. Stivala, P. Stuckey, A. Wirth, "Fast and accurate protein substructure searching with simulated annealing and GPUs" *BMC Bioinformatics* 2010 **11**:446

- A brief summary ("application note") of Chapter 7 has been submitted for publication as:

  A. Stivala, M. Wybrow, P. Stuckey, A. Wirth, J. Whisstock, "Automatic generation of protein structure cartoons with Pro-origami", submitted to *Bioinformatics* (under review)

Acknowledgements

Thanks are due first to my supervisors, Prof. Peter Stuckey and Dr Tony Wirth. Without their advice and guidance none of this thesis would have been possible. Thanks are also due to Dr Linda Stern, the third member of my thesis committee, who has also provided me with useful advice on bioinformatics in general and RNA structure in particular.

I am grateful to the Australian Government and the University of Melbourne for an Australian Postgraduate Award, which has funded my PhD studies.

I owe A/Prof Maria Garcia de la Banda and Prof. Manuel Hermenegildo a great deal for their contribution, with Peter, of the original idea of using randomization and lock-free hash tables for parallel dynamic programming, which led to Chapter 4.

Likewise, I am indebted to Prof. James Whisstock for the suggestion of a new system for protein structure cartoons, and his advice and feedback on my many prototypes of the "Pro-origami" system, described in Chapter 7. James's advice on structural biology has been invaluable to my understanding of protein structure and to the development of the Pro-origami system. James is also responsible for the "Pro-origami" name itself. Also for Pro-origami, Dr Michael Wybrow's contributions have been invaluable, and I appreciate the time he has spent adding yet more features and "tweaks" to Dunnart, without which Pro-origami could not have existed. Prof. Kim Marriott and Dr Tim Dwyer have also provided useful advice on constraint-based diagram layout and Pro-origami.

For much advice on everything to do with tableaux, and more generally protein structure, structural alignment, bioinformatics, and writing and publishing academic papers, I am greatly indebted to Dr Arun Konagurthu. In fact it was his PhD completion seminar in 2006 that was my introduction to protein structural alignment. Chapter 5 in particular benefited greatly from Arun's advice.

Dr Mike Kuiper and the other staff and interns at VPAC are responsible for introducing me to GPU programming during my internship there in the summer of 2008/2009. I have subsequently used VPAC facilities for all of the high performance computing work I have required, and in particular their GPU facilities were essential to the work in Chapter 6. I am grateful for the advice and assistance of VPAC staff, which has been invaluable in the programming and benchmarking work behind much of this thesis. Thanks are also due to Lucas, Thomas and the other Department

technical service staff who (in between fixing other things for me) installed the web server for Pro-origami, and to Dr Tony Wirth for paying for it.

I am grateful to Prof. Justin Zobel for his advice on research and academic writing, and for particular discussions on RNA structural alignment. All the other members of the NICTA genomics group, and the COMBINE bioinformatics group (now the Australian Regional Student Group of the ISCB) have provided stimulating discussions which have contributed to my understanding of bioinformatics. In this context I particularly acknowledge the efforts of Mr Geoff Macintyre, Dr Justin Bedo, Mr Greg Paperin and other members of the COMBINE committee in organizing various social and academic events for the group.

Thanks to Dana and Sabrina for proof reading this thesis (or parts thereof), and Tara for helping me out with printing. Any remaining errors are, of course, entirely my fault.

I am grateful to A/Prof. Steven Bird, Dr Tim Baldwin, Dr Linda Stern and Prof. Justin Zobel, for employing me as a tutor or assistant at various times, giving me experience in teaching (and useful income). I am also indebted to Pinoo, who knows how everything really works in the Department.

My officemates Kian, Abdulla, Graeme and Ben, and Olya, have provided much discussion, some of it even relevant to our theses. Likewise all the other graduate students and members of the Department with whom I have had many a conversation in the staff room.

Thanks are also in order for everyone who has given me a place to live or shared a house with me during my PhD candidature: Alison & David, John & Jackie, Sacha, Tim, Joe and Joy.

Contents

1 Introduction **21**

1.1 Thesis overview 22

2 Background **25**

2.1 Dynamic programming 25

2.1.1 Bounding for dynamic programs 29

2.2 Introduction to RNA structure 33

2.3 Introduction to protein structure 38

2.4 Sequence alignment and structural alignment 46

2.5 Evaluating database searching methods 52

3 RNA Structural Alignment **57**

3.1 Introduction 57

3.1.1 Tree representations of RNA secondary structure 59

3.1.2 Tree and arc alignment algorithms 61

3.1.3 Other approaches to RNA structural alignment 62

3.1.4 RNA base pairing probability matrix alignment 64

3.1.5 SimulFold 66

3.1.6 RNA Sampler 66

3.1.7 MASTR 67

3.1.8 Pruning of the dynamic programming matrix 69

3.1.9 FoldalignM 70

3.1.10 Lara 70

3.1.11 Aln3nn 71

3.1.12 BlockMSA 71

3.2 Benchmarking RNA structural alignment algorithms 72

3.2.1 Methods 73

3.2.2 Results and discussion 74

3.3 Improving the performance using bounding on a simplified case . . 78

3.3.1 Methods 78

3.3.2 Results and discussion 80

3.4 Using bounding to attempt to improve performance on the real problem 81

3.4.1 Methods . . . 81
3.4.2 Results and discussion . . . 84
3.5 Bounding for purely sequence alignment . . . 89
3.5.1 Methods . . . 90
3.5.2 Results and discussion . . . 91
3.6 Conclusions . . . 93

4 Parallel Dynamic Programming **95**
4.1 Introduction . . . 96
4.2 Methods . . . 97
4.2.1 Lock-free concurrent hash tables . . . 99
4.3 Results and discussion . . . 109
4.3.1 Knapsack . . . 113
4.3.2 Shortest paths . . . 114
4.3.3 RNA base pairing probability matrix alignment . . . 116
4.3.4 Minimization of Open Stacks . . . 121
4.4 Conclusions . . . 125

5 Protein Substructure Searching **127**
5.1 Introduction . . . 128
5.2 Methods . . . 132
5.2.1 Tableaux . . . 132
5.2.2 Quadratic integer programming formulation of extraction of maximally-similar subtableaux . . . 136
5.2.3 Relaxed quadratic programming formulation and solution by interior point method . . . 141
5.2.4 Evaluation . . . 142
5.2.5 Implementation . . . 146
5.3 Results and discussion . . . 147
5.3.1 Comparison with MAX-CMO heuristic . . . 153
5.3.2 Examples . . . 157
5.3.3 Substructure search . . . 159
5.3.4 Non-sequential matchings . . . 164
5.3.5 Performance improvements . . . 166

5.4 Conclusions 168

6 Accelerating Protein Substructure Searching 171

6.1 Methods 172

6.1.1 CUDA 172

6.1.2 Application of simulated annealing to the problem 173

6.1.3 Parallel implementation on a GPU 176

6.1.4 Evaluation 178

6.1.5 Implementation 179

6.2 Results and discussion 181

6.2.1 Protein structure search (fold classification) 181

6.2.2 Substructure queries 191

6.2.3 Statistical analysis of tableau matching scores 193

6.3 Conclusions 196

7 Pro-origami: Protein Structure Cartoons 197

7.1 Introduction 197

7.2 Constraint-based diagram layout 211

7.3 Methods 212

7.3.1 Graph representation of secondary structure 214

7.3.2 SSE and sheet orientation and tableaux 216

7.3.3 Distance matrices 216

7.3.4 Sheet and barrel detection 216

7.3.5 Secondary structure element placement and orientation . . . 218

7.3.6 Helix clustering 225

7.3.7 Domain placement 227

7.3.8 Shape colouring and labelling 228

7.3.9 Connector colouring 229

7.3.10 Diagram layout 231

7.3.11 Implementation 231

7.4 Results and discussion 232

7.5 Pro-origami web server interface 245

7.5.1 The Pro-origami web server 246

7.5.2 Building substructure queries 247

7.5.3 Job scheduling and results presentation 247
7.5.4 Implementation 251
7.6 Conclusions 253

8 Conclusions **255**

List of Figures

2.1 Call tree for top-down computation of example 0/1 knapsack problem. 29

2.2 Call tree for computation of example 0/1 knapsack problem with local bounding. 32

2.3 Call tree for computation of example 0/1 knapsack problem with global bounding and trivial initial lower bound. 32

2.4 RNA secondary structure. 35

2.5 Three-dimensional structure of a tRNA. 36

2.6 Linear (base pairing arc) representation of a tRNA. 36

2.7 Example of an RNA pseudoknot structure. 37

2.8 Base pairing probability matrix dot plot. 38

2.9 Two residues in a peptide chain showing the ϕ and ψ angles. 40

2.10 Ramachandran plot. 41

2.11 Secondary structure. 42

2.12 Protein three-dimensional structure 43

2.13 Protein contact map. 44

2.14 The two chiral forms of the β-α-β motif. 45

2.15 Secondary structure of two tRNA sequences. 49

2.16 Example RNA structural alignment. 49

2.17 Example protein structural alignment. 51

3.1 Tree representation of RNA secondary structure. 60

3.2 Tree edit and alignment distance. 63

3.3 BRALISCORE (SPS×SCI) versus APSI (Average Percentage Sequence Identity) for multiple alignments of $k = 2, 3, 5, 7, 10$ and 15 sequences using different methods. 75

3.4 CPU time versus average sequence length for multiple alignments of $k = 2, 3, 5, 7, 10$ and 15 sequences using different methods. 76

3.5 Number of cells computed with and without bounding on simplified problem. 80

3.6 CPU seconds used for different bounding options (C implementation) 84

3.7 Cells computed for different bounding options (C implementation) . 85

3.8 Number of times value computed is not greater than lowbound (C implementation) 86
3.9 Number of times value computed is not greater than lowbound, for "perfect" bounding perturbed by different fractions (Python implementation) 87
3.10 Number of calls to dynamic programming recursive function for "perfect" bounding perturbed by different fractions (Python implementation) 88
3.11 User CPU seconds for local bounding and no bounding (Python implementation) 89
3.12 Scatter plot of upper bound versus actual value for M26977 and M87833 90
3.13 CPU usage for sequence alignment with no, local, and global bounding 91
3.14 Number of cells computed for sequence alignment with no, local, and global bounding 92
3.15 Scatter plot of upper bound versus actual Needleman-Wunsch score for M26977 and M87833 93

4.1 Generic top-down dynamic programming code on the left, and the parallelized version on the right. 98
4.2 Semantics of the CompareAndSwap operation. 101
4.3 Algorithms for the par_insert and par_lookup operations on the concurrent lock-free open addressing hash table. 103
4.4 Speedup for lock-free open addressing and separate chaining hash tables on UltraSPARC T1 (SunFire T2000, 1 GHz, 8 cores, 32 total concurrent threads, 16 GB RAM), 2/3 lookup and 1/3 insert. 106
4.5 Speedup for lock-free open addressing and separate chaining hash tables on PowerPC 64-bit (IBM eServer pSeries 650, 1.45 GHz, 8 cores, 16 GB RAM), 2/3 lookup and 1/3 insert. 107
4.6 Speedup of different concurrent hash table implementations on AMD Quad Core Opteron (2.3GHz, 2 processors, total 8 cores, 32 GB RAM), 2/3 lookup and 1/3 insert. 108
4.7 Speedup for parallelized dynamic programs using the lock-free open addressing hash table on UltraSPARC T1. 110

4.8 Speedup for parallelized dynamic programs using the lock-free open addressing hash table on IBM PowerPC (8 cores). 111
4.9 Speedup for parallelized dynamic programs using the lock-free open addressing hash table on AMD Quad Core Opteron (2.3GHz, 2 processors, total 8 cores, 32 GB RAM). 112
4.10 Total computations (number of times the par_insert function is called, h) for the knapsack dynamic program using the lock-free open addressing hash table on UltraSPARC T1. 114
4.11 Increase in total computations. 115
4.12 Speedup for parallelized RNA base pairing matrix alignment using an array to store computed values on the UltraSPARC T1 (32 cores). . 117
4.13 Speedup for parallelized RNA base pairing matrix alignment using an array to store computed values on the IBM PowerPC (8 cores). . . 118
4.14 Speedup for parallelized RNA base pairing matrix alignment using an array to store computed values on the AMD Opteron (8 cores). . . 119
4.15 The A^* dynamic programming algorithm for the open stacks problem (Garcia de la Banda and Stuckey, 2007). 123

5.1 β-grasp query structure and tableau. 133
5.2 Computing the relative angle ω between two SSE axes. 136
5.3 Axes fitted to strands in a sheet. 137
5.4 Axes fitted to helices 138
5.5 Double-quadrant encoding of orientations. 139
5.6 ROC curves without using distance information. 148
5.7 ROC curves using distance information. 148
5.8 Tableau for `d1ubia_` (left) and its maximally-similar subtableau found in `d1or0.1` (right). 149
5.9 `d1ubia_` structure (left) and the corresponding SSEs found in `d1or0.1` (right). 149
5.10 Histogram of tableau sizes. 151
5.11 ROC curves for different methods. 152
5.12 ROC curves for MSVNS3 and QP Tableau Search with different normalization functions on the Fischer data set at fold level. 154

5.13 ROC curves for MSVNS3 and QP Tableau Search with different normalization functions on the Fischer data set at class level. 155
5.14 ROC curves for MSVNS3 and QP Tableau Search with different normalization functions on the Nh3D data set at architecture level. . . . 156
5.15 ROC curves for MSVNS3 and QP Tableau Search with different normalization functions on the Nh3D data set at class level. 156
5.16 Superposition of β-grasp query hits. 158
5.17 Serpin B/C sheet query substructure. 158
5.18 The serpin B/C sheet query substructure found in a serpin from a thermophilic prokaryote. 160
5.19 Superposition of serpin B/C sheet query hits. 161
5.20 Superposition of 2JMK and d1kb9b1_. 165
5.21 Histograms of tableau sizes for ASTRAL SCOP version 1.75 95% sequence identity non-redundant database. 168

6.1 The CUDA thread programming model. 174
6.2 AUC for different methods for 200 queries in the ASTRAL 95% data set. 182
6.3 AUC versus elapsed time for SA Tableau Search. 185
6.4 AUC for different methods in the Fischer data set. 186
6.5 AUC for different methods in the COPS benchmark data set. 187
6.6 The β-grasp motif query. The SSEs forming the query are in red. . . . 192
6.7 Distribution of SA Tableau Search scores. 194
6.8 Coverage versus Error plot for SA Tableau Search scores. 195

7.1 Early protein topology diagram style used by Schulz and Schirmer (1974). 199
7.2 Early protein sequence/topology style used by Schulz *et al.* (1974). . . 199
7.3 Topology/packing diagram used by Levitt and Chothia (1976). 201
7.4 Topology diagram used by Sternberg and Thornton (1977). 201
7.5 TOPS cartoon for thioredoxin. 202
7.6 β-sheet topology diagram used by Richardson (1977). 202
7.7 Holliday junction resolvase, PDB id 1HH1 (Bond *et al.*, 2001). Image generated with PyMOL (DeLano, 2002). 203

7.8 HERA Hydrogen bonding diagram of Holliday junction resolvase Hjc (Bond *et al.*, 2001), PDB code 1HH1, drawn with PROMOTIF version 2. 204
7.9 PDBsum topology diagram of Holliday junction resolvase Hjc (Bond *et al.*, 2001), PDB code 1HH1, from the PDBsum website. 205
7.10 TOPS cartoon of 1HH1. 206
7.11 Example of a cartoon with overlapping symbols. 209
7.12 Example of a cartoon of a β/α-barrel structure drawn with helices between strands in a sheet. 209
7.13 Example of a parallel sheet structure with the helices drawn below the strands of the sheet. 210
7.14 The graph representation of 1HH1 used internally and drawn with the Graphviz `neato` program (Gansner and North, 2000). 215
7.15 Hydrogen bond overlap criterion for determining if two strands are on the same or opposite sides of a reference strand. 219
7.16 Using dihedral angle between strand axes to determine if two strands are on the same or opposite sides of a common neighbour strand. . . 220
7.17 The strand offset value is c where $c = a - b$. 220
7.18 Strand offsets used as exact spacing constraints. 221
7.19 Side-by-side comparison of a 3D cartoon and a Pro-origami cartoon of β-sandwich structure (the human T cell coreceptor CD8 (Leahy *et al.*, 1992), PDB identifier 1CD8). 223
7.20 A 3D cartoon and Pro-origami cartoon of the N-terminal domain of Plu-MACPF (Rosado *et al.*, 2007), PDB identifier 2QP2. 224
7.21 A 3D PyMOL cartoon side-by-side with a Pro-origami cartoon for α_1-antitrypsin (Elliott *et al.*, 2000), PDB id 1QLP, with helix clustering and helix proximity colouring enabled. 226
7.22 Comparison of a manually drawn cartoon of 1HH1 with those automatically generated by Pro-origami and Dunnart (Wybrow, 2008; Dwyer *et al.*, 2009). 234
7.23 Pro-origami and TOPS cartoons for two serpins, one of which has two chains in the PDB structure, showing that the structural similarity between them is still strongly conveyed in these cartoons. 235

7.24 Comparison of different cartoon styles for *E. coli* MobB. 236
7.25 Comparison of different cartoon styles for a TIM-barrel structure. . . 239
7.26 The structure of α_1-antitrypsin, PDB identifier 1QLP. 240
7.27 Comparison of Pro-origami cartoons for 1QLP when the sheets and helices are defined by DSSP and by records in the PDB file. 241
7.28 Comparison of different cartoon styles for subtilisin. 242
7.29 Some chirality information is conveyed neither by TOPS nor by Pro-origami cartoons. 244
7.30 Web server hovertext example. 248
7.31 Web server connector overlap example. 249
7.32 Web server substructure query building example. 250
7.33 Web server search results example. 252

List of Tables

2.1 Example 0/1 knapsack problem. 27

2.2 Dynamic programming matrix for example 0/1 knapsack problem. . 28

2.3 IUPAC-IUB standard abbreviations for the 20 naturally occurring amino acids. 39

3.1 Methods evaluated in the RNA structural alignment benchmarking. 73

4.1 Maximum speedup for parallelized dynamic programs using randomization and the lock-free open addressing hash table on UltraSPARC T1. 109

4.2 Maximum speedup for parallelized dynamic programs using randomization and the lock-free open addressing hash table on IBM PowerPC (8 cores). 110

4.3 Maximum speedup for parallelized dynamic programs using randomization and the lock-free open addressing hash table on AMD Quad Core Opteron (2.3GHz, 2 processors, total 8 cores, 32 GB RAM). 110

5.1 AUC and time for some widespread folds. 147

5.2 AUC for the 200 query set. 152

5.3 Area under the ROC curve (AUC) for the Fischer data set at fold level. 154

5.4 Area under the ROC curve (AUC) for the Fischer data set at class level. 154

5.5 Area under the ROC curve (AUC) for the Nh3D data set at architecture level. 155

5.6 Area under the ROC curve (AUC) for the Nh3D data set at class level. 157

5.7 Comparison of methods for substructure searching. 161

5.8 Comparison of the unique hits from each method for substructure searching. 163

5.9 AUC for non-sequential matchings averaged over five permutations of each of the fold query tableaux. 164

5.10 AUC, elapsed time and speedup relative to the UMFPACK implementation for the 200 query set with different implementations. . . . 167

6.1 AUC and elapsed time for 200 queries in the ASTRAL 95% data set. 181

6.2 ΔAUC relative to SA Tableau Search for 200 queries in the ASTRAL 95% data set. 183
6.3 AUC and elapsed time in the Fischer data set. 184
6.4 ΔAUC relative to SA Tableau Search in the Fischer data set. 184
6.5 AUC and elapsed time in COPS benchmark data set. 188
6.6 ΔAUC relative to SA Tableau Search in the COPS benchmark data set. 188
6.7 AUC and elapsed time for some motif (substructure) queries. 193

1

Introduction

The *central dogma of molecular biology* (Crick, 1970) (and its associated *sequence hypothesis*) is that the direction of information flow is from DNA to protein via RNA, and, specifically, not from protein backwards to RNA or DNA. Together with *Anfinsen's dogma* (Anfinsen, 1973), that the structure of a protein is determined by its amino acid sequence, this means that (at least in theory), everything about the structure (and hence function) of a protein could be deduced from the sequence encoding that protein. However, prediction of protein structure from sequence (the *protein folding problem*) is an unsolved (or, at least, incompletely solved) problem (Dill *et al.*, 2007) and therefore the process of solving the structure of proteins experimentally is still required.

For this reason, and together with advances in DNA sequencing technology giving rise to very large amounts of sequence data, the relatively new field of *bioinformatics* has come into being, and traditionally has dealt largely with sequence analysis, often with statistical methods (for example, Durbin *et al.* (1998)). Efficient methods for sequence similarity search, such as BLAST (Altschul *et al.*, 1990, 1997) are some of the most important and widely used tools in bioinformatics.

Recently, the importance of *ncRNA* (noncoding RNA — that is, RNA that does not encode a protein) has become increasingly apparent, and sequence similarity search is difficult for ncRNA as structure may be more conserved than sequence (Eddy, 2002). Existing algorithms are often too computationally intensive for large-scale applications and therefore improving the efficiency of pairwise and multiple

structural alignment of RNA is of some importance in the area of functional genomics.

Therefore, there is a need for bioinformatics tools not only for sequence analysis, but also for analysis and comparison of the structure of both RNA and proteins. The research described in this thesis aims to contribute to the fields of bioinformatics and computational molecular biology, and computer science, by demonstrating a general technique for improving the performance of *dynamic programming* (introduced in Chapter 2), and its application to RNA structural alignment, an improved algorithm for protein structural search, including substructure and non-sequential structure, and a new technique for automatic generation of (two-dimensional) protein structure diagrams. The latter two contributions are combined in the creation of a system which allows the selection of parts of a protein structure as shown in its automatically generated cartoon to be used as a substructure query against a database of protein structures.

1.1 Thesis overview

Chapter 2 describes the basics of RNA and protein structure, and dynamic programming. This background is important for understanding the remainder of the thesis.

Chapter 3 reviews a range of algorithms for RNA structural alignment and describes the methods and results of an exercise in benchmarking some existing implementations of these methods for speed and a particular measure of accuracy. Based on these results, we select one technique, which uses dynamic programming, as the most promising, and apply bounding techniques to attempt to improve its performance, with little success. Investigation as to why this should be so leads us to experiment with bounding for purely sequence alignment, with some limited success in small cases, which are apparently not scalable.

This leads us, in Chapter 4, to investigate an alternative method, parallelization, to improve the performance of dynamic programs. Reasonable speedups are obtained for the RNA structural alignment problem. However, Chapter 4 is a more general contribution to computer science rather than to bioinformatics or computa-

tional molecular biology specifically, and the best results of the problems we apply the parallelization to are for a problem in manufacturing logistics.

We then move from RNA to protein structure, and Chapter 5 describes a method for finding structurally similar proteins, or proteins that contain similar substructures, based on their tableaux representation (Lesk, 1995; Konagurthu *et al.*, 2008) and *quadratic programming*.

Chapter 6 extends the work on protein structure and substructure searching described in Chapter 5 by using a different heuristic technique, improving both speed and accuracy. Further, this technique can be accelerated by implementation on a graphics processing unit, improving the speed even more.

Chapter 7 reviews the history of protein structure cartoons and previous techniques for generating them, and describes a new system, which we call Pro-origami, for the automatic generation of protein structure cartoons from atomic co-ordinates. This technique is based on a constraint-based approach to diagram layout (Wybrow, 2008). Chapter 7 also describes a web server for Pro-origami, which is then used as a user interface to the protein substructure search systems described in Chapter 5 and Chapter 6.

Finally, Chapter 8 presents the conclusions of our research and suggests some future directions.

2

Background

This chapter introduces some important concepts used throughout the thesis. First, we describe dynamic programming, which we use extensively in Chapters 3 and 4. In particular, we introduce the technique of using bounding functions to improve the performance of dynamic programming by branch-and-bound, which is used in Chapter 3. Then, in Section 2.2, we introduce the basics of RNA structure, also important for Chapter 3, and in Section 2.3, we provide an introduction to protein structure, required for Chapters 5, 6 and 7. In Section 2.4 we introduce the concepts of sequence and structural alignment, which are used throughout bioinformatics. In Section 2.5, we describe a method of evaluating the accuracy of database searching methods, which we use in Chapters 5 and 6, where we describe methods for searching a database of protein structures.

2.1 Dynamic programming

Dynamic programming (Bellman, 1957) is a powerful technique for solving any optimization problem for which an optimal solution can be efficiently computed from optimal solutions to its subproblems. Such problems are said to have *optimal substructure*, derived from Bellman's "Principle of Optimality":

> P O . An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.

(Bellman, 1957, p. 83)

In implementing an algorithm to solve a dynamic program, the idea is to avoid recomputing the optimal solution to subproblems by reusing previously computed values. Thus, for dynamic programming to be useful, the same subproblems must be encountered sufficiently often while solving the original problem.

Dynamic programming can be easily implemented using either a *"bottom-up"* or *"top-down"* approach. In the "bottom-up" approach, the solution to every single subproblem is computed and stored in the dynamic programming matrix, starting from the smallest subproblems until the solution to the entire problem is finally computed. This approach is particularly simple to implement; it requires no recursion and no data structure more sophisticated than an array. It is also efficient if (a) the problem is small enough for the entire matrix to be stored in memory, and (b) the computation of unnecessary cells does not introduce too much overhead. The classic bioinformatics sequence alignment algorithms of Needleman and Wunsch (1970) and Smith and Waterman (1981), which will be described in Section 2.4 below, are generally implemented in this way, for example, and hence dynamic programming is essential to bioinformatics.

In contrast, the "top-down" approach starts from the function call to compute the solution to the original problem, and uses recursion to only compute the solution to those subproblems that are actually encountered when solving the original problem. Note that here we are considering the recurrence relation describing the dynamic program in computer science terms as evaluation of a recursive function. Previously computed values are reused by applying a technique called *memoization* (Michie, 1968). In this technique each computed value is stored in an associative array (implemented, for example, by a hash table). Then, the recursive function tests if the value it is called for has been previously computed (and therefore exists in the associative array) and, if so, simply reuses the value rather than recomputing it. This approach to implementing dynamic programming avoids the computation of unnecessary values and is particularly effective when combined with branch-and-bound techniques to further reduce unnecessary computations (Puchinger and Stuckey, 2008).

In this section, we will use the *0/1 knapsack problem*, a classic "textbook" example of dynamic programming to illustrate various aspects of dynamic programming.

In Chapter 4, we will also use this example to demonstrate the application of our technique for parallelizing dynamic programs.

In the 0/1 knapsack problem, we are given the total weight (or capacity) W of the knapsack, and a set of n items $\{1, \ldots, n\}$, where each item i has been assigned a weight w_i and a profit p_i. The problem is to choose the subset of items $I \subseteq \{1, \ldots, n\}$ such that $\sum_{i \in I} w_i \leq W$ and profit $\sum_{i \in I} p_i$ is maximized. The dynamic programming formulation computes the optimal profit $k(i, w)$ using only items in $1, \ldots, i$ with a capacity w as:

$$k(i,w) = \begin{cases} 0 & \text{if } i = 0 \vee w = 0 \\ k(i-1,w) & \text{if } w < w_i \\ \max\{k(i-1,w), k(i-1,w-w_i) + p_i\} & \text{otherwise} \end{cases} \tag{2.1}$$

An example instance of the 0/1 knapsack problem is shown in Table 2.1. In this example, there are 4 items, and the knapsack has a total capacity $W = 5$. It is fairly easy to see in this small example that the optimal solution is to choose items 1 and 2 for a total profit of 7.

item	profit	weight	
1	3	2	
2	4	3	Capacity: $W = 5$
3	5	4	
4	6	5	

Table 2.1: *Example 0/1 knapsack problem.*

The dynamic program matrix (or table) resulting from bottom-up computation of the example instance is shown in Table 2.2. This matrix is computed by filling in the cells starting from the base case ($i = 0 \vee j = 0$), that is, filling in the first row and column with all zeros. Then each element of the subsequent rows can be computed (from left to right) according to the recursive cases of Equation 2.1. The optimal solution is then the last entry in the matrix, $k(4, 5) = 7$.

To use the bottom-up technique, an order in which to compute the cells must be defined. This ordering must be such that, for each cell, the cells on which its value depends must already have been computed. In the case of the 0/1 knapsack problem, such an ordering is easy to find. However, for more complicated dynamic programs this may not be the case. As we shall see in Chapter 4, this analysis of the data dependencies in the bottom-up dynamic programming matrix means that

		w					
		0	1	2	3	4	5
	0	0	0	**0**	0	0	0
	1	0	0	**3**	3	3	3
i	2	0	0	3	4	4	**7**
	3	0	0	3	4	5	**7**
	4	0	0	3	4	5	**7**

TABLE 2.2: *Dynamic programming matrix for example 0/1 knapsack problem.*

previous work on parallelizing dynamic programs has required analysis specific to each particular problem.

The entries in boldface in Table 2.2 show the *traceback* procedure used to find the set of items that result in the optimal profit. This tracing back procedure defines a path through the matrix which starts at the optimal solution, and, for each cell, determines which case of the recurrence relation was used to arrive at that cell. The case used indicates whether or not the item (i) was added to the knapsack for each cell. In this example instance it shows that the case corresponding to adding an item ($k(i-1, w-w_i)+p_i$, that is, the only case where an item's profit p_i is added to the current solution) was used only for items 1 and 2. The optimal profit of 7, therefore, is achieved by selecting items 1 and 2.

This tracing back procedure is necessary if we want to know not just the optimum value of the function, but the values of its variables that result in that solution. However time to perform the traceback, once the optimal solution has been found, is not significant compared to that of actually computing the solution, and so we will not consider it further. We will concentrate only on the computation of the optimal solution.

Figure 2.1 shows the call tree for a top-down computation of the example instance from Table 2.1. The root node is the value that needs to be computed for the overall solution $k(4,5)$. The children of each node are the values that need to be computed in order to compute the value at that node; that is, the instances of the function called recursively by the instance represented by the node. In the case of the 0/1 knapsack problem, this is a binary tree, since there are at most two recursive calls in any case specified by Equation 2.1. Notice that while the bottom-up computation (Table 2.2) computes the $k(i,w)$ function at 30 different points (all 6 values of w multiplied by all 5 values of i), the top-down computation (Figure 2.1)

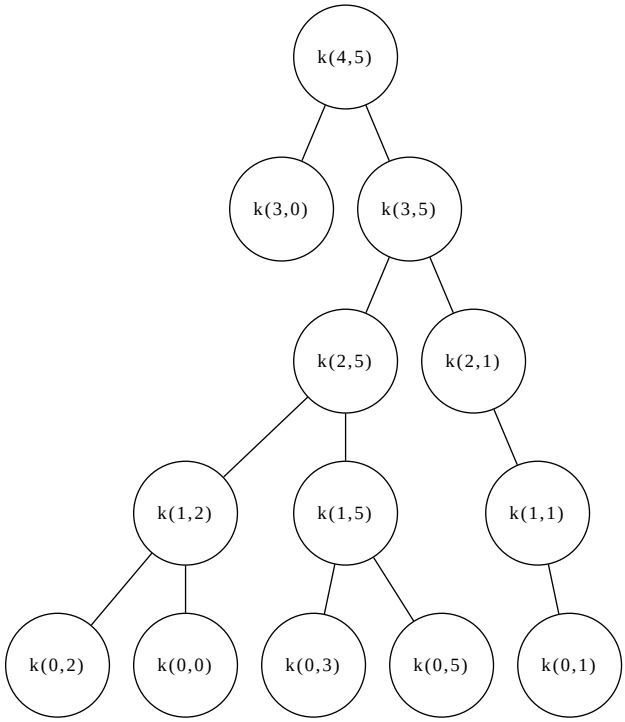


FIGURE 2.1: *Call tree for top-down computation of example 0/1 knapsack problem.*

computes the function at only 13 different points. Specifically, it computes $k(i, w)$ for only those values of i and w that are required to compute the overall solution at the root node $k(4, 5)$.

2.1.1 Bounding for dynamic programs

As we have just seen, the use of the top-down, rather than bottom-up, implementation of dynamic programming can result in a reduction in the total number of computations of the function being optimized. This is due to the top-down implementation needing only to compute values for those subproblems that are actually necessary for computing the optimal value of the entire problem. Once we no longer have to compute the values for all subproblems, this opens up the possibility of using *bounding* to reduce the number of computations even further.

Bounding refers to the use of lower or upper bounds on the value of the function being optimized. The bounding function can be used to prune branches of the top-down computation that cannot lead to an optimal solution. That is, the classic computer science technique of "branch-and-bound".

The use of bounds to improve dynamic programming goes back at least as far as Morin and Marsten (1976). In that paper, which considers dynamic programs in the form of minimization or maximization over a sum of incremental costs, a global lower bound is used. Then, if the incremental costs required to reach a subproblem, plus the upper bound on the subproblem, is less the global lower bound, that subproblem need not be considered. This technique was extended by Carraway and Schmidt (1991), by using a lower bounding function to improve the global lower bound. Puchinger and Stuckey (2008) describe a general technique to improve the performance of any dynamic program (for which we have a bounding function) using local bounding, ordering, or argument bounding. In the following descriptions, as in Puchinger and Stuckey (2008), we will assume the dynamic program is a maximization problem (such as knapsack) and that we have an upper bounding function.

Local bounding allows us to omit computation of one or more of the alternatives in the recurrence relation, given the upper bounding function and the already computed values of some of the other alternatives. If we have computed one case, with a value x, and an upper bound on a second case, with a value u, then the second case need not be computed at all if $u \leq x$. The value x is a known lower bound (it is the value of one of the cases of the recurrence relation), and so if the upper bound u on the second case is not greater than the known lower bound x on the function value, that case cannot be the optimal value and so need not be computed. Note that the use of local bounding on a dynamic program cannot increase the number of calls to the recursive function (Puchinger and Stuckey, 2008). Therefore, the only possible additional cost of using local bounding is the cost of computing the bounding function itself.

Ordering refers to the technique of ordering the evaluations of the subproblems in such a way as to evaluate those subproblems that are more likely to be the optimal solution first. We can use the upper bounding function to estimate which of the subproblems is more likely to lead to the optimal solution. If there are two subproblems, $f(\bar{x_1})$ and $f(\bar{x_2})$ with upper bounds u_1 and u_2 respectively, then we would compute $f(\bar{x_1})$ before $f(\bar{x_2})$ if $u_1 > u_2$ and vice versa. This technique is easily combined with local bounding.

Argument bounding, also referred to as *global bounding*, passes lower bounds (values of already computed subproblems) to the function call for each subproblem, using upper bounds to prune the computation of subproblems that cannot lead to the optimal solution. The memoization of the optimal value is reused to also store the lower bounds. In each function call, if the upper bound u is no greater than the lower bound l (passed in as an argument) then the upper bound is returned. Otherwise the subproblems are computed using the lower bound l, and if the result is greater than l it is stored as the optimal value, otherwise it is stored as a bound. On the initial function call, a heuristic lower bound is passed in (or a trivial lower bound if no such heuristic is available). Argument bounding is also easily combined with ordering. One potential problem with argument bounding is that, unlike local bounding, the number of calls to the dynamic programming function can actually increase. This can occur when the same function call $f(\bar{x}, l)$ is called for different values of l, when l occurs in an decreasing sequence but is still greater than the optimal value (Puchinger and Stuckey, 2008). Although this situation, which we will refer to as *recomputation*, was found to be rare in the work of Puchinger and Stuckey (2008), in Chapter 3 (Section 3.4.2) we show a dynamic program for which it occurs to such an extent that the use of global bounding is deleterious.

As in Puchinger and Stuckey (2008), we will use an upper bound for the 0/1 knapsack problem due to Dantzig (1957) as an example. This upper bound is the sum of the k most profitable items that fit within the knapsack capacity W, plus the proportion of the $(k+1)$th most profitable item that fits in the remainder. Here we define profitability as the profit-to-weight ratio p_i/w_i. Note that the items in the example instance in Table 2.1 are sorted by profitabililty; item 1 has the highest profitability ($p_1/w_1 = 1.5$) and item 4 the worst ($p_4/w_4 = 1.2$). A lower bound can be similarly derived: the optimal profit must be at least the sum of the k most profitable items that fit within the knapsack capacity W.

We are now in a position to demonstrate the application of bounding to the 0/1 knapsack problem.

Figure 2.2 shows the call tree of a local bounding implementation for the example instance in Table 2.1. Only 9 calls to the function are required, compared to 13 for the version without bounding (Figure 2.1); the $k(3, 0)$ branch from the root is pruned, as is the $k(2, 1)$ branch from $k(3, 5)$.

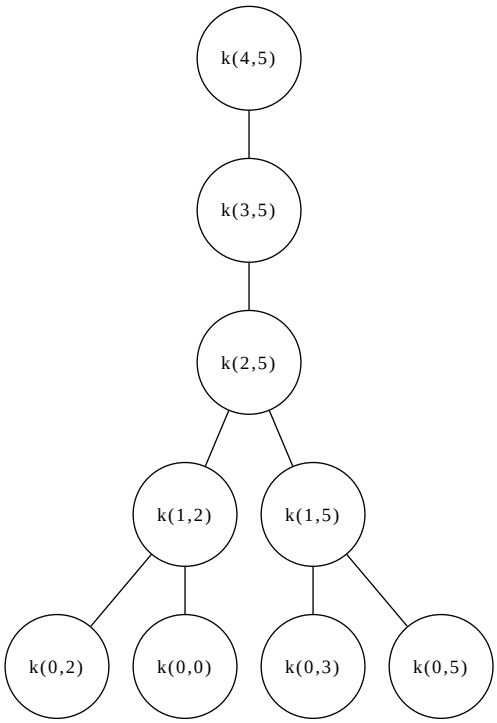


FIGURE 2.2: *Call tree for computation of example 0/1 knapsack problem with local bounding.*

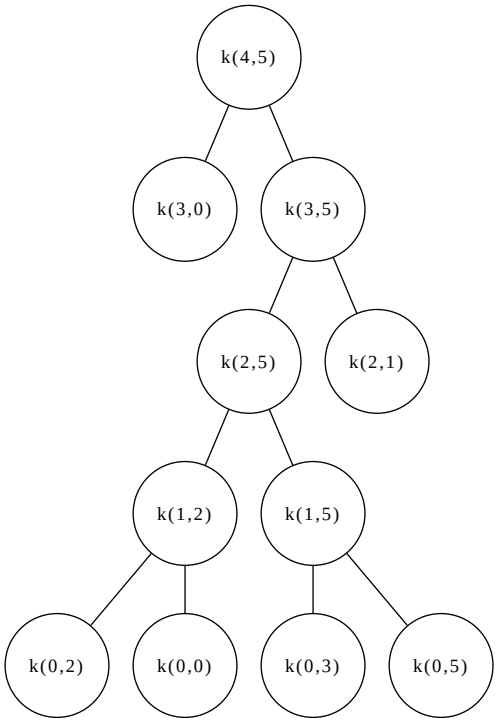


FIGURE 2.3: *Call tree for computation of example 0/1 knapsack problem with global bounding and trivial initial lower bound.*

In global bounding, the new lower bound is now passed as a parameter to each subproblem. The benefits of a good initial lower bound are demonstrated by the example instance (Table 2.1) of this problem. If we use a trivial lower bound of 0, the call tree is that shown in Figure 2.3. This shows 11 calls to the function, which is fewer calls than the unbounded version (Figure 2.1) but more than the locally bounded version. However, if we use the heuristic lower bound described earlier, then in this instance no actual computations are required (apart from computing the bounds themselves) as the lower and upper bounds are equal.

Although we have not discussed ordering for this example, it should now be clear how we can use the bounds to order the computations in the knapsack dynamic program. More details can be found in Puchinger and Stuckey (2008), which also describes program transformations that are able to automatically apply these bounding techniques to dynamic programs expressed in a declarative manner. In addition, the ordering of subproblems is considered in Chapter 3 for the RNA base pairing matrix alignment dynamic program, and in a somewhat different context in Chapter 4 where the ordering of subproblems is randomized as part of a method for parallelizing the implementations of dynamic programs.

2.2 Introduction to RNA structure

As was mentioned in Chapter 1, the flow of information in molecular biology is essentially from DNA sequence, via RNA sequence to the *amino acid* sequence of proteins, which determines how the protein *folds* in three dimensions. Here we describe RNA structure; proteins are described in the next section. A standard reference for molecular biology in which more details can be found is Alberts *et al.* (2008), and for the details of nucleic acids, Saenger (1984). Clote and Backofen (2000) provides an introduction to the application of mathematics and computational methods to molecular biology, including background material on molecular biology and mathematics.

As is well known, DNA (deoxyribonucleic acid) encodes information by the sequence of its four *nucleotides* adenine, thymine, cytosine and guanine, conventionally denoted by the single letters A, T, C, and G, respectively. Strictly speaking, A, T, C, and G themselves are *bases*; they are referred to as nucleotides when at-

tached to the sugar and phosphate group that allows them to form a strand on the DNA. The bases have complementary structures such that A forms bonds with T, and C forms bonds with G, giving rise to the *complementary* strand (whereby every A is matched with a T in the complementary strand, and every C with a G, and vice versa), and the ability of DNA to be replicated by the process known as *templated polymerization*.

The process of *transcription* results in this sequence being transcribed into an RNA (*ribonucleic acid*) sequence. This is similar to DNA, but with the thymine (T) base replaced by the chemically similar U (uracil) base. Hence the "canonical", or Watson-Crick, base pairings for RNA are C-G and A-U. Non-canonical base pairings, such as G-U, also occur in RNA molecules. Unlike DNA, which generally has two strands and folds only into the famous double-helix shape, RNA tends to fold into varied structures with only small regions being helical where complementary bases form hydrogen bonds.

Certain transcribed RNA sequences known as *mRNA* (messenger RNA)[1] may undergo a further process called *translation* whereby sequences of three consecutive bases (*codons*), each of which codes for one of the 20 naturally occurring amino acids, build a string (*polymer*) of amino acids via another kind of RNA called *tRNA* (transfer RNA).

Although the "traditional" view of RNA is that it is mostly a vehicle for the production of proteins via the translation mechanism, recent research suggests it has many other important functions (Eddy, 2002; Mattick, 2003, 2005; Mattick and Makunin, 2006). RNAs that do not code for a protein are known as *noncoding RNA (ncRNA)*, of which tRNA is merely one example (albeit a very important one in molecular biology). Therefore the study of the structure of such RNA molecules is motivated by its necessity in determining their function (Laederach, 2007).

As with DNA, the simplest way to view RNA is simply by its sequence. Unlike DNA, however, RNA molecules, as has been mentioned above, rather than generally having the same structure (such as the DNA double helix), fold into a variety of different structures, determined by their sequence. The hydrogen bonding between particular bases in an RNA molecule, resulting in the helical regions mentioned

[1] I am omitting details such as splicing and promoters, and the details of the genetic code itself — the purpose of this section is only to introduce RNA structure. Details on transcription and translation can be found in any genetics or molecular biology textbook, such as Alberts *et al.* (2008).

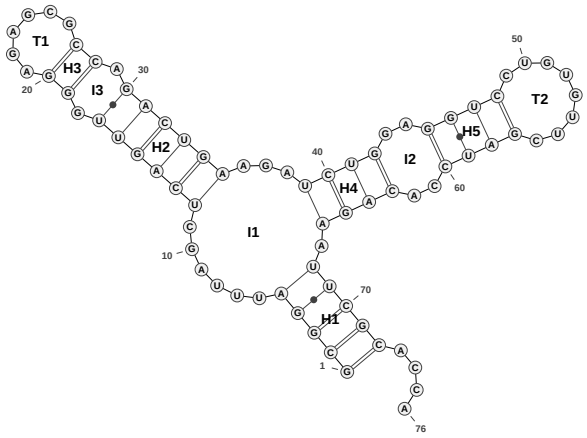


FIGURE 2.4: *RNA secondary structure.*

Secondary structure diagram of a Phenylalanine tRNA from yeast, NDB (Berman et al.*, 1992) id TR0001, as drawn by the* `VARNA` *program (Darty* et al.*, 2009). This figure illustrates some common RNA secondary structures. There are two stem-loop (or hairpin loop) structures: one consisting of the terminal loop* T1 *together with stem (or helix)* H3*, and one consisting of the terminal loop* T2 *together with the stem* H5*. There are three interior loops labelled* I1*,* I2 *and* I3*.*

above, determines its *secondary structure*. Secondary structure can be represented by drawing the molecule in two dimensions (i.e. on the plane) without any base pairs "crossing over" each other. An example is shown in Figure 2.4. This is the secondary structure as predicted by the `RNAfold` (Hofacker *et al.*, 1994) program for a particular tRNA molecule. Figure 2.5 shows the experimentally determined three-dimensional (*tertiary*) structure of this tRNA.

RNA secondary structure consists largely of *stem* and *loop* structures. A common motif is the *stem-loop* or *hairpin loop* structure, where a stem is terminated by a closed loop. The terminating loop is also known as a *terminal loop*. A loop that is not such a terminal (hairpin) loop structure is sometimes called an *interior loop*. Stems are also referred to as helices, since, as can be seen from Figure 2.5, the stem regions which appear simply as "ladders" of base pairs in the two-dimensional secondary structure drawing style of Figure 2.4 are actually helices in three dimensions. These RNA secondary structure motifs are indicated in Figure 2.4 by the annotations T1, T2, etc. for terminal loops, I1, I2, etc. for interior loops, and H1, H2, etc. for stems (helices). Stems are not infrequently interrupted by *bulges*, in which one strand of the stem has a base "inserted" that is not paired with a base in the other strand.

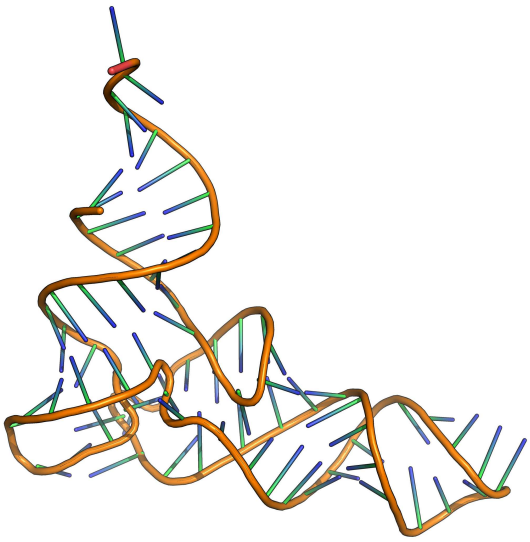


FIGURE 2.5: *Three-dimensional structure of a tRNA.*

3D rendering of the structure of a Phenylalanine tRNA from yeast, NDB id TR0001. This figure was generated with PyMOL (DeLano, 2002).

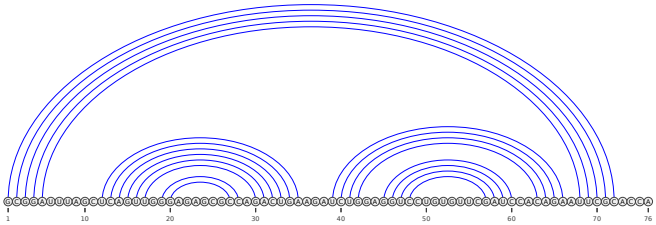


FIGURE 2.6: *Linear (base pairing arc) representation of a tRNA.*

Linear (base pairing arc) representation of the secondary structure of a Phenylalanine tRNA from yeast, NDB id TR0001. This figure was generated with `VARNA`.

As well as the base pairing interactions that constitute the stem regions, base *stacking* interactions between adjacent bases are important in RNA secondary structure. Specifically, the stem-loop structure is stabilised by base stacking interactions.

An alternative, more abstract, representation of RNA secondary structure is shown in Figure 2.6. In this representation, base pairing interactions are shown by arcs connecting the two bases, which are simply drawn along a horizontal line according to the RNA sequence. This can also be represented as a tree, as will be described in Section 3.1.1 of Chapter 3. Note that these base pairing arcs do not cross each other; they are all nested or completely disjoint.

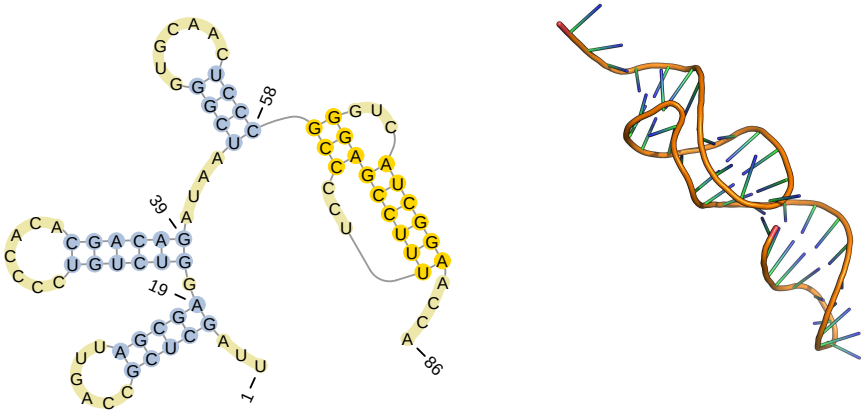


FIGURE 2.7: *Example of an RNA pseudoknot structure.*

Structure of turnip yellow mosaic virus. The planar secondary structure representation (left) is drawn with PseudoViewer3 (Byun and Han, 2009); the pseudoknot is highlighted in yellow. The 3D rendering of the pseudoknot structure (right) was generated with PyMOL from the structure with PDB identifier 1A60 (Kolk et al.*, 1998).*

One structure that arises in RNA molecules but which cannot be represented in a planar diagram, with no arc crossings such as that in Figure 2.6, is the *pseudoknot*. Such a structure consists of base pairing between bases in a loop region and bases outside that loop. A pseudoknot structure would result in a crossing of two arcs in a base pairing arc representation such as that of Figure 2.6. An example is shown in Figure 2.7. Traditionally, an assumption of the absence of pseudoknots has been made in order to render RNA structure prediction and alignment algorithms more tractable (McCaskill, 1990), although more recently more work has been done on predicting pseudoknots, for example Ji *et al.* (2004); Huang and Ali (2007); Meyer and Miklós (2007); Möhl *et al.* (2010). In the remainder of this thesis, when discussing RNA, we will assume an absence of pseudoknots unless otherwise stated.

RNA secondary structure can be predicted from sequence by means of dynamic programming algorithms on the basis of *minimum free energy* (*MFE*). That is, the free energy of the molecular structure is minimized, taking into account the reduction in free energy due to base pairing and stacking. Such algorithms are described in Nussinov *et al.* (1978); Zuker and Stiegler (1981); Sankoff (1985); Hofacker *et al.* (1994); Mathews *et al.* (1999), for example. An alternative technique is to compute the *partition function*. This fully describes the thermodynamic properties of the ensemble of structures at thermodynamic equilibrium. That is, rather than assuming

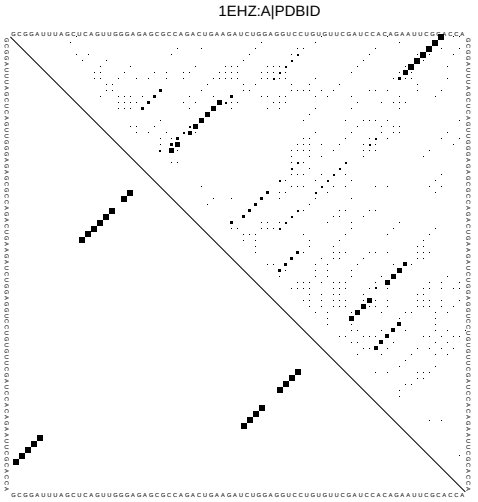


FIGURE 2.8: *Base pairing probability matrix dot plot.*

The base pairing probability matrix for Phenylalanine tRNA from yeast, NDB id TR0001. The upper triangle shows the base pairing log probability matrix, and the lower triangle the MFE (minimum free energy) structure. As one would expect, the MFE base pairings correspond largely with the highest probability base pairings in the base pairing probability matrix. This matrix was computed, and the dot plot drawn, by `RNAfold`*.*

the single MFE structure is the "real" one, the statistical mechanics view of an ensemble of structures is used. The result of this computation is the equilibrium *base pairing probability matrix*. This matrix describes, for each base in the RNA molecule, the probability of it pairing with each one of the other bases in that molecule, in the equilibrium ensemble of structures. This is the approach taken by McCaskill (1990), where a "box matrix" (dot plot) representation, showing the probabilities by boxes whose size increases (logarithmically) with base pairing probability, is also introduced. The algorithm described in McCaskill (1990) is also a dynamic programming algorithm. An example of such a dot plot is shown in Figure 2.8. These base pairing probability matrices are the basis of an RNA structural alignment algorithm (Hofacker *et al.*, 2004) discussed in detail in Chapter 3.

2.3 Introduction to protein structure

Proteins have more complicated structures than RNA. Here we introduce some fundamentals of protein structure; a standard reference for this subject is Branden and Tooze (1999), or the much older but still relevant overview of protein structure in Richardson (1981).

Three-letter symbol	Name	One-letter symbol
Ala	Alanine	A
Arg	Arginine	R
Asn	Asparagine	N
Asp	Aspartic acid	D
Cys	Cysteine	C
Gln	Glutamine	Q
Glu	Glutamic acid	E
Gly	Glycine	G
His	Histidine	H
Ile	Isoleucine	I
Leu	Leucine	L
Lys	Lysine	K
Met	Methionine	M
Phe	Phenylalanine	F
Pro	Proline	P
Ser	Serine	S
Thr	Threonine	T
Trp	Tryptophan	W
Tyr	Tyrosine	Y
Val	Valine	V

TABLE 2.3: *IUPAC-IUB standard abbreviations for the 20 naturally occurring amino acids.*

As with DNA and RNA, the simplest way to think of a protein is as its sequence, or *primary* structure. Proteins are *polypeptides*, that is, they consist of chains of amino acids, connected by peptide bonds (bonds between the amine and carboxyl groups of the amino acids). Amino acids are often referred to as *residues*, when part of such a protein chain. As has been previously mentioned in Section 2.2, there are 20 naturally occurring amino acids. These are listed, with their standard one and three letter abbreviations, in Table 2.3. Each amino acid consists of a central carbon atom (denoted C_α), an *amine* ($-NH_2$) and *carboxyl* ($-COOH$) group. The carbon atom in the carboxyl group is denoted C′. The amino acids differ in that each one has a different *side chain* also attached to its C_α atom.

Peptide bonds form between the carboxyl group of one amino acid and the amine group of the next in the chain. This results in the *main chain* or *backbone* of the protein having two ends, one with an amine group, called the *N-terminus*, and the other with a carboxyl group, called the *C-terminus*. By convention, the residues are always numbered from the N- to the C-terminus.

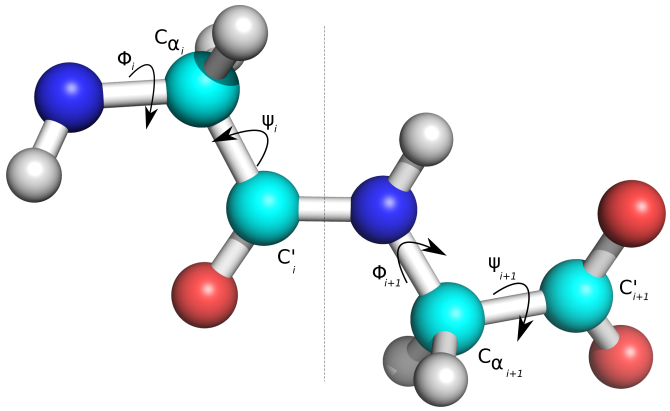


FIGURE 2.9: *Two residues in a peptide chain showing the ϕ and ψ angles.*

Two glycine residues in a peptide chain. Glycine is the simplest amino acid, with no side chain (the C_α atom has a "side chain" consisting of nothing but two hydrogen atoms), simplifying this figure. Carbon atoms are shown in cyan, nitrogen in blue, oxygen in red, and hydrogen atoms in white. Glycine residue i and glycine residue $i+1$ are separated by the vertical dotted line, which runs through the peptide bond between the two residues. Glycine residue $i+1$ is the C-terminal residue of the chain while glycine residue i is an intermediate residue in the chain (the N-terminal residues have been omitted from the figure, leaving only a single hydrogen on its nitrogen atom, which forms a peptide bond to residue $i-1$). This figure was generated using PyMOL.

The protein chain is not a straight line as the two bonds at each C_α atom can each be rotated around, allowing the chain to fold into complex shapes. These two angles of rotation are denoted ψ for the rotation around the $C_\alpha - C'$ bond and ϕ for the rotation around the $N - C_\alpha$ bond. They are often referred to as the *dihedral angles* as they are the angles between the two planes containing the atoms between which the bonds are formed. Figure 2.9 illustrates these concepts.

Only certain values of ϕ and ψ are possible, as most are prevented from existing by collisions between the protein main chain and side chains. These angles can be represented by a *Ramachandran plot*, which plots the values of ϕ and ψ that exist in a protein structure. An example of a Ramachandran plot is shown in Figure 2.10.

Proteins fold into complicated structures, which can be considered on several levels. The primary structure, which we have already mentioned, is simply the sequence of amino acids. The *secondary structure* of a protein consists of two main kinds of structures that regions of the protein fold into: *helices* and *β-strands*. Helices are formed by hydrogen bonding between residues near each other in sequence. They are of three types (depending on the exact sequence position of the bonded residues): *α-helices*, *π-helices*, and *3_{10}-helices*. β-strands are formed by sequences of bonds between residues that may be distant in sequence, forming the β-strands into

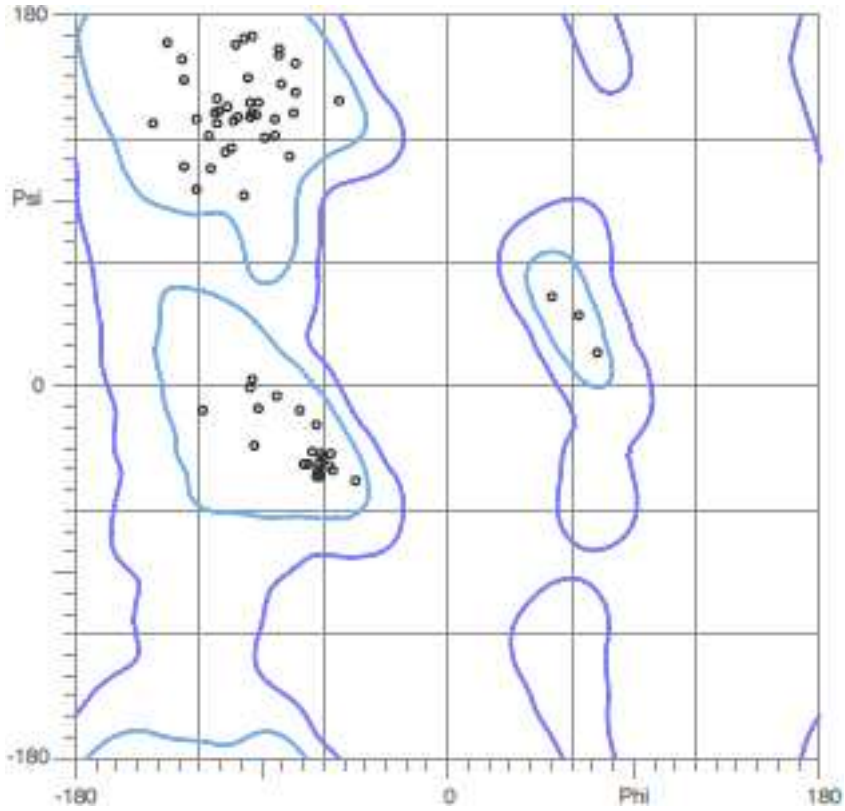


FIGURE 2.10: *Ramachandran plot.*

Ramachandran plot for ubiquitin, PDB identifier 1UBI (Ramage et al.*, 1994). The regions outlined in dark blue show the possible values of* (ϕ, ψ) *(98% of structures are inside these regions) and those in lighter blue the values that contain 99.95% of structures. These allowed regions correspond generally to secondary structures, namely* β *strands (top left), helices (middle left) and coil regions (right). This plot was generated with MolProbity (Chen* et al.*, 2010) from the PDB website (Berman* et al.*, 2000).*

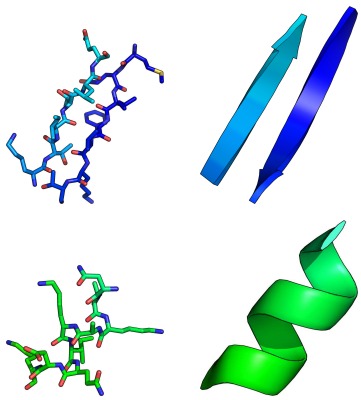


FIGURE 2.11: *Secondary structure.*

"Stick" (left) and "cartoon" (right) representation of common secondary structures: two anti-parallel β-strands (top) and an α-helix (bottom). Because these secondary structures are so common, and important to protein structure, they are often represented in the "cartoon" format (on the right of the figure) to abstract away the detail of backbone angles and sidechains and show only the secondary structure element (SSE) that is formed by the residues. Images generated with PyMOL.

β-sheets, where the strands that form the sheet are held together by hydrogen bonds between the strands. These two types of secondary structure (helices and strands) are generically referred to as *secondary structure elements* (*SSEs*).

We will later (in Chapter 7) refer to adjacent strands as *bridge partners* after the term *bridge*, used to describe the hydrogen bonding pattern between strands used in Kabsch and Sander (1983). Two adjacent strands in a sheet may be either *parallel* or *anti-parallel*, depending on whether they run in the same or opposite directions. A sheet that consists entirely of parallel or anti-parallel strands is described as a parallel or anti-parallel sheet; if it contains both it is described as *mixed*.

Some of these secondary structures are illustrated in Figure 2.11.

Regions that do not form either helical or strand secondary structures are known as *coil*, *loop* or *linker* regions.

Sometimes we refer to *supersecondary* structure; these are recurrent motifs of nearby SSEs, such as two strands separated by a helix (a *β-α-β motif*).

Tertiary structure refers to the full three-dimensional structure of a protein. Figure 2.12 shows two ways of representing the three-dimensional structure of a protein. Note that a protein may consist of more than one polypeptide chain; these multiple chains can interact as part of a single sheet for example.

Quaternary structure refers to multiple protein structures that interact and form a larger structure.

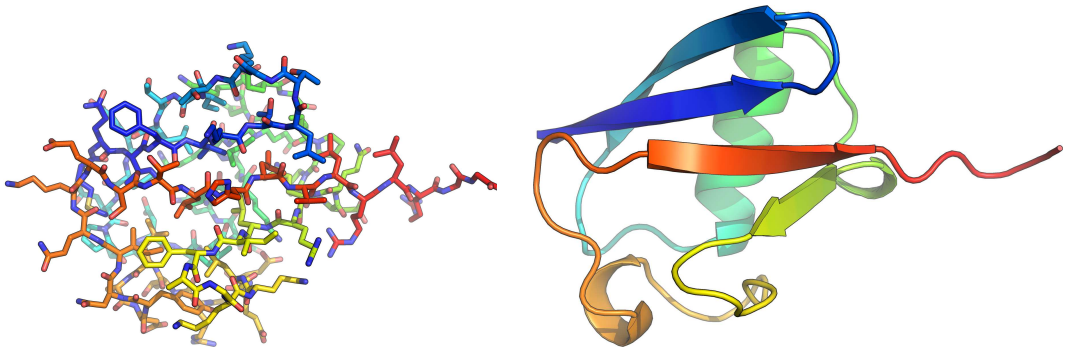


FIGURE 2.12: *Protein three-dimensional structure*

Three-dimensional structure of ubiquitin, PDB identifier 1UBI, in "stick" (left) and "cartoon" (right) format. The protein chain is coloured in a colour gradient from blue at the N-terminus to red at the C-terminus. This structure contains a mixed β-sheet, with two parallel β-strands in the middle, each of which also has anti-parallel β-strand. This is much easier to see in the cartoon representation (in which the direction of the β-strands is indicated by the arrows) than in the more detailed stick representation. Images generated with PyMOL.

The structure of a protein can be represented by a *distance map*. This is a square matrix M where each entry M_{ij} is the distance between residues i and j. Another useful representation is the *contact map*, which is a binary version of the distance map. That is, each entry C_{ij} in the contact map is 1 if $M_{ij} < \tau$ for some threshold distance τ, and otherwise 0. The three-dimensional structure of a protein can be reconstructed from its contact map, although this is equivalent to the unit disk graph realization problem, which is NP-hard (Breu and Kirkpatrick, 1998; Vassura *et al.*, 2008). Contact maps can be used for protein structural comparison, for example by maximum contact map overlap (MAX-CMO), which is discussed in Section 5.3.1 of Chapter 5. An example contact map is shown in Figure 2.13. In protein structure, it is conventional to measure distance in *Ångströms*, denoted Å, where 1 Å = 10^{-10} metres.

Proteins can be composed of multiple *domains*. There is no single consistent definition of what exactly a domain is (Veretnik *et al.*, 2004), although for our purposes we may think of it as a compact spatially distinct unit of protein structure. Many proteins consist of two or more domains, and it is often useful to consider the domains separately as individual units. Indeed one way to think of domains is to consider them as units for which such a decomposition "makes sense" structurally or functionally.

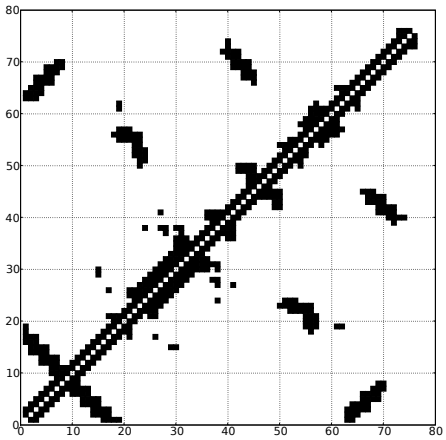


FIGURE 2.13: *Protein contact map.*

Contact map for ubiquitin, PDB identifier 1UBI. This figure was generated with PConPy (Ho et al.*, 2008) with the default options of C_α distance and threshold 8.0Å.*

One more aspect of protein structure that we need to consider is *chirality*, otherwise known as *"handedness"*, or (particularly in chemistry) *enantiomorphism*. A chiral object is one that is not superimposable on its mirror image. Human hands for instance are chiral: a left hand cannot be superimposed on a right hand in three-dimensional space. A chiral molecule and its mirror image are also sometimes called *optical isomers*.

Chirality appears in various contexts in protein structure. First, amino acids have two chiral forms, the L- and D-form, although all amino acids that occur in proteins have the L-form and we will not consider this further. Second, helices can be left- or right-handed. However, α-helices in proteins are almost always right-handed due to collisions between side chains and the C'O group of the amino acids usually forbidding the left-handed form. Again, we will not need to consider this further. A third context in which chirality appears is that arrangements of three or more SSEs can be chiral. The β-α-β motif, for example, can have one of two chiral forms, illustrated in Figure 2.14. Only the "right-handed" form is found in most proteins (Sternberg and Thornton, 1977; Branden and Tooze, 1999, pp. 28,48), however there are some interesting exceptions. We will consider this type of chirality further in Chapter 7, particularly in Section 7.4. Although we mentioned above that the full three-dimensional structure of a protein can be reconstructed from its contact map, note that this is only up to chirality. That is, two chiral forms of a structure have the same contact map.

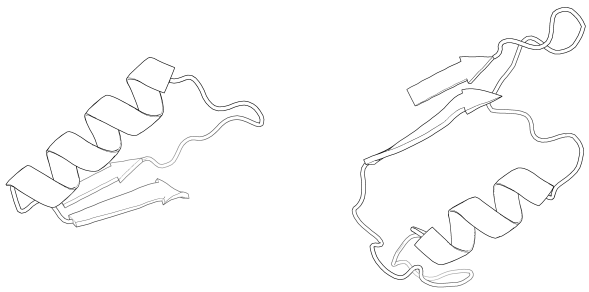


FIGURE 2.14: *The two chiral forms of the β-α-β motif.*

The right-handed form (left) has the helix above the sheet, while the left-handed form (right) has the helix below the sheet, when drawn with the sheet in the same orientation. The right-handed form is far more frequent than the left-handed form. These two motifs were extracted from the structure of Bacillus amyloliquefaciens subtilisin, PDB identifier 1ST2 (Bott et al.*, 1988). This structure is discussed further in Section 7.4 of Chapter 7. Images generated with PyMOL.*

An important part of studying protein structure is the classification of proteins based on their structural (and other) characteristics. There are two main hierarchical classifications of protein structure: SCOP (Murzin *et al.*, 1995; Andreeva *et al.*, 2004) and CATH (Pearl *et al.*, 2005). Both SCOP and CATH classify protein domains rather than entire protein structures, so as a first step they decompose proteins into their constituent domains.

Protein domains were divided into four classes all-α, all-β, α+β and α/β by Levitt and Chothia (1976), based on their SSE composition and packing. The first two classes consist of only helices or only β-sheets, respectively. The α+β class consists of mainly anti-parallel β-sheets, with the α and β regions segregated. The α/β class consists of mainly parallel β-sheets, largely composed of β-α-β motifs.

SCOP classifies domains according to the hierarchy (from most general to most specific) class (as above, although with some additional classes such as "small proteins" and "cell surface proteins and peptides"), *fold*, *superfamily* and *family*, and then the lowest level, the individual domain. Domains in the same fold have major structural similarity; the major SSEs are in the same arrangement with the same topological connections. Domains in the same superfamily have a probable common evolutionary origin, although they have low sequence identity. Domains in the same family have a clear evolutionary relationship, generally (but not always) indicated by relatively high sequence identity. The SCOP classification relies on visual inspection and comparison of structures.

CATH classifies domains by (again in order of decreasing generality) class (C) *architecture* (A), *topology* (T) and *homology* (H). The class is as above, although with the α+β and α/β classes combined to form the α-β class. The architecture describes the overall shape of the structure as determined by SSE orientation, but without (unlike the SCOP fold level) considering the topological connectivity between SSEs. The topology level groups together structures that have the same topology or fold in the core of the structure, possibly with different structures "decorating" the core. The homology ("homologous superfamily") level groups together structures thought to share a common ancestor, not dissimilar to the SCOP family level. CATH is manually curated, based on a combination of automated and manual techniques.

In Chapters 5 and 6 we will make use of SCOP and CATH as "gold standards" for measuring the performance of structural comparison (structural database searching) methods, using the techniques described below in Section 2.5.

2.4 Sequence alignment and structural alignment

The concepts of *sequence alignment* and *structural alignment* are very important in bioinformatics. These techniques compare RNA or proteins on the basis of their sequence or structural similarity respectively, on the assumptions that structural similarity implies functional similarity and structural composition is closely related to sequence composition (Krissinel, 2007).

Both a sequence and a structural alignment may be a *global alignment* or a *local alignment*. We will discuss the difference in terms of sequence alignment of RNA molecules, but the situation for structural alignment is analogous. In a global alignment, we seek a correspondence between the two RNA sequences that minimizes the edit distance between them in terms of *insertion*, *deletion*, and *substitution*. A deletion is the case that a *gap* is added to a sequence; as this is equivalent to an insertion in the other sequence, insertions and deletions are often referred to collectively as *indels*. In representing an alignment, a gap is usually denoted with a - symbol. This is associated with some cost, called the *gap cost* or *gap penalty*. In the simplest case, we use a *linear gap cost*, where there is a fixed cost for each gap symbol. However, more realistic models usually use an *affine gap cost*, in which there is one cost for adding a single gap symbol, and a second (lower)

cost for each extension of the gap. A substitution is the substitution of one base for a different one. Again there is some cost associated with this, depending on the nature of the substitution. These costs are represented by a *substitution matrix*, which gives a score for each possible type of substitution. These matrices are generally derived from analysis of known multiple alignments from which the probability of the different substitutions can be deduced.

The classic algorithm for global alignment of two sequences is the Needleman-Wunsch algorithm (Needleman and Wunsch, 1970). This is a dynamic program to find the maximum score for aligning two sequences given the gap cost and substitution scores. For aligning two sequences A and B of length m and n respectively, this score $M(m, n)$ is:

$$M(i,j) \quad = \quad \max \begin{cases} M(i+1,j) + \gamma, \\ M(i,j+1) + \gamma, \\ M(i,j) + \sigma(A_i, B_j) \end{cases} \tag{2.2}$$

with base cases $M(0, j) = j \cdot \gamma$ and $M(i, 0) = i \cdot \gamma$, where $\gamma < 0$ is the gap cost and $\sigma(x, y)$ is a substitution matrix giving the score for substituting symbol y for x.

As a simple example, consider the following two RNA sequences:

```
ACCGUAAA
ACGUGAA
```

If we set $\gamma = -2$ and use a simple substitution matrix defined by

$$\sigma(x,y) \quad = \quad \begin{cases} 1 & \text{if } x = y \\ 0 & \text{otherwise} \end{cases}$$

then an optimal alignment of these two sequences computed according to the Needleman-Wunsch algorithm (Equation 2.2) is:

```
ACCGUAAA
A-CGUGAA
```

with a score $M(8, 7) = 4$.

A local alignment, rather than considering the two sequences in their entirety, seeks to find the best alignment of two subsequences within the sequences being

aligned. Most commonly the task is actually to find the best alignment of a query sequence within a longer database sequence (or sequences). The classic algorithm for local alignment is the Smith-Waterman algorithm (Smith and Waterman, 1981). BLAST (Altschul *et al.*, 1990, 1997), probably the most widely used bioinformatics application, is a much faster heuristic method for performing the same task.

In order to illustrate the difference between a global and local alignment, we will use the same sequences and scores as the example of global alignment just given. If we now use the Smith-Waterman algorithm (Smith and Waterman, 1981) to obtain a local alignment, rather than a global alignment, we obtain:

```
ACCGUAAA
 ACGUGAA
```

with a score of 5. The higher score is due to the fact that, rather than having to insert a gap to make the sequences the same length, as in a global alignment, the local alignment finds the best subsequence of the longer sequence with which to align the shorter sequence. The space on the left of the shorter sequence is not a gap, and so has no penalty associated with it. Note that gaps may still appear in one or both sequences in a local alignment, they just happen not to in this example. Also, in a global alignment, a gap may well appear at the beginning or end of one or both of the sequences, in which case it is still a gap, with the associated cost.

A structural alignment takes account of not just the sequence, but also the structure of the two molecules (RNA or proteins) being aligned. In the case of RNA, the structural alignment should take into account the stem and loop structures of the RNAs being aligned, and find an alignment that causes corresponding structural parts to be aligned. Algorithms for RNA structural alignment are discussed in detail in Chapter 3, but an example will help illustrate the concept. Figure 2.15 shows the secondary structure of two tRNAs, and Figure 2.16 shows a structural alignment of the two.

In the case of protein structural alignment, we seek an alignment that allows the two structures to be superimposed in three-dimensional space so that the distances between corresponding residues are minimized. This is generally quantified by the

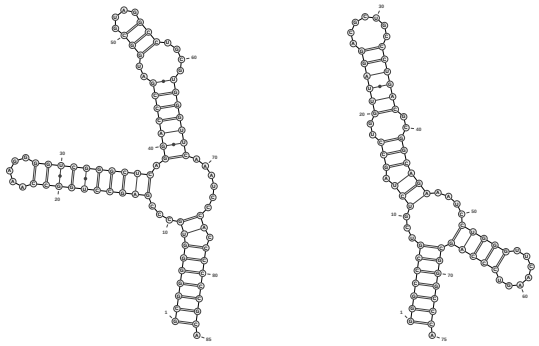


FIGURE 2.15: *Secondary structure of two tRNA sequences.*

Secondary structure of tRNA sequences AP000063 (left) and X06054 (right) from the BRAliBase II data set (Gardner et al.*, 2005). Base number 14 (A) in the structure on the right is an example of a "bulge"; it interrupts the stem by having no base pair on the other strand of the stem. The secondary structure was generated from sequences with the* `RNAfold` *program and the figures generated using* `VARNA`*.*

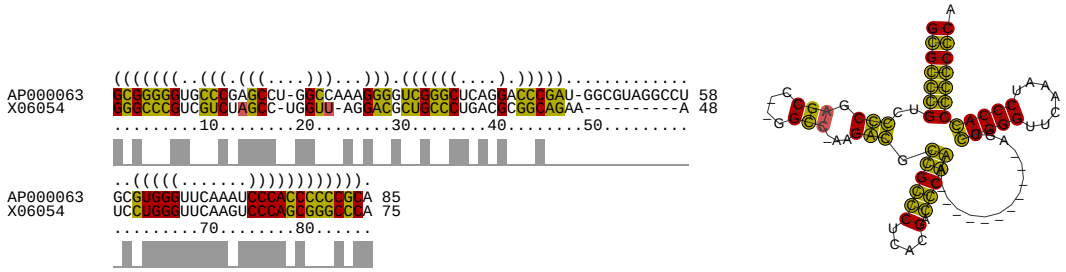


FIGURE 2.16: *Example RNA structural alignment.*

Structural alignment of the two tRNAs from Figure 2.15 according to the reference alignment in BRAliBase II. The alignment is shown on the left and the consensus secondary structure on the right. Grey bars below the sequence alignment show conserved bases. This also illustrates the commonly used "bracket" notation for RNA secondary structure: matching left and right parentheses indicate the base pairings (compare with Figure 2.6: non-crossing arcs are equivalent to the ability to use this bracket notation — the absence of pseudoknots is required). The colour annotations show the base pair mutations with respect to the structure: the red and ochre colours are showing that a base pair is formed by several combinations of nucleotides, or consistent or compensatory mutations have occurred. For example, the first base in each sequence is G, and in the consensus structure they pair with the second-last base, which is C. This is coloured red to indicate this consistency. The second base in the two sequences is C and G, respectively, which is paired with the third-last base in the consensus structure, which is G in the first sequence and C in the second. This pairing is coloured ochre to represent this compensatory mutation (a C-G pairing in the first structure and a G-C pairing in the second). Pale colours indicate base pairs that cannot be formed in one of the sequences. An example is the pink A in the second sequence at position 14; this is part of a stem in the first structure (left of Figure 2.15) but is a "bulge" in the second structure (right of Figure 2.15). Consensus structure and figures generated with the `RNAalifold` *program (Hofacker* et al.*, 2002).*

root mean squared deviation (*RMSD*) between the structures:

$$\text{RMSD} = \sqrt{\frac{\sum_{i=1}^{n} d_i^2}{n}} \tag{2.3}$$

where n is the number of aligned residues and d_i is the distance between the ith pair of corresponding residues. This formulation of the protein structural alignment problem is to find both a set of correspondences between residues, and a superposition in three-dimensional space that minimizes the RMSD. Such alignments are important since, for example, for distantly-related proteins it is necessary to use structural information to find a "good" alignment that reflects evolutionary conservation, since the sequences may be very highly diverged, but not so the structures (Krissinel, 2007; Konagurthu *et al.*, 2008). An example protein structural alignment is shown in Figure 2.17. We could use such alignments to search databases of protein structure, but (as we will see in Chapter 5), this can be excessively computationally demanding. Searching for structural similarity between a query protein structure and a database of protein structures can be made much faster by other methods, such as finding corresponding SSEs, rather than residues. We will examine this in detail in Chapter 5.

So far we have mentioned only pairwise alignments, that is, alignments between two sequences (or structures). A *multiple alignment* is an alignment of three or more sequences or structures. Such alignments are very important, as they reveal patterns of conservation among families of proteins (or RNAs) that are not evident in only pairwise alignments, however they are much more computationally challenging as the number of sequences (or structures) in the alignment increases (Konagurthu, 2006). For this reason, rather than solving the multiple alignment problem exactly, heuristics are generally used.

A common method, often referred to as *progressive alignment*, is to compute pairwise alignments, and then extend them to a multiple alignment by means of a *guide tree*. This tree is an approximate phylogenetic tree built by using the pairwise distances derived from pairwise alignments to approximate evolutionary distance. The tree is then used to determine the order in which sequences and alignments are aligned with each other. This technique was introduced by Feng and Doolittle (1987) and is used by CLUSTAL W (Thompson *et al.*, 1994) for example. T-Coffee

```
d1ubia_.ent          1  --MQIFVKTLTGKTITLEVEPSDTIENVKAKIQDKEGIPPDQQRLIFAGKQLEDGRTLSD   58
d2uyzbI.ent          1  EYIKLKVIGQDSSEIHFKVKMTTHLKKLKESYCQRQGVPMNSLRFLFEGQRIADNHTPKE   60
                           V       I   V        K       G P    R  F G    D  T

d1ubia_.ent         59  YNIQKESTLHLVLRLRGG   76
d2uyzbI.ent         61  LGMEEEDVIEVYQEQTG-   77
                          E         G
```

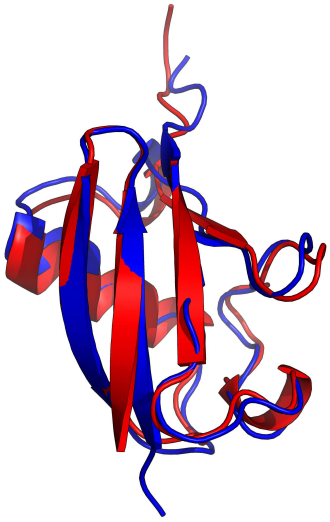


FIGURE 2.17: *Example protein structural alignment.*

Structural alignment of two structures with the ubiquitin fold: ubiquitin, PDB identifier 1UBI, and SUMO-1, PDB identifier 2UYZ chain B (Knipscheer et al.*, 2007). The alignment is shown on the top, and the structural superposition below it, with 1UBI in red and 2UYZ chain B in blue. The colours in the sequences indicate the chemical properties of the residues (red: hydrophobic, blue: acidic, magenta: basic, or green: basic with hydroxyl or amine groups). Conserved residues are noted below the sequences. Note that these two proteins have a sequence similarity of only 16.7%, but their structural similarity is evident from the structural superposition. Alignment generated with MUSTANG (Konagurthu* et al.*, 2006) and image rendered with PyMOL.*

(Notredame *et al.*, 2000) is an enhanced version of this strategy, which uses a so-called *extended library* of pairwise alignments in order to consider the alignments between all pairs during the progressive alignment. In this way, suboptimal alignments are less likely to be made at early stages of the progressive alignment (and therefore be carried through all subsequent stages).

T-Coffee is implemented in such a way that any pairwise sequence or structural alignment system can be used to provide the pairwise alignments. For this reason it is highly useful for extending pairwise alignment methods to multiple alignments, and, as we shall see in Chapter 3, many pairwise RNA structural alignment methods are extended to multiple alignment methods in this way. T-Coffee when used on its own uses CLUSTAL W to provide pairwise global alignments and the Lalign program from the FASTA package (Pearson and Lipman, 1988) to provide pairwise local alignments. One example of using a different pairwise alignment algorithm as the input to T-Coffee is R-Coffee (Wilm *et al.*, 2008). R-Coffee is a multiple RNA structural alignment program that incorporates RNA secondary structure information into the T-Coffee pairwise alignment library. It does so by using base pairing information from the `RNAfold` or `RNAplfold` (Bernhart *et al.*, 2006) programs, so that information about base pairs not observed in sequence alignments but which secondary structure prediction suggests as relevant are incorporated into the T-Coffee library. In addition, the scoring scheme for the progressive alignment phase is enhanced by including this base pairing information.

2.5 Evaluating database searching methods

In Chapter 5 and Chapter 6, we describe two methods for searching a database of protein structures, and compare them with some existing methods. In order to provide an objective comparison of the evaluation of these methods, we require some "gold standard" of truth against which we compare the results, and a method of evaluating the results of each method against this truth.

For evaluating these database searches on the basis of fold classification (or fold recognition), that is, determining whether the query structure is a member of the same fold as a database structure, we can use the SCOP or CATH databases (described above). Specifically, we can say that if our algorithm determines that the

query is in the same fold as the database structure, and they are also in the same fold in the SCOP database, then that constitutes a *true positive*. A *false positive* arises if they are not in the same SCOP fold (but the algorithm says they are). Similarly, if our algorithm decides that two structures are not in the same fold, and they are also in different folds in the SCOP database, then this is a *true negative*. A *false negative* arises if the algorithm decides the two structures are not in the same fold, however they are in the same fold in the SCOP database. By using SCOP as the gold standard in this way, large scale automatic evaluation on a large number of different queries is possible.

We can then compute the true positive rate (*TPR*), or sensitivity, as $TPR = TP/N$ where TP is the number of true positives and N is the number of structures that match the query according to the gold standard. The false positive rate (*FPR*), which is equal to 1 − specificity, is $FPR = FP/(TN + FP)$ where FP is the number of false positives and TN is the number of true negatives.

If the fold classification technique we are evaluating gives (as we have implicitly assumed above) a binary answer to the question "are these two structures in the same fold?", then we can simply use the TPR and FPR values described above (or some combination of them) as our evaluation measure. However, a much more common situation is that the algorithm does not give a binary output, but rather some score for each comparison; the output can then be sorted by this score so the database structures at the top of the list are the most likely according to that algorithm to be in the same fold as the query, and those at the bottom of the list the least likely. In this case, we need a somewhat more sophisticated way of comparing the accuracy of two or more such methods. One standard way of doing this is to use the *receiver operating characteristic* (*ROC*) curve. This curve plots the TPR against the FPR as the threshold score (the value above which we are considering a score to indicate a positive classification) is varied from the minimum to maximum possible value.

In this way we evaluate the accuracy of structural search by counting a hit (a score above the threshold) as correct (a true positive) if the structure is in the same SCOP fold as the query structure, and incorrect (a false positive) otherwise.

The ROC curve is then constructed by plotting the *TPR* against the *FPR* for all values of the score threshold. The area under the ROC curve (*AUC*) is an overall

measure of the quality of a classification method; a perfect classifier has $AUC = 1.0$, and a random classifier has $AUC = 0.5$.

When multiple queries are being evaluated in one ROC curve, all the scores are combined together (after normalization, if required), with each labelled as either a positive or negative according to the appropriate gold standard. That is, each individual query has a list of scores, one for each entry in the database. Each one of these entries is either positive or negative according to the relevant gold standard, that is, each individual database entry either is, or is not, in the same fold as the query. So each query has a set of tuples (s, P) where s is the score for that pairwise comparison and P is a Boolean variable that is *True* if the query and that database entry are in the same fold (according to the gold standard for the database being used) and otherwise *False*. Each of these sets of tuples (one for each query) are then all combined into one large set. That is, the end result is simply a large set of (s, P) tuples associating a score with a Boolean variable indicating whether that score is associated with a positive (query and database structures are in the same fold) or negative (query and database structures are not in the same fold) classification according to the gold standard. This entire set of tuples can then be sorted by score for computation of the ROC curve.

It might seem that we can simply take the AUC value for two different methods and compare them, declaring that the one with the higher AUC is "better" (on the criteria we have used for evaluation). However there are (at least) two subtleties we should contend with here: first, the error in the estimation of the AUC value for each method (due to not having an infinite number of samples along the ROC curve), and second, the correlation between the ROC curves for different algorithms that arises since they are applied to the same data.

A method for calculating a confidence interval for an AUC value is described by Hanley and McNeil (1982). This is derived from the fact that the AUC measures the probability θ that two randomly paired positive and negative instances will be correctly identified as such by the method being evaluated. This, in turn, is equivalent to the Wilcoxon, or Mann-Whitney, statistical test which measures this same probability θ that two randomly chosen positive and negative instances will be correctly ranked. From the statistical properties of the Mann-Whitney U statistic (Mann and Whitney, 1947), which we will denote U here, we can compute the

standard error in the AUC (Hanley and McNeil, 1982):

$$\mathrm{SE(AUC)} = \sqrt{\frac{\theta(1-\theta) + (n_A - 1)(Q_1 - \theta^2) + (n_N - 1)(Q_2 - \theta^2)}{n_A n_N}} \tag{2.4}$$

where $\theta = U/n_A n_N$, $Q_1 = \theta/(2-\theta)$, and $Q_2 = 2\theta^2/(1+\theta)$.

In the above, n_A is the number of positive samples ("abnormal" in the Hanley and McNeil (1982) terminology) and n_N is the number of negative samples ("normal" in the Hanley and McNeil (1982) terminology).

We could use this standard error to create confidence intervals and do statistical significance testing of the AUC values for two different methods. However, when we are running the methods on the same data (as we will do when comparing the performance of different fold recognition methods on the same query and database), we should take account of the correlation between their ROC curves due to this. DeLong *et al.* (1988) present a method for computing the statistical significance of differences in AUC values (which we denote ΔAUC) using a nonparametric approach based on the properties of the Mann-Whitney statistic U. An $R \times R$ covariance matrix is estimated for the AUC estimates of R methods and used to compute a χ^2 statistic for testing if there is a significant difference between AUC values for different methods. This method has been implemented in the StAR program (Vergara *et al.*, 2008), which we use to estimate the significance of ΔAUC values in Chapter 6.

Another technique we use in Chapter 6 is the *coverage versus error plot*. Coverage versus error plots were introduced by Brenner *et al.* (1998) as an alternative to ROC curves for comparing sequence comparison methods which have a huge background of false positives (non-homologues), and were subsequently also used by Rognes (2001) and Hulsen *et al.* (2006). Coverage is defined as the number of actual true positives divided by the total number of possible true positives. Errors per Query (*EPQ*) is defined as the number of false positives divided by the number of queries. The plot of EPQ against coverage for each score threshold is the coverage versus error graph.

The research field of *Information Retrieval* (*IR*) has developed a large number of methods of measuring the effectiveness of IR systems (see, for example, Moffat and Zobel (2008) for one recent addition). In bioinformatics the use of the ROC

curve and related techniques such as ROC_{50} scores (Gribskov and Robinson, 1996) for assessing sequence and structural database search is widespread and generally accepted. Hence we have chosen to use these techniques in this thesis, although we are aware of their deficiencies as effectiveness measures and the existence of alternatives which can be usefully applied in bioinformatics research, as described by Carroll *et al.* (2010), for example.

3

RNA Structural Alignment

In this chapter we review existing techniques for RNA structural alignment and show the results of a benchmarking exercise on several published methods. We find that a technique based on alignment of base pairing probability matrices with dynamic programming is the most promising, and attempt to improve its efficiency by bounding techniques (Puchinger and Stuckey, 2008). This is, however, not very successful, and, in investigating the reasons why, we find some moderately successful results in improving sequence alignment with similar techniques.

This chapter is organized as follows: in Section 3.1 we review existing algorithms for RNA structural alignment. In Section 3.2 we perform a benchmarking exercise on some existing implementations of these algorithms, and find quite divergent performance in both computational resources and accuracy. In Section 3.3 we use the bounding technique on a simplified version of the RNA base pairing matrix alignment problem, and in Section 3.4 we apply it to the full algorithm. The lack of success on this problems leads us to investigate its application to the simpler problem of purely sequence alignment in Section 3.5.

3.1 Introduction

To structurally align RNA molecules, there are several possibilities. One method is to, for each sequence, infer its structure using algorithms such as those discussed in Section 2.2 of Chapter 2, followed by an alignment that takes account of both sequence and this computed structure. Another method, if the structure is already known (by some experimental method) for one or both of the sequences, is to use this

known structure instead of a computed structure. However since it is much more common to have sequences but not experimentally determined structures[1], this approach is less commonly useful. A third method is to simultaneously align and infer structure from RNA sequences, and the classical algorithm for simultaneous alignment and secondary structure prediction is Sankoff's algorithm (Sankoff, 1985). This dynamic programming algorithm is $O(n^{3m})$ for m RNA sequences, each of length n. Sankoff's algorithm works by minimizing a linear combination of the alignment cost and minimum free energy (basepairing energy). Specifically (in the pairwise case for two sequences A and B) it minimizes the cost $F(i, j, k, l)$ of a sum of free energies and (constrained) alignment cost of subsequences $i \dots j$ in sequence A and $k \dots l$ in sequence B (Sankoff, 1985):

$$F(i,j,k,l) = \min \begin{cases} C(i,j,k,l) + D(i,i,k,k) + D(j,j,l,l), \\ \min_{i \le h_1 < j, k \le h_2 < l} \{F(i,h_1,k,h_2) + F(h_1+1,j,h_2+1,l)\}, \\ D(i,j,k,l) \end{cases} \quad (3.1)$$

where

$$C(i,j,k,l) = \min \begin{cases} e(s_A(i,j)) + e(s_B(k,l)) + D(i+1,j-1,k+1,l-1), \\ \min_{p_1,q_1,p_2,q_2} \left\{ \begin{array}{l} e(s_A(i,j)) + e(s_B(k,l)) + C(p_1,q_1,p_2,q_2) \\ +D(i+1,p_1,k+1,p_2) + D(q_1,j-1,q_2,l-1) \end{array} \right\}; \\ \qquad p_1 - i + j - q_1 - 2 \le U, p_2 - k + l - q_2 \le U, \\ \min_{h_1,h_2} \{G(i+1,h_1,k+1,h_2) + G(h_1+1,j-1,h_2+1,l-1) + 2A\}; \\ \qquad i < h_1 < j-1, k < h_2 < l-1 \end{cases} \quad (3.2)$$

and

$$G(i,j,k,l) = \min \begin{cases} C(i,j,k,l) + 2P + D(i,i,k,k) + D(j,j,l,l), \\ \min_{h_1,h2} \left\{ \begin{array}{l} G(i,h_1,k,h_2) + (j - h_1 + l - h_2)Q + D(h_1+1,j,h_2+1,l), \\ G(i,h_1,k,h_2) + G(h_1+1,j,h_2+1,l), \\ (h_1 - i + 1 + h_2 - k + 1)Q + G(h_1+1,j,h_2+1,l) + D(i,h_1,k,h_2) \end{array} \right. \\ \qquad i < h_1 < j, k < h_2 < l \end{cases} \quad (3.3)$$

[1] As of 26 July 2010, RNAdb (Pang *et al.*, 2007) contains over 304 000 RNA sequences, and these are only mammalian noncoding RNA sequences. In contrast, the Nucleic Acid Database (NDB) (Berman *et al.*, 1992) contains only 4 840 experimentally determined RNA structures.

and D is the sequence alignment cost

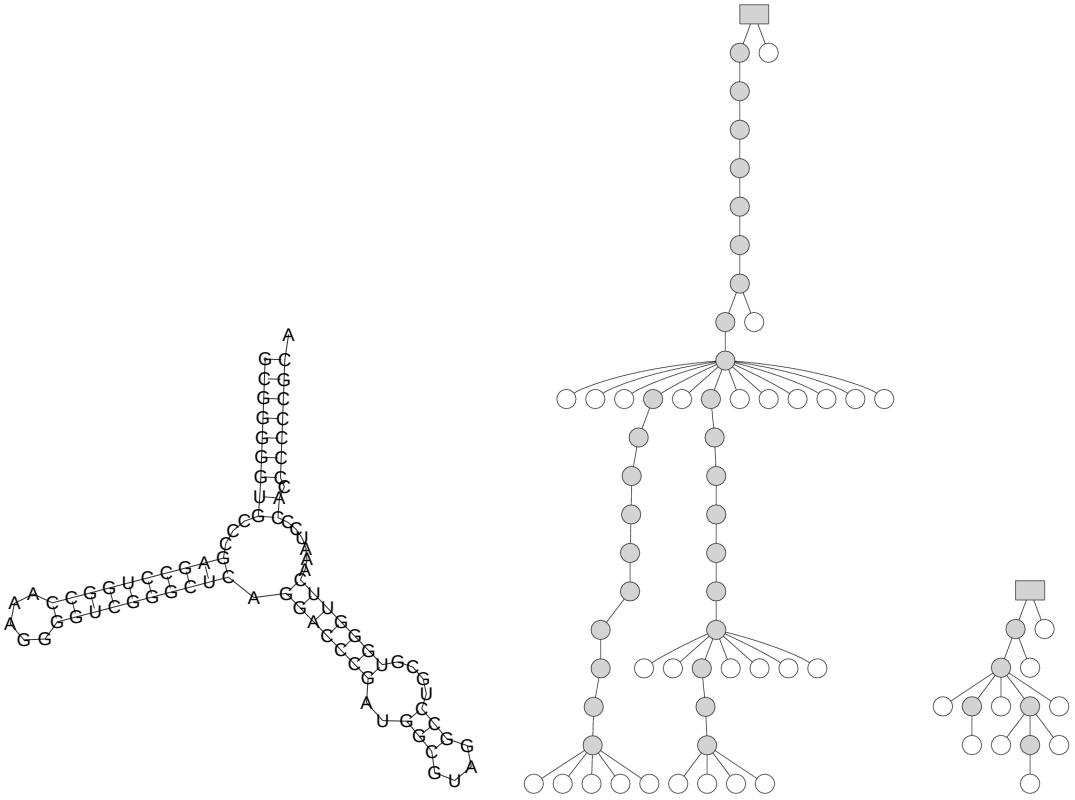
$$D(i,j,k,l) = \min \begin{cases} D(i+1,j,k,l) + y, & \\ D(i+1,j,k+1,l) + x & \text{if } A_i \neq B_k, \\ D(i+1,j,k+1,l) & \text{if } A_i = B_k, \\ D(i,j,k+1,l) + y & \end{cases} \tag{3.4}$$

where x is the cost of substitution of a base and $y > 0$ is the gap cost. In the equations above, A, P and Q are the free energies of multiple loops, paired bases and unpaired bases, respectively, $e(s)$ is the free energy of a hairpin loop (so $e(s_A(i,j))$ is the free energy of a hairpin loop in sequence A enclosed by the basepair (i,j)), and U is the maximum number of unpaired bases in a loop.

Many RNA structural alignment programs, for example Foldalign (Havgaard *et al.*, 2005), are based on simplified variants of this algorithm, with constraints or heuristics to improve the running time, although this can result in some reduction in accuracy. Often, as in Foldalign, and the basepairing matrix alignment algorithm (Hofacker *et al.*, 2004) described in Section 3.1.4 below, the problem is transformed from a minimization to a maximization problem. That is, rather than minimizing a sum of free energy and alignment cost, a score based on sequence similarity and some (simple) function of structure, such as number of base pairs (Havgaard *et al.*, 2005) or basepairing log probability (Hofacker *et al.*, 2004) is maximized. Because of the complexity of this algorithm, it is typically (as we shall see in the methods discussed below) only computed for the pairwise case, and extended to multiple alignments by other methods such as T-Coffee, as discussed in Section 2.4 of Chapter 2.

3.1.1 Tree representations of RNA secondary structure

The Vienna RNA package uses a tree representation of RNA secondary structures in which an internal node corresponds to a base pair, and external nodes correspond to unpaired bases (Hofacker *et al.*, 1994). This representation, proposed by Fontana *et al.* (1993), includes a reduction to a "homeomorphically irreducible tree" (HIT), which contracts multiple nodes into a single node with a weight (while preserving relative traversal order), allowing "coarse-graining" of secondary structure to a predefined resolution. These trees are illustrated in Figure 3.1.



(a) RNA secondary structure (b) Tree representation (c) HIT representation

FIGURE 3.1: *Tree representation of RNA secondary structure.*

Figure 3.1(a) shows the secondary structure of an RNA molecule as drawn by the `RNAfold` *program (Hofacker* et al.*, 1994). Figure 3.1(b) shows the tree representation of this RNA secondary structure. In this figure, internal nodes (shaded circles) represent base pairs and leaf nodes (open circles) represent unpaired bases. The (shaded square) root node does not represent any part of the RNA and is added only to ensure the tree is connected and rooted. Figure 3.1(c) shows the "homeomorphically irreducible tree" (HIT) representation, in which multiple nodes have been contracted into single nodes.*

An earlier coarse-graining technique for tree representation of RNA secondary structure is due to Shapiro (1988). In this representation, only "high level" secondary structure is represented. There are four types of nodes; 'B', 'I', 'M' and 'H' for bulges, internal loops, multibranch loops and hairpin loops, respectively. Arcs then represent base-paired regions, and the alignment is by string alignment of a linear representation of such trees, and is therefore only an approximation and not a formal tree matching (Shapiro, 1988). This technique was later improved (Shapiro and Zhang, 1990) by annotating the tree with properties such as size and energy, and using these as heuristic edit weights in the Zhang-Shasha tree edit algorithm (Zhang and Shasha, 1989).

Another algorithm that works by decomposing RNA secondary structure into a tree is RSMatch (Liu *et al.*, 2005). RSMatch is an O(mn) algorithm for structures of size m and n. Rather than matching full structure trees or a reduced representation such as a HIT, RSMatch decomposes the structure into so-called "circles", which are sets of bases reachable from each other on a planar representation of the structure (such as that in Figure 3.1(a)) without crossing a basepair. These "circles" are then formed into a tree, and dynamic programming is used to align progressively larger partial structures in these trees.

3.1.2 Tree and arc alignment algorithms

The Zhang-Shasha tree edit algorithm induces a distance metric on the set of trees by an edit distance in terms of three kinds of edit operations (change, delete and insert). The distance between two trees is then the minimum number of such operations required to convert one into the other. This distance for two trees T_1 and T_2 can be computed in time $O(|T_1| \cdot |T_2| \cdot \min(\mathit{depth}(T_1), \mathit{leaves}(T_1)) \cdot \min(\mathit{depth}(T_2), \mathit{leaves}(T_2)))$ (Zhang and Shasha, 1989). This is an improvement on an earlier algorithm due to Tai that has time complexity $O(|T_1| \cdot |T_2| \cdot \mathit{depth}(T_1) \cdot \mathit{depth}(T_2))$ (Tai, 1979). The MARNA (Siebert and Backofen, 2005) system for multiple alignment of RNA is based on this edit distance, extended from pairwise to multiple alignment using T-Coffee (Notredame *et al.*, 2000).

An alternative to edit distance as a measure of similarity between trees is *tree alignment*. Tree alignment between two (ordered, i.e. the children of each node are ordered) trees T_1 and T_2 consists of inserting "space" (λ) nodes into T_1 and T_2 until

the two trees have the same structure (ignoring labels) and then overlaying the two (isomorphic) trees, resulting in a single aligned tree where each node is labelled with a tuple (x, y) where x is a label from T_1, or λ, and y is a label from T_2, or λ. The cost is then the number of labels in the aligned tree where $x \neq y$, and the optimal alignment is the alignment with minimum cost. This is illustrated in Figure 3.2.

Edit and alignment are not equivalent for trees (unlike sequences), and in general the edit distance is smaller than the alignment distance (Jiang *et al.*, 1995), as illustrated in Figure 3.2. There is an $O(|T_1| \cdot |T_2| \cdot (deg(T_1) + deg(T_2))^2)$ algorithm for alignment of labelled ordered trees (Jiang *et al.*, 1995); this algorithm was subsequently shown by Herrbach *et al.* (2010) to have average complexity $O(|T_1| \cdot |T_2|)$.

A survey of algorithms for tree edit distance and related problems can be found in Bille (2005).

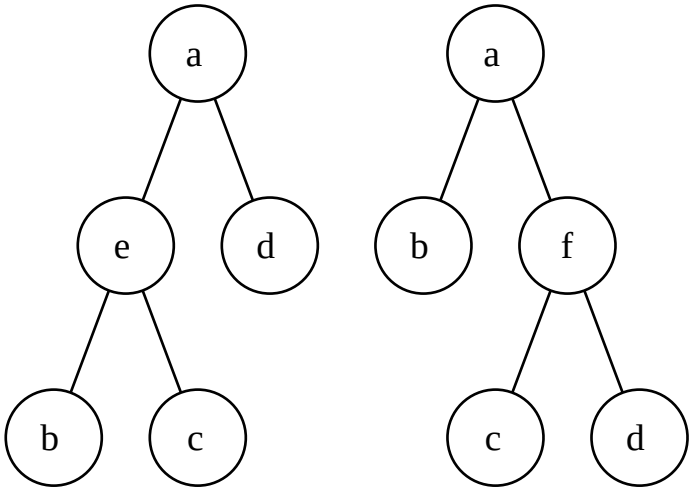
Tree representations are not necessary for RNA secondary structure matching problems; they can be considered as problems on arc-annotated sequences directly. For example, an algorithm has been published which determines if one sequence is an arc-preserving subsequence of another in $O(mn)$ time (Gramm *et al.*, 2006).

3.1.3 Other approaches to RNA structural alignment

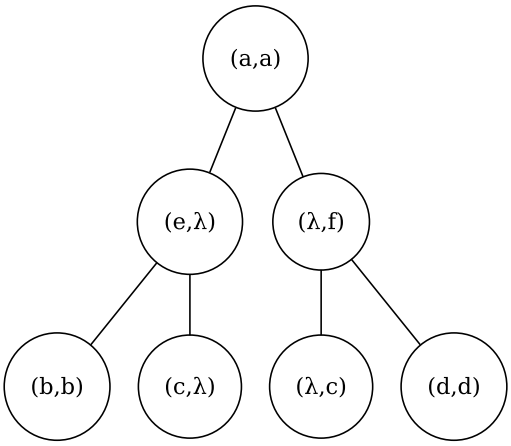
Pairwise stochastic context-free grammars (*SCFG*s) can be used to model conserved RNA structure, with certain constraints to improve efficiency, as in the Stemloc (Holmes, 2005) and Consan (Dowell and Eddy, 2006) programs. These algorithms are equivalent to Sankoff's algorithm, with the constraints to improve effective efficiency. SCFGs can also be used to describe covariance models for matching RNAs against a database (Weinberg and Ruzzo, 2004). Covariance models (Eddy and Durbin, 1994; Durbin *et al.*, 1998) are statistical models that describe both the sequence and secondary structure consensus of an RNA multiple alignment, allowing a candidate sequence to be scored as to its similarity with the consensus.

SCARNA (Tabei *et al.*, 2006) is a pairwise structural alignment algorithm, which, unlike many, can handle pseudoknots. It is a heuristic method based on fixed-length fragments of stem candidates, and has cubic time complexity.

Another technique which allows pseudoknots is that of T-Lara (Bauer and Klau, 2005), which uses a graph model and solves the resulting system of constraints by Lagrangian relaxation. This is extended to multiple alignment using T-Coffee



(a) Tree T_1 (b) Tree T_2



(c) Optimal alignment of T_1 and T_2

FIGURE 3.2: *Tree edit and alignment distance.*

Example showing that tree edit and alignment distance are not equivalent (Jiang et al.*, 1995). T_1 can be transformed into T_2 by deleting node e and inserting node f, for an edit distance of 2. The optimal alignment, shown in Figure 3.2(c), however, has a cost of 4 (there are 4 nodes where the two labels differ).*

(Bauer *et al.*, 2005a,b). Some more recent related work to perform alignment with arbitrary structures using constraint programming is described by Palù *et al.* (2010).

An alternative technique is to convert secondary structure into a linear representation and use existing distance measures on sequences, such as the Lempel-Ziv algorithm, an approach taken by Liu and Wang (2006).

A review and performance comparison of multiple RNA structural alignment algorithms as of 2005 can be found in Gardner *et al.* (2005).

3.1.4 RNA base pairing probability matrix alignment

Another technique for RNA structural alignment is to align base pairing probability matrices calculated from the equilibrium partition function of RNA secondary structure (McCaskill, 1990). This is the approach taken by `pmcomp` (Hofacker *et al.*, 2004) and StrAl (Dalli *et al.*, 2006).

`Pmcomp` finds the consensus secondary structure of two base pairing probability matrices generated by the `RNAfold` program in the Vienna RNA Package (Hofacker *et al.*, 1994; Hofacker, 2003). `Pmcomp` uses dynamic programming to compute the following recurrence:

$$S(i,j,k,l) = \max \begin{cases} S(i+1,j,k,l)+\gamma, \\ S(i,j,k+1,l)+\gamma, \\ S(i+1,j,k+1,l)+\sigma(A_i,B_k), \\ \max_{h\le j,q\le l}\left\{S^M(i,h,k,q)+S(h+1,j,q+1,l)\right\} \end{cases} \quad (3.5)$$

$$S^M(i,j,k,l) = S(i+1,j-1,k+1,l-1)+\Psi^A_{ij}+\Psi^B_{kl}+\tau(A_i,A_j,B_k,B_l) \quad (3.6)$$

In the above, $S(i,j,k,l)$ is the best score of matching subsequences from $i \ldots j$ in sequence A and $k \ldots l$ in sequence B. $\gamma < 0$ is the gap penalty and σ and τ are score functions for unpaired bases and base pairs, respectively. Ψ^A and Ψ^B are the base pairing log probability matrices for sequence A and sequence B respectively.

This algorithm is $O(n^6)$ and is equivalent to Sankoff's algorithm (Sankoff, 1985) when the base pairing probability matrices are binary, i.e. elements are 1 when a base pair can be formed at that position and 0 otherwise. It is reduced to $O(n^4)$ by

"banding", i.e. restricting the difference in the span of matching base pairs for all partial alignments (Hofacker *et al.*, 2004).

A more recent implementation of this algorithm is `LocARNA` (Will *et al.*, 2007), which significantly improves efficiency by rearranging the dynamic programing equations to avoid recomputation and by being implemented in `C++` rather than `Perl`. Importantly, its time complexity, without banding, is effectively O(n^4). This is achieved by filtering out base pairs with probability less than a threshold, the sparseness of the probability matrices then meaning that the number of remaining nonzero base pair probabilities is effectively linear (Will *et al.*, 2007). `LocARNA` can be used for multiple (not just pairwise) alignments with the `mLocARNA` script that builds a multiple alignment using `LocARNA` pairwise alignments by iterative refinement using a guide tree.

S A (Dalli *et al.*, 2006) is a type of method described as a "structure enhanced" approach by Kruspe and Stadler (2007). That is, it makes use of sequence alignment algorithms but incorporates structural information into the sequence match and mismatch scores. Specifically, S A uses three vectors to project the (two-dimensional) base pairing probability matrix onto the (one-dimensional) sequence, a technique first described in Bonhoeffer *et al.* (1993). One vector contains (for each base in the sequence) the probability of pairing downstream (i.e. earlier in the sequence), the second the probability of pairing upstream, and the third the probability of the base being unpaired. A similarity score is then constructed by a particular combination of these vectors and the usual sequence substitution score, and this is used in a conventional sequence alignment algorithm (with affine gap costs).

The recent appreciation of the importance of RNA has led to renewed interest in efficient algorithms for prediction of RNA structure and structural alignment of multiple RNA sequences on a large scale. Here we give a brief summary of some of those published during the last three years. An overview of recent work on multiple sequence alignment more generally, particularly "meta-methods", can be found in Notredame (2007).

3.1.5 SimulFold

SimulFold (Meyer and Miklós, 2007) is a thorough and mathematically principled approach to simultaneously inferring RNA structures, alignments, and trees. It uses a Bayesian Markov Chain Monte Carlo (MCMC) approach and allows pseudoknots. Due to doubts that the functional structure of RNA necessarily corresponds to the thermodynamic minimum free energy (*MFE*) secondary structure traditionally used in RNA structure prediction, SimulFold instead predicts the evolutionarily conserved secondary structure. Many methods require a high quality alignment to accurately predict secondary structure, yet the construction of such an alignment requires knowledge of a conserved secondary structure. Several shortcomings of most existing methods are noted, such as working only pairwise rather than on many sequences, the exclusion of pseudoknots, and the lack of (explicit) modelling of evolutionary relationships between sequences.

SimulFold overcomes these problems by using a joint distribution of RNA structures, alignments, and trees. As well as co-estimation of all three properties, any combination of them may be kept fixed, so for example if high confidence structures are available, alignments and trees can be estimated from them.

SimulFold is evaluated as a multiple alignment program against CLUSTAL X. An advantage of SimulFold is that it assigns probabilities to columns of the alignments, allowing poorly estimated parts of the alignment to be found.

The efficiency of SimulFold is difficult to evaluate analytically as a whole due to the probabilistic nature of MCMC algorithms, although big-O complexity is given for individual components of the system. The total time for a dataset consisting of 15 sequences is given as 1.5 days on a dual 3 GHz Intel Xeon when using parallel tempering.

3.1.6 RNA Sampler

RNA Sampler (Xu *et al.*, 2007) is an RNA multiple structural alignment program based on probabilistic sampling of stem alignments. It can also handle pseudoknots. Like several other methods (such as SCARNA (Tabei *et al.*, 2006), Stemloc, and CARNAC (Perriquet *et al.*, 2003)), RNA Sampler uses stem alignments to construct a structural alignment.

RNA Sampler starts by computing base alignment probabilities and base pairing probabilities, and generating a list of all possible stems for each sequence. Iteratively, blocks (pairs of aligned stems) are generated and block conservation scores computed. Compatible blocks are sampled to generate common structures, and the base pairing probabilities and alignment probabilities are updated. The sampled structures tend to converge to compatible conserved blocks. A greedy algorithm is then used to assemble converged blocks into a consensus structure, and CLUSTAL W is used to realign single-stranded regions separately.

This process is pairwise, and is extended to multiple sequences by sampling common structures between all pairwise sequences and using a consistency-based method to update probabilities and recalculate conservation scores.

RNA Sampler is implemented in C. It is compared with CARNAC, Dynalign, FoldalignM and Stemloc, and compares favourably with them in terms of structural alignment accuracy. It is not compared to SCARNA, or PMcomp (Hofacker *et al.*, 2004), and was published before the LocARNA paper (Will *et al.*, 2007).

RNA Sampler takes $O(m^2rSN^2)$ time for N sequences with m stems each, where r is the total number of iterations and S is the sample size. On the test set that was used, CARNAC, RNAalifold (Hofacker *et al.*, 2002) and Stemloc all ran faster than, or at a similar speed to, RNA Sampler. RNA Sampler was much faster than Dynalign or FoldalignM (Torarinsson *et al.*, 2007) however.

Another method that uses stems is described in Fang *et al.* (2007). This technique detects conserved stems with a position matrix using a CLUSTAL W alignment and computes their signal-to-noise ratio using randomized alignments generated with a profile stochastic context-free grammar (SCFG). The structural alignment is then constructed by incorporating conserved stems into the CLUSTAL W alignment. The results are assessed only against CLUSTAL W and reference alignments in Rfam (Griffiths-Jones *et al.*, 2003), and time is not measured.

3.1.7 MASTR

MASTR (Lindgreen *et al.*, 2007) uses MCMC with simulated annealing to simultaneously predict structure and perform multiple alignment of RNA sequences. The MASTR paper was accepted only 10 days after the SimulFold paper was published,

and Lindgreen *et al.* (2007) note the similarities, they also note that SimulFold takes quite a different approach, which is more computationally complex.

MASTR uses a heuristic sampling approach, where the cost function is defined as a sum of three terms: an alignment term, a structure term, and a covariance term. This function is minimized using simulated annealing where the moves are to change the structure on the basepair level or by moving gaps around in the sequence alignment.

The alignment cost is the negative log-likelihood of the alignment probability calculated using gap open, gap extension and base substitution probabilities.

The structure cost is calculated from base pairing probability matrices (McCaskill, 1990), transformed into a cost function using the negative logarithm of the mean probability and a threshold to account for the background probability of random basepairs being found.

The covariation cost is computed based on Hamming distances between aligned base pairs, and stacking information is incorporated.

Simulated annealing is used to optimize this combined cost function. The moves in the state space consist of sequence moves and structure moves. The first few iterations are purely sequence moves since the initial alignment is random and so common structure cannot be deduced from it.

Sequence moves consist of gap block moves, gap insertion and gap deletion. Structure moves consist of adding a basepair, extending a stem, and deleting a basepair.

MASTR has a large number of parameters (move probabilities, etc.) and thresholds, which are optimized by hand (on a different set of data than that used in the evaluation).

MASTR is implemented in C++ and compared with FoldalignM, LocARNA and RNA Sampler. Structure prediction and alignment quality are comparable with the other methods, with only RNA Sampler outperforming MASTR on low sequence similarity datasets.

In terms of efficiency, no complexity analysis is presented, but it is shown in Lindgreen *et al.* (2007) on the datasets tested that MASTR is faster than RNA Sampler and RNAforester, but slower than LocARNA.

It is stated (without explanation) that MASTR can handle larger datasets than RNA Sampler or FoldalignM. However, as is noted elsewhere in the paper, MASTR depends on the partition function to calculate basepair probability matrices, which has known problems for long sequences, and hence only short (less than 250 base-pairs) sequences are used in the tests. This is of course a problem shared with other techniques that use the partition function (for example PMComp, LocARNA, FoldalignM), and is problematic as we may well want to align sequences up to 10 000 nucleotides in length (Jossinet *et al.*, 2007).

3.1.8 Pruning of the dynamic programming matrix

Havgaard *et al.* (2007) describes several algorithms and heuristics for improving the performance of Foldalign (Havgaard *et al.*, 2005). These techniques are subsequently applied in FoldalignM to improve the performance of the PMComp algorithm.

Pruning consists in comparing any dynamic programming score to a local threshold, and not storing that score in the matrix if it is less than the threshold (note that Dynalign scores are being maximized, not minimized). The local threshold depends on the lengths of the subsequences being aligned. For global alignments, a different threshold is used, combining local thresholds and a gap-elongation cost. This pruned sub-alignment can then not be part of any longer sub-alignment.

In addition to pruning, constraints on motif length and maximum length difference between subsequences are employed.

In evaluating the version of Foldalign with pruning against Stemloc, Dynalign, LocARNA and Consan, it is found that LocARNA is the fastest and uses the least memory, but least accurate (by correlation coefficient). Foldalign, while slower than LocARNA, is more accurate, and the other methods are slower still for no gain in accuracy.

Murlet (Kiryu *et al.*, 2007) is another method based on Sankoff's algorithm. Rather than pruning, it relies on preprocessing to reduce the complexity of the dynamic program. This preprocessing computes the probability of matching two positions in an alignment, computed with a Pair Hidden Markov Model (Durbin *et al.*, 1998), and the base pairing probability, computed with McCaskill's algorithm (McCaskill, 1990). In addition, it employs a heuristic, and distinguishes between the problems of multiple alignment and consensus structure prediction, solving

only the former problem rather than both simultaneously, in order to obtain a more efficiently solved variant of Sankoff's algorithm.

3.1.9 FoldalignM

FoldalignM (Torarinsson *et al.*, 2007) is a reimplementation in Java of PMComp (Hofacker *et al.*, 2004), with improvements to the dynamic programming recurrence to allow pruning of sub-alignments from the matrix when their score is below a threshold, and the addition of a maximum allowed length difference in the alignment. This pruning saves 20 – 60% of CPU time.

FoldalignM is extended to multiple sequence alignment by computing the most frequently occurring base pairs in all progressive alignments and assigning a probability of 1.0 to these base pairs in a new base pairing probability matrix, then aligning each sequence again to this consensus matrix.

FoldalignM is evaluated only against Foldalign and CMFinder (Yao *et al.*, 2005). It is noted that FoldalignM was to be extended to local alignment, but the authors became aware of LocARNA (another reimplementation of PMComp) which does so in a way that uses on O(L^2) memory, only when finalizing their work.

3.1.10 Lara

Lara (Bauer *et al.*, 2007) uses a graph-theoretic model for pairwise alignment, which is transformed into an integer linear programming (ILP) problem and solved by Lagrangian relaxation. Lara can handle pseudoknots.

Lara has a time complexity of O(n^2) for the pairwise case, and is extended to multiple alignment using T-Coffee (Notredame *et al.*, 2000).

Lara is compared to MARNA (Siebert and Backofen, 2005), S A (Dalli *et al.*, 2006), FoldalignM and MAFFT (Katoh *et al.*, 2005). Note that MAFFT is a purely sequence-based method. Lara is found to perform the best as the number of input sequences increases. In the pairwise case, FoldalignM is the best approximation to the reference. Strangely, MAFFT (which uses no structure information) is closer to the manually curated reference alignment than any method except Lara, for alignment of more than 10 sequences. As we will see in Section 3.2.2 below, we also find that MAFFT (G-INS-i) has surprisingly good performance for a non-structural method on multiple alignments with larger numbers of sequences.

The running time of Lara is compared to that of MARNA and FoldalignM for increasing numbers of sequences in the multiple alignment. FoldalignM is consistently faster, especially as the number of sequences increases, due to it using a progressive alignment strategy which does not require the computation of all pairwise alignments, unlike MARNA and Lara.

3.1.11 Aln3nn

Aln3nn (Kruspe and Stadler, 2007) is, like S A , a "structure enhanced" approach, and in fact it uses the same technique as S A to incorporate RNA structural information into a sequence alignment. Aln3nn, however, rather than performing pairwise alignment, uses the dynamic program described by Konagurthu *et al.* (2004) to perform an exact three-way multiple alignment with affine gap costs. These three-way exact alignments are then combined using the Neighbor-Net algorithm (Bryant and Moulton, 2004) for multiple alignments of more than three sequences.

Aln3nn also makes use of a divide-and-conquer technique to reduce the time (and space) complexity of the three-way alignment. The sequences are divided into smaller subsequences of a threshold length l, and these subsequences are aligned separately and the results concatenated. This can result in much reduced time and memory requirements, but means that the obtained alignments are only approximations, rather than the optimal alignments produced by the algorithm described originally by Konagurthu *et al.* (2004).

Aln3nn has a time complexity of $O(Nn^2 + Nnl^2)$ where N is the number of sequences to align, n is the sequence length, and l is the cutoff length used in the divide-and-conquer technique.

3.1.12 BlockMSA

BlockMSA (Wang *et al.*, 2007) uses *biclustering* for multiple RNA sequence alignment. In biclustering, data in a matrix is clustered by rows (data objects) and columns (features) simultaneously. BlockMSA transforms multiple sequence alignment into a biclustering problem by identifying "blocks", which are candidate multiple local alignments (i.e. subsequences from multiple sequences that can be aligned with each other). These blocks are then formed into a binary matrix where each row is a block and each column is a sequence, with a value of 1 indicating the block is in

the sequence and 0 that it is not. The biclustering procedure identifies "biclusters", submatrices with all elements equal to 1, whose columns are a subset of sequences and whose rows are a subset of blocks. These correspond to the conserved features in a set of sequences, so the identified blocks are locally aligned regions in that set of sequences.

The candidate blocks are identified by dividing the sequences into overlapping fragments of fixed length and generating a library of pairwise alignments for all possible sequence pairs using CLUSTAL W (Thompson *et al.*, 1994). The blocks are then constructed from ungapped subregions with similarity scores above a threshold by a greedy clustering procedure that adds the fragment to each cluster with maximum sum of similarity scores with each fragment in the cluster.

BlockMSA has a time complexity that is in the worst case $O(n^2l^2)$ where n is the number of sequences in the multiple alignment and l is the average unaligned sequence length. As we will see in Section 3.2.2 below, BlockMSA is in practice one of the fastest methods, but, as we might expect from its purely sequence (not RNA structure) based method, its accuracy is not competitive with methods that take structure into account. It should, however, be noted (as it is in Wang *et al.* (2007)) that BlockMSA is intended specifically for local, not global, alignments, and BRAliBase 2.1 is not especially suited for benchmarking the kind of application for which BlockMSA is intended, since it has at most 15 sequences in a multiple alignment, and the reference alignments do not generally have cases with a large number of insertions and deletions. Hence Wang *et al.* (2007) construct a specific benchmark, however they compare BlockMSA only with other sequence-based methods, not any specific RNA structural alignment methods such as S AL.

3.2 Benchmarking RNA structural alignment algorithms

As we have just seen, there is a large variety of algorithms for RNA structural alignment. The paper describing a new method may compare it to only a few existing methods, and different data sets and evaluation techniques may be used in each paper. In order to more thoroughly evaluate and compare methods, a single benchmark, against which all methods are evaluated on the same criteria, is desirable. As has already been mentioned, Gardner *et al.* (2005) provides a such

Method	Version	Options
S A	0.5.4	default
MAFFT	6.520b	G-INSI-i
T-Coffee	5.65	default
mLocARNA	1.2.2a	default
mLocARNA	1.2.2a	`--indel=-200 --indel-open=0`
FoldAlignM	2.1.0	default
aln3nn	0.6.0	default
CLUSTAL W	1.83	default
MARNA	January 2008	default
MASTR	December 2007	default
Lara	1.3	default
Murlet	0.1.1	default
Stemloc	0.19b	default
RNA Sampler	1.3	`-S 100 -i 15`
BlockMSA	January 2008	default
R-Coffee	5.65	default

Table 3.1: *Methods evaluated in the RNA structural alignment benchmarking.*

a benchmark, however many new methods have been published since then. An updated benchmark, BRAliBase 2.1 (Wilm *et al.*, 2006) has since been published, and used to evaluate sequence (not structural) alignment of structural RNAs. Here we use this benchmark data set to evaluate a number of the methods described previously, in a consistent fashion. Hence in order to ensure the programs were run on the same data set and the same hardware for comparing execution times, we only evaluate methods that are available to download and run locally (rather than on a webserver).

Table 3.1 lists the methods we evaluated, each of which has been previously described in this chapter or in Chapter 2.

3.2.1 Methods

The BRAliBase 2.1 benchmark (Wilm *et al.*, 2006) contains 18 900 aligned sets of sequences. There are sets with 2, 3, 5, 7, 10, and 15 aligned sequences, and average pairwise sequence identity (*APSI*) ranging from 20% to 95%. The alignments are based on hand-curated seed alignments of 36 RNA families.

To evaluate RNA structural alignment methods, we use the same methods as described by Wilm *et al.* (2006); the methods are run on the BRAliBase 2.1 sequences, and the resulting alignments compared with the reference alignment using the

BRALISCORE (Gardner *et al.*, 2005; Wilm *et al.*, 2006). This score is defined as the product of the sum-of-pairs score (*SPS*) and the structure conservation index (*SCI*). The SPS, commonly used in sequence alignment benchmarks, is the fraction of all possible pairs aligned in both the evaluation and reference alignments. The perfect SPS is 1, and the worst possible SPS is 0. The SCI (Washietl *et al.*, 2005) measures the structural conservation in the alignments to be evaluated. This is computed by comparing the consensus minimum free energy (MFE) of the alignments with the MFE of each individual sequence in the alignment, as calculated by `RNAalifold` (Hofacker *et al.*, 2002). The SCI is defined as the ratio of the consensus MFE to the average of the individual sequence MFEs. An SCI close to 0 means that no consensus structure was found, and a perfectly conserved structure has an SCI close to 1. Note that the SCI can be greater than 1, indicating the secondary structure is perfectly conserved and is also supported by mutations in the sequences that maintain the secondary structure. The SCI specifically measures the structural (not sequence) part of the alignment accuracy, and, unlike the SPS, is independent of the reference alignment.

As did Wilm *et al.* (2006), we used the modified `compalignp` program[2] to compute the SPS, and the `scif` program[3], based on `RNAZ` (Washietl *et al.*, 2005) to compute the SCI. We calculated the APSI using the `alistat` program[4]. The BRALISCORE and CPU time graphs of the results were generated using R (R Development Core Team, 2007) and the curves fitted to the data points using locally weighted polynomial regression with the `lowess` function.

3.2.2 Results and discussion

Figure 3.3 shows the BRALISCORE against the APSI for the different methods, and Figure 3.4 shows CPU time against average sequence length for the different methods, each for the different number of sequences in the alignment in the benchmark from $k = 2$ (i.e. pairwise) to $k = 15$.

We can see that `mLocARNA` is consistently the best method measured by BRALISCORE on this benchmark. For pairwise alignments, FoldalignM , Lara and RNA Sampler (for low APSI) also perform well, but as the number of sequences in-

[2] `http://www.biophys.uni-duesseldorf.de/bralibase/compalignp.tgz`
[3] `http://www.biophys.uni-duesseldorf.de/bralibase/scif.tgz`
[4] `ftp://ftp.genetics.wustl.edu/pub/eddy/software/squid.tar.gz`

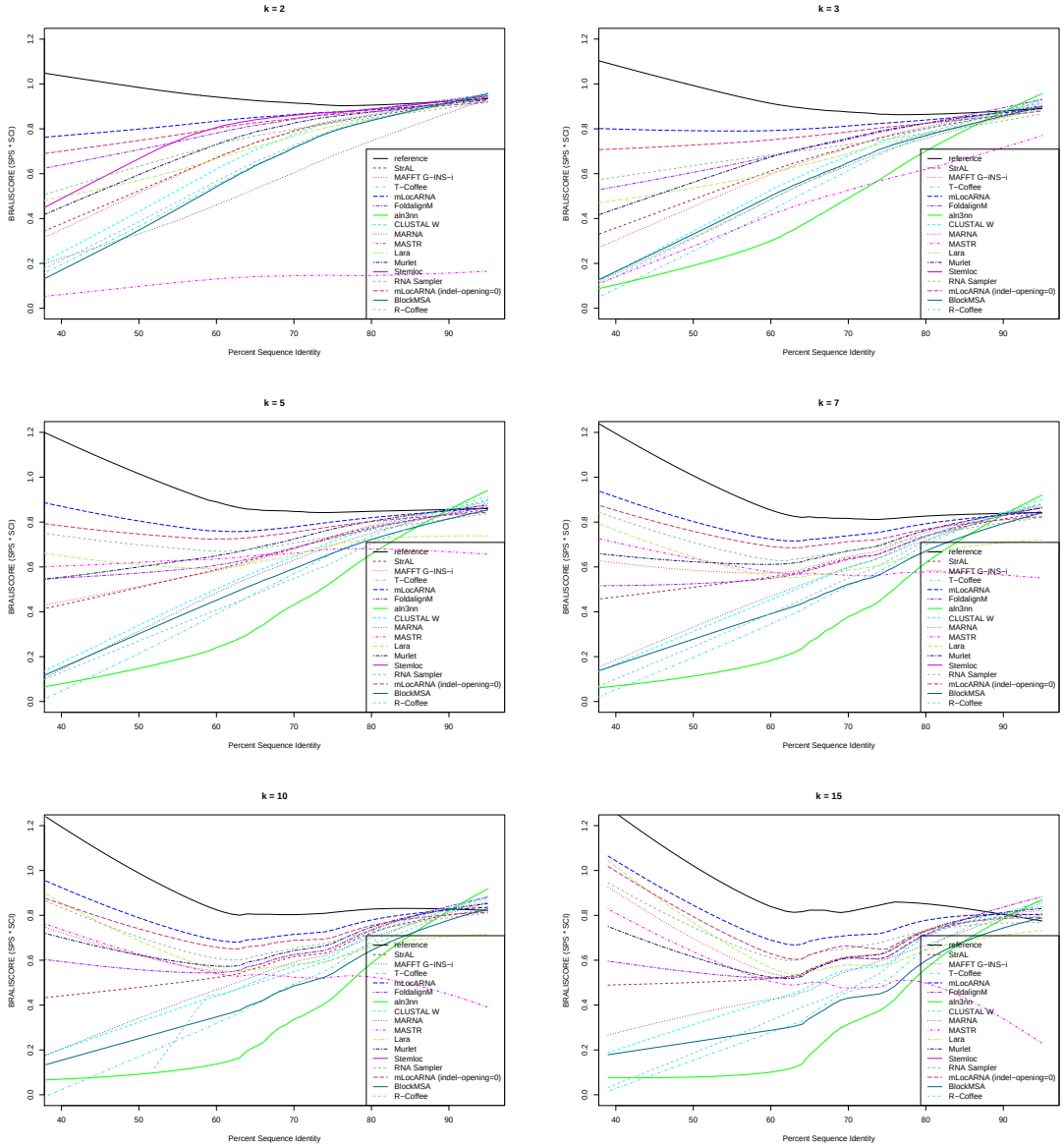


FIGURE 3.3: *BRALISCORE (SPS × SCI) versus APSI (Average Percentage Sequence Identity) for multiple alignments of $k = 2, 3, 5, 7, 10$ and 15 sequences using different methods.*

`aln3nn` *is only included for* $k \geq 3$ *as it requires a minimum of 3 sequences, and* `Stemloc` *is only included for* $k \leq 5$ *due to running out of memory for* $k > 5$.

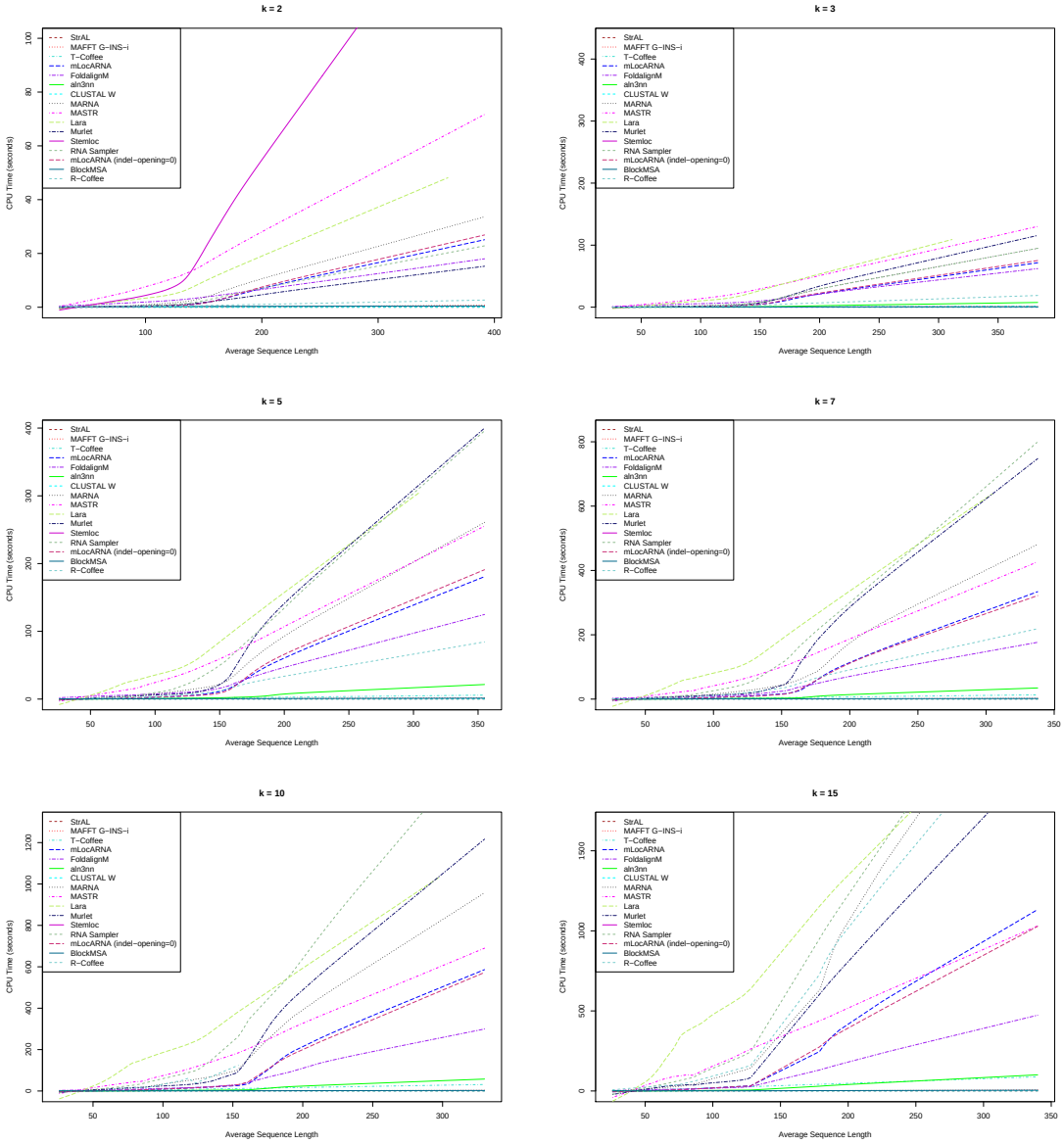


FIGURE 3.4: *CPU time versus average sequence length for multiple alignments of $k = 2, 3, 5, 7, 10$ and 15 sequences using different methods.*

The CPU time is seconds of user CPU time on a single core of an Intel Xeon Pro 2.4 GHz system with 4 GB memory running Linux. `aln3nn` *is only included for $k \geq 3$ as it requires a minimum of 3 sequences, and* `Stemloc` *is only included for $k \leq 5$ due to running out of memory for $k > 5$.*

creases, MARNA starts to become competitive at low APSI values. The sequence only alignment methods, such as `aln3nn`, CLUSTAL W, and T-Coffee have much lower BRALISCORE values, especially at low APSI values, reflecting, as we would expect, the fact that they do not specifically take RNA structure into account. An exception is MAFFT (G-INS-i), which, like other sequence-only methods has low BRALISCORE for small values of k, but which improves greatly as k increases. At $k \geq 7$, it has higher BRALISCORE for low APSI values than FoldalignM, and at $k = 15$ is nearly as good as Lara.

The behaviour of MASTR is worth noting; we can see that at $k = 2$ its BRALISCORE is clearly much lower than other methods, but it is vastly improved for $k = 3$, and as k increases, MASTR gets better at low sequence identity alignments, but its performance for high sequence identity alignments declines. The performance evaluation of MASTR in Lindgreen *et al.* (2007) is over a set of multiple alignments with varying numbers of sequences, so this behaviour is not apparent in that assessment.

Somewhat ironically, given the results we show here, neither the `LocARNA` paper (Will *et al.*, 2007) nor the paper for the algorithm on which it is based (Hofacker *et al.*, 2004) do any benchmarking against other methods or make any particular claims to its accuracy or speed compared to other methods. Our results show that, using the BRAliBase 2.1 benchmark, `LocARNA` is the most accurate method (especially for low sequence identity alignments).

Considering the CPU time requirements, the sequence only (not structure) alignment methods, as we would expect, are much faster. CLUSTAL W and MAFFT are clearly the fastest methods, showing little increase in time visible on the scale of these graphs even for $k = 15$. CLUSTAL W does not perform well (measured by BRALISCORE) on low sequence identity alignments, however, as described above, MAFFT G-INS-i performs surprisingly well (for larger numbers of sequences). As noted by Wilm *et al.* (2006), for APSI $\leq$ 55%, sequence alignment may not be sufficient for aligning homologous RNA sequences and structure alignment is necessary. Given the results shown here, we would be inclined to move this threshold somewhat higher, to say 60%, given the greatly increased BRALISCORE achievable by some structural alignment methods, such as `LocARNA` and `Lara` at that APSI, although for very long sequences the resource requirements could become prohibitive. In such cases, especially when a multiple alignment of 7 or more sequences is required,

MAFFT G-INS-i would be a good choice, given its speed and its very good performance on BRALISCORE for $k \geq 7$. BlockMSA is comparable with the other sequence only methods in CPU time, however its BRALISCORE reflects this, being worse than CLUSTAL W, especially for low APSI sequences. T-Coffee and `aln3nn`, also sequence only methods, are the next fastest tested, but, this being the case, also have very low BRALISCORE for low APSI sequences.

In summary, `LocARNA` is found to be consistently the most accurate method on this benchmark, although as sequence lengths increase it cannot compete on speed with sequence-only methods. Hence we now turn to investigations into how we might find an improved method for solving the dynamic program on which `LocARNA` is based (Equation 3.6) that does not compromise the solution (i.e. is an exact solution not a heuristic) but can be computed much more quickly.

3.3 Improving the performance using bounding on a simplified case

In order to explore the possibilities of applying bounding, described in Section 2.1, to the dynamic program for RNA base pairing matrix alignment, we will first start with a simplified case.

In the following, we will use the RNA base pairing matrix alignment dynamic program described in Section 3.1.4 with the simple scoring scheme $\tau = \sigma = 0$, and use binary base pairing probabilities. In addition, we will set the gap cost to zero ($\gamma = 0$). When we use the simple scoring scheme $\sigma = \tau = 0$, matches of unpaired bases do not contribute to the score.

3.3.1 Methods

Under these assumptions, we can quickly compute a simple upper bound on the value S^M in Equation 3.6. $S^M(i, j, k, l)$ is the score of the best match subject to the constraint that (i, j) and (k, l) are matched base pairs (Hofacker *et al.*, 2004). $S^M(i, j, k, l)$ must be bounded above by the total possible number of matches of paired bases inside (i, j) in A and (k, l) in B. This is the minimum of the number of paired bases inside (i, j) of A and the number of paired bases inside (k, l) of B:

$$S = \left\{(h,q) : h > i \wedge q < j \wedge \Psi^A_{h,q} > 0\right\} \quad (3.7)$$

$$T = \left\{(h,q) : h > k \wedge q < l \wedge \Psi^B_{h,q} > 0\right\} \quad (3.8)$$

$$U_{pairs}(i,j,k,l) = \min\{|S|, |T|\} \quad (3.9)$$

Using this upper bound, which can be computed in linear time, we can reduce the number of computations in the dynamic programming implementation (Puchinger and Stuckey, 2008) by computing the maximum of the first three cases of Equation 3.5, and then only computing the fourth case (involving S^M) if this value is less than the upper bound we have computed for S^M.

The `pmcomp` algorithm described by Hofacker *et al.* (2004) was re-implemented in Python.[5] In order to operate with a binary base pairing probability matrix, it is necessary to use a minimum free energy structure, or other consistent set of base pairings, rather than a base pairing probability matrix representing a partition equilibrium function. We use the minimum free energy structure for this purpose.

In order to take advantage of bounding, this implementation, unlike `pmcomp.pl`, computes the dynamic programming "top-down" rather than "bottom-up". That is, rather than computing every element of the dynamic programming array, a recursive function is used, which makes use of "memoization" to record computed values and return previously computed values rather than recompute them. The advantage of this is that, unlike the "bottom-up" method, values that are not actually required will never be computed. Because not every value in the array is computed, it is also therefore possible to use a hash table rather than an array to reduce the space requirement.

For experiments with the RNA structural alignment dynamic programs, we use the BRAliBase II (Gardner *et al.*, 2005) pairwise RNA structural alignment benchmark (data set 2), which consists of 118 pairs of RNA sequences (from a total of 78 individual sequences) to be aligned. This data set was specifically developed for testing pairwise RNA structural alignment methods, and contains pairwise alignments of tRNA sequences, from 10% to 90% sequence identity.

[5] `http://www.python.org`; with the NumPy library `http://www.scipy.org`.

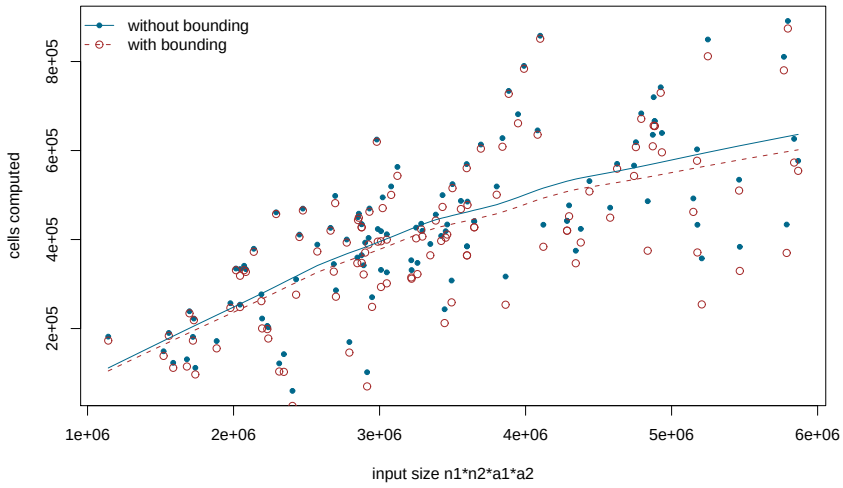


FIGURE 3.5: *Number of cells computed with and without bounding on simplified problem.*

We used the `RNAfold` program in the Vienna RNA Package (Zuker and Stiegler, 1981; Hofacker *et al.*, 1994) to generate the base pairing probability matrix and minimum free energy structure for each sequence in the data set. Then the MFE structure can be used for the simplified case that requires a binary base pairing matrix, and the partition equilibrium (McCaskill) base pairing probability matrix can be used for the actual problem, as described in Section 3.4 below.

3.3.2 Results and discussion

The reduction in the number of cells computed by using the simple bounding described in Section 3.3.1 is shown in Figure 3.5.

In this graph the x-axis is the size of the input measured as $n_1 n_2 a_1 a_2$ where n_1 and n_2 are the lengths of the two input sequences and a_1 and a_2 are the number of base pairs in the MFE structure of the two sequences. Therefore (assuming similar input lengths), a straight line on such a graph represents a value that is quartic ($O(n^4)$) in the input size. The graphs were generated using `R` and the curves fitted to the data points using locally weighted polynomial regression with the `lowess` function.

It can be seen that this is not a very impressive result; the maximum reduction in the number of cells computed is 57% and the mean is only 6.4%. This is not very

encouraging, but the full problem has more scope for including extra information in the bounding function, so in the next section we investigate whether or not that will provide better opportunities for using the bounding technique.

3.4 Using bounding to attempt to improve performance on the real problem

The bounding function described in Section 3.3.1 can only be used in the simplified version of the problem where the base pairing probability matrix has been made discrete (and consistent), and the gap cost γ and base substitution matrix σ are zero. A more complex bounding function is required for the general case of base pairing probability matrices and nonzero gap costs and substitution matrices.

3.4.1 Methods

We can compute an upper bound on the value of the dynamic programming recurrence $S(i, j, k, l)$ of Equation 3.5 as the sum of two parts: a sequence part consisting of the score contributed by the purely sequence alignment, and a structural part, consisting of the score contributed by the purely structural (base pair) matching. I.e.:

$$U(i, j, k, l) = U_{seq}(i, j, k, l) + U_{struct}(i, j, k, l) \tag{3.10}$$

The structural part U_{struct} may be computed in a similar manner to the upper bound for the simplified discrete case described in Section 3.3.1. Specifically, the argument there as to the maximum number of possible matches of paired bases inside the intervals (i, j) in A and (k, l) in B still holds. Therefore, the value $U_{pairs}(i, j, k, l)$ of Equation 3.9 is still an upper bound on the *number* of possible matches of paired bases. However, this is no longer necessarily an upper bound on the structural component of the score in the interval, as the values of the base pairing probability matrices Ψ are no longer constrained to be either 0 or 1. Such an upper bound can be obtained by multiplying the maximum number of matched base pairings possible in an interval by the sum of the two maximum values of Ψ^A and Ψ^B in the intervals (i, j) of A and (k, l) of B respectively:

$$U_{struct}(i,j,k,l) = U_{pairs}(i,j,k,l) \cdot (mw_A(i,j) + mw_B(k,l)) \tag{3.11}$$

where

$$mw_A(i,i) = 0 \tag{3.12}$$

$$mw_A(i,j) = \max_{i \le h \le j} \left\{ \Psi^A_{ih}, mw(i+1,j) \right\} \tag{3.13}$$

and similarly for mw_B. Note that $mw_A = \max_{i \le h < q \le j} \Psi^A_{hq}$ but it has been defined as a recurrence relation in Equation 3.13 in order to be implemented by dynamic programming which allows reuse of previously computed values.

Note that, since this computation is quadratic, the purely sequence score upper bound $S_{seq}(i,j,k,l)$ could in fact be computed as the exact sequence alignment score, i.e. the first three terms of Equation 3.5 which is effectively equivalent to the Needleman-Wunsch algorithm (Needleman and Wunsch, 1970), since this is also quadratic. However, a faster (linear) upper bound on the sequence score can be computed by observing that the score cannot be greater than the number of symbols in common between the two subsequences, multiplied by the matching score, plus their difference in length multiplied by the gap penalty:

$$U_{seq}(i,j,k,l) = \left(\sum_{x \in \{A,C,G,U\}} \min \left\{ \text{Count}_x(A_{i\ldots j}), \text{Count}_x(B_{k\ldots l}) \right\} \right) \cdot \sigma_{match} + |(j-i) - (l-k)| \cdot \gamma \tag{3.14}$$

where the subsequences $A_{i\ldots j}$ and $B_{k\ldots l}$ are regarded as strings over the RNA alphabet $\{A, C, G, U\}$ and $\text{Count}_x(s)$ counts the number of occurrences of character x in string s.

In practice, the contribution of the sequence score is insignificant compared to that of the structural component, and so the difference between using the exact sequence score and the fast upper bound on it is insignificant.

Bounding as discussed so far has referred to "local" bounding, in which upper bounds are computed on the values of subsequent terms in the dynamic programming recurrence, and these terms are not evaluated if the upper bound is not greater than the value of the terms so far (inside a maximization).

"Global" or "argument" bounding makes use of an extra argument to the dynamic programming function in order to communicate the previously calculated lower bound. The function then returns immediately the (computed or retrieved) upper bound if it is less then or equal to the (passed in as an argument) lower bound. Otherwise the dynamic programming recurrence is computed, and if the value is greater than the lower bound it is stored as the optimal value and returned, else it is stored as a bound and returned. This is both neater than local bounding, and has greater potential for reducing computations (Puchinger and Stuckey, 2008).

Ordering refers to rearranging the order of computation of subproblems in such a way as to (hopefully) maximize the probability that optimal values are computed earlier and thereby avoid the need to compute many suboptimal values. We note that in Equation 3.5, the last term involves computing the recurrence for all subsequences of current intervals in the two sequences. In many cases, these subsequences will be of quite different lengths, and therefore less likely to be the optimal sub-alignments than subsequences of similar lengths. We may therefore expect that re-ordering these computations to compute more similarly sized subsequences first would be beneficial.

Bounding and ordering are described in detail by Puchinger and Stuckey (2008).

As described in Section 3.3.1, the dynamic programming algorithm described in Hofacker *et al.* (2004) was reimplemented in Python. Enhancement of this dynamic programming algorithm by means of (local and also global) bounding, using the function described in Section 3.4.1 was implemented in this program.

In addition, the same algorithms were also reimplemented in the C programming language, using static arrays to store the dynamic programming values, in order to remove any implementation artifacts that may have been caused by recursion or the use of dictionaries in Python.

In order to test sensitivity of performance to the accuracy of the upper bounding function, an option was added to the Python implementation to use a "perfect" upper bounding function i.e. one which uses the actual score, but which can be increased by some factor.

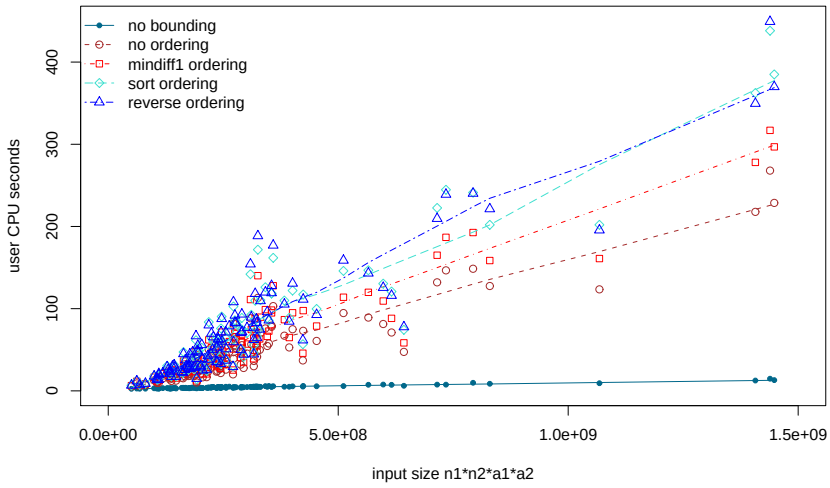


FIGURE 3.6: *CPU seconds used for different bounding options (C implementation)*

"user CPU seconds" are seconds of user CPU time on a single core of an AMD Quad Core Opteron (2.3GHz, 32 GB RAM) running Linux.

3.4.2 Results and discussion

In all cases the various bounding options used do *not* change the value computed (i.e. are not approximations, heuristics, etc.); the bounding removes redundant computations and does not affect the final computed value. Hence part of the process for generating these results involved asserting that the final value was equal for the same inputs in all cases. The gap penalty γ was set at -3 and the unpaired base substitution value σ was set to 0.05 for a match and 0 for a mismatch. The paired base substitution matrix τ was set to always zero.

In all the graphs in this section the x-axis is the size of the input measured as $n_1 n_2 a_1 a_2$ where n_1 and n_2 are the lengths of the two input sequences and a_1 and a_2 are the number of above-threshold probability base pairs in the sequences. Therefore (assuming similar input lengths), a straight line on such a graph represents a value that is quartic ($O(n^4)$) in the input size. The graphs were generated using `R` and the curves fitted to the data points using locally weighted polynomial regression with the `lowess` function.

Figure 3.6 shows the user mode CPU usage, and Figure 3.7 the number of "cells" (different values of $S(i, j, k, l)$ computed) for the C implementation for some different bounding options, as follows:

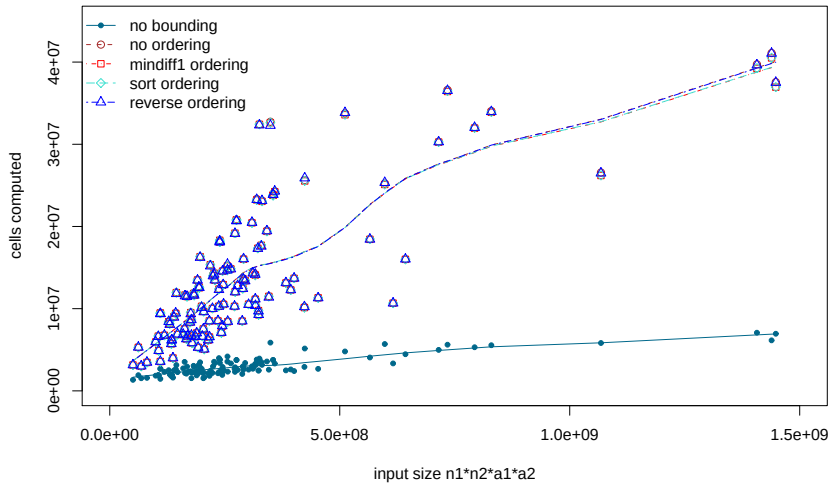


FIGURE 3.7: *Cells computed for different bounding options (C implementation)*

no bounding No bounding is used.

no ordering Global bounding, with no ordering of the computations in the inner loop. The order is simply by ascending i and k.

mindiff1 ordering The computations are ordered by finding the minimum difference between the lengths of the two subsequences, and computing with these two intervals first.

sort ordering The list of intervals is sorted by length difference, ascending, and computed in that order.

reverse ordering Like sort ordering, but the order is reversed, to make it in some sense the *least* optimal order of computation.

We observe from Figure 3.6 that not using bounding at all has the lowest CPU usage, and also a slower rate of growth in the input size. Of the different bounding options, using no ordering is the most efficient, while the "mindiff1" ordering is less efficient, and both sorting options are much less efficient, and about the same as each other. The inefficiency of global bounding on this problem will be addressed further below, and here we just consider the effect of ordering. We account for the fact that having no ordering is the most efficient by the fact that the data structures built from the list of above-threshold base pairing probabilities are actually pre-sorted

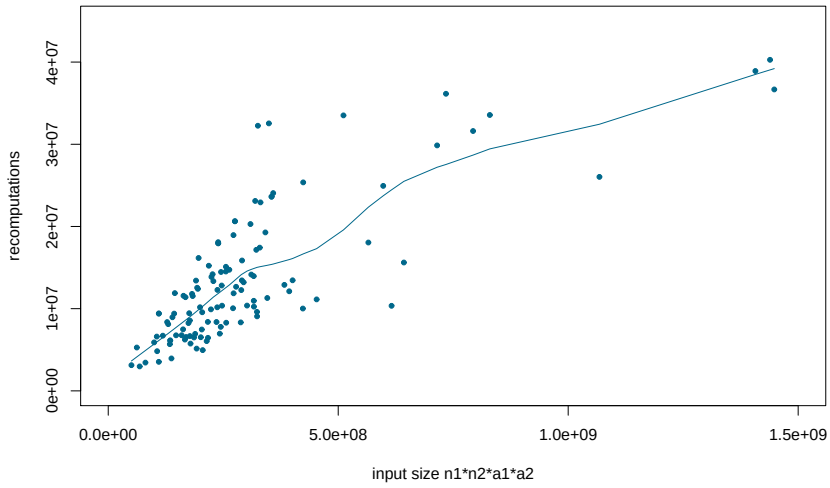


FIGURE 3.8: *Number of times value computed is not greater than lowbound (C implementation)*

by sequence position ascending, and in fact this ordering usually corresponds to the "optimal" ordering anyway. Hence the overhead of the sorting for the two sort orderings causes an increase in CPU usage for little or no gain. The "mindiff1" ordering requires much less overhead (as there is no sorting) but similarly does not often differ from the default ordering anyway.

From Figure 3.7 we cannot really distinguish any of the bounding options from each other, just that they all in fact compute significantly *more* cells than not using bounding at all, which is not at all what we would have expected. Note that the number of cells computed refers to the sum of the total number of S values computed, and the number of times the dynamic programming function computed a value not greater than the passed-in lower bound and therefore stored this value only as a bound, not the optimal value. It is this latter case that appears to be occurring frequently and causing (global) bounding to perform badly.

Figure 3.8 shows the number of times this situation occurs for different input sizes. It can be seen that it increases (at least) linearly on this graph, i.e. in $O(n^4)$ for sequences of length n, and that therefore bounding the dynamic programming with this upper bounding function will be deleterious rather than beneficial with respect to the total number of computations made.

Given this, it seems useful to determine just how good an upper bounding function must be in order to reduce rather than increase the number of computations.

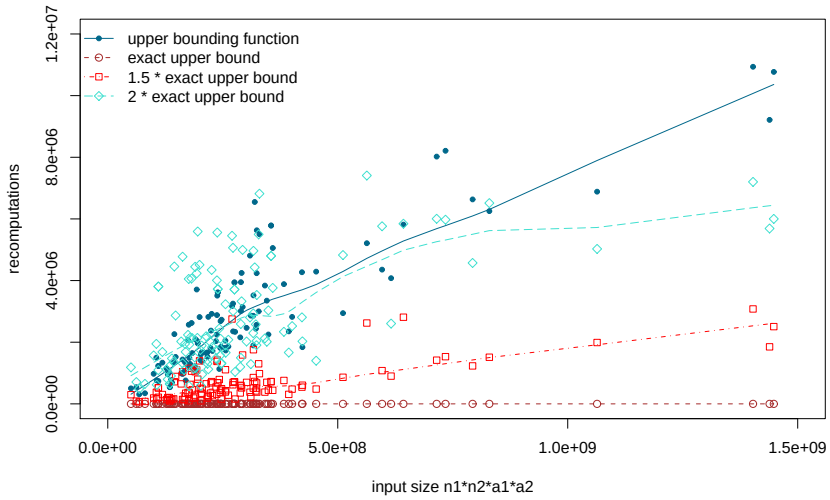


FIGURE 3.9: *Number of times value computed is not greater than lowbound, for "perfect" bounding perturbed by different fractions (Python implementation)*

We investigated this by using a "perfect" bounding function, that is, one that returns the exact value of the function $S(i, j, k, l)$ as the upper bound. This is clearly the best possible case; it can then be adjusted by increasing it by various factors to be "less perfect". Figure 3.9 shows the results:

upper bounding function The upper bounding function that has been used so far (described in Section 3.4.1).

exact upper bound The perfect upper bounding function, which always returns the exact value of the function as the upper bound.

1.5 * exact upper bound The perfect upper bound increased by a factor of 1.5 (i.e. 50%).

2 * exact upper bound The perfect upper bound increased by a factor of 2 (i.e. 100%).

It can be seen that, as one would expect, the perfect upper bounding function results in effectively no recomputations. Increasing the upper bound by 100% or even 50% causes the number of recomputations to increase with input size, indicating that an upper bounding function to work at all on this problem would have to be a very good approximation of the exact solution.

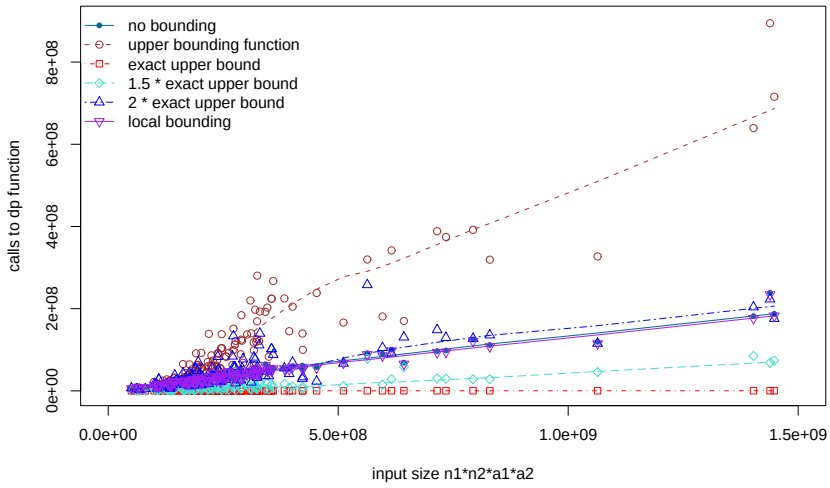


FIGURE 3.10: *Number of calls to dynamic programming recursive function for "perfect" bounding perturbed by different fractions (Python implementation)*

The effects of this recomputation can be seen clearly in Figure 3.10, which shows, in addition to the perfect bounding function perturbed by different factors as described previously, the results for the bounding function of Section 3.4.1 and also for no bounding, and local bounding. Counting the number of calls to the dynamic programming function is a more useful measure than CPU time in this case, since CPU time includes the time spent calculating the upper bound, which in the case of the "perfect" upper bounds means the time spent calculating the actual value to be used as the upper bound. The actual computation time can only be decreased by an upper bounding function that results in a reduction in the number of recursive calls made to the dynamic programming function, which can be seen on this graph. It can be seen that the number of calls increases linearly with input size (i.e. quartic) when no bounding is used, and that only the "perfect" bounding function and the perfect bounding function increased by a factor of 50% do better than this when global bounding is used. However, when *local* bounding is used (with the bounding function of Section 3.4.1), the number of calls is also linear, and is a slight improvement on not using bounding.

For local bounding, unlike global bounding, it is guaranteed that the number of computations cannot increase (Puchinger and Stuckey, 2008), and from Figure 3.10 it would seem that local bounding gives the best chance of improving the performance

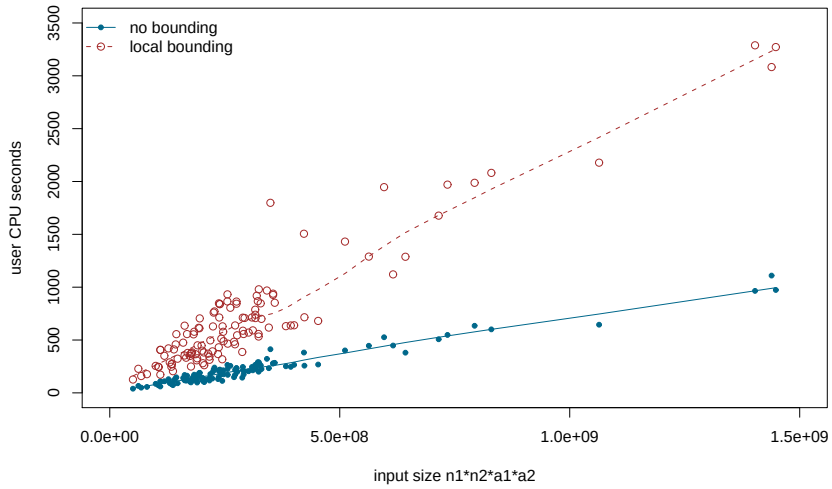


FIGURE 3.11: *User CPU seconds for local bounding and no bounding (Python implementation)*

"user CPU seconds" are seconds of user CPU time on a single core of an AMD Quad Core Opteron (2.3GHz, 32 GB RAM) running Linux.

of the dynamic programming implementation (other than the "perfect" bound, which is of course unrealistic). However, as can be seen from Figure 3.11, the cost of computing the upper bound in fact means that the CPU usage increases faster with local bounding than without bounding at all.

Figure 3.12 shows a scatter plot of values of the upper bounding function described in Section 3.4.1 against the actual function value. It can be seen that these upper bound values are often much more than 50% above the true value (indeed, often an order of magnitude more than the true value), therefore resulting in the large increase in number of recomputations and consequent poor performance that has been observed.

3.5 Bounding for purely sequence alignment

The lack of success in reducing computation using bounding in the RNA base pair probability matrix alignment dynamic programming algorithm naturally leads to the question of whether it may succeed in a related but somewhat less complex problem. The obvious problem is purely sequence (i.e. with no structural component) alignment, for example the well-known Needleman-Wunsch (Needleman

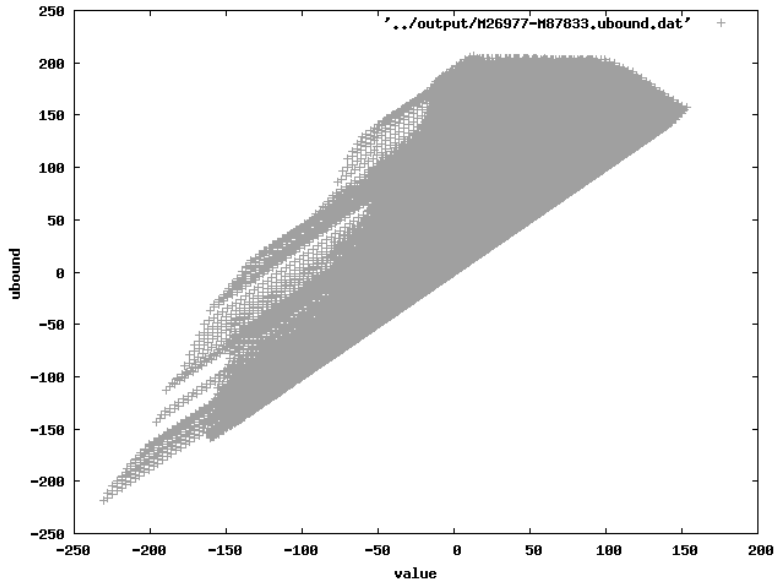


FIGURE 3.12: *Scatter plot of upper bound versus actual value for M26977 and M87833*

and Wunsch, 1970) dynamic programming algorithm, as described in Section 2.4 (specifically, Equation 2.2) of Chapter 2.

3.5.1 Methods

The Needleman-Wunsch algorithm was implemented in Python as a top-down (recursive) dynamic programming function using Python dictionaries to store the values for memoization. Both local and global bounding were implemented for comparison.

Similarly to the sequence component of the upper bound for structural alignment described in Section 3.4.1, a (computable in linear time) upper bound on the sequence score can be computed by observing that the score cannot be greater than the number of symbols in common between the two subsequences, multiplied by the matching score, plus their difference in length multiplied by the gap penalty:

$$U_{nw}(i,j) = \left(\sum_{x \in \{A,C,G,U\}} \min \left\{ \text{Count}_x(A_{0\ldots i}), \text{Count}_x(B_{0\ldots j}) \right\} \right) \cdot \sigma_{match} + |j-i| \cdot \gamma \quad (3.15)$$

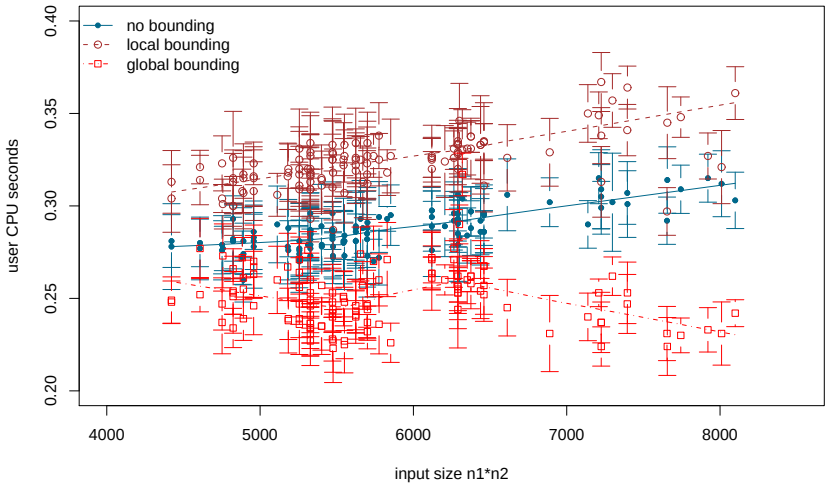


FIGURE 3.13: *CPU usage for sequence alignment with no, local, and global bounding*

"user CPU seconds" are seconds of user CPU time on a single core of an AMD Quad Core Opteron (2.3GHz, 32 GB RAM) running Linux. Each data point is the mean of the user CPU seconds for all sequence pairs with the same input size (product of the two sequence lengths), with each alignment run 10 times, and the error bars show the 95% confidence interval.

where the subsequences $A_{0...i}$ and $B_{0...j}$ are regarded as strings over the RNA alphabet $\{A, C, G, U\}$ and $\text{Count}_x(s)$ counts the number of occurrences of character x in string s.

3.5.2 Results and discussion

In all the graphs in this section the x-axis is the size of the input measured as $n_1 n_2$ where n_1 and n_2 are the lengths of the two input sequences. Therefore (assuming similar input lengths), a straight line on such a graph represents a value that is quadratic ($O(n^2)$) in the input size. The graphs were generated using `R` and the curves fitted to the data points using locally weighted polynomial regression with the `lowess` function.

The sequences used are those described in Section 3.3.1, but self-alignments were excluded as the upper bounding function is perfect in those cases (the value of the upper bound $U_{nw}(i, j)$ in Equation 3.15 is equal to the actual Needleman-Wunsch score $M(i, j)$), reducing the complexity from quadratic to linear, thereby severely biasing the resulting graphs.

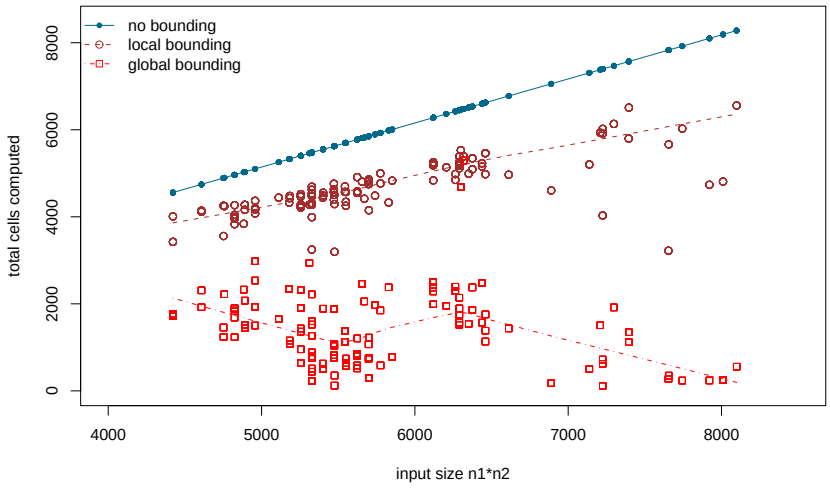


FIGURE 3.14: *Number of cells computed for sequence alignment with no, local, and global bounding*

Figure 3.13 shows that local bounding uses more CPU time than not using bounding at all, however global bounding results in a slight reduction, but more promisingly, it breaks the linear relationship between input size and CPU time. Figure 3.14 gives a clearer picture of what is going on. The Needleman-Wunsch algorithm without bounding has an exactly linear relationship between the input size and the number of cells computed, while local bounding reduces the rate of growth of the number of cells computed as input size increases, but (as we see in Figure 3.13), it is not a sufficient reduction to improve on the total time given the overhead of computing the upper bound. Global bounding, however, results in a much greater reduction in the number of cells computed, and the relationship between the input size and the number of cells computed is no longer linear at all, but depends on the particular input pairs.

Figure 3.15 shows a scatter plot of the upper bound computed according to Equation 3.15 against the actual Needleman-Wunsch sequence alignment score for a particular input sequence pair. It can be seen that the upper bound for purely sequence alignment is much better than for RNA structural alignment (Figure 3.12) on the same input sequences, giving much more useful results for global bounding in the former case.

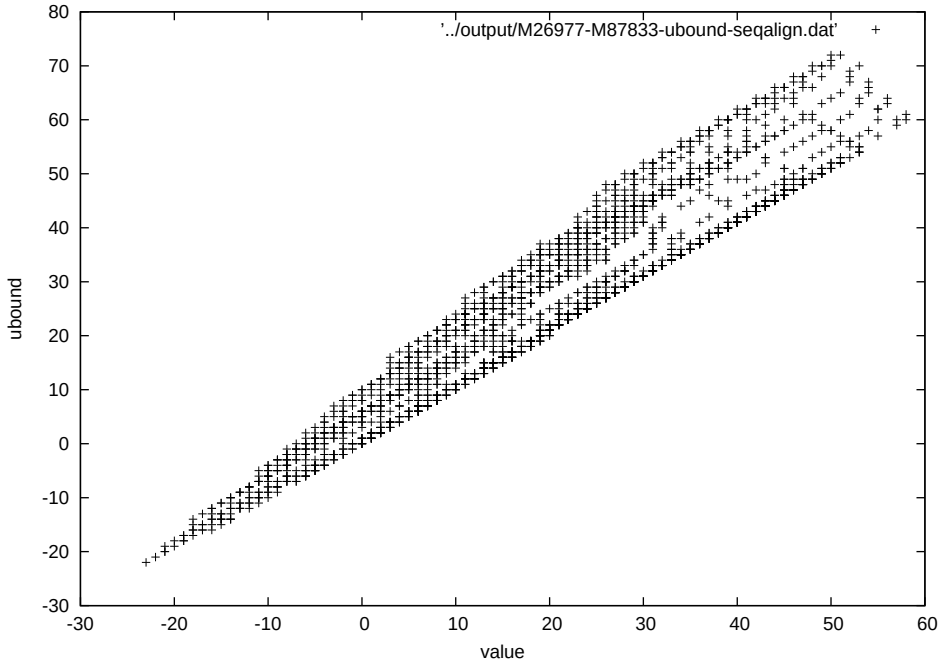


FIGURE 3.15: *Scatter plot of upper bound versus actual Needleman-Wunsch score for M26977 and M87833*

3.6 Conclusions

Despite its success in improving the performance of several other dynamic programming algorithms (Puchinger and Stuckey, 2008), the use of bounding in the dynamic programming algorithm for base pair probability matrix alignment of Hofacker *et al.* (2004) has been found to be deleterious to performance, at least for the comparatively simple upper bounding function we implemented.

In the simpler case of purely sequence alignment using the Needleman-Wunsch algorithm, it was found that a simple linear time upper bound with global bounding can significantly reduce the CPU time required to compute an exact global sequence alignment. However, for the upper bounding to be most effective, the sequences must be relatively closely related (and in the extreme case of identical sequences the upper bound is equal to the Needleman-Wunsch score), and specialized algorithms for comparison of entire closely related genomes, such as MUMmer (Kurtz *et al.*, 2004) handle this task more efficiently, and have even been parallelized for graphics programming units (Schatz *et al.*, 2007; Trapnell and Schatz, 2009).

Therefore, we conclude that the use of bounding has not been successful for the RNA structural alignment dynamic program, and in the next chapter we consider a more general technique for accelerating dynamic programs by parallelization.

4

Parallel Dynamic Programming

As shown in the previous chapter, bounding techniques, while often highly successful in improving the performance of dynamic programming algorithms, were of little use in improving the RNA base pairing probability matrix alignment dynamic program. In this chapter, we investigate a technique to speed up dynamic programs by parallelization, and describe a general technique for doing so on modern multicore processor architectures with shared memory.

We find that we are able to achieve reasonable speedups (over both a top-down and bottom-up implementation) by parallelizing the RNA base pairing probability matrix dynamic program with our technique. The technique is, moreover, completely general for dynamic programs, and we apply it to three other problems, finding the best speedup, of over 10 times for 32 threads, on a dynamic programming algorithm for the manufacturing logistics problem of minimizing the maximum number of open stacks.

In Section 4.2 we describe our approach to the parallelization of top-down dynamic programs. In Section 4.2.1 we examine previous work on concurrent hash tables, define our own implementations, and show their effectiveness in the case where the ratio of inserts to lookups is quite high. In Section 4.3 we give the results of experiments on four different dynamic programs on three different architectures, illustrating the effectiveness of the parallelization. In Section 4.4, we conclude by summarizing these results and their significance in being able to parallelize any dynamic program without regard to its specific properties or data dependencies.

4.1 Introduction

Previous efforts at parallelizing dynamic programming have focused on the "bottom-up" style dynamic programming matrix, by computing in parallel cells known to have no data dependencies. For example, the Smith-Waterman algorithm has been accelerated by the parallel computation of cells in the matrix that can be computed independently by the use of SIMD vector instructions (Wozniak, 1997; Rognes and Seeberg, 2000; Farrar, 2007), special-purpose hardware (Oliver *et al.*, 2005), general-purpose graphics processing units (*GPGPUs*) (Liu *et al.*, 2007; Manavski and Valle, 2008), or other parallel processors such as the Cell Broadband Engine (Wirawan *et al.*, 2008). More generally, Tan *et al.* (2007) describe a parallel pipelined algorithm to exploit fine-grained parallelism in dynamic programs, and apply it to Zuker's algorithm (Zuker and Stiegler, 1981; Lyngsø *et al.*, 1999) for predicting RNA secondary structure. Subsequently, Xia *et al.* (2009) implemented their own specific parallelization of the Zuker algorithm on FPGA hardware. Chowdhury and Ramachandran (2008) describe tiling sequences (recursive decompositions) for several classes of dynamic programs for cache-efficient implementation on multicore architectures.

All these techniques require careful analysis of each particular algorithm to find the data dependencies in the dynamic programming matrix, resulting in a parallelization that is specific to each individual problem. Furthermore, they only work on the "bottom-up" approach and, therefore, can only be applied to problems for which computing every cell is feasible.

The contributions of this chapter are:

- a generic approach to parallelizing "top-down" dynamic programming approach by using
  - a *lock-free* hash table for the memoization, where each thread computes the entire problem but shares results through the hash table;
  - the randomization of the order in which the dynamic program computes its subproblems to encourage divergence of the thread computations, so that fewer subproblems are computed by more than one thread simultaneously;

- an effective algorithm for a lock-free hash table supporting only insertions and lookups; and
- experimental results showing that this approach can produce substantial speedups on a variety of dynamic programs.

4.2 Methods

Our approach to parallelizing top-down dynamic programs is simple. Each thread solves the entire dynamic program independently *except that* whenever it determines a result to a subproblem it places it in a shared hash table, and whenever it begins to compute the answer of a subproblem it checks whether the result already exists in the shared hash table. When one thread has found the solution to the entire dynamic program, we have the answer and simply terminate all other threads.

As previously mentioned, an advantage of the "top-down" versus the simpler "bottom-up" dynamic programming approach, is that the former might not need to compute a value for every subproblem. This opens up the question of the order in which to compute the subproblems, since this order can make a large difference to the number of cells computed (Garcia de la Banda and Stuckey, 2007; Puchinger and Stuckey, 2008).

In a serial (single-threaded) implementation, we are constrained to choosing a single order in which to compute the subproblems. However, now that we have multiple threads available and the means to safely share values between them, we can parallelize the dynamic program by simply starting several threads at the initial function call with a randomized ordering choice. That is, each thread runs exactly the same function, starting at the same point, but the randomization of the choice of subproblems results in the threads diverging to compute different subproblems, while still reusing any value that has already been computed by a thread. In this way we take advantage of whatever parallel computing power is available to us to compute different subproblems simultaneously.

Throughout this chapter we will use the 0/1 knapsack problem, a classic problem for dynamic programming, described in Section 2.1 of Chapter 2, as an example to demonstrate our technique. In this problem, we are given the total weight (or capacity) W of the knapsack, and a set of n items $\{1, \ldots, n\}$, where each item i has

```
f(x̄)
    v ← lookup(x̄)
    if v ≠ KEY_NOT_FOUND
        return v
    if b(x̄) then v ← g(x̄)
    else
        for i ∈ 1...n
            v[i] ← f(x̄_i)
        v ← F(v[1],...,v[n])
    insert(x̄,v)
    return v
```

```
f(x̄)
    v ← par_lookup(x̄)
    if v ≠ KEY_NOT_FOUND
        return v
    if b(x̄) then v ← g(x̄)
    else
        for i ∈ 1...n in random order
            v[i] ← f(x̄_i)
        v ← F(v[1],...,v[n])
    par_insert(x̄,v)
    return v
```

FIGURE 4.1: *Generic top-down dynamic programming code on the left, and the parallelized version on the right.*

been assigned a weight w_i and a profit p_i. The problem is to choose the subset of items $I \subseteq \{1, \ldots, n\}$ such that $\sum_{i \in I} w_i \leq W$ and profit $\sum_{i \in I} p_i$ is maximized. The dynamic programming formulation computes the optimal profit $k(i, w)$ using only items in $1, \ldots, i$ with a weight limit w as:

$$k(i,w) = \begin{cases} 0 & \text{if } i = 0 \vee w = 0 \\ k(i-1,w) & \text{if } w < w_i \\ \max\{k(i-1,w), k(i-1,w-w_i) + p_i\} & \text{otherwise} \end{cases}$$

The above dynamic program is presented as a recurrence relation, where the order of computation of the two subproblems in the last case is not defined. However, an implementation using the "top-down" approach and memoization needs to determine an order. In the simple dynamic programming formulation of the 0/1 knapsack problem presented above, our technique randomly chooses, with equal probability, one of $k(i-1,w)$ and $k(i-1,w-w_i)+p_i$ to compute first.

In general, consider a dynamic program f defined as follows:

$$\begin{aligned} f(\bar{x}) \quad = \quad & \textbf{if } b(\bar{x}) \textbf{ then } g(\bar{x}) \\ & \textbf{else } F(f(\bar{x}_1), \ldots, f(\bar{x}_n)) \end{aligned}$$

where $b(\bar{x})$ holds for the base cases (which do not require considering subproblems), $g(\bar{x})$ is the result for base cases, and F is a function combining the optimal answers to a number of sub-problems $\bar{x}_1, \ldots, \bar{x}_n$. The pseudo-code to implement f as a recursive top-down dynamic program is shown on the left of Figure 4.1, where

insert($\bar{x}, v$) and lookup($\bar{x}$) respectively insert a value v for key $\bar{x}$ in a hash table and look up the stored value.

The key insight of this chapter is that we can run f in parallel simply by (a) using a shared parallel hash table and (b) randomizing the order in which we compute the subproblems. The resulting pseudo-code is shown on the right of Figure 4.1 with (the very few) changes shown in *italics*. Each of the parallel threads executes this version of f.

4.2.1 Lock-free concurrent hash tables

The hash table needed to implement a top-down dynamic programming approach only requires two operations: insert a (key,value) pair, and look up a value given a key. Once a (key,value) pair has been inserted, any further insertions of the same key must have the same value (since any computed optimal value for a subproblem is the optimal value, and does not change). Hence, updates and deletions are not required.

Furthermore, to implement efficiently the dynamic programming approach previously described, we require a concurrent hash table that scales well in the number of threads inserting into and reading from it. Thus, the use of critical sections is undesirable, given they require access controlled by locks (mutexes). Also, we would like our hash table to be not only *non-blocking*, that is, an arbitrary delay of one thread cannot halt progress of others (thereby excluding the use of mutual exclusion), but also meet the stronger condition of being *lock-free*. Although lock-freedom has had several historical definitions, we use the definition of Herlihy and Shavit (2008), that "[a] method is *lock-free* if it guarantees that infinitely often *some* method call finishes in a finite number of steps." (Herlihy and Shavit, 2008, p. 60). That is, there must be guaranteed system-wide progress.

Various implementations of hash tables are possible, including separate chaining (open hashing) and open addressing (closed hashing). It was not clear *a priori* which implementation would work better for the associative array we require for parallel dynamic programming. Thus, we experimented with several implementations on three different processors: AMD Opteron (Intel IA-64), IBM PowerPC, and UltraSPARC T1.

Previously defined lock-free hash tables

A number of alternative algorithms and implementations of lock-free hash tables have been described in the literature. Michael (2002) described a lock-free hash table using separate chaining, which is compatible with any lock-free memory management method. As described below, we have implemented a simplified version of this method (not supporting a delete operation). Purcell and Harris (2005) describe a lock-free open addressing hash table which, when compared with separate chaining, has better cache locality and has no pointer and memory allocator overhead. However, their algorithm cannot implement a dictionary, and it is complicated by the necessity of supporting a lock-free delete operation, which we do not require. Hence, we describe below a quite different (and simpler) algorithm for a lock-free open addressing hash table, which cannot (as given) support a delete operation, but allows keys to have values. Gao *et al.* (2005) describe a lock-free open addressing hash table which supports dynamic resizing, together with correctness proofs (supported by an automated theorem proving system). As our usage requires neither dynamic resizing nor the delete operation (also supported in this algorithm), this hash table is considered overly complex for us. Shalev and Shavit (2006) introduced split-ordered lists, a lock-free extensible hash table that implements extensibility by keeping all items in a lock-free linked list, which can be repeatedly split into different buckets atomically by moving "bucket pointers" rather than the items themselves. Herlihy *et al.* (2008) describe hopscotch hashing, an open addressed resizable concurrent hash table, and a concurrent version of cuckoo hashing (Pagh and Rodler, 2004) can be found in Herlihy and Shavit (2008). Again, these hash tables are overly complicated for us since we do not require a delete operation, and will use an atypical operation distribution, with many more inserts relative to lookups than is generally assumed to be the case in hash tables.

Our implementations

In order to implement lock-free data structures and algorithms efficiently, hardware support in the form of atomic instructions is required. We only consider here the *compare-and-swap* (or CAS) operation, which we will denote as CompareAndSwap, with the semantics detailed in Figure 4.2.

Support for the CompareAndSwap operation dates back to the IBM S/370 and it is still available on many modern processors including Intel IA-64 (x86) and Sun SPARC. Processors, like the IBM PowerPC, that do not support CompareAndSwap, often directly support Load-Linked and Store-Conditional (LL/SC) instead. Since CompareAndSwap can be implemented using LL/SC (Michael, 2004), it is sufficient to focus on CompareAndSwap. Use of CompareAndSwap can result in the *ABA problem*, whereby the value at an address is changed from a to b and then back to a after another thread, P has already read a from it. Then thread P can succeed in using CompareAndSwap on the same address with an expected value of a, as if the value at the address had never changed. This can result in incorrect semantics if, for example, the values are addresses, and the second use of a is due to reuse of a deleted node in a list. As we will show later, the ABA problem does not arise in our implementations.

As we require only insert and lookup, our implementations of the separate chaining and open addressing hash tables are quite simple.

Separate chaining: The lookup operation consists of simply hashing the key, and comparing the key with each element in the linked list anchored at the entry for that hash value. The insert operation requires the use of the CompareAndSwap operation to insert the newly allocated element at the head of the list anchored at the hash value for the new key; if the head of the list has changed during the insertion, we must retry it.

As mentioned before, this is essentially a simplified version of the lock-free hash table described by Michael (2002). As we do not implement the delete operation, the ABA problem does not arise. To allocate new cells, our implementation

```
CompareAndSwap(address, expectedval, newval)
  atomically:
    load value_at(address) into oldval
    if oldval = expectedval then
      store newval at address
    return oldval
```

FIGURE 4.2: *Semantics of the* CompareAndSwap *operation.*

The new value newval *is only stored at* address *if it contained the expected value* expectedval*. The value that was at* address *is returned, allowing the caller to detect failure of the operation, when* expectedval *is not returned.*

uses the Streamflow scalable locality-conscious multithreaded allocator (Schneider *et al.*, 2006). As Streamflow was not available for SPARC, on this platform we used a simple cellpool allocator, in which we allocated a large amount of memory at initialization time, and allocated cells in a lock-free manner by using the CompareAndSwap operation to advance the "next cell" pointer for each request.

Since the CompareAndSwap operation is used only on pointers (rather than the actual key), there is no limit on key size imposed by the maximum size of the operand for this instruction. Functions to compare keys and to copy keys and values can be provided as callback functions, allowing any data type to be used for keys and values.

If the allocator is lock-free then this hash table is lock-free. The only operation that could cause it not to be is the insert operation, which must be retried only if the CompareAndSwap operation fails (i.e., when another entry with the same hash value for the key has been inserted during the operation). But since the CompareAndSwap operation can only fail because it has succeeded in another thread, there is global progress and therefore the algorithm is lock-free.

Open addressing: We made the further simplifying assumption that the hash table does not need to be resized and simply allocate a very large (2^{26}-entry) hash table at initialization. Hence, this implementation requires no thread-safe memory allocator and memory allocation is not required at all during its use, both simplifying its implementation and removing a possible impediment to scalability in the number of threads. Each entry in the hash table is a 64-bit key and a 64-bit value (thus, a key is always immediately followed in memory by its value). We use linear probing to resolve hash collisions. As with separate chaining, the lookup operation requires no special instructions, but the insert operation uses the CompareAndSwap operation to ensure the slot for the key it is about to insert did not have another key inserted during its operation. The algorithms for the par_insert and par_lookup operations in the open addressing hash table are shown in Figure 4.3. The essential invariant in this concurrent data structure is that once an entry has changed from the initial empty state (marked with the NO_KEY constant), to containing a key, the key field never changes: the entry is forever occupied by that key. The CompareAndSwap operation ensures that if another thread has taken the entry during the par_insert, then the current attempt to use it is abandoned, and we restart the

```
par_insert(key,value)
    ent ← get_entry(key)
    if ent = NULL then
        error_exit("hash table full")
    if ent.key = NO_KEY then
        if CompareAndSwap(ent.key, NO_KEY, key)
                ≠ NO_KEY then
            return par_insert(key, value)
    ent.value ← value
    return TRUE

par_lookup(key)
    ent ← get_entry(key)
    if ent ≠ NULL ∧ ent.value ≠ NO_VALUE then
        return ent.value
    else
        return KEY_NOT_FOUND

get_entry(key)
    h ← hash(key)
    ent ← hashtable[h]
    probes ← 0
    while probes < TABLE_SIZE − 1 ∧ ent.key ≠ key
            ∧ ent.key ≠ NO_KEY do
        probes ← probes + 1
        h ← (h + 1) mod TABLE_SIZE
        ent ← hashtable[h]
    if probes ≥ TABLE_SIZE − 1 then
        return NULL
    else
        return ent
```

FIGURE 4.3: *Algorithms for the* par_insert *and* par_lookup *operations on the concurrent lock-free open addressing hash table.*

insertion operation. Because the key value can only change from NO_KEY to some other value, and never back again, the ABA problem cannot occur with this use of CompareAndSwap. Note also that (as is typically the case in hash tables) there can be no duplicate keys in the table: once an entry has been taken by a key, not only will it never again be empty, but any insertion of the same key will find that same entry.

Note that the key and the value fields are operated on separately. This allows a thread to look up a key while another thread is in the process of inserting that key and, hence, read the value field before it has been set by the par_insert function. This simply results in the par_lookup operation finding the NO_VALUE constant if the key did not previously exist in the table, since all entries are initialized to have NO_KEY and NO_VALUE in the key and value fields respectively. This is

interpreted as the key not being found in the table, just as if the par_lookup operation had been serialized to occur prior to the par_insert operation that is in progress.

This algorithm is clearly non-blocking: an arbitrary delay of any thread will not halt the progress of any other threads. We claim that this open addressing hash table is also lock-free. To see that this is the case, we need only consider the par_insert operation, since par_lookup can clearly not be delayed by any other thread. Consider the case that two threads are simultaneously trying to insert the same key in an empty entry. Only one of them will execute CompareAndSwap successfully, as the other will find that the entry is no longer empty and fail. When this thread retries (the recursive call), it will find the entry no longer empty and complete. In general, we need to show that it is not possible for two (or more) threads to continually delay each other and cause livelock. To see that this cannot occur, observe that the CompareAndSwap operation only fails in one thread if it succeeds in another (because the contentious entry has been changed from NO_KEY to some key value). Hence, at least one thread has completed its operation (since if the CompareAndSwap succeeds, that par_insert operation will succeed), and the algorithm is lock-free.

Various optimizations are possible when implementing this algorithm. For example, we do not need to recursively call par_insert on failure of the CompareAndSwap operation, but could instead use a loop and not recompute the hash value for the key. In the interests of simplicity and clarity we do not show here or implement such optimizations, but use a straightforward implementation of the algorithm as shown.

Evaluation of the time performance of hash tables

We built and tested our two hash tables (separate chaining and open addressing) on three different processors: AMD Opteron (Intel IA-64) and IBM PowerPC processors running Linux, and on the UltraSPARC T1 processor running Solaris. These hash table implementations used the `gcc` (version 4.1.2) compiler builtins for atomic operations, except on Solaris where we used the Sun Studio 12 C compiler and the Solaris (version 5.10) atomic operation library functions.

For the AMD Opteron processor, we also compared our hash tables to a publicly available C implementation of a Java non-blocking data structure library called

nbds,[1] and the concurrent hash map with scalable allocator from the Intel Threading Building Blocks 2.1 (TBB) for Open Source.[2] The latter is unlike all other implementations in that it is not lock-free, and is in C++ rather than C. Note that TBB and nbds only work on Intel compatible processors.

In order to evaluate the time performance of each hash table, we used a test program that, for 10^7 uniformly distributed random 64-bit integer keys, first performs a lookup operation, then, if the key is not already in the table, performs a lookup on a new random key (also a 64-bit integer chosen uniformly at random), then inserts the original key, thereby generating on average twice as many lookup as insert operations. If the key is already in the table it simply asserts that the value is the correct one for the key (the values are inserted as the same as the key to make this correctness test straightforward). We use this ratio of inserts and lookups in order to model the dynamic programming usage we have observed in practice of twice as many lookups as inserts. Note that this ratio is quite different from those generally cited as typical for hash tables, such as 90% lookup, 9% insert, 1% delete (Herlihy and Shavit, 2008, p. 300). Thus, we require a hash table that is efficient for much more frequent insertions than typical usages (and which does not need to support a delete operation at all).

On the UltraSPARC T1 platform, Figure 4.4 shows that the open addressing hash table scales quite well in the number of threads, up to a peak speedup of 32 times for 30 threads (note this is relative to the baseline sequential implementation of separate chaining, and even with 1 thread is faster). The separate chaining hash table, however, scales very poorly, with a peak speedup of less than 4 times, and no significant increase after 8 threads. This result was achieved with our own trivial cell pool allocator, as described in Section 4.2.1, as the Streamflow allocator (Schneider *et al.*, 2006) was not available for SPARC. We also experimented with the standard `malloc()` and the `umem_cache_*()` object cache allocator (Bonwick, 1994, 2001) but found that the trivial cell pool allocator was faster for this purpose (data not shown).

On the PowerPC platform, Figure 4.5 shows that the separate chaining and open addressing hash tables achieve similarly good (linear) speedup. Note that the speedup actually appears to be superlinear; this is because the baseline uses

[1] `http://code.google.com/p/nbds/`
[2] `http://www.threadingbuildingblocks.org/`

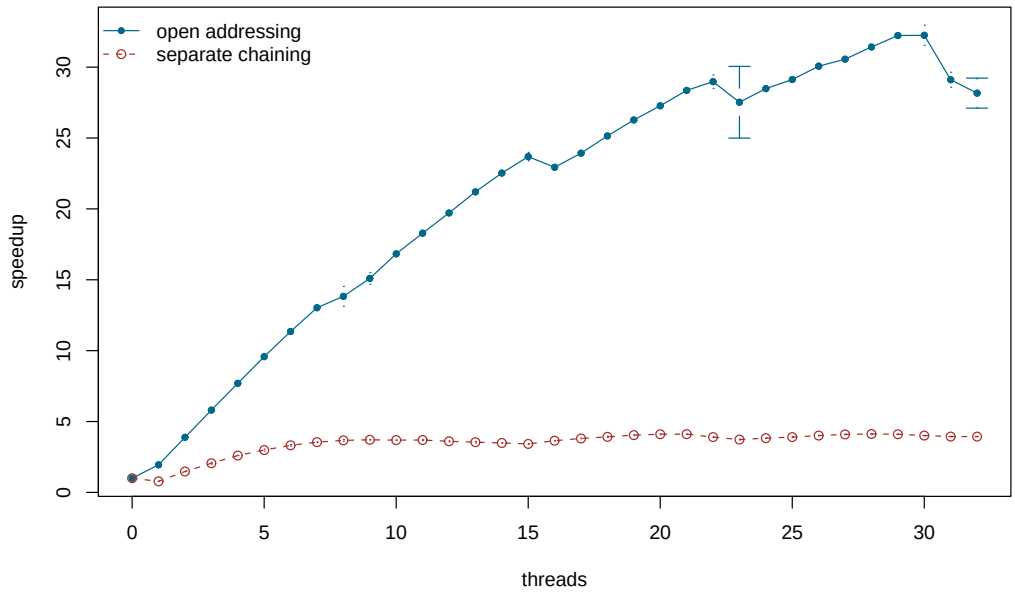


FIGURE 4.4: *Speedup for lock-free open addressing and separate chaining hash tables on UltraSPARC T1 (SunFire T2000, 1 GHz, 8 cores, 32 total concurrent threads, 16 GB RAM),* 2/3 *lookup and* 1/3 *insert.*

The baseline (0 threads value) is the separate chaining hash table implemented with no atomic instructions and compiled and linked with no threading support, and the standard library `malloc()`*. Each test was repeated 10 times and the average value used to compute the speedup.*

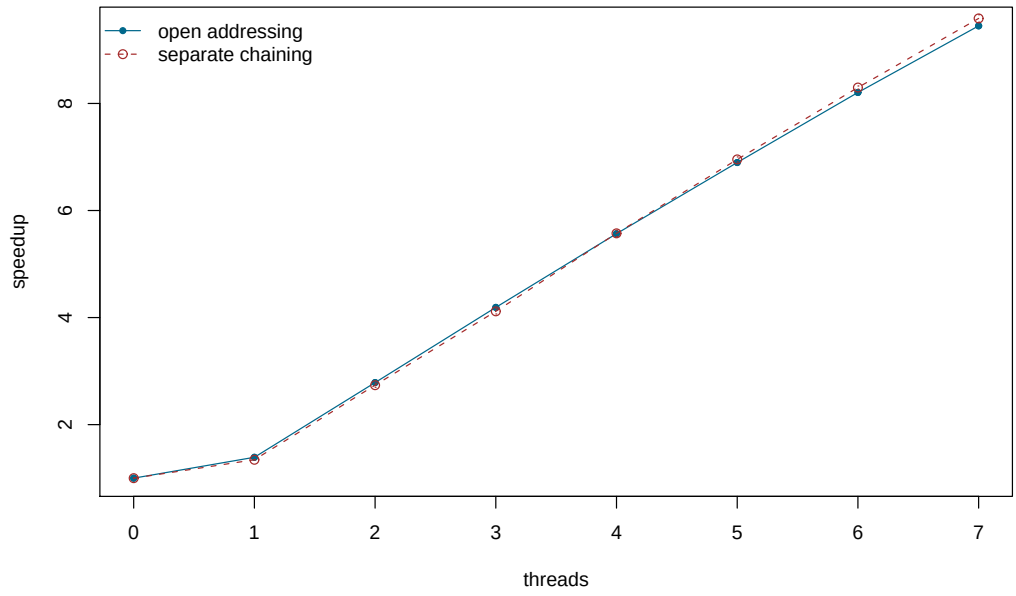


FIGURE 4.5: *Speedup for lock-free open addressing and separate chaining hash tables on PowerPC 64-bit (IBM eServer pSeries 650, 1.45 GHz, 8 cores, 16 GB RAM), 2/3 lookup and 1/3 insert.*

The baseline (0 threads value) is the separate chaining hash table implemented with no atomic instructions and compiled and linked with no threading support, and the standard library `malloc()`*. Each test was repeated 10 times and the average value used to compute the speedup.*

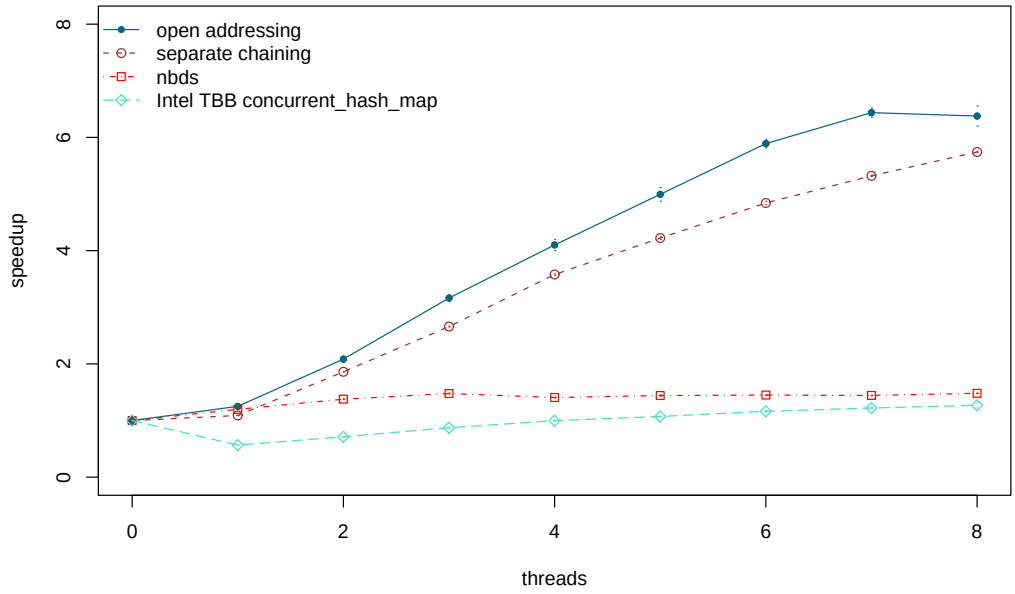


FIGURE 4.6: *Speedup of different concurrent hash table implementations on AMD Quad Core Opteron (2.3GHz, 2 processors, total 8 cores, 32 GB RAM),* 2/3 *lookup and* 1/3 *insert.*

The baseline (0 threads value) is a simple separate chaining hash table implemented with no atomic instructions and compiled and linked with no threading support, and the standard library `malloc()`*. Each test was repeated 10 times and the average value used to compute the speedup.*

the standard library `malloc()` for the memory allocator, while the lock-free implementation uses the Streamflow (Schneider *et al.*, 2006) allocator, which gives better performance for even a single thread on this platform. Hence, even the multithreaded implementation with only one thread is faster than the baseline.

On the AMD Opteron platform, Figure 4.6 shows the speedup of several concurrent hash table implementations. Speedup for the existing publicly available implementations is disappointing, with both the Intel TBB library (which uses locks) and `nbds` achieving no significant speedup with our high-insertion workload. Both the separate chaining and open addressing hash tables achieve quite good speedup on this platform, the former having a peak speedup of 5.7 times and the latter 6.4 times.

The results detailed in this section lead us to choose the lock-free open addressing hash table for implementing the parallel dynamic programming algorithm, since the same, quite simple, code achieves good scalable speedup on the three systems

problem	speedup	threads
knapsack	8.97	31
shortest paths	9.88	32
RNA struct. alignment	9.17	31
open stacks (tiebreak)	6.96	27
open stacks (full)	10.83	32

Table 4.1: *Maximum speedup for parallelized dynamic programs using randomization and the lock-free open addressing hash table on UltraSPARC T1.*

tested, and does not require the added complication of a scalable multithreaded allocator. The separate chaining hash table performs slightly better than the open addressing hash table on the PowerPC, but significantly worse on the other two platforms, although if Streamflow were available on the SPARC platform it may well change the results on that platform significantly. We note that the UltraSPARC T1 is particularly suitable for running scalable multithreading applications as it was designed specifically for scalable multithreaded performance rather than for optimal performance for single threads (Sheahan, 2007).

4.3 Results and discussion

We now show the results of using our method to parallelize the dynamic programming solutions to four very different problems. These dynamic programs were implemented in C with the same compilers described in Section 4.2.1. We note, however, that in testing the hash table implementation, as described in Section 4.2.1, we used uniformly randomly distributed keys. It is well known (see, e.g., Askitis (2009) and citations therein) that uniform distribution of hash values is important in the performance of hash tables, in order to prevent too-frequent collisions. For the tests described in Section 4.2.1, the hash function was unimportant, since the keys were generated from a uniform random distribution. However, in implementing dynamic programs, this is certainly not the case. To obtain a reasonably uniform distribution of hash values, we have used a hash function that combines bit shifts and multiplication.[3]

[3] `http://www.concentric.net/~Ttwang/tech/inthash.htm`

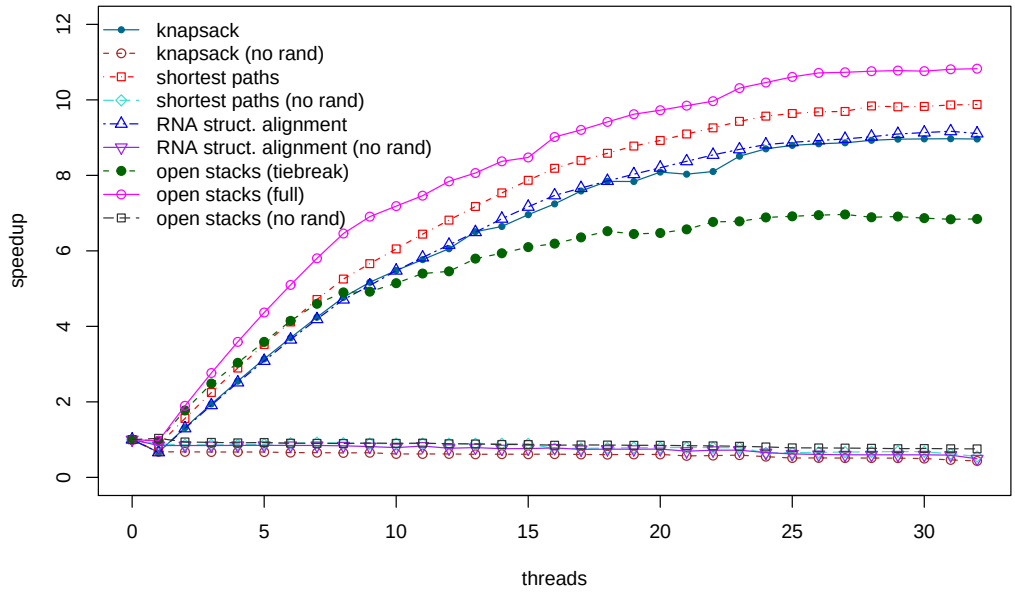


FIGURE 4.7: *Speedup for parallelized dynamic programs using the lock-free open addressing hash table on UltraSPARC T1.*

The baseline (0 threads value) for open stacks is the implementation described in Garcia de la Banda and Stuckey (2007), and for the other problems are the same implementations as the parallelized versions, but without randomization of subproblem orderings and using a simple (not lock-free) hash table implementation. All baseline implementations use no atomic instructions and are compiled and linked with no threading support, and the standard library `malloc()`*. Implementations labelled with "(no rand)" refer to implementations where no randomization is used, so all threads solve the same subproblems in the same order.*

problem	speedup	threads
knapsack	3.11	8
shortest paths	7.68	8
RNA struct. alignment	5.64	8
open stacks (tiebreak)	3.81	6
open stacks (full)	5.57	8

TABLE 4.2: *Maximum speedup for parallelized dynamic programs using randomization and the lock-free open addressing hash table on IBM PowerPC (8 cores).*

problem	speedup	threads
knapsack	3.21	8
shortest paths	4.81	8
RNA struct. alignment	4.40	8
open stacks (tiebreak)	3.95	8
open stacks (full)	4.34	8

TABLE 4.3: *Maximum speedup for parallelized dynamic programs using randomization and the lock-free open addressing hash table on AMD Quad Core Opteron (2.3GHz, 2 processors, total 8 cores, 32 GB RAM).*

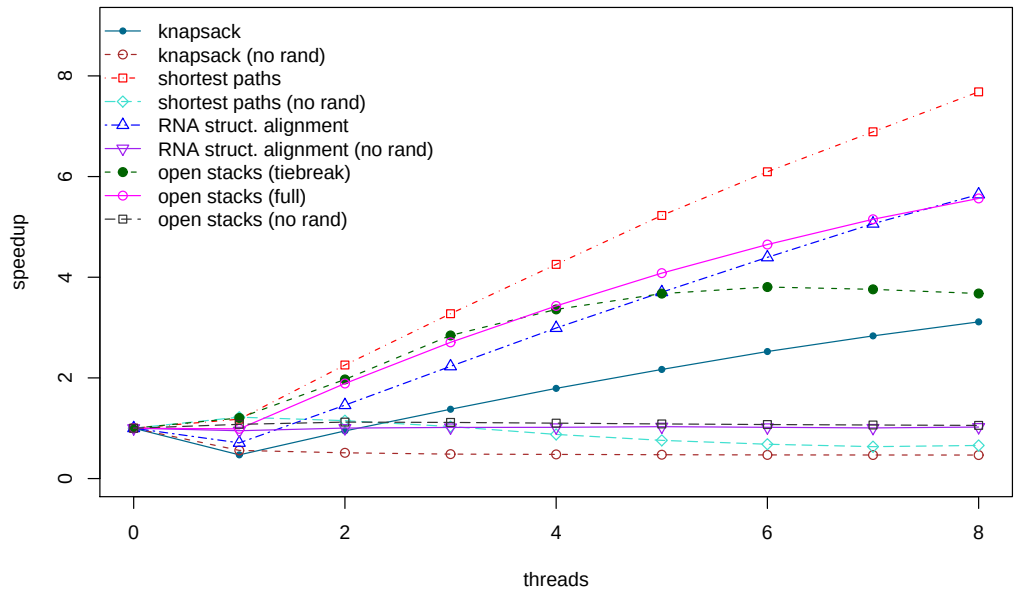


FIGURE 4.8: *Speedup for parallelized dynamic programs using the lock-free open addressing hash table on IBM PowerPC (8 cores).*

The baseline (0 threads value) for open stacks is the implementation described in Garcia de la Banda and Stuckey (2007), and for the other problems are the same implementations as the parallelized versions, but without randomization of subproblem orderings and using a simple (not lock-free) hash table implementation. All baseline implementations use no atomic instructions and are compiled and linked with no threading support, and the standard library `malloc()`.

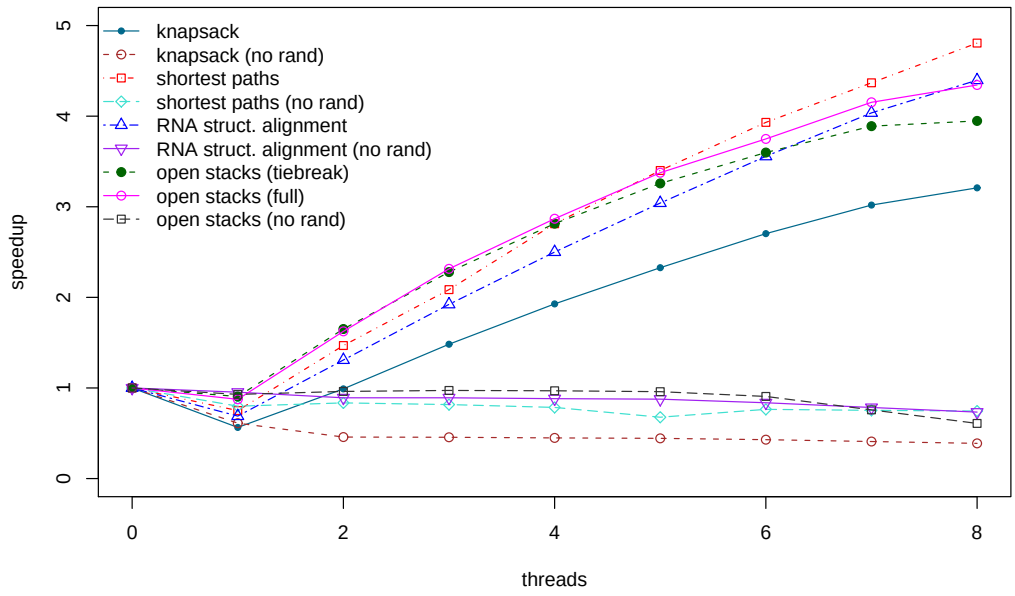


FIGURE 4.9: *Speedup for parallelized dynamic programs using the lock-free open addressing hash table on AMD Quad Core Opteron (2.3GHz, 2 processors, total 8 cores, 32 GB RAM).*

The baseline (0 threads value) for open stacks is the implementation described in Garcia de la Banda and Stuckey (2007), and for the other problems are the same implementations as the parallelized versions, but without randomization of subproblem orderings and using a simple (not lock-free) hash table implementation. All baseline implementations use no atomic instructions and are compiled and linked with no threading support, and the standard library `malloc()`.

4.3.1 Knapsack

We implemented in C the knapsack dynamic program formulation given in Section 4.2 using the top-down recursive approach. We then parallelized this program using randomization and ran it using our lock-free open addressing hash table described in Section 4.2.1. We used the test generator `gen2.c` used in Martello *et al.* (1999) and available from `http://www.diku.dk/~pisinger/gen2.c` to create 100 instances each of uncorrelated, weakly correlated, strongly correlated, inverse strongly correlated and almost strongly correlated knapsack problems, each with 500 items and weights in the interval $[1, 500]$.

On the UltraSPARC T1 platform, Figure 4.7 and Table 4.1 show that randomizing the subproblem ordering provides a speedup of 8.97 times (for 31 threads) for the knapsack dynamic program. Figure 4.10 sheds some light on why this does not achieve the speedup of up to 19 times achieved in Figure 4.4 for simple insertions and lookups: the total number of computations increases in approximately a straight line from 1.03×10^{10} for a single thread to 1.56×10^{10} for 32 threads. Thus, at 32 threads, 5.305×10^9 more subproblems are computed in total, a 51% increase in total computations.

This shows that we cannot achieve indefinite linear scalability with our technique, even if we had a perfectly linearly scalable lock-free concurrent hash table. As we increase the number of threads, we inevitably recompute some subproblems, when two or more threads compute the same subproblem simultaneously. Hence, although we have more threads exploring the solution space, we also have in aggregate more computations than we would have with fewer threads.

On the PowerPC platform, Figure 4.8 and Table 4.2 show that a speedup of 3.1 times is achieved for 8 threads, and for the AMD Opteron platform, Figure 4.9 and Table 4.3 show a speedup of 3.2 times for 8 threads (as opposed to 4.2 times for 8 threads in the UltraSPARC T1). We note that the speedup appears to be nearly perfectly linear in Figure 4.8, since the test system only has 8 cores. However, as seen for the UltraSPARC T1 platform (Figure 4.7), and to a lesser extent (for the last data point only, for 8 threads) in Figure 4.9, it is likely that the speedup will level out; its rate of increase will decline after 8 threads.

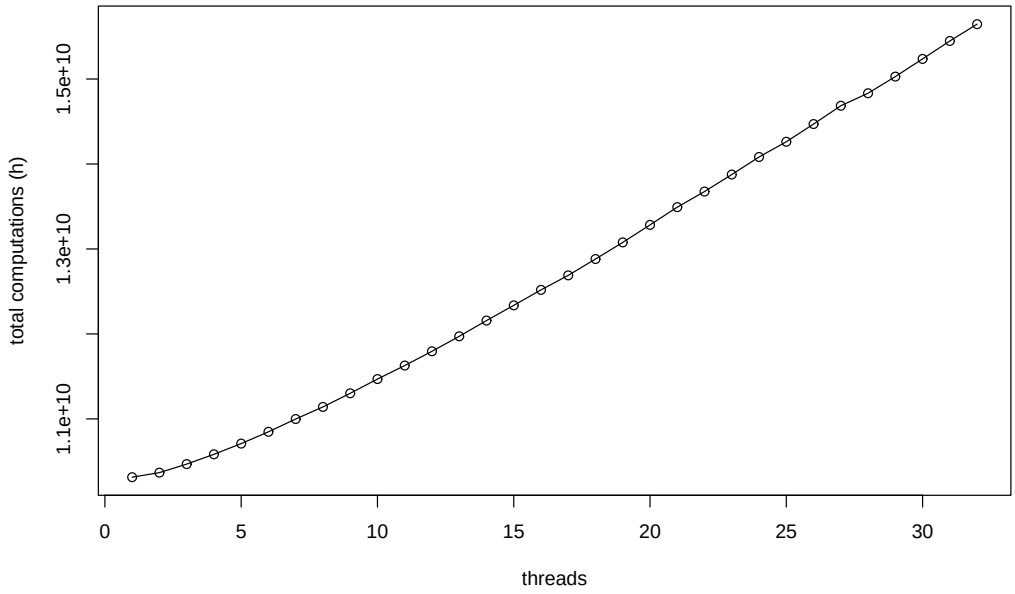


FIGURE 4.10: *Total computations (number of times the* par_insert *function is called, h) for the knapsack dynamic program using the lock-free open addressing hash table on UltraSPARC T1.*

4.3.2 Shortest paths

The shortest path between two nodes i and j in a directed graph can be found with the dynamic programming Floyd-Warshall algorithm (Floyd, 1962; Warshall, 1962). This algorithm is based on the function $s(i, j, k)$, which returns the length of the shortest path from node i to node j using only $1, \ldots, k$ as intermediate nodes. This function is defined using dynamic programming as follows:

$$s(i,j,k) = \begin{cases} 0, & \text{if } i = j \\ d_{ij}, & \text{if } k = 0 \\ \min\{s(i,j,k-1), s(i,k,k-1) + s(k,j,k-1)\} & \text{otherwise} \end{cases}$$

where d_{ij} is the weight (distance) of the directed edge from i to j. Once k reaches the total number of nodes in the graph, say n, the value returned by $s(i, j, n)$ represents the optimal solution to the initial problem.

As before, we implemented this dynamic program in C using the recursive top-down approach, parallelized it using randomization, and ran it using our lock-free open addressing hash table. To randomize we randomly choose, with equal

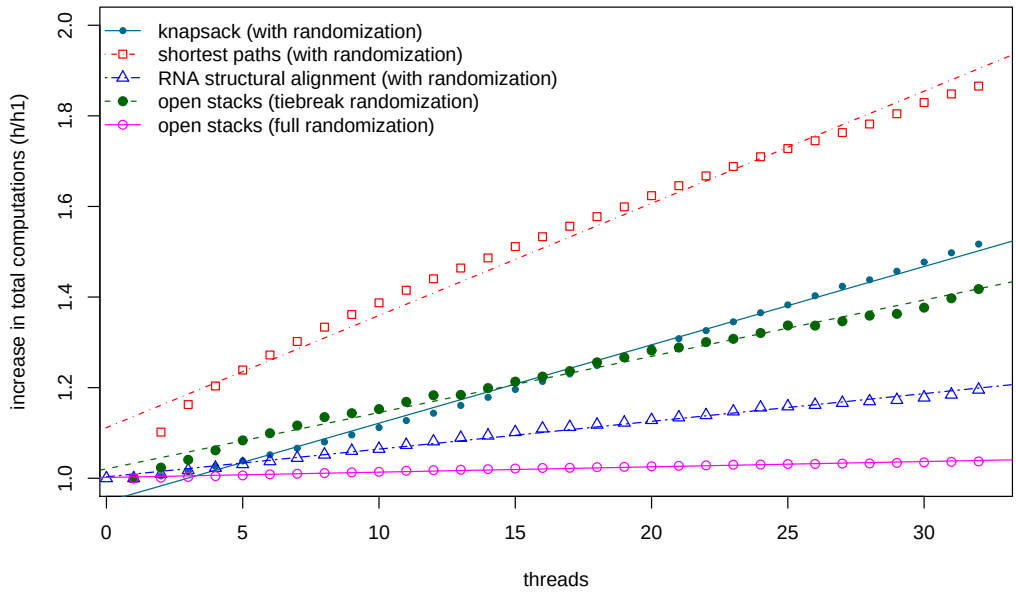


FIGURE 4.11: *Increase in total computations.*

Increase in total computations (number of times the par_insert *function is called, h) relative to the number for a single thread (i.e., h/h_1) for the dynamic programs using the lock-free open addressing hash table on UltraSPARC T1. Straight lines were fitted to the data points by linear least squares regression with the* `lm` *function in R.*

probability, one of the 3! = 6 possible orderings of the three subproblems in the recurrence relation just described. We derived the test data for this example from the 9th DIMACS Implementation Challenge — Shortest Paths data from `http://www.dis.uniroma1.it/~challenge9/`. We chose three 500-node slices of the New York City dataset, with 100 queries per instance.

Figures 4.7, 4.8, and 4.9 and Tables 4.1, 4.2, and 4.3 show that the resulting program achieves a speedup of 9.88 times with 32 threads on the UltraSPARC T1 platform, 7.68 times with 8 threads on the IBM PowerPC platform, and 4.81 times with 8 threads on the AMD Opteron platform.

4.3.3 RNA base pairing probability matrix alignment

The dynamic program for aligning two RNA base pairing probability matrices is described in Section 3.1.4 of Chapter 3, and specifically by Equations 3.5 and 3.6.

As before, we implemented this dynamic program in C using the recursive top-down approach, parallelized it using randomization, and ran it using our lock-free open addressing hash table. To randomize we ordered the subproblem computations according to a random permutation of the entire list of subproblems in the $S(\cdot)$ and $S^M(\cdot)$ computation.

We used the probability threshold as described by Will *et al.* (2007) to reduce the computational complexity of the problem, and, as does `LocARNA` (Will *et al.*, 2007), normalized the base pairing log probabilities and converted them to integers in order to avoid floating point computations. This is particularly important on the UltraSPARC T1 processor, since it has a single floating point unit per chip, shared by all cores (Sheahan, 2007), which limits scalability of programs using floating point operations.

We run the parallelized program with the BRAliBase II (Gardner *et al.*, 2005) pairwise RNA structural alignment benchmark data set, which consists of 118 pairs of RNA sequences (from a total of 78 individual sequences) to be aligned. We used the `RNAfold` program in the Vienna RNA Package (Zuker and Stiegler, 1981; Hofacker *et al.*, 1994) to generate the base pairing probability matrix for each sequence, which is then processed by a script into a form used as input to our program.

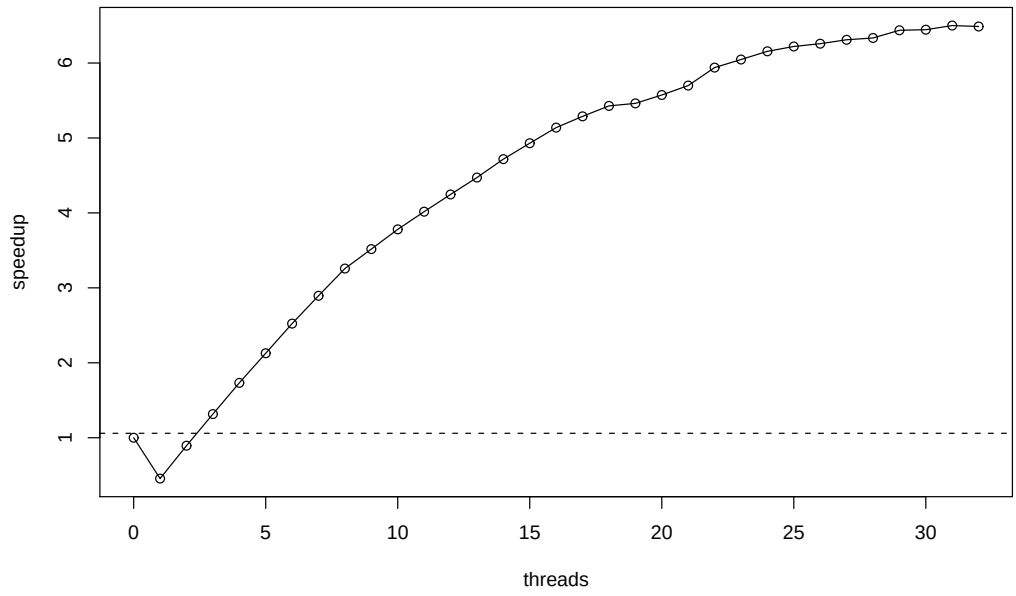


FIGURE 4.12: *Speedup for parallelized RNA base pairing matrix alignment using an array to store computed values on the UltraSPARC T1 (32 cores).*

The baseline (0 threads value) is a top-down implementation of the dynamic program using an array to store computed values, with no randomization, and compiled and linked with no threading support. The dashed horizontal line represents the speedup relative to the same baseline of a bottom-up implementation of the dynamic program, showing that our parallelized implementation is faster than the bottom-up implementation on this platform when 3 or more threads are used.

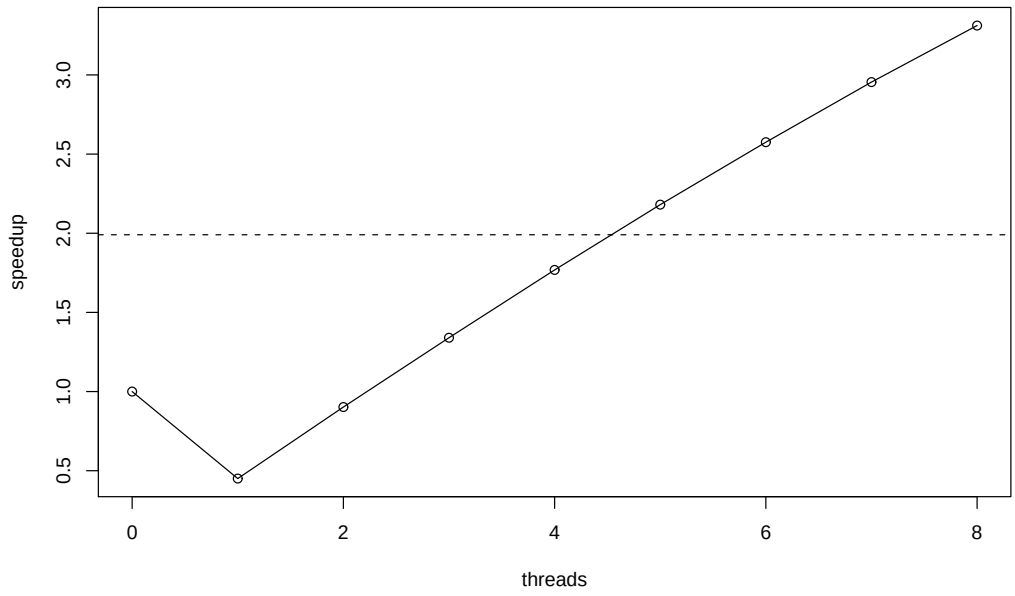


FIGURE 4.13: *Speedup for parallelized RNA base pairing matrix alignment using an array to store computed values on the IBM PowerPC (8 cores).*

The baseline (0 threads value) is a top-down implementation of the dynamic program using an array to store computed values, with no randomization, and compiled and linked with no threading support. The dashed horizontal line represents the speedup relative to the same baseline of a bottom-up implementation of the dynamic program, showing that our parallelized implementation is faster than the bottom-up implementation on this platform when 5 or more threads are used.

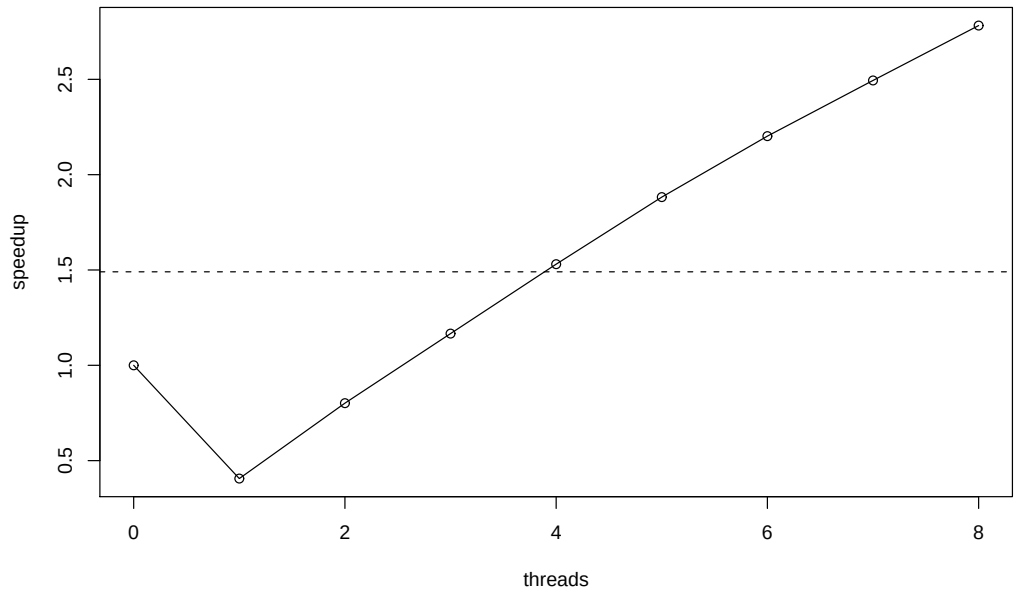


FIGURE 4.14: *Speedup for parallelized RNA base pairing matrix alignment using an array to store computed values on the AMD Opteron (8 cores).*

The baseline (0 threads value) is a top-down implementation of the dynamic program using an array to store computed values, with no randomization, and compiled and linked with no threading support. The dashed horizontal line represents the speedup relative to the same baseline of a bottom-up implementation of the dynamic program, showing that our parallelized implementation is faster than the bottom-up implementation on this platform when 4 or more threads are used.

Figures 4.7, 4.8, and 4.9 and Tables 4.1, 4.2, and 4.3 show that the top-down implementation with the concurrent lock-free open addressing hash table achieves a speedup of 9.17 times with 31 threads on the UltraSPARC T1 platform, 5.64 times with 8 threads on the IBM PowerPC platform, and 4.40 times with 8 threads on the AMD Opteron platform for this implementation. In these figures, the decline in performance from the "0 threads" value (the baseline, with speedup 1.0 by definition) to the single-thread value, is due to the performance penalty incurred by the overhead of thread support and randomization. On all the platforms tested, it requires at least three threads to achieve better performance than the non-threaded baseline.

We note that this problem is different from the ones previously shown, in that it is perfectly feasible to compute and store every possible subproblem (i.e., use the bottom-up technique) for realistic problem instances. Hence, a hash table is not necessary. Furthermore, a careful arrangement of the computation order and efficient implementation of the bottom-up technique, such as `LocARNA` (Will *et al.*, 2007), is considerably faster than the top-down technique (certainly for a single thread). Therefore, we also implemented a bottom-up version of the algorithm, as well as a top-down version that uses an array rather than the concurrent hash table to store results. Figure 4.12 shows that the top-down parallelization using an array rather than a hash table achieves a maximum speedup of 6.50 times on the UltraSPARC T1 processor and is faster than the bottom-up implementation for 3 or more threads. Figures 4.13 and 4.14 show that the corresponding results on the PowerPC and AMD Opteron processors respectively are a maximum speedup of 3.31 times and 2.78 times, and that on the IBM PowerPC and AMD Opteron, 5 and 4 or more threads,respectively, are required in order to be faster than the bottom-up implementation.

This example demonstrates that our technique can be used to accelerate any top-down implementation of a dynamic program (whether storing the results in a hash table or an array), without having to find an efficient ordering of the (bottom-up) computation.

4.3.4 Minimization of Open Stacks

The Minimization of Open Stacks Problem (Yuen, 1991; Yuen and Richardson, 1995) is a scheduling problem in manufacturing. A factory manufactures a number of different products in batches, each of which needs to be finished before a different one is manufactured. Customers place orders for one or more different products, and once a product in one customer's order starts being manufactured, a "stack" is opened for that customer to store all products in the order. Once the customer's entire order is finished, the stack can be closed and used for another customer. The aim is to find a sequence in which to manufacture products so that the maximum number of open stacks (i.e. maximum number of customers whose orders are simultaneously active) is minimized. Since there is a one-to-one relationship between stacks and customers, this is equivalent to minimizing the maximum number of *active customers*.

A highly effective dynamic programming solution to this problem was recently described by Garcia de la Banda and Stuckey (2007). Briefly, the problem is formalized as a set P of products, a set C of customers, and a function $c(p)$ which returns the set of customers who have ordered product $p \in P$. The products ordered by each customer are placed in a different stack from that of any other customer, so c denotes both a customer and the customer's associated stack. A customer c is active (or the stack c is open) at time k if there is a product manufactured at or before time k and also there is a product manufactured at time k or afterwards. Then the aim is to find a schedule for manufacturing products, that is, a permutation of the products in P, which minimizes the number of open stacks at any time.

The key insight of Garcia de la Banda and Stuckey (2007) is that the order in which previous products have been manufactured is not important, and therefore they can be considered as a set. This reduces the search space from $|P|!$ to $2^{|P|}$ (since only the minimum stacks for each subset, rather than permutation, of P needs to be computed) and opens up the possibility of using dynamic programming.

To use this in expressing the problem as a dynamic program, the function $c(p)$ is extended to handle a set of products $S \subseteq P$. That is, $c(S) = \cup_{p \in S} c(p)$ is a function which returns the set of customers ordering products from the set $S \subseteq P$. If $A \subset P$ is the set of all products scheduled to be manufactured after product p, then the set

of active customers at the time p is manufactured is

$$a(p, A) = c(p) \cup (c(A) \cap c(P - A - \{p\}))$$

that is, those who ordered p, plus those whose orders include some products scheduled after p and some scheduled before p. It is the fact that $a(p, A)$ does not depend on the order of products in A or $P - A - \{p\}$ that makes this problem amenable to dynamic programming. Let $S \subseteq P$ denote the set of products that still need to be manufactured and let $stacks_P(S)$ be the minimum number of stacks required to schedule the products in S. Then the dynamic programming formulation of $stacks_P(S)$ is:

$$stacks_P(S) = \min_{p \in S} \max\{a(p, S - \{p\}), stacks_P(S - \{p\})\}$$

which computes, for each product p, the maximum number of open stacks needed if p was scheduled first, and then takes the minimum of this over all products.

In the dynamic programming formulation of the open stacks problem described by Garcia de la Banda and Stuckey (2007), the order in which the products are tried significantly affects the amount of work performed, and a heuristic is used which selects the product with the least number of active customers (its pseudo code is given in Figure 4.15). We demonstrate two different approaches to randomization. Our first approach, stacks_tiebreak, retains the heuristic and only introduces randomization for breaking ties. This is achieved by changing the third line of the **while** loop in Figure 4.15 to

$p \leftarrow$ *RandomElementOf(Q)*

Our second approach, stacks_full, discards the heuristic and randomly selects any product that has the same or fewer number of active customers than the currently known minimum. In addition to the change above, the 2nd line in the **while** loop is replaced by

$$Q \leftarrow \{p \mid p \in T, a(p, S - \{p\}) \leq min\}$$

We used the same C code described in Garcia de la Banda and Stuckey (2007), adding the randomization techniques just described, and replacing the simple separate chaining hash table of Garcia de la Banda and Stuckey (2007) with our lock-free

```
stacks(S, L, U)
    if S = ∅ then
        return 0
    min ← lookup(S)
    if min ≠ KEY_NOT_FOUND then
        return min
    min ← U + 1
    T ← S
    if ∃p ∈ S.c(p) ⊆ c(P − S) ∩ c(S) then
        return stacks(S − {p}, L, U)
    while min > L ∧ T ≠ ∅ do
        min_now ← min{a(p, S − {p}) | p ∈ T}
        Q ← {p | p ∈ T, a(p, S − {p}) =min_now}
        p ← FirstElementOf(Q)
        T ← T − {p}
        if a(p, S − {p}) ≥ min then
            break
        sp ← max{a(p, S − {p}), stacks(S − {p}, L, U)}
        if sp < min then
            min ← sp
    insert(S,min)
    return min
```

FIGURE 4.15: *The A* dynamic programming algorithm for the open stacks problem (Garcia de la Banda and Stuckey, 2007).*

stacks(S, L, U) *returns the minimal number of open stacks required for scheduling the set of products* S *given a lower bound on the number of stacks* L *and an upper bound* U*. A heuristic choice of the order in which to try products is made by selecting a product* p *which results in the least number of active customers if scheduled immediately.*

open addressing hash table described in Section 4.2.1. We use the same problem instances from the Constraint Modelling Challenge 2005,[4] omitting the two instances that exceeded the search limit in Garcia de la Banda and Stuckey (2007), as well as a single instance that requires a set of more than 64 products after preprocessing (since their implementation represents the product set as a bit set, and is currently restricted to a single (64 bit) machine word as the hash table key).

Figures 4.7, 4.8, and 4.9 and Tables 4.1, 4.2, and 4.3 show that the tiebreak randomization achieves a speedup of up to 6.96 times (for 27 threads) on the UltraSPARC T1 platform, 3.81 times (for 6 threads) on the PowerPC platform, and 3.95 (for 8 threads) on the AMD Opteron platform. For the full randomization (stacks_full) the corresponding speedups are 10.83, 5.57, and 4.34 times for 32, 8 and 8 threads, respectively.

Note that, for comparison, not randomizing the choice so that each thread performs the same computation results in no speedup and, in fact, the addition of more threads causes a slight slowdown.

Just as for the knapsack problem, we can see from Figure 4.11 that increasing the number of threads also linearly increases the total number of subproblems computed. At the maximum value of 32 threads, this is a 51% increase for knapsack and a 43% increase for open stacks with the tiebreak randomization, but only a 3.7% increase for open stacks with the full randomization.

The difference in speedup between stacks_tiebreak and stacks_full illustrates the importance of ensuring that the path threads take through the set of subproblems diverges as much as possible. In the extreme case of no randomization (so that each thread computes exactly the same subproblems in exactly the same order), we clearly obtain no speedup. In general, the more divergent that the paths that the threads take through the set of possible subproblems becomes, the better the speedup will be. This is because more divergent paths cause a slower growth (lesser slope of the h/h_1 lines as plotted in Figure 4.11) in the number of duplicated computations (when two or more threads compute the same subproblem). In the case of the open stacks problem, this reduction in duplicated computation more than makes up for the loss of the heuristic which selects the product with the least number of open stacks if selected first.

[4] http://www.dcs.st-and.ac.uk/~ipg/challenge

4.4 Conclusions

We have described a technique for parallelizing dynamic programs on shared memory multiprocessors, and demonstrated its application to dynamic programming formulations of the well-known knapsack and shortest paths problems, as well as the bioinformatics problem of RNA structural alignment, and the problem of minimizing the maximum number of open stacks. Our technique is applicable to any dynamic program, since it operates on the top-down (i.e., recursive) implementation of the dynamic program, which is a direct implementation of the recurrence relation (Bellman equation) expression of the problem. This is in contrast to previous work on parallelizing dynamic programs, which focuses on vectorizing the operations in filling in the dynamic programming matrix in the bottom-up technique.

Much greater speedups (orders of magnitude) can be achieved for specific dynamic programming problems by careful analysis of the problem structure and properties of optimal solutions, in order to apply, for example, bounding techniques. Although the parallelization technique we have described results in much more modest speedups, it can be applied immediately to any dynamic program, without the need for further analysis.

For dynamic programs that are too large to implement in the bottom-up manner (filling in every entry of a dynamic programming matrix), such as open stacks, vectorization approaches are inapplicable and a method, such as the one presented here, that is applicable to the top-down implementation is required. We have shown a speedup greater than 10 times (for 32 threads) by applying this method to a dynamic programming algorithm for the minimization of maximum number of open stacks problem, although we note that much greater improvements have since been achieved for this problem by using a branch and bound strategy (Chu and Stuckey, 2009).

For problems that can be practically implemented with the bottom-up technique, such as sequence alignment and RNA structure prediction and alignment, vectorization techniques have been successful. However, these techniques require careful analysis of the data dependencies in the particular problem being parallelized and result in increased complexity of the implementation. Our method, in contrast, can be applied directly to a simple implementation of the recurrence relation defining

the problem as a recursive function, without any analysis of the particular problem. In these cases an array can be used to store results, without even the need for a lock-free hash table.

In order to simplify our algorithm and implementation we made the assumption that dynamic resizing of the hash table is unnecessary, and simply allocated a very large hash table at initialization. This is clearly wasteful for dynamic programming problem instances that do not require a large number of entries. A more sophisticated implementation would use a dynamically resizable lock-free hash table, such as that provided by split-ordered lists (Shalev and Shavit, 2006). Further experiments would have to be carried out to determine if the increase in complexity and use of indirection (resulting in reduced cache efficiency) is a winning trade-off for the advantage of not allocating unnecessarily large amounts of memory.

5

Protein Substructure Searching

Having discussed RNA structural comparisons in previous chapters, we now move on to protein structure, and in this chapter we describe a technique for automated structural comparison of proteins.

This technique is based on the tableau representation of protein structure, described in Section 5.2.1. Tableaux are compared efficiently by solving the quadratic program (QP) corresponding to the quadratic integer program (QIP) formulation for the extraction of maximally similar tableaux, and we find that additional constraints based on the separation of secondary structure elements increases the accuracy of the method. We compare the accuracy of this method with several other methods in classifying protein folds, and demonstrate its ability to find substructural and non-sequential similarities.

In Section 5.1 we review existing algorithms for protein structural search, and in Section 5.2 we describe our algorithm and its implementation, as well as the evaluation methods that we use. In Section 5.3 we compare our method with some others, and demonstrate some examples. We also discuss some improvements to the speed of the method by reducing the number of SSEs considered or using more efficient solvers; in Chapter 6, however, we will describe a different approach to finding a locally optimal solution of the same problem that is both much faster and can give more accurate results. Finally, in Section 5.4 we conclude by describing the significance of our results for protein structural search, and placing them in the context of some current and ongoing related work.

5.1 Introduction

Searching a database of protein structures for structures that are similar to, or contain substructures that are similar to, a query structure is a significant problem in structural biology and bioinformatics.

There are many existing methods for finding structurally similar proteins, such as: structural alignment at the level of residues or backbone atoms (for example, Dali and DaliLite (Holm and Sander, 1996; Holm and Park, 2000) and MUSTANG (Konagurthu *et al.*, 2006)) or (as an initial step) secondary structure elements (Madej *et al.*, 1995; Gibrat *et al.*, 1996; Alexandrov, 1996; Singh and Brutlag, 1997; Shapiro and Brutlag, 2004; Krissinel and Henrick, 2004), purely topological matching (Gilbert *et al.*, 1999; Torrance *et al.*, 2005), and probabilistic approaches (Carugo and Pongor, 2002; Gáspári *et al.*, 2005; Kirillova and Carugo, 2008).

SHEBA (Jung and Lee, 2000) and YAKUSA (Carpentier *et al.*, 2005) both use a one-dimensional representation of protein structure to accelerate structural searching. SHEBA uses "environmental profiles" containing information about sequence homology and residue-dependent information such as solvent accessibility, hydrogen bonds, and side-chain packing, which is then refined for three-dimensional geometry by dynamic programming. YAKUSA is a fast method that uses a one-dimensional representation based on protein internal angles.

Some recent methods use information about secondary structure element (*SSE*) orientation to match proteins on the basis of global geometric information. TableauSearch (Konagurthu *et al.*, 2008) and IR Tableau (Zhang *et al.*, 2010) make use of tableaux (Lesk, 1995) to perform rapid and accurate comparison of whole protein structures. ProSMoS (Shi *et al.*, 2007) also makes use of SSE orientations, but, as is described below, takes quite a different approach from most other methods.

Methods that use SSEs require some method for assigning secondary structure to proteins. That is, some method to classify residues in the protein as belonging to helices (of various kinds) or strands (as part of a β-sheet), or not part of an SSE. This can be done by a variety of methods, such as pattern recognition of hydrogen bonds and geometrical features, as in probably the best-known method, DSSP (Kabsch and Sander, 1983), or by hydrogen bonds and statistically derived backbone torsional angle information as in STRIDE (Frishman and Argos, 1995).

Assignment of secondary structure is not an exact procedure, with various methods disagreeing about the exact beginning and end of SSEs, and particular SSEs depart significantly from the "ideal" model of Pauling and Corey (Pauling *et al.*, 1951; Pauling and Corey, 1951; Martin *et al.*, 2005). In addition, using SSEs (only) means that regions of protein structure not defined as being part of an SSE are not used at all, which can lead to less sensitive results (or, in some cases, excluding a structure from being processed at all, if the method of assignment determines that it contains no SSEs). At least a partial solution to this can be to use a method of defining SSEs that assigns a larger fraction of the structure to SSEs, as is done by ProSMoS, which uses the PALSSE method (Majumdar *et al.*, 2005), specifically designed for use in protein structure similarity searching.

Another protein structure comparison method is maximum contact map overlap (*MAX-CMO*), which consists of finding an alignment of residues in two proteins that maximizes the overlap between their contact maps. That is, it maximizes the number of contacts where residues that are in contact in one protein are aligned with residues that are also in contact in the other (Lancia *et al.*, 2001). MAX-CMO has been solved exactly by various methods including Lagrangian relaxation (Caprara and Lancia, 2002; Caprara *et al.*, 2004) and branch-and-bound (Xie and Sahinidis, 2006), but these methods are often impractically slow and so heuristics have been used to approximate the solution (Pelta *et al.*, 2008).

We can classify methods for protein structural searches into four categories. First, methods that align proteins directly at the level of residues. Dali and DaliLite fall into this category, as does any use of MAX-CMO as a protein structural search method. Second, methods that align proteins at the level of SSEs. TableauSearch, ProSMoS, and the TOPS-based methods (Gilbert *et al.*, 1999; Torrance *et al.*, 2005) fall into this category. Third, methods that perform an initial alignment at the level of SSEs, and then extend it to a residue level alignment. VAST (Madej *et al.*, 1995; Gibrat *et al.*, 1996), SSM (Krissinel and Henrick, 2004), LOCK2 (Shapiro and Brutlag, 2004), and SARF2 (Alexandrov, 1996) fall into this category. Fourth, methods that do not perform an alignment at all, but use some other means of providing a similarity score. YAKUSA and PRIDE (Carugo and Pongor, 2002; Gáspári *et al.*, 2005; Kirillova and Carugo, 2008) fall into this category.

Methods in the first category tend to be the slowest, since they are not necessarily designed solely or primarily for database scanning, but also to provide a set of correspondences between residues. Since the number of residues is naturally much larger than the number of SSEs, these methods must solve problems of a larger size than SSE-based methods. In this chapter, we compare our new method, QP Tableau Search, with two methods from this class: DaliLite, the standalone implementation of the widely-used method Dali, and SHEBA, a much faster method.

We note that this classification is not strict or even exclusive, for example we have placed SHEBA in the first category since it provides a residue-level alignment, but it does not do so directly (using its "environmental profiles" as a first step), so could be considered as partly belonging to the fourth category, with an extra stage bringing it into the first category. However it certainly does not belong to the second or third categories since it does not perform matching at the SSE level. Note also that any method in the second category (SSE alignment) can be transformed into a method in the third category by adding to it some method of extending the SSE alignment to a residue alignment.

Of the methods described so far, only ProSMoS and TOPS are capable of substructure (motif) queries, and in fact ProSMoS is designed specifically to search for manually defined motifs in a database of structures, rather than structure similarity searching. In both these cases the motifs are defined at the SSE (not residue) level. None of the residue-based methods (such as DaliLite and SHEBA) are designed to allow a motif (substructure) query, as they assess structural similarity to find common regions between two structures and find an optimal superposition. As discussed in the description of ProSMoS (Shi *et al.*, 2007), motifs, in contrast, are defined by "the main-chain topology, and general orientation and packing of secondary structural elements" (Shi *et al.*, 2007, p. 1331). TOPS allows specification of such motifs but operates on topological similarity only, while the algorithms described in this chapter and Chapter 6 use tableaux, which are based on SSE orientation, and distance matrices. ProSMoS makes use of "meta-matrices" which include information on SSE orientation and contacts including hydrogen-bonding.

ProSMoS, although it is similar to tableau matching in its use of SSE orientations, takes quite a different approach from most existing structural search methods in that, rather than taking a structure (or substructure) definition as a query, the

query meta-matrix is constructed manually (or at least modified manually from one generated by the supplied scripts) by the user. This is clearly useful for finding user-specified motifs in a database of structures, but creates challenges in assessing the performance of the method since the results are so dependent on the user-specified query. ProSMoS, in contrast to our method, returns a list of hits to the query structure, rather than a matching score for each structure in the database. This is often simpler for the user, but it does have the disadvantage that finding more (or fewer) hits requires editing the query meta-matrix, which can be quite difficult to calibrate. Returning a score for each database structure means that adjusting the sensitivity or specificity required is simply a matter of varying the cutoff score for a match to be considered significant. The ProSMoS web server (Shi *et al.*, 2009) extends ProSMoS to return a score, but only for the list of hits returned, that is, it only ranks those structures that it already considers a hit.

An advantage of the maximally-similar subtableaux formulation is that it allows the discovery of similar substructures within two structures, without requiring that the two structures are themselves similar as a whole, or that one of the structures must match as a whole some substructure within the other structure. We may choose to use one structure as a "query" motif, usually a small well-defined structural folding pattern, and find structures that contain this entire motif as a substructure, but it is also possible to find common substructures in two unrelated folds.

The two most similar methods to tableau matching are perhaps LOCK (Singh and Brutlag, 1997), and its newer version LOCK 2 (Shapiro and Brutlag, 2004), and ProSMoS. LOCK and LOCK 2 also match SSE vectors between structures, but use a more complex set of seven scoring functions, both orientation dependent and orientation independent, and use iterative dynamic programming requiring parameters for each of the scoring functions (Singh and Brutlag, 1997). In contrast, the tableau matching formulation is simpler and more elegant, although to obtain higher accuracy we extend it with a distance difference constraint that requires a parameter.

TableauSearch is described in the original tableau-based protein structure searching paper (Konagurthu *et al.*, 2008) as an approximation to the maximally-similar subtableaux extraction problem, which is an NP-hard problem (Konagurthu *et al.*, 2008). It uses an alignment-like approach with two phases of dynamic program-

ming, and is fast, but is inherently sequential (it cannot find non-sequential structural matchings), and cannot find substructure matchings (motifs).

IR Tableau (Zhang *et al.*, 2010) is a more recent protein structure search method based on tableaux. By reducing the tableau representation to feature vectors, IR Tableau can compare structures by *cosine similarity*, that is, just the cosine of the angle between the feature vectors. This is extremely fast, but, like TableauSearch, is no longer capable of finding substructural matches. In addition, it is capable only of providing a matching score, and not an alignment or set of correspondences between SSEs. As we show in Chapter 6, IR Tableau is the fastest method of those tested, but on some data sets is significantly less accurate than existing methods.

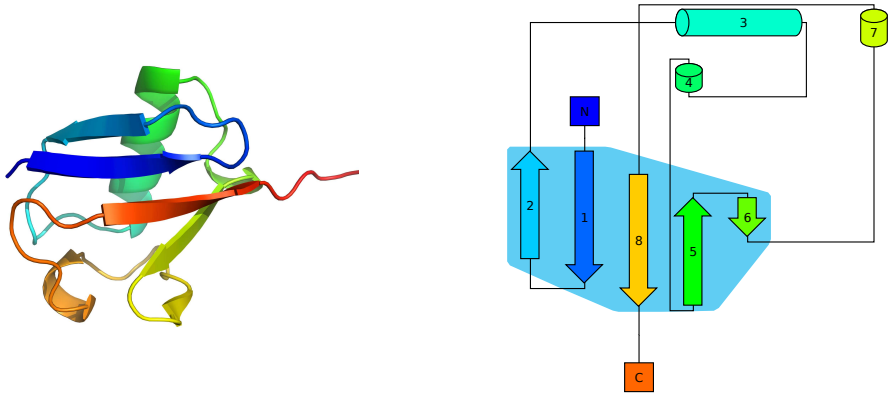
Here we present a method, based on the exact tableaux matching formulation (Konagurthu *et al.*, 2008) and recent work in alignment of molecular networks (Li *et al.*, 2007), that allows searches for occurrences of a query structure as substructures of structures in a database such as SCOP in practical time, allows non-sequential matchings, and is able to provide a set of correspondences between SSEs.

5.2 Methods

We built a database of tableaux, which is a file containing the tableau representation for each structure in the database. By pre-computing the tableaux in this way, only the query structure needs to have its tableau built when searching for occurrences of that structure. The search procedure is then to compute a matching score between the query tableau and each tableau in the database. Sorting the results by score allows the desired balance of sensitivity and specificity to be found by choosing a threshold score above which a match is considered a "hit" of the query to the database structure.

5.2.1 Tableaux

Tableaux (Lesk, 1995, 2003; Kamat and Lesk, 2007; Konagurthu *et al.*, 2008) describe the relative orientations of strands and helices. A tableau is an encoding of the orientation matrix, which is a square symmetric matrix in which each element is the angle between the axes of two SSEs, where such axes are defined to have a direction from the N- to the C-terminus of the SSE. Figure 5.1 shows a structure



(a) (b)

```
 0.000
 2.654  0.000
-1.173  2.147  1.000
 0.389 -2.757  1.352  3.000
 2.040 -1.438  2.079 -1.651  0.000
-1.258  1.556 -1.108 -1.647  2.985  0.000
 1.693 -1.808 -1.851  1.309 -0.370 -2.913  3.000
-0.590  2.102 -1.231  0.959  2.566 -0.721  2.264  0.000
```

(c)

```
e
OT e
LE RT xa
PD OS RD xg
RT LE RT LS e
LE RD LE LS OT e
RT LS LS RD PE OS xg
PE RT LE RD OT PE RT e
```

(d)

FIGURE 5.1: *β-grasp query structure and tableau.*

(a) 3D structure of ubiquitin, PDB identifier 1UBI. Image generated with PyMOL. (b) Topology cartoon of 1UBI. (c) Orientation matrix Ω for 1UBI. Each angle is in radians between $-\pi$ and π. The main diagonal denotes the SSE type by 0.000, 1.000, 2.000, or 3.000 for β-strands, α-helices, π-helices, and 3_{10}-helices, respectively. (d) Tableau for 1UBI. The main diagonal denotes the SSE type by `e`, `xa`, `xi`, *or* `xg` *for β-strands, α-helices, π-helices, and 3_{10}-helices, respectively.*

and its orientation matrix and tableau. The construction of orientation matrices and tableaux will now be described in detail; it may be useful to refer back to Figure 5.1 after reading the remainder of this section.

Fitting axes to helices for the purposes of computing angles between them is a standard technique going back at least to Chothia *et al.* (1981), with more detailed helix packing angle calculations implemented more recently in programs such as Helix Packing Pair (Dalton *et al.*, 2003). We follow the method used in TableauCreator (Konagurthu *et al.*, 2008) for defining the axis for a helix. A total least squares line is fitted through the midpoints of the C_α atoms of all consecutive residue triplets in the helix. This is done by computing the singular value decomposition of the matrix A, each row of which is the vector from the centroid of all of such

midpoints to each single midpoint. This results in the matrices U, Σ and V such that $A = U\Sigma V^*$ where U and V are unitary matrices and Σ is a diagonal matrix with the singular values on the main diagonal. The columns of V (the right singular vectors) form an orthonormal basis for the space spanned by A. The basis vector (column of V, i.e. row of V^*) corresponding to the largest singular value is then a vector pointing in the direction in which variance of the data in A is greatest; that is, it points along the major axis of the helix.

This vector points along the helix axis, but there is no guarantee that it points from the N- to the C- terminus (rather than from the C- to the N- terminus). We compute the cosine of the angle between this vector and the vector from the most N-terminal to the most C-terminal midpoint (i.e. the dot product of these two vectors) and if this cosine is positive then the first vector (the axis vector) is pointing in the wrong direction (i.e. from C- to N-terminus) so we reverse it.

For helices consisting of fewer than four residues, this computation cannot be performed. If the helix consists of three residues, the axis is computed as the vector between the two triplet midpoints that can be computed, however no axis can be computed for fewer than three residues.

Several methods for fitting axes to β-strands have been used. The simplest is probably that described by Richards and Kundrot (1988), which is used by TOPS (Flores *et al.*, 1994), and a variation of which is used in TableauCreator (Konagurthu *et al.*, 2008). In this method, vectors between consecutive C_α atoms are computed, and averaged to obtain an overall axis vector. The most common method, however, is to fit a (total) least squares line to the C_α atoms of residues in the strand, or some variation such as their midpoints. This method is described for example by Cohen *et al.* (1981), Lesk (1995), Taylor *et al.* (1983); Taylor (2001) and Kamat and Lesk (2007), and is the method we employ. Following Cohen *et al.* (1981), we fit the line to the midpoints of each consecutive pair of C_α atoms in order to reduce the influence of the pleat of the strand. In the same manner as for the helix axis, the SVD is used to compute the direction cosine of the axis, which is then reversed so that it points from the N- to C-terminus if it does not already do so.

If the strand has fewer than four residues, we use the simpler method of computing the vector from the most N-terminal to the most C-terminal midpoint (for

three residues) or from the most N-terminal to the most C-terminal C_α atom (for two residues).

After axes have been fitted to all SSEs, the relative angle between each pair of SSEs is computed. This interaxial angle ω is defined as the smallest angle required to reorient one vector so that it eclipses the other, around the mutual perpendicular between the two vectors; or, equivalently, the angle between the two vectors projected onto a plane normal to their mutual perpendicular (Konagurthu *et al.*, 2008). This is illustrated in Figures 5.2, 5.3 and 5.4. In Figure 5.2, BC is the shortest line segment between the axes for SSE 1 and SSE 2, defining the vector v_2. The vectors v_1 and v_3 define the axes of SSE 1 and SSE 2 respectively. The relative angle ω (the interaxial angle) is the smallest angle required to reorient v_1 so that it eclipses v_3 (or vice versa) when "sighting along" the mutual perpendicular BC (vector v_2):

$$\omega = \begin{cases} -\theta, & \text{if } s < 0, \\ \theta, & \text{otherwise.} \end{cases} \tag{5.1}$$

where

$$\theta = \arccos\left(\frac{v_1 \times v_2}{|v_1 \times v_2|} \cdot \frac{v_2 \times v_3}{|v_2 \times v_3|}\right) \tag{5.2}$$

$$s = v_2 \cdot \left(\frac{v_1 \times v_2}{|v_1 \times v_2|} \times \frac{v_2 \times v_3}{|v_2 \times v_3|}\right) \tag{5.3}$$

Figures 5.3 and 5.4 show the application of these methods to strands and helices respectively.

The interaxial angles between each pair of SSEs gives the orientation matrix. This is converted into a tableau by means of a double-quadrant encoding of the angle as described by Lesk (1995). The first quadrant encoding classifies the angle as P (parallel), O (anti-parallel ("opposite")), R (crossing-right), or L (crossing-left). The second quadrant encoding is rotated 45° to the first and the second character given by this encoding ensures that angles differing by 45° or less must have at least one of the two characters of the encoding in common. This is illustrated in Figure 5.5.

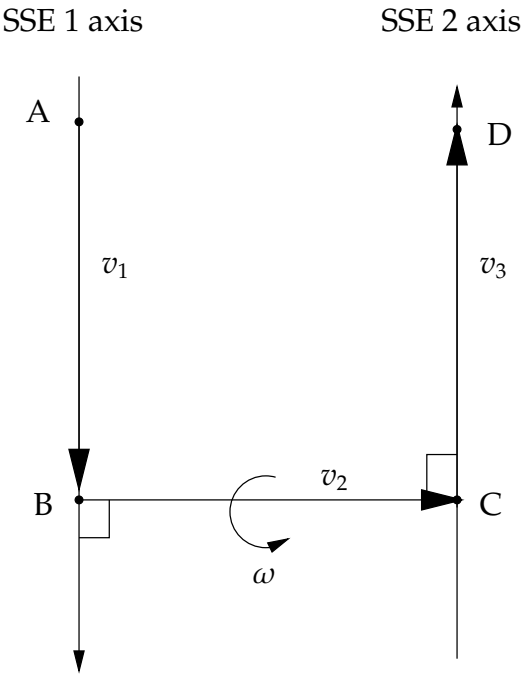


FIGURE 5.2: *Computing the relative angle ω between two SSE axes.*

Because the orientation matrix and tableau are symmetric, we need only store one triangle, and since the main diagonal is the meaningless self-angle, we use it to store the type of SSE represented by that row and column (see Figure 5.1).

As originally defined by Lesk (1995) and used in Kamat and Lesk (2007), tableaux only contain entries for SSEs that are "in contact" with each other, that is, the corresponding entry in the distance matrix is below some threshold. Here, as in Konagurthu *et al.* (2008), we use a version of tableaux in which every entry has a value, regardless of the distance between the two SSEs.

5.2.2 Quadratic integer programming formulation of extraction of maximally-similar subtableaux

The extraction of maximally-similar tableaux by quadratic integer programming (QIP) was described by Konagurthu *et al.* (2008). We use the same formulation:

Let $\Omega_A = (\omega^A_{ij}), 1 \leq i, j \leq N_A$ be the orientation matrix for protein A with N_A SSEs and $\Omega_B = (\omega^B_{ij}), 1 \leq i, j \leq N_B$ the orientation matrix for protein B with N_B SSEs. Similarly let $T_A = (t^A_{ij})$ and $T_B = (t^B_{ij})$ be tableaux.

Define Boolean variables x_{ij}, $1 \leq i \leq N_A$, $1 \leq j \leq N_B$ where $x_{ij} = 1$ indicates that the ith SSE in structure A is matched with the jth SSE in structure B.

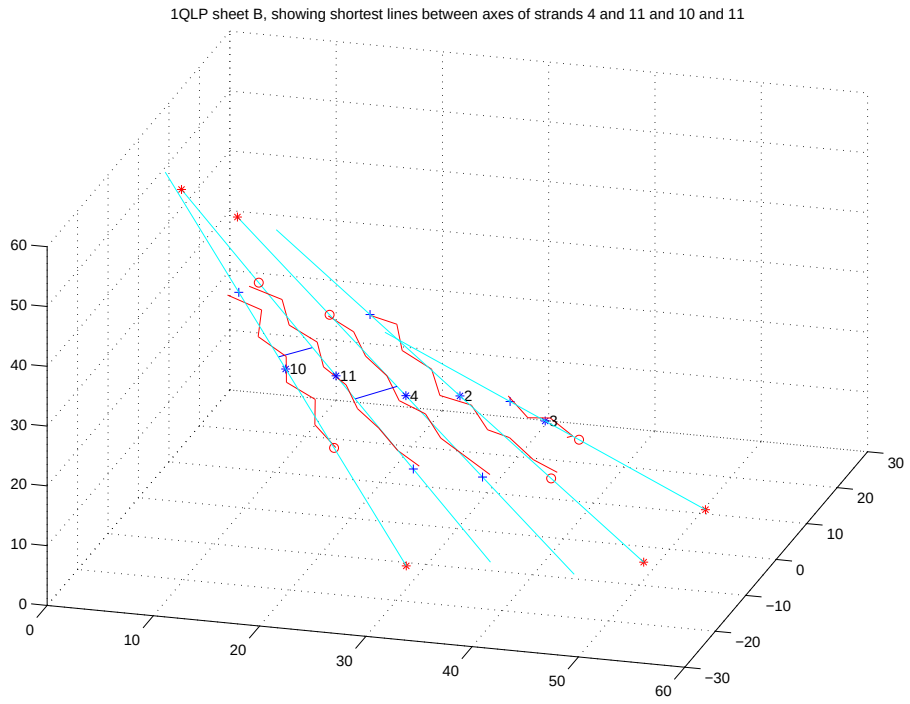


FIGURE 5.3: *Axes fitted to strands in a sheet.*

In each strand, the red line is plotted between C_α co-ordinates of each residue, and the cyan line is drawn along the axis fitted to the strand. The (blue) asterisk is plotted at the centroid of the C_α atoms in the strand, and the (red) asterisk on the axis line is plotted on the C-terminal side of the centroid to show the direction of the axis line. The (blue) plus and (red) circle are plotted at the projections of the most N-terminal and most C-terminal C_α atoms respectively onto the axis line. The mutual perpendiculars to (i.e. the shortest lines between) strands 10 and 11 and 4 and 11 are also shown (in blue). This diagram was plotted from Pro-origami output (see Chapter 7) using MATLAB (MathWorks, 2006).

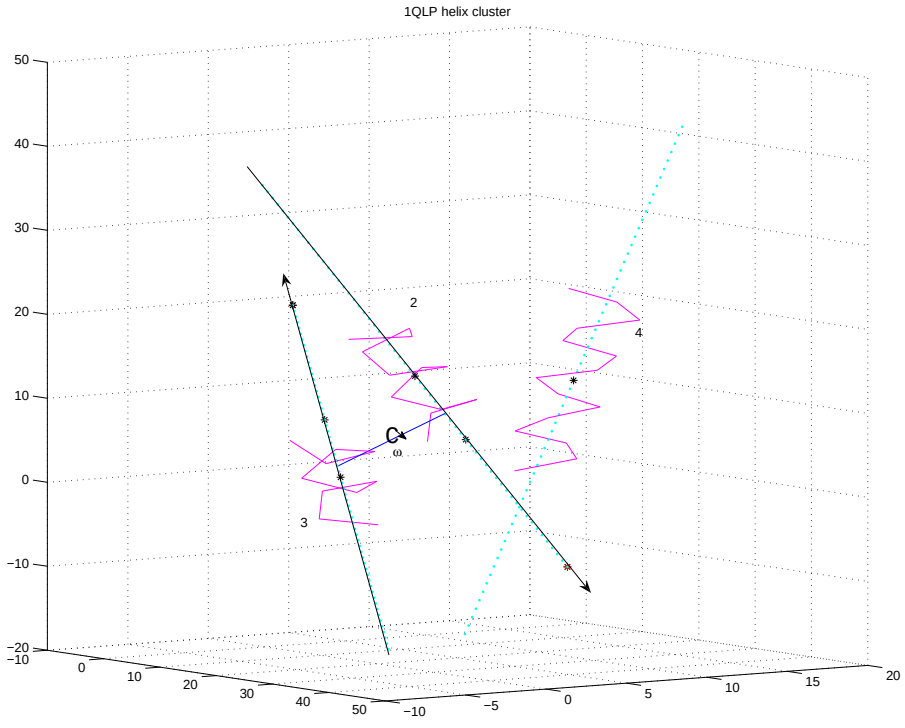


FIGURE 5.4: *Axes fitted to helices*

The interaxial angle ω between the axes fitted to helices 2 and 3 is shown. The line between the two axes is the shortest line between them, which is perpendicular to both axes, and the angle ω is the smallest rotation around this line that eclipses one axis by the other when "sighting along" the mutual perpendicular. This diagram was generated from Pro-origami (Chapter 7) output using MATLAB.

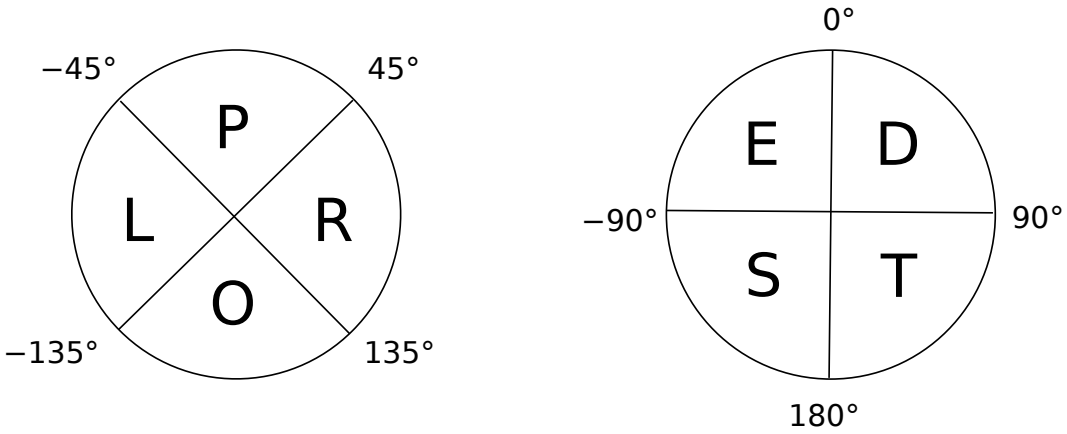


Figure 5.5: *Double-quadrant encoding of orientations.*

The relative orientation of two SSEs is encoded as a two-character string for the tableau by means of the two quadrant schemes shown, which differ in orientation by 45° *in order to prevent small variations resulting in completely different encodings. For example, two SSEs that are anti-parallel would be encoded as either* `OS` *or* `OT`*, and two SSEs that are "crossing-right" would be encoded as either* `RD` *or* `RT`*. Note how this prevents small variations in the angle resulting in completely different encodings: a relative angle of just under* 135° *is encoded as* `RT` *and a relative angle of just over* 135° *is encoded as* `OT` *— they have the second character in common, whereas if only the first quadrant were used, they would have completely different (single-character) codes.*

Define a scoring function ζ which gives high scores to similar orientations, as follows:

$$\zeta(\omega^A_{ik}, \omega^B_{jl}) = \frac{\pi}{4} - \Delta\omega \tag{5.4}$$

where

$$\Delta\omega = \min\{|\omega^A_{ik} - \omega^B_{jl}|, 2\pi - |\omega^A_{ik} - \omega^B_{jl}|\} \tag{5.5}$$

When comparing (discrete) tableaux, the scoring function is defined as:

$$\zeta(t^A_{ik}, t^B_{jl}) \quad = \quad \begin{cases} 2, & \text{if } t^A_{ik} \equiv t^B_{jl} \\ 1, & \text{if } t^A_{ik} \simeq t^B_{jl} \\ -2, & \text{otherwise.} \end{cases} \tag{5.6}$$

where $t^A_{ik} \equiv t^B_{jl}$ means the two tableau codes are identical, and $t^A_{ik} \simeq t^B_{jl}$ means they differ in only one quadrant, for example `OS` and `OT`, or `OT` and `RT`.

Then the QIP is:

maximize

$$f(x) = \sum_{1 \le i,k \le N_A, 1 \le j,l \le N_B} \zeta(\omega^A_{ik}, \omega^B_{jl}) x_{ij} x_{kl} \tag{5.7}$$

subject to

$$\sum_{j=1}^{N_B} x_{ij} \leq 1, \qquad 1 \leq i \leq N_A \tag{5.8}$$

$$\sum_{i=1}^{N_A} x_{ij} \leq 1, \qquad 1 \leq j \leq N_B \tag{5.9}$$

Constraints (5.8) and (5.9) ensure each SSE in one tableau is matched with at most one SSE in the other.

We introduce a further condition that two SSEs of different types (for example an α-helix and a β-strand) should not be matched, by assigning a low score to such a matching, for which we use the SSE type information encoded on the diagonal of the tableau or Ω matrix.

We may optionally avoid non-sequential matchings by assigning a low score to matches between SSEs whose indices i, k in one structure and j, l in the other satisfy both of the following inequalities:

$$1 \leq i < k \leq N_A \tag{5.10}$$

$$1 \leq l < j \leq N_B \tag{5.11}$$

Without this condition, non-sequential matchings are found.

In order to avoid false positives when SSEs in two structures have similar orientations relative to other SSEs in their respective structures, but are at very different distances from those other SSEs, we introduce a distance difference constraint, disallowing matches between SSEs where the difference in distances between the SSEs exceeds a threshold distance τ:

$$x_{ij} + x_{kl} \leq 1 \text{ if } |d^A_{ik} - d^B_{jl}| > \tau, \qquad 1 \leq i, k \leq N_A, 1 \leq j, l \leq N_B \tag{5.12}$$

where $D^A = (d^A_{ik}), 1 \leq i, k \leq N_A$ and $D^B = (d^B_{jl}), 1 \leq j, l \leq N_B$ are SSE midpoint distance matrices. These are square symmetric matrices, of the same dimensions as the orientation matrices and tableaux, where each entry is the distance (in Ångströms)

between the centroids of the C_α atoms used in computing the respective SSEs' axes. We use the value $\tau = 4.0$Å for the distance difference threshold. This value was found empirically to give good results after testing various values between 2.0Å and 8.0Å on a subset of the queries in Table 5.1. As with tableaux, these distance matrices are pre-computed and stored as a triangle with SSE information on the main diagonal. As before, we do not implement this constraint directly, but instead penalize the objective function when it is violated.

5.2.3 Relaxed quadratic programming formulation and solution by interior point method

The QIP just described is NP-hard (Konagurthu *et al.*, 2008). Even though the instances are quite small, direct solution with ILOG CPLEX[1], a state-of-the-art commercial solver, is too slow for practical use in searching a structure database (Konagurthu *et al.*, 2008). A solution to this problem is provided by the work of Li *et al.* (2007), whose formulation of biological network alignment is strikingly similar to the QIP for extracting maximally-similar tableaux. They show that the constraints (5.8)–(5.9) are *totally unimodular*, that is, the determinant of every square submatrix of the constraint matrix (A in Equation 5.14) is 0, 1, or -1. This allows the QIP to be relaxed to a quadratic program (QP) by removing the integrality constraints on the Boolean variables x_{ij}, and that the QP will have an integer solution under certain conditions. This allows this (nonconvex) QP to be solved with an efficient interior ellipsoidal trust region method (Ye and Tse, 1989; Ye, 1992, 1997).

The standard form of a QP is

$$\text{minimize} \quad \tfrac{1}{2}x^T Q x + c^T x \tag{5.13}$$

$$\text{subject to} \quad Ax \leq b, x \geq 0 \tag{5.14}$$

where Q is the symmetric $n \times n$ objective matrix, A is the constraint left-hand $m \times n$ matrix, b is the constraint right-hand $m \times 1$ vector, c is the objective $n \times 1$ vector, and x is the solution $n \times 1$ vector.

[1] `http://www.ilog.com/products/cplex/`

In expressing the maximally-similar subtableaux QIP (5.7)–(5.9) in standard form, the vector c is zero as there is no linear term in the QIP objective function (5.7). The coefficient matrix A contains only 1s and 0s, since the constraints (5.8) and (5.9) are all of the form $\sum_{i=1}^{n} x_i \leq 1$; hence A is totally unimodular as shown by Li *et al.* (2007). The objective matrix Q contains the values of the scoring function ζ; these values are simply negated to transform the maximization problem (5.7) to the minimization problem (5.13).

Constraints (5.8) and (5.9), to ensure each SSE in one tableau is matched with at most one SSE in the other, are expressed in the standard form, that is, in the A matrix and b vector in equation (5.14). Constraints on SSE midpoint distance differences, mismatching SSE types and linearity of matchings, when desired, are not expressed directly, due to the infeasibly large matrices that would result from so doing. Rather, we relax them and penalize their violation by assigning a low score to such matches. This results in the following modified objective function for the discrete (tableau) version, where the negation to transform the problem to a minimization problem has also been shown:

$$f_p(x) = \sum_{1 \leq i,k \leq N_A, 1 \leq j,l \leq N_B} x_{ij} x_{kl} \cdot \begin{cases} 0 & \text{if} \quad t^A_{ii} \neq t^B_{jj} \vee t^A_{kk} \neq t^B_{ll} \\ 1 & \text{if} \quad (i < k \wedge j > l) \vee (i > k \wedge j < l) \\ 0 & \text{if} \quad |d^A_{ik} - d^B_{jl}| > \tau \\ -\zeta(t^A_{ik}, t^B_{jl}) & \text{otherwise} \end{cases} \tag{5.15}$$

The choice of 0 and 1 as the penalties in conjunction with the discrete tableau scoring function ζ (5.6) was found empirically to give good results.

We find in common with Li *et al.* (2007), that although the sufficient conditions described in the Supplementary Materials of Li *et al.* (2007), are not always met, that nevertheless an integer solution is almost always obtained.

5.2.4 Evaluation

As a structural database for evaluating QP Tableau Search and other protein structural search methods, we use the 15273 domains in the 95% sequence identity non-redundant subset of the ASTRAL SCOP 1.73 database (Chandonia *et al.*, 2004;

Andreeva *et al.*, 2008). Unless otherwise stated, all queries, other than those for comparison with MAX-CMO using the Fischer or Nh3D data sets, discussed in the results were against this database. In the case of the tableau-based methods (QP Tableau Search and TableauSearch) the structural database to be searched consists of a file containing the tableau (and, in the case of QP Tableau Search, also the distance matrix) for each structure.

The larger scale query set is a set of 200 queries chosen from the ASTRAL SCOP 1.73 95% sequence identity non-redundant data set. The queries were chosen so that each of the SCOP classes α, β, α/β, $\alpha + \beta$ is represented in the query set in the same ratio as it is in the database. The list of queries is available with the source code and other data as described Section 5.2.5.

The Fischer data set, described in Table II of Fischer *et al.* (1996), consists of 68 proteins. Several PDB identifiers in this table have since been obsoleted, and we replaced these with their new versions according to the RCSB PDB website (Berman *et al.*, 2000). As was done by Pelta *et al.* (2008), we performed an all-against-all comparison in this data set, including redundant comparisons, resulting in 4624 comparisons.

The Nh3D v3.0 data set (Thiruv *et al.*, 2005) consists of 806 structures, each representing a different CATH (Pearl *et al.*, 2005) topology. We performed the same 58838 comparisons as Pelta *et al.* (2008) by comparing each of the 73 structures listed in the Supplementary Material of Pelta *et al.* (2008) against every structure in the Nh3D v3.0 data set.

The MSVNS3 implementation (described in Section 5.3.1) and tableau search implementations produce unnormalized scores. MSVNS3 provides an overlap value, QP Tableau Search provides the maximum value of the tableau scoring function, and TableauSearch also provides an approximation of the maximum value of the tableau scoring function. For comparing sets of pairwise scores between proteins of different sizes, a normalization function is required. We use the same

three normalization functions as Pelta *et al.* (2008), namely:

$$\text{norm1}(P_i, P_j) = \frac{\text{score}(P_i, P_j)}{\min\{\text{size}(P_i), \text{size}(P_j)\}} \tag{5.16}$$

$$\text{norm2}(P_i, P_j) = 2 \cdot \frac{\text{score}(P_i, P_j)}{\text{size}(P_i) + \text{size}(P_j)} \tag{5.17}$$

$$\text{norm3}(P_i, P_j) = \begin{cases} 0 & \text{if } \frac{|\text{size}(P_i) - \text{size}(P_j)|}{\max\{\text{size}(P_i), \text{size}(P_j)\}} > 0.75 \\ \text{norm1}(P_i, P_j) & \text{otherwise} \end{cases} \tag{5.18}$$

where score is the overlap value or tableau matching score for MSVNS3 or tableau search, respectively, and size is the number of contacts or number of SSEs for MSVNS3 or tableau search, respectively.

We evaluated the accuracy of structural search by counting a hit (a score above the threshold) as correct (a true positive) if the structure is in the same SCOP fold as the query structure, and incorrect (a false positive) otherwise. By using SCOP as the gold standard in this way, large scale automatic evaluation on a large number of different queries is possible.

For the Fischer data set, we evaluated at both the fold and class level. At the fold level, a true positive is counted when the score is above the current cutoff and the two structures are in the same fold according to Table II of Fischer *et al.* (1996). At the class level, a true positive is counted when the score is above the current cutoff and the two structures are in the same class (α, β, α/β, $\alpha + \beta$, or "other") according to Table II of Fischer *et al.* (1996). For the Nh3D data set, we evaluated at both the architecture and class levels in CATH. At the architecture level, a true positive is counted when the score is above the current cutoff and the two structures have the same CATH architecture identifier and the same CATH class identifier. At the class level, they need only have the same class identifier.

Evaluation of the accuracy of substructure queries is more challenging, since we require as our gold standard a database of structures that contain a motif as a substructure. By using `d1ubia_`, an exemplar of the β-grasp fold, as the query, we used the data from Table 1 of Shi *et al.* (2007) as the gold standard. A hit is considered a true positive if it is in the same SCOP superfamily as the exemplars listed in Table 1 of Shi *et al.* (2007) for the β-grasp core and gregarious fold (Harrison

et al., 2002) categories, or if it is one of the structures considered by Shi *et al.* (2007) to contain the β-grasp motif by structural drift (Krishna and Grishin, 2005).

The Area under the ROC Curve (AUC, see Section 2.5 of Chapter 2) is computed with the ROCR package (Sing *et al.*, 2005) in R (R Development Core Team, 2007). 95% confidence intervals for the AUC values are calculated by the Hanley-McNeil method (Hanley and McNeil, 1982).

For comparisons with other methods we used SHEBA version 3.1.1, VAST downloaded from `http://migale.jouy.inra.fr/outils/mig/vast`, ProSMoS downloaded from `ftp://iole.swmed.edu/pub/ProSMoS`, and the TOPS matching software downloaded from `http://balabio.dcs.gla.ac.uk/tops/software.html`.

TableauSearch was supplied by Dr Arun Konagurthu (personal communication). The authors' implementation of the VNS heuristic for MAX-CMO (Pelta *et al.*, 2008), MSVNS4MaxCMO, was downloaded from `http://modo.ugr.es/jrgonzalez/msvns4maxcmo`. We used MSVNS3, the best performing version of the heuristic according to Pelta *et al.* (2008), for all tests.

For MAX-CMO, we generated contact maps for each structure with a threshold of 7.0Å and sequence separation of 2 residues using a modified version of `PConPy` (Ho *et al.*, 2008). For QP Tableau Search, we generated tableaux and distance matrices for each structure with our own implementation of the tableau creation algorithm, including π and 3_{10} helices and using DSSP to define secondary structure elements. We built the TOPS database for the ASTRAL SCOP 1.73 95% sequence identity non-redundant subset using TOPS downloaded from `http://www.tops.leeds.ac.uk` (July 2007) with default parameters (DSSP is used to define SSEs).

For the comparison with SSM, the SSM web server[2] running SSM v2.36 and searching the SCOP 1.73 database was used, with default parameters. The search was restricted to the 95% sequence identity non-redundant subset by uploading the relevant ASTRAL SCOP identifier list as the list of SCOP 1.73 codes for the target.

For the comparison with ProSMoS, we found that the query meta-matrices produced by the scripts included with ProSMoS applied to the query structures resulted in no hits, even when extensively edited to make them less specific. Therefore, we manually constructed the query meta-matrices based on the following information:

- DSSP SSE assignments

[2] `http://www.ebi.ac.uk/msd-srv/ssm/`

- automatically generated topology cartoons
- 3D structure as shown by PyMOL (DeLano, 2002)
- SCOP description of the fold
- the list file generated by the ProSMoS matrix generation scripts for the query structure.

When comparing methods for substructure search, apart from the detailed analysis of occurrences of the β-grasp motif available in Table 1 of Shi *et al.* (2007), for constructing the comparisons detailed in Table 5.7 and Table 5.8 we follow a procedure similar to that described for Table 2 of Shi *et al.* (2007), and count SCOP superfamily and fold descriptions that mention the query structure in question. ProSMoS and SSM give a list of hits to the query structure, and so we can simply count the number of superfamilies represented by these hits. Our method, however, and also the version of TOPS we are using, does not return such a list of hits but rather assigns a score to each database structure. This makes the computation of ROC curves, as previously described, a useful and appropriate method of assessment, but creates a difficulty for the superfamily counting method: we need a method to determine a score above which a hit is considered significant. In order to do this, we compute a Z-score for each matching of a query to a database structure as

$$Z = \frac{s - \mu}{\sigma} \tag{5.19}$$

where s is the score assigned to the matching, μ is the arithmetic mean of the scores for all database structures for that query, and σ is the standard deviation. For both our method and TOPS, we choose a significant hit to be a matching with $Z \geq 3.0$, a value which was found empirically to give a reasonable number of hits without excluding too many true positives amongst our set of example queries.

5.2.5 Implementation

We implemented scripts for creating tableaux, building the tableaux database, evaluating results against SCOP and converting search output for visualization with PyMOL in Python. Our implementation of the tableaux creation algorithm optionally allows a list of SSEs in the structure to be represented in the tableau, rather than

			distance information			
			without		with	
Fold	SCOP sid	# SSEs	AUC	time	AUC	time
β-grasp	d1ubia_	8	0.80	0 h 47 m	0.92	0 h 32 m
Key-barrel	d1tttb1	9	0.80	0 h 50 m	0.97	0 h 50 m
Immunoglobulin	d1ae6h1	13	0.89	1 h 47 m	0.95	1 h 53 m
Plait (ferredoxin)	d1bhne_	15	0.61	1 h 53 m	0.85	2 h 18 m
GFP-like	d1h6rb_	17	1.00	3 h 06 m	0.99	3 h 06 m
Jelly-roll	d2phlb1	19	0.87	4 h 24 m	0.93	5 h 13 m
Tim-barrel	d1tima_	21	0.99	4 h 31 m	1.00	4 h 30 m
NAD-binding fold	d1f6dc_	30	0.98	14 h 45 m	0.99	16 h 33 m

TABLE 5.1: *AUC and time for some widespread folds.*

Area under ROC curve (AUC) and time taken for searches for several structural folding patterns against the ASTRAL SCOP 95% sequence identity non-redundant database consisting of 15273 domains using discrete tableaux, both with and without incorporating SSE distance information. Times (in hours and minutes) reported are elapsed times using the sparse matrix (UMFPACK) implementation on an Intel Xeon 3.2GHz machine with 8GB memory running Linux. The table is sorted by query size (number of SSEs in query).

all SSEs in the structure, in order to generate tableaux for substructure queries. We used the BioPython library (Cock *et al.*, 2009) and the Bio.PDB file parsing and structure class (Hamelryck and Manderick, 2003) to parse PDB files and the Bio.SCOP interface (Casbon *et al.*, 2006) to read SCOP and ASTRAL data. We re-implemented the QP solving algorithm (Ye and Tse, 1989; Ye, 1992), originally implemented by the author[3] in MATLAB, in Fortran 77 with the BLAS (Dongarra *et al.*, 1988) and LAPACK (Anderson *et al.*, 1992) libraries for dense matrices, and the UMFPACK 5.2 (Davis and Duff, 1997, 1999; Davis, 2004a,b) library for sparse matrices. The tableau searching program itself was written in Fortran 77.

Source code, data sets, and executable binaries are available from `http://www.csse.unimelb.edu.au/~astivala/qpprotein/`.

5.3 Results and discussion

We evaluate the accuracy of our QP tableau matching algorithm as a method for determining the fold of a structure, using SCOP as the truth. The tradeoff between sensitivity and specificity for such a classification task can be shown as a Receiver Operating Characteristic (ROC) curve (Sam *et al.*, 2006).

[3] `http://www.stanford.edu/~yyye/matlab.html`

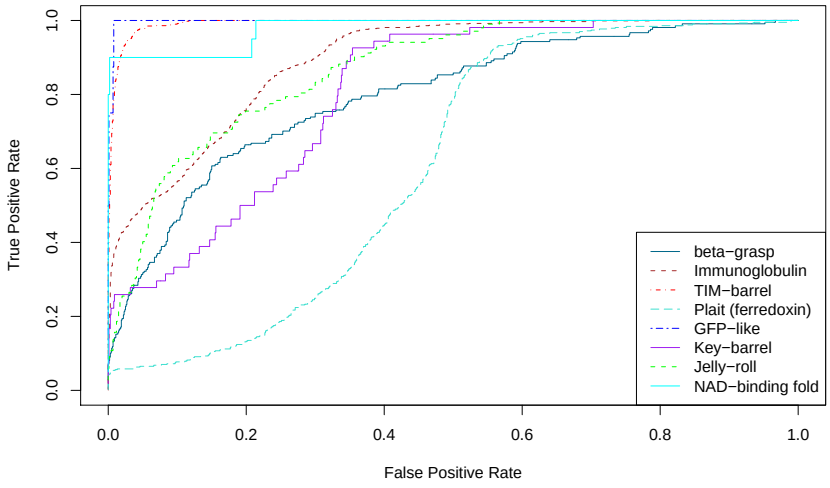


FIGURE 5.6: *ROC curves without using distance information.*

ROC curves for the structural folding patterns in Table 5.1 as the query structure, using the discrete tableau encoding and no distance information. A true positive is a "hit" on a structure that is in the same SCOP fold as the query.

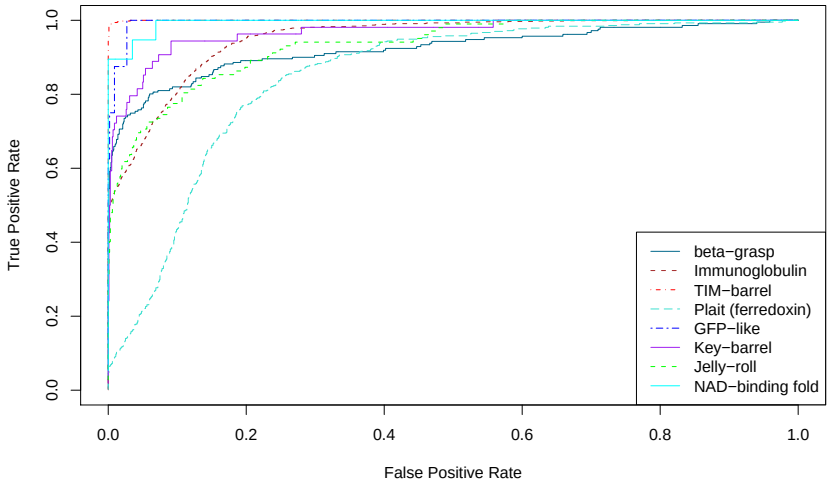


FIGURE 5.7: *ROC curves using distance information.*

ROC curves for the structural folding patterns in Table 5.1 as the query structure, using the discrete tableau encoding and distance information. A true positive is a "hit" on a structure that is in the same SCOP fold as the query.

```
e                          e
OT e                       OT e
LE RT xa                   LE RT xa
PD OS RD xg                PD OS RD xg
RT LE RT LS e              RT LE RT RD e
LE RD LE LS OT e           LE RT LE LS OS e
RT LS LS RD PE OS xg       RD LE OT RD PE OT xg
PE RT LE RD OT PE RT e     LE RT LE LE OT PE RT e
```

FIGURE 5.8: *Tableau for* `d1ubia_` *(left) and its maximally-similar subtableau found in* `d1or0.1` *(right).*

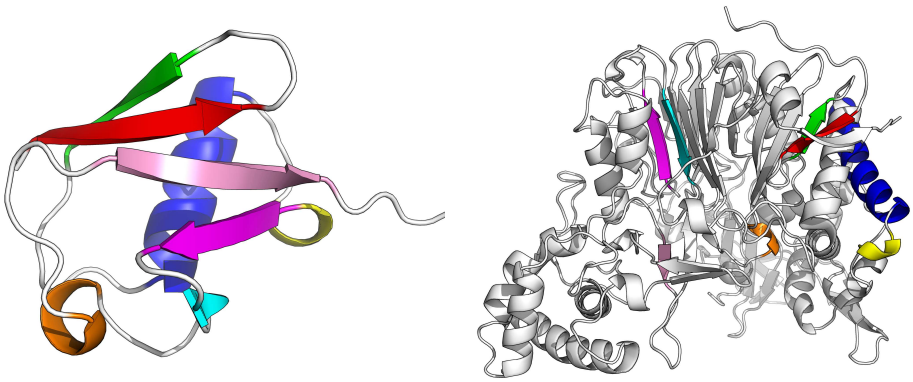


FIGURE 5.9: `d1ubia_` *structure (left) and the corresponding SSEs found in* `d1or0.1` *(right).*

SSEs from each structure that are found to correspond according to maximally-similar subtableaux extraction, without the distance difference constraint, have the same colour. Both cartoons were generated with PyMOL.

ROC curves for several different structural queries are shown in Figures 5.6 and 5.7. Figure 5.6 shows results obtained using only tableau information, while Figure 5.7 shows results obtained using tableau and distance information. The area under the ROC curve (AUC) for each of these curves is shown in Table 5.1. Incorporating distance information results in consistently higher AUC values without significantly affecting the elapsed time.

To demonstrate why the distance difference constraint improves the accuracy of structural database searching by reducing false positives, we show an example. In this example, maximally-similar subtableaux extraction by QP tableau matching *without* the distance difference constraint finds a subtableau of `d1or0.1`, cephalosporin acylase, that is very similar to the β-grasp structure `d1ubia_` (see Fig-

ure 5.8). It is easy to see by eye that these two tableaux are very similar. However, consider the corresponding matching SSEs illustrated in Figure 5.9. It is clear that the β-grasp motif is not really present in `d1or0.1` at all, since the SSEs that make it up, although they are oriented in the same way as in the β-grasp motif, are at very different distances from each other in this structure. The distance difference constraint prevents this situation by disallowing matches between SSEs that exceed the distance difference threshold.

We find that the best-performing variation of our method, using the discrete tableau encoding rather than numeric Ω matrices, and incorporating distance information, has an AUC of 0.95 averaged over the eight queries in Table 5.1.

We can see in Table 5.1 that the ferredoxin fold query (`d1bhne_`) performs significantly worse than the others. We examined the results from this query and found that a large number of false negatives occur (many members of this fold are not given a high score by our method). Examining some of these false negatives in detail, we find that it is often due to DSSP (Kabsch and Sander, 1983), which we use to define SSEs, not defining some of the SSEs required to match the query structure (in an extreme case, `d2atcb1`, DSSP defines only a single helix and nothing else). Although we have the capability of using STRIDE (Frishman and Argos, 1995) rather than DSSP, the results are often similar (as in the `d2atcb1` example). This is a shortcoming of any method that depends on SSE definitions, although ProSMoS solves it to some degree by using PALSSE (Majumdar *et al.*, 2005), a secondary structure assignment method that assigns many more residues to SSEs precisely in order to avoid this problem (Shi *et al.*, 2007). False negatives can also occur independently of the SSE definition algorithm, if a structure does not have some SSEs not considered essential to the fold according to SCOP (but which are included in our query structure) and/or sufficiently different in their orientation that tableau matching does not assign them a high score. An example of this is `d1q8ba_`, which does not have all the helices in the query structure, and some which it does include have rather different orientations from those in the query, but it is nevertheless classified as a member of the ferredoxin-like fold.

Table 5.1 shows that a search in a database of 15273 structures takes approximately one hour for a small (10 or fewer SSEs) query structure on a single CPU of a standard PC, and under four hours for query structures with fewer than 18 SSEs,

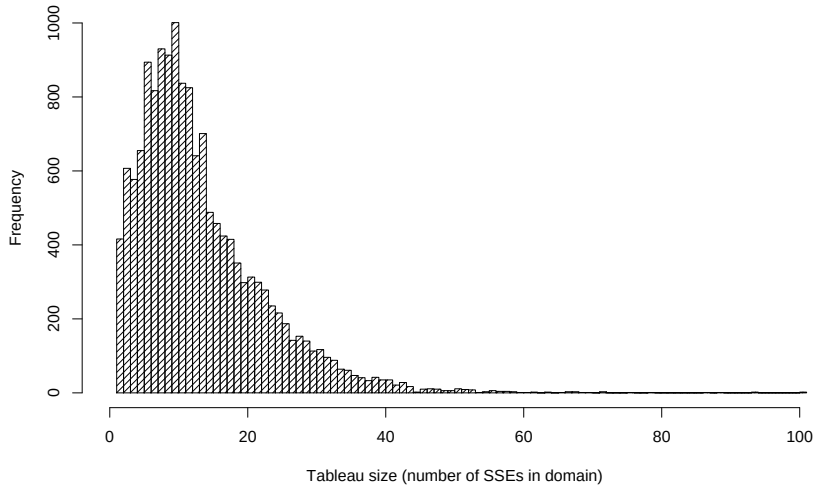


FIGURE 5.10: *Histogram of tableau sizes.*

but can take more than 16 hours for a query structure with 30 SSEs. Since 75% of domains in the database have fewer than 20 SSEs, and the most frequent number of SSEs is 10 (see Figure 5.10), most queries for a structure drawn from a set of structures with the same distribution of tableau size as the database will complete within 4 hours. We note that the peak at 10 differs from the results of Kamat and Lesk (2007), who find the peak is at 6, as we have used DSSP to define SSEs and have included π- and 3_{10}-helices, while Kamat and Lesk (Kamat and Lesk, 2007) used the assignments of helices and strands from PDB files.

For a larger-scale test, we used a set of 200 queries chosen from the ASTRAL SCOP 1.73 95% sequence identity non-redundant data set (Chandonia *et al.*, 2004; Andreeva *et al.*, 2008). The queries were chosen so that each of the SCOP classes α, β, α/β, $\alpha + \beta$ is represented in the query set in the same ratio as it is in the database. The overall AUC for different normalizations and different methods are shown in Table 5.2. It can be seen that normalization `norm2` is the best normalization function for the tableau search methods, and that SHEBA (Jung and Lee, 2000) is the best performing method, followed by our method, then VAST (Madej *et al.*, 1995; Gibrat *et al.*, 1996), and lastly TableauSearch (Konagurthu *et al.*, 2008) and TOPS. Figure 5.11 shows the ROC curves for the different methods (using the best normalization function where appropriate). It can be seen that the curve for SHEBA lies above that for VAST at every point, consistent with the results for all-against-all

Method	Normalization	AUC
SHEBA	None	0.941
QP Tableau Search	norm2	0.925
QP Tableau Search	norm1	0.904
QP Tableau Search	norm3	0.904
VAST	None	0.890
TableauSearch	norm2	0.871
TOPS	None	0.871
TableauSearch	norm1	0.869
QP Tableau Search	None	0.854
TableauSearch	None	0.846
TableauSearch	norm3	0.832

TABLE 5.2: *AUC for the 200 query set.*

Area under ROC curve (AUC) for the 200 query set against the ASTRAL SCOP 95% sequence identity non-redundant database consisting of 15273 domains, for different methods and normalizations. Note that only the tableau search methods (our method (QP Tableau Search) and TableauSearch) require normalization, as VAST, SHEBA and TOPS report normalized scores themselves and so do not require an external normalization step to combine results for different queries. The table is ordered by decreasing AUC.

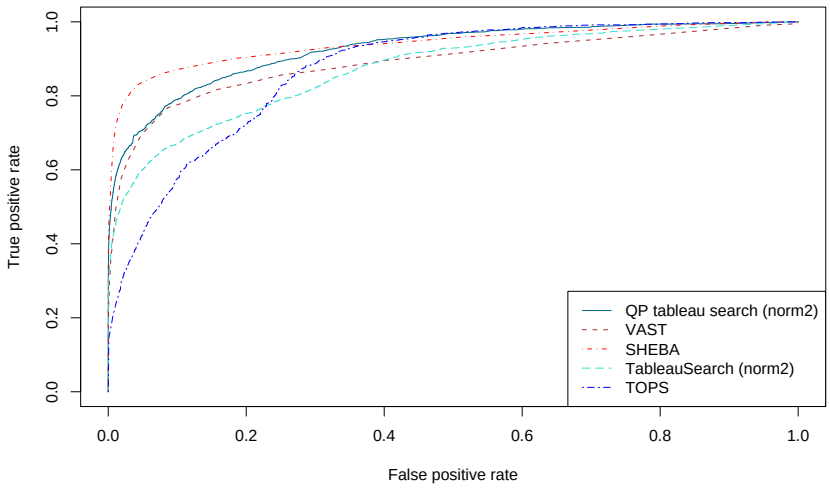


FIGURE 5.11: *ROC curves for different methods.*

ROC curves for the 200 query set against the ASTRAL SCOP 95% sequence identity non-redundant database consisting of 15273 domains.

comparisons in the ASTRAL SCOP 1.63 40% sequence identity non-redundant data set reported in Sam *et al.* (2006). The curve for our method lies between the two at low false positive rates, but crosses over the SHEBA curve at a false positive rate of approximately 0.4, indicating it has a slightly higher sensitivity at high false positive rates than SHEBA. SHEBA, however, is more sensitive at low false positive rates, a generally more useful attribute.

In terms of elapsed time (for a single processor core), TableauSearch is by far the fastest method. On our system, it has a total elapsed time for the 200 query set of only 1 hour 25 minutes, compared to 28 hours for VAST, 52 hours for SHEBA, and 741 hours for our method. Large scale comparisons with the exact solution of the QIP or ILP with CPLEX are not practical, as a single comparison takes at least several seconds and can take up to several days, and in some cases exhausts the virtual memory of our machine.

5.3.1 Comparison with MAX-CMO heuristic

Maximum Contact Map Overlap (MAX-CMO) is a formulation of the problem of finding the similarity of two protein structures. MAX-CMO uses the contact map representation of proteins, in which a protein with n residues is represented as a square symmetric matrix $C_{n\times n}$ where $C_{ij} = 1$ when the distance between residues i and j is less than some threshold, and $C_{ij} = 0$ otherwise. Typically this distance is defined as the C_α distance, and the threshold is for example 7Å. The MAX-CMO problem is then to find a (non-crossing) alignment of residues that maximizes the overlap between two contact maps. The value (or score) of the alignment is the number of contacts in one protein whose residues are aligned with residues that are also in contact in the other protein (Caprara *et al.*, 2004).

Recently, a variable neighbourhood search (VNS) algorithm for approximating MAX-CMO has been published, with an analysis of its effectiveness in ranking protein similarity (Pelta *et al.*, 2008). Here we compare the performance of the QP formulation of maximally-similar subtableaux extraction with the VNS heuristic for MAX-CMO of Pelta *et al.* (2008).

Figure 5.12 shows the ROC curves for the Fischer data set (Fischer *et al.*, 1996) at the fold level, and Table 5.3 shows the corresponding AUC values. Figure 5.13 and Table 5.4 show the corresponding results at the class level. It is apparent that

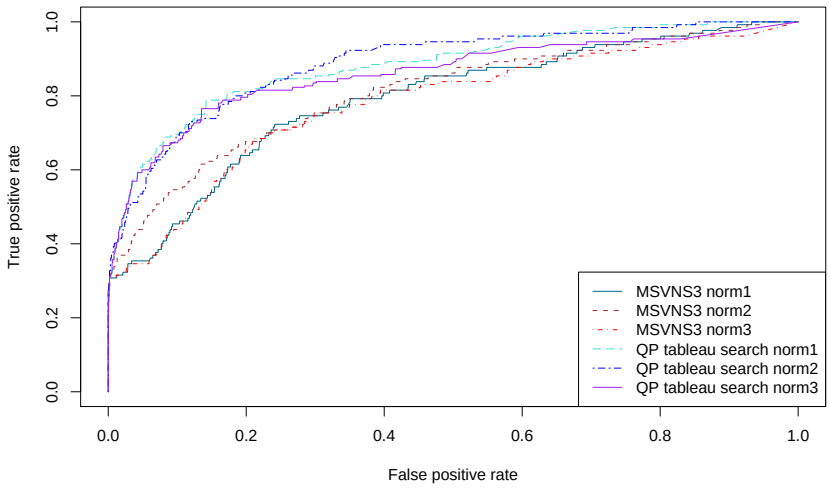


FIGURE 5.12: *ROC curves for MSVNS3 and QP Tableau Search with different normalization functions on the Fischer data set at fold level.*

Method	Normalization	AUC	standard error	95% confidence interval lower	upper
MSVNS3	None	0.788	0.017	0.754	0.821
MSVNS3	norm1	0.791	0.017	0.758	0.824
MSVNS3	norm2	0.809	0.016	0.777	0.842
MSVNS3	norm3	0.781	0.017	0.747	0.815
QP Tableau Search	None	0.837	0.016	0.807	0.868
QP Tableau Search	norm1	0.882	0.014	0.855	0.909
QP Tableau Search	norm2	**0.887**	0.014	0.861	0.914
QP Tableau Search	norm3	0.860	0.015	0.831	0.889

TABLE 5.3: *Area under the ROC curve (AUC) for the Fischer data set at fold level.*

Method	Normalization	AUC	standard error	95% confidence interval lower	upper
MSVNS3	None	0.666	0.009	0.647	0.684
MSVNS3	norm1	0.604	0.010	0.586	0.623
MSVNS3	norm2	0.696	0.009	0.678	0.714
MSVNS3	norm3	0.628	0.010	0.610	0.647
QP Tableau Search	None	0.789	0.008	0.773	0.805
QP Tableau Search	norm1	0.833	0.008	0.819	0.848
QP Tableau Search	norm2	**0.851**	0.007	0.837	0.865
QP Tableau Search	norm3	0.824	0.008	0.809	0.839

TABLE 5.4: *Area under the ROC curve (AUC) for the Fischer data set at class level.*

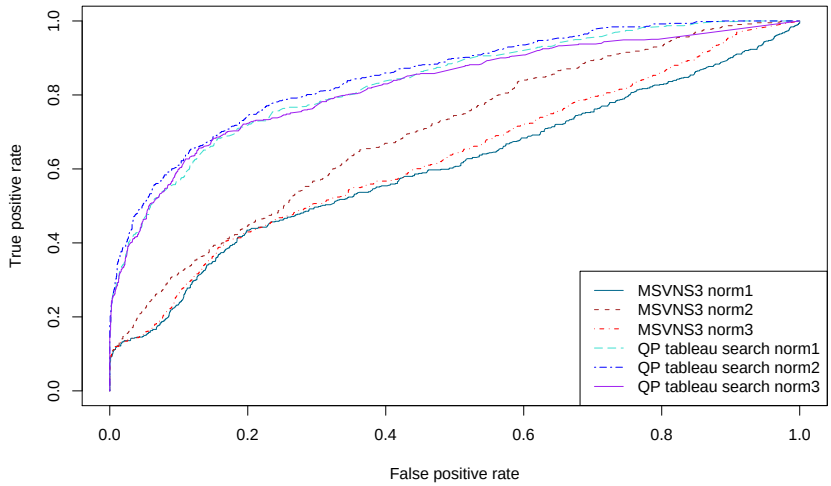


FIGURE 5.13: *ROC curves for MSVNS3 and QP Tableau Search with different normalization functions on the Fischer data set at class level.*

Method	Normalization	AUC	standard error	95% confidence interval lower	upper
MSVNS3	None	0.537	0.005	0.528	0.547
MSVNS3	norm1	**0.617**	0.005	0.607	0.627
MSVNS3	norm2	0.583	0.005	0.573	0.593
MSVNS3	norm3	0.598	0.005	0.588	0.608
QP Tableau Search	None	0.578	0.005	0.568	0.588
QP Tableau Search	norm1	**0.617**	0.005	0.607	0.626
QP Tableau Search	norm2	0.608	0.005	0.598	0.618
QP Tableau Search	norm3	0.599	0.005	0.589	0.608

TABLE 5.5: *Area under the ROC curve (AUC) for the Nh3D data set at architecture level.*

for the Fischer data set, the QP Tableau Search method achieves significantly higher accuracy at both levels than the MSVNS3 method, regardless of normalization type.

Figure 5.14 shows the ROC curves for queries in the Nh3D data set (Thiruv *et al.*, 2005; Pelta *et al.*, 2008) at the architecture level, and Table 5.5 shows the corresponding AUC values. Figure 5.15 and Table 5.6 show the corresponding results at the class level. At the architecture level, there is no significant difference in the performance of the two methods, but at the class level again QP Tableau Search has significantly higher accuracy when measured by AUC.

We should perhaps discount any superiority in the performance of the tableau search method at the class level, as this level of classification in the CATH hierarchy indicates only the percentage of α-helices and β-strands in the domain (Pearl *et al.*,

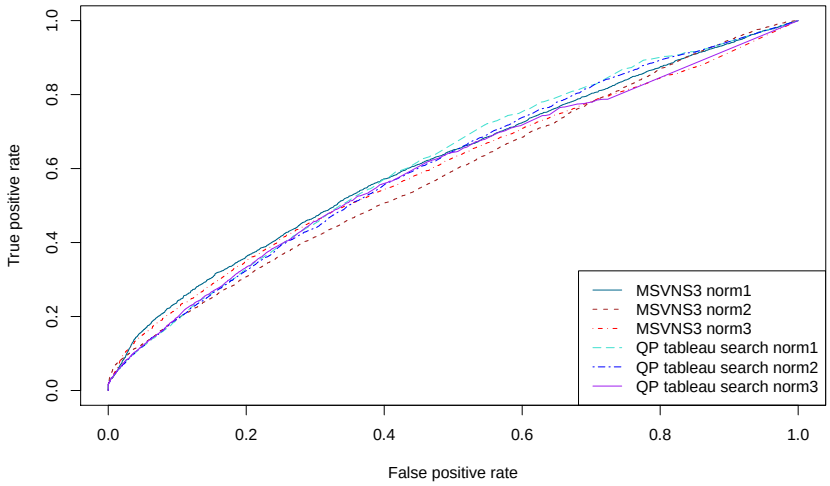


FIGURE 5.14: *ROC curves for MSVNS3 and QP Tableau Search with different normalization functions on the Nh3D data set at architecture level.*

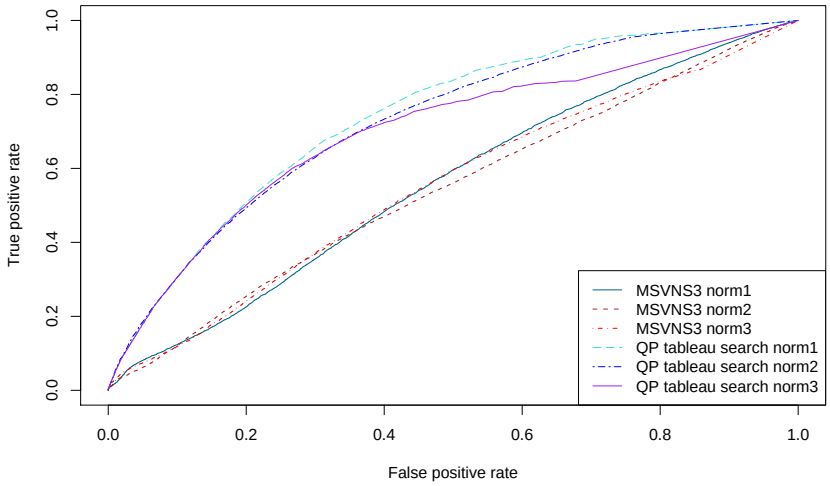


FIGURE 5.15: *ROC curves for MSVNS3 and QP Tableau Search with different normalization functions on the Nh3D data set at class level.*

Method	Normalization	AUC	standard error	95% confidence interval lower	upper
MSVNS3	None	0.590	0.003	0.585	0.595
MSVNS3	norm1	0.559	0.003	0.554	0.564
MSVNS3	norm2	0.543	0.003	0.538	0.548
MSVNS3	norm3	0.551	0.003	0.546	0.557
QP Tableau Search	None	0.708	0.002	0.703	0.712
QP Tableau Search	norm1	**0.740**	0.002	0.735	0.744
QP Tableau Search	norm2	0.726	0.002	0.722	0.731
QP Tableau Search	norm3	0.700	0.002	0.695	0.704

TABLE 5.6: *Area under the ROC curve (AUC) for the Nh3D data set at class level.*

2005). Since tableaux are based on SSEs (defined by DSSP) we could trivially obtain good classification performance at this level just from the DSSP classification, while MAX-CMO uses only residue contact information, and so must score protein similarity at this high level without having SSEs defined by an existing method.

Ignoring the class level comparisons therefore, we find that QP Tableau Search has significantly superior accuracy compared to MSVNS3 on the Fischer data set, and similar accuracy to MSVNS3 on the Nh3D data set.

For the Fischer data set, MSVNS3 took 8 hours while the sparse matrix (UMFPACK (Davis and Duff, 1997, 1999; Davis, 2004a,b)) implementation of QP Tableau Search took 2 hours on a PC with an Intel Core 2 Duo processor and 2 GB of memory running 32-bit Linux. For the Nh3D data set, MSVNS3 took 62 hours while QP Tableau Search took 8 hours.

5.3.2 Examples

Figure 5.16(a) shows the superposition of the top 20 hits from the β-grasp query on the query structure (`d1ubia_`), showing that these high scoring matches are correctly matching corresponding SSEs in similar structures. This is not a multiple alignment, but simply a superposition of each structure onto the query structure according to the SSEs matched by our method. Figure 5.16(b) is a true multiple alignment of the top 20 hits from our method, generated with MUSTANG (Konagurthu *et al.*, 2006).

Substructure queries need not be entire structures themselves. In order to illustrate the ability of our method to find substructural matches, we chose the B/C sheet of the canonical active serpin, α_1-antitrypsin, PDB id 1QLP (Elliott *et al.*, 2000) as a query tableau. Figure 5.17 shows the substructure represented by this query.

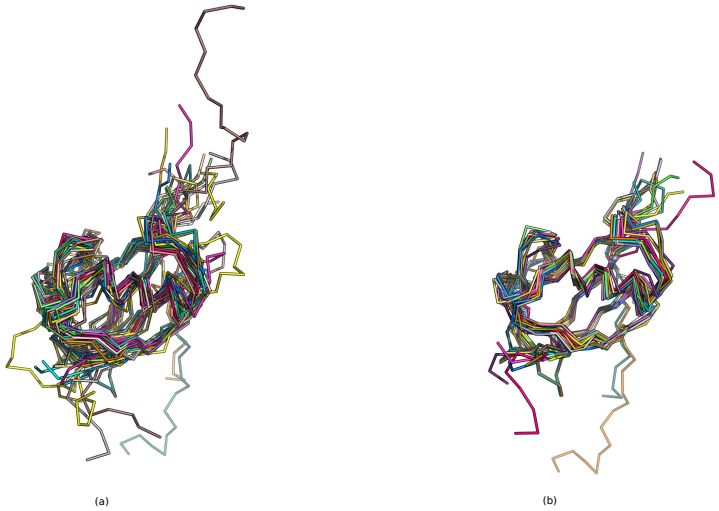


FIGURE 5.16: *Superposition of β-grasp query hits.*

(a) Superposition of the top 20 hits to the β-grasp query structure `d1ubia_`*. This superposition is generated by simply finding the orthogonal transformation that minimizes the RMSD between the C_α atoms in the most central residues of the SSEs that are matched between the query and the hit structure. (b) Multiple alignment of the top 20 hits generated with MUSTANG (Konagurthu* et al.*, 2006). Five structures are missing as they have multiple NMR models, which MUSTANG cannot currently handle. Figures were generated with PyMOL.*

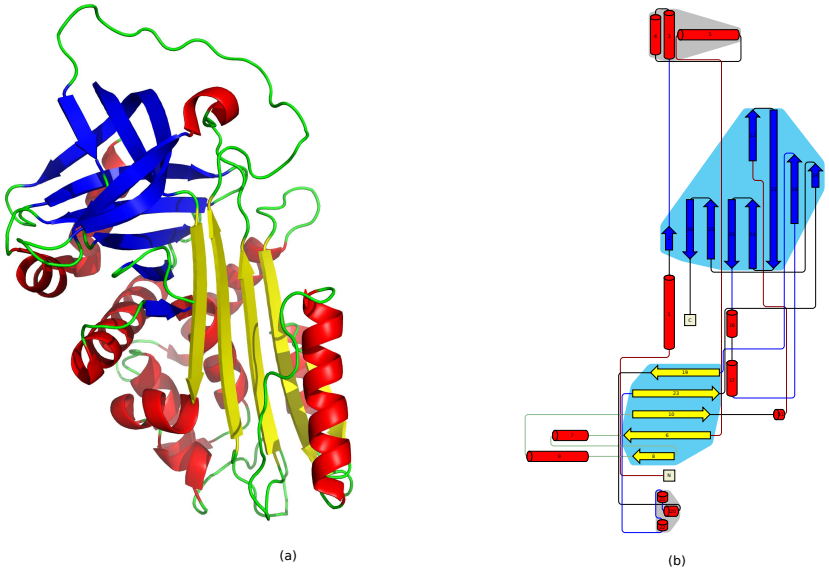


FIGURE 5.17: *Serpin B/C sheet query substructure.*

(a) 3D structure of the canonical active serpin, α_1-antitrypsin, PDB id 1QLP. Figure generated with PyMOL. (b) Topology cartoon of 1QLP. In both cartoons, the B/C sheet used as the substructure query is coloured blue.

Of the total 18 structures in our database classified by SCOP as belonging to the serpin fold, 17 are matched as the top 17 hits (`d1qlpa_` itself is the top hit). One, however, `d1m93.1`, is only at rank 1411 in the sorted hit list. This is the cleaved form of the serpin, but this does not account for the failure to detect its similarity, as, for example, `d1jjo.1` is also the cleaved form and it is at rank 11 in the sorted hit list. Inspection of the tableaux for the B/C sheets of these two serpins shows that while that for `d1jjo.1` is visibly similar to the query tableau, the relative angles of the strands in the B/C sheet of `d1m93.1` are different enough from the query that the tableau is no longer sufficiently similar to find the correct matching of the two sheets. Using the entire serpin structure `d1qlpa_` as the query, however, results in all 18 serpins as the top 18 hits.

Figure 5.18 shows the SSEs found as corresponding between the query substructure and a serpin from a thermophilic prokaryote, SCOP identifier `d1mtp.1`, at rank 7 in the sorted hit list. This demonstrates that the β-strands in the B/C sheet have been correctly identified by our method. Figure 5.19 shows superpositions and multiple alignments of the top 10 hits, showing that the B/C sheet has been correctly identified by our method in these serpins.

5.3.3 Substructure search

Evaluated as a substructure (motif) query, the β-grasp query (`d1ubia_`) using the discrete tableau encoding has an AUC of 0.94. Since the data set used as the gold standard in this case is that defined by ProSMoS (Shi *et al.*, 2007), that method by definition has an AUC of 1.00 on this query.

Table 5.7 shows the results of using the eight query structures to query the ASTRAL SCOP 1.73 95% sequence identity non-redundant database with ProSMoS, SSM (Krissinel and Henrick, 2004), TOPS and our method. In all methods except ProSMoS, the exemplar SCOP structure is used directly as the query. ProSMoS, however, requires a manually edited query meta-matrix. We found that the query meta-matrices produced by the scripts included with ProSMoS applied to the query structures resulted in no hits, even when extensively edited to make them less specific, and so we used manually constructed meta-matrices instead. Therefore the ProSMoS results reflect not only the performance of ProSMoS, but also our construction of the relevant query matrices.

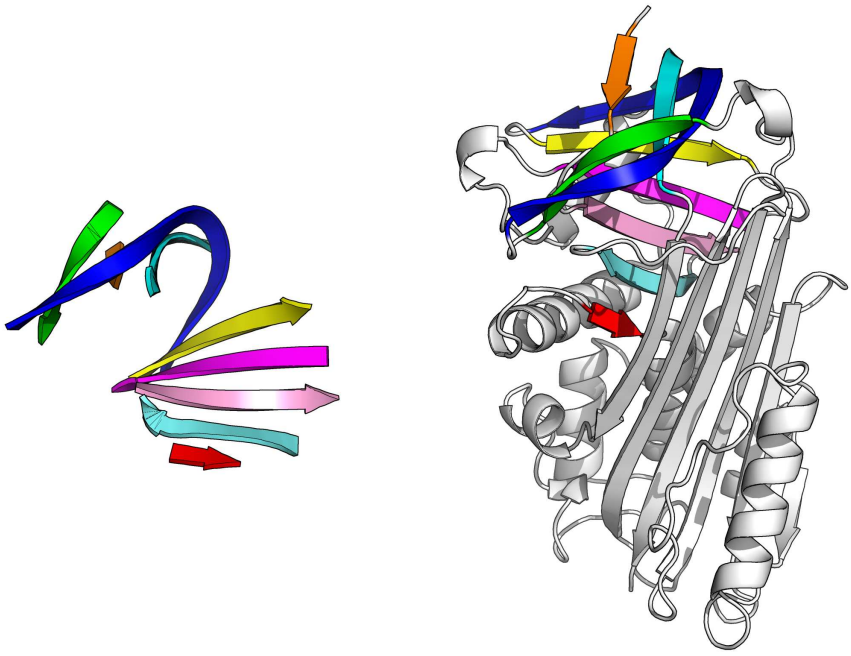


FIGURE 5.18: *The serpin B/C sheet query substructure found in a serpin from a thermophilic prokaryote.*

(a) 3D cartoon of the strands in the serpin B/C sheet from PDB identifier 1QLP used as the query structure. (b) 3D cartoon of a serpin from a thermophilic prokaryote, SCOP identifier `d1mtp.1`*, one of the top 10 hits to this query structure. SSEs from each structure that are found to be corresponding according to our method have the same colour. Both cartoons were generated with PyMOL.*

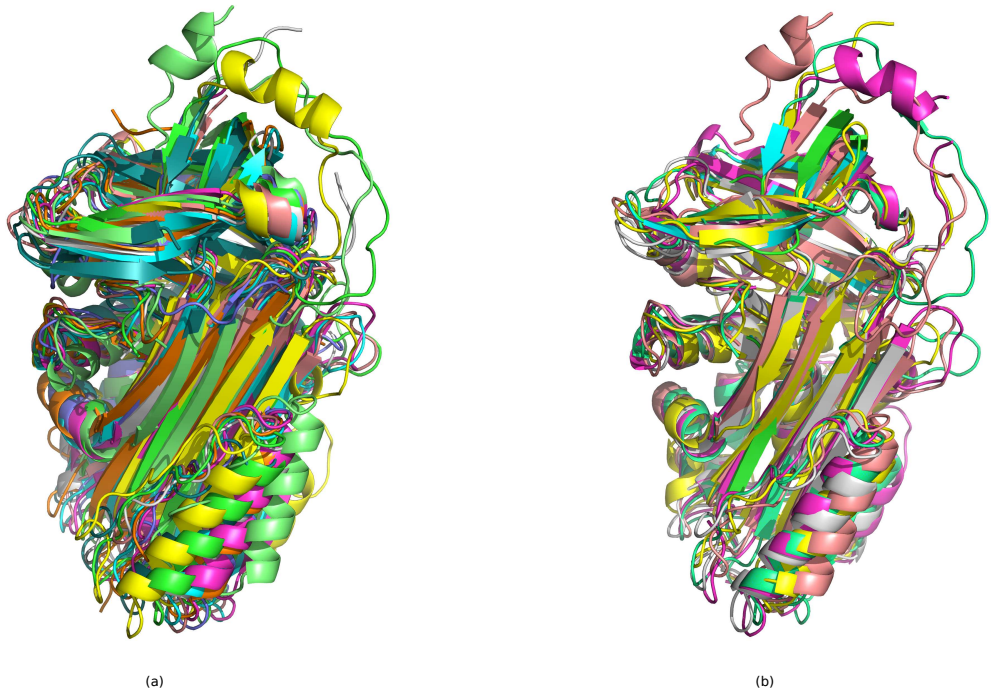


FIGURE 5.19: *Superposition of serpin B/C sheet query hits.*

(a) Superposition of the top 10 hits to the serpin B/C sheet query substructure, generated by simple orthogonal transformation to minimize RMSD between C_α atoms of the central residue in SSEs that are matched to each other between the query and the hit structure according to our method. (b) Multiple alignment of 5 of the top 10 hits generated by MUSTANG. Five are missing as they contain multiple chains which MUSTANG currently cannot process. Both images were generated with PyMOL.

Fold	SCOP sid	P	S	T	Q	R	P/R	S/R	T/R	Q/R
β-grasp	d1ubia_	33	9	42	14	15	15/15	9/15	13/15	10/15
Key-barrel	d1tttb1	10	3	0	17	5	5/5	1/5	0/5	4/5
Immunoglobulin	d1ae6h1	27	1	4	1	11	9/11	1/11	4/11	1/11
Plait (ferredoxin)	d1bhne_	20	1	61	14	28	7/28	1/28	24/28	6/28
GFP-like	d1h6rb_	1	1	58	21	1	1/1	1/1	1/1	1/1
Jelly-roll	d2phlb1	1	1	19	15	12	1/12	1/12	10/12	5/12
TIM-barrel	d1tima_	16	16	40	33	32	16/32	16/32	28/32	32/32
NAD-binding fold	d1f6dc_	1	1	42	19	8	1/8	1/8	7/8	5/8

TABLE 5.7: *Comparison of methods for substructure searching.*

The number of superfamilies in the ASTRAL SCOP 95% sequence identity nonredundant database found for the eight query structures by ProSMoS (P), SSM (S), TOPS (T) and our method, QP Tableau Search (Q). ProSMoS and SSM return a list of significant hits, so the number of superfamilies found by these hits is shown. Other methods return scores for all database structures, and the number of superfamilies in the hits with a Z-score ≥ 3.0 are shown. The R column is the number of superfamilies, from the combined list of those found by all methods, in which the relevant query pattern or fold description is explicitly mentioned in the SCOP structure description. The ratio columns show the number of superfamilies in the R column found by the specified method.

We note that our results here differ significantly from those in Table 2 of Shi *et al.* (2007): our method of constructing this table is similar, but not identical, to that of Shi *et al.* (2007), we have used slightly different queries (with the exception of the β-grasp query, where we used the meta-matrix described in Shi *et al.* (2007)) and different versions of the software and a different database have been used. Consistently with Shi *et al.* (2007), SSM finds the least number of matches. In our results, however, ProSMoS does not always return the greatest number of matches: sometimes TOPS does, since we are using a version of TOPS that computes scores for all matches, rather than the pre-computed "classic" structure patterns.

TOPS also tends to have more false positives than ProSMoS or our method, that is, superfamilies found by TOPS that are not considered by the SCOP descriptions to contain the fold in question. This is consistent with TOPS being a purely topological method, which does not take account of other structural properties. Sometimes this also results in TOPS finding true positives which the other methods do not, for example when using the ferredoxin query, only TOPS finds the monooxygenase (hydroxylase) regulatory protein superfamily, d.137.1, which SCOP describes as having "some topological similarity to the ferredoxin-like fold". SCOP also notes in the family description for d.137.1.1 that "the solution structure determinations disagree in the relative orientations of two motifs", so topological similarity without taking into account more detailed structural similarity (specifically, SSE orientation, as used by our method and ProSMoS) is a more appropriate method to find matches to this structure, reflected in the relatively better performance of TOPS, and the previously discussed poor performance of our method on this query.

In order to better examine the unique properties of each method, Table 5.8 shows, for each method, the number of superfamilies found only by that method, and the number of these for which SCOP explicitly mentions the relevant query pattern. SSM finds no hits that the other methods do not, and TOPS usually finds the most. For the ferredoxin query, we can see that TOPS finds a large number of true positives (17) that the other methods do not; our method finds 3 and ProSMoS 1. However, on the TIM-barrel query, TOPS finds 11 unique hits, none of which are considered to contain the TIM-barrel motif according to SCOP, while our method finds 3 unique hits that all contain the TIM-barrel motif according to SCOP.

Fold	SCOP sid	Pu	Su	Tu	Qu	R	Pu/R	Su/R	Tu/R	Qu/R
β-grasp	d1ubia_	17	0	27	4	1	1/1	0/1	0/1	0/1
Key-barrel	d1tttb1	4	0	0	10	1	1/1	0/1	0/1	0/1
Immunoglobulin	d1ae6h1	25	0	2	0	9	7/9	0/9	2/9	0/9
Plait (ferredoxin)	d1bhne_	8	0	48	7	21	1/21	0/21	17/21	3/21
GFP-like	d1h6rb_	0	0	56	19	0	0/0	0/0	0/0	0/0
Jelly-roll	d2phlb1	0	0	15	11	9	0/9	0/9	7/9	2/9
TIM-barrel	d1tima_	0	0	11	3	3	0/3	0/3	0/3	3/3
NAD-binding fold	d1f6dc_	0	0	33	10	4	0/4	0/4	3/4	1/4

TABLE 5.8: *Comparison of the unique hits from each method for substructure searching.*

The number of superfamilies in the ASTRAL SCOP 95% sequence identity nonredundant database found by only that method for the eight query structures by ProSMoS (Pu), SSM (Su), TOPS (Tu) and our method, QP Tableau Search (Qu). The R column is the number of superfamilies, from the combined list of those found uniquely by each method, in which the relevant query pattern or fold description is explicitly mentioned in the SCOP structure description. The ratio columns show the number of superfamilies in the R column represented by the superfamilies found only by the specified method.

With the exception of SSM, we can see that each method finds some unique true matches that the others do not. As an example of true positive hits that only our method finds, consider the jelly-roll query. Only QP Tableau Search finds the viral protein domain superfamily b.19.1, and the membrane penetration protein μ1 superfamily e.35.1, both of which are described by SCOP as containing a jelly-roll motif.

An interesting example is the ferredoxin fold, where the performance of our method as a structural search method is relatively poor. However, as a substructure search technique, some true positives are found only by our method. Only QP Tableau Search finds the peptide methionine sulfoxide reductase superfamily d.58.28, the CcmK-like superfamily d.58.56, and the release factor superfamily e.38.1. The first two are members of the ferredoxin-like fold but d.58.28 is described by SCOP as having the common fold "elaborated with additional secondary structures". The release factor superfamily (e.38.1) is described by SCOP as having 4 domains, one of which is a ferredoxin-like fold.

It is important to note several caveats in interpreting Table 5.7 and Table 5.8. First, as already discussed, ProSMoS queries were manually constructed, which is not the case for the other methods. Second, ProSMoS and SSM return a set of hits for a query, whereas the other methods return a matching score between the query and every database structure. Hence, in order to construct the tables, a cutoff score needs to be chosen. The values in the tables are therefore very sensitive to the method used

Fold	SCOP sid	Average AUC
β-grasp	d1ubia_	0.84
Key-barrel	d1tttb1	0.90
Immunoglobulin	d1ae6h1	0.92
Plait (ferredoxin)	d1bhne_	0.65
GFP-like	d1h6rb_	1.00
Jelly-roll	d2phlb1	0.90
Tim-barrel	d1tima_	1.00
NAD-binding fold	d1f6dc_	0.99

TABLE 5.9: *AUC for non-sequential matchings averaged over five permutations of each of the fold query tableaux.*

to choose the cutoff score: we could find arbitrarily many superfamilies simply by decreasing the value at which a score is considered a hit. Third, as discussed in Shi *et al.* (2007), the lack of explicit mention of a structure in the SCOP description does not necessarily mean the structural motif is absent.

5.3.4 Non-sequential matchings

In order to verify the capability of our method to find non-sequential matchings when the SSE ordering constraint is disabled, we first use an artificial test. Five different random permutations of the eight tableaux (that is, unique random reorderings of the rows, and columns to preserve symmetry, of the tableaux) previously discussed and shown in Table 5.1 were generated. These were then used as queries, and the AUC for each calculated in the same way as for Table 5.1. The average AUC for each fold over its five permutations is shown in Table 5.9. From this table we can see that, despite the tableaux being permuted so that the SSEs in the query are no longer in the same sequence as in the database structures, the structures are still matched, albeit sometimes with a lower AUC than for the correctly ordered SSEs with the ordering constraint enabled.

As a demonstration of a real case of a non-sequential matching, we use as the query the hypothetical novel-fold protein TA0956 (PDB id 2JMK) (Koo *et al.*, 2007) which was recently found to have several non-sequential alignments (Guerler and Knapp, 2008). When ordering constraints are enabled, our method finds no significant matches in the ASTRAL SCOP 1.73 95% sequence identity non-redundant database. However, when ordering constraints are disabled, some high-scoring hits are found, in particular the top-scoring hit is to d1kb9b1, cytochrome bc_1 core

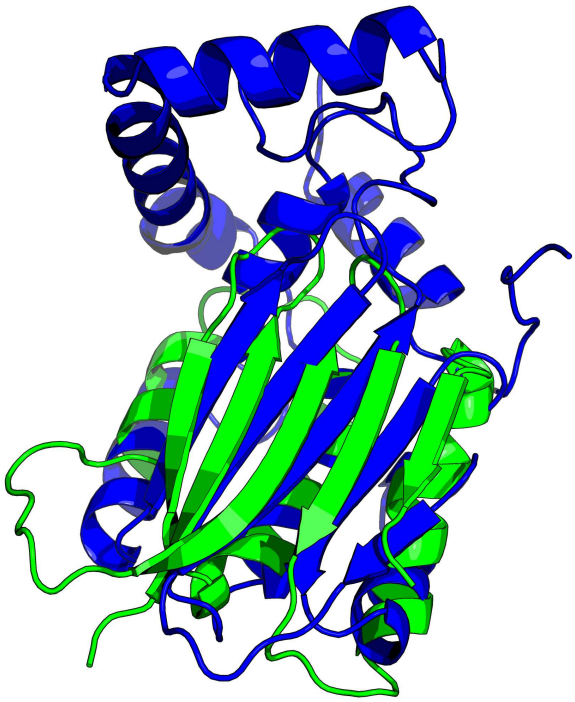


FIGURE 5.20: *Superposition of 2JMK and d1kb9b1_*

Superposition resulting from the non-sequential matching of the query structure 2JMK (green) to the top hit `d1kb9b1_` *(blue). The figure was generated with PyMOL.*

subunit 2. The superposition of these two structures is shown in Figure 5.20. This is different from the result reported by Guerler and Knapp (2008) using GANGSTA+ (Kolbeck *et al.*, 2006; Guerler and Knapp, 2008), who find that PDB identifier 1GO4 is the most similar structure (`d1go4a_` is at rank 244 in the sorted hit list with our method). Therefore, we used the GANGSTA+ web server[4] with 2JMK as the query, and found that after setting the selection criteria to require at least 10 matching SSEs, `d1kb9b1` is the fourth most significant hit, demonstrating that GANGSTA+ is in agreement with our method that this non-sequential matching is significant.

5.3.5 Performance improvements

All the results discussed to this point were with the implementation of the interior point solver using UMFPACK as the solver, as described in Section 5.2.5. Although the UMFPACK solver is primarily designed for unsymmetric matrices, it can also use a symmetric pivoting strategy for symmetric matrices (Gould *et al.*, 2007, p. 17), and so is a reasonable choice for the interior point solver we re-implemented, which solves a sequence of (indefinite) sparse linear symmetric systems. As noted by Gould *et al.* (2007, p. 7), the need to solve such a sequence of systems, where the sparsity pattern remains fixed but the numerical values change, is quite common and hence solvers (such as UMFPACK) are often designed with separate analysis and factorization phases so that factorizations can be reused for matrices with different coefficients but the same sparsity structure. We take advantage of this feature in our implementation.

In seeking to improve the performance of our implementation, we referred to a paper giving a thorough evaluation of the performance of several sparse solvers for symmetric linear systems (Gould *et al.*, 2007). This evaluation finds that the MA57 (Duff, 2004; HSL, 2007) and PARDISO (Schenk and Gärtner, 2004, 2006) solvers offer significantly better performance on indefinite systems than the other solvers (including UMFPACK) evaluated (Gould *et al.*, 2007, pp. 25–28). PARDISO is included in the Intel Math Kernel Library (MKL) (version 10.1),[5] but is also available free for academic use.[6] We used the Intel MKL (10.1) version. We used the nested dissection algorithm implemented in the METIS package (Karypis and

[4] `http://gangsta.chemie.fu-berlin.de`
[5] `http://software.intel.com/en-us/intel-mkl/`
[6] `http://www.pardiso-project.org/`

Method	min. SSE length	AUC	time	speedup
UMFPACK	1	0.92	293 h 19 m 38 s	1.00
PARDISO	1	0.92	190 h 42 m 39 s	1.54
MA57	1	0.91	157 h 50 m 58 s	1.86
UMFPACK	4	0.89	68 h 53 m 30 s	4.26

TABLE 5.10: *AUC, elapsed time and speedup relative to the UMFPACK implementation for the 200 query set with different implementations.*

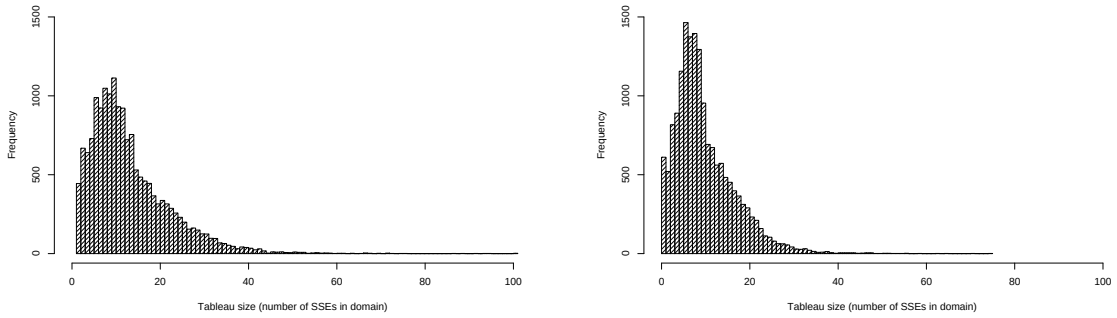
Area under ROC curve (AUC) and times for the 200 query set against the ASTRAL SCOP version 1.75 95% sequence identity non-redundant database consisting of 16712 domains, with different implementations of the QP Tableau Search program. All results used the norm2 normalization. "UMFPACK", "MA57", and "PARDISO" are the interior point solver using the UMFPACK, MA57, and PARDISO sparse linear solvers, respectively. Hence the first row is the implementation discussed previously, forming the baseline result for this comparison. When the "min. SSE length" is greater than 1, this means that the tableau for each structure only includes SSEs with at least that many residues (in previous results this was always 1, that is, all SSEs were included). Note also that these results are also on a newer release of ASTRAL, and are run on a different (faster) system; hence only direct comparisons within this table, not with other tables in this chapter, are valid. These tests were run on a single core of an AMD Quad Core Opteron (2.3GHz, 2 processors, total 8 cores, 32 GB RAM) running Linux. All programs were compiled with the Intel Fortran compiler (version 11.1). The table is ordered by decreasing elapsed time.

Kumar, 1998) for the symmetric fill-in reducing ordering in both solvers. Note that, although PARDISO is a parallel solver, we configured it to use only a single thread, so that all results presented here are for a single processor core.

Table 5.10 shows that, for the same accuracy (measured by AUC), the use of PARDISO gives a 1.54 times speedup over the UMFPACK implementation, and MA57 a 1.86 times speedup, but with slightly reduced accuracy in this case.

Minimum SSE length restriction

A greater improvement in speed, but at a cost to accuracy, can be achieved by reducing the sizes of the tableaux (and distance matrices) that must be matched. If we make the assumption that SSEs with fewer than some number of residues are "not significant" in matchings, then we can remove any such SSEs from the tableaux and distance matrices, reducing their dimensions (recall that for n SSEs the tableau and distance matrix are both $n \times n$ matrices). Figure 5.21 shows the effect on tableaux size of removing all SSEs with fewer than four residues. Doing so reduces the maximum tableau dimension from 101 to 60, and the median from 11 to 9. When all SSEs are included, 75% of the domains have 20 or fewer SSEs, and when only SSEs with four or more residues are included, 90% of the domains have 20 or fewer SSEs, and 75% have 15 or fewer.



(a) All SSEs included.

(b) Only SSEs containing at least 4 residues included.

FIGURE 5.21: *Histograms of tableau sizes for ASTRAL SCOP version 1.75 95% sequence identity non-redundant database.*

Table 5.10 shows that restricting the minimum SSE size to four residues gives a 4.26 times speedup over using all SSEs, but at some cost to accuracy compared to using the same solver with no SSE size restriction.

5.4 Conclusions

We have introduced an improved method of searching for protein structures with similar folds using tableaux, incorporating constraints on the distances between SSEs to improve accuracy. This method is capable of finding either matches of an entire structure to the query, or matches where the query is a substructure of a larger structure. It is capable of finding non-sequential matchings, where structurally equivalent parts do not have the same relative positions in the sequences of the two proteins. It also provides a set of corresponding SSEs, useful for manual validation of the result or as a seed for a more detailed structural alignment.

In assessing their VNS heuristic for MAX-CMO, Pelta *et al.* (2008) ask whether it is necessary to solve MAX-CMO exactly in order to perform structure classification, and conclude that it is not: the heuristic solution is sufficient. We have shown that, consistent with previous work using the tableau representation of protein folds (Kamat and Lesk, 2007; Konagurthu *et al.*, 2008), the much more coarse-grained (and hence smaller and faster to solve) tableau representation is sufficient to accurately represent protein folds and perform structure classification. Specifically, we have shown that the efficient approximation of maximally-similar subtableaux extraction

by relaxed quadratic programming is able to consistently classify folds at least as accurately as the VNS heuristic for MAX-CMO. In addition, our implementation is able to do so in less time than the MSVNS3 implementation described by Pelta *et al.* (2008).

We have demonstrated that the accuracy of our technique assessed as a protein fold recognition method compares favourably with some existing methods, and that it is fast enough to scan protein structure databases in a practical time, unlike the exact solution using CPLEX. It is, however, not as fast as some existing methods such as SHEBA and VAST, and the TableauSearch dynamic programming approximation introduced by Konagurthu *et al.* (2008) and the IR Tableau feature vector based approach (Zhang *et al.*, 2010) are faster still. These methods, however, cannot be used to find substructures or non-sequential matchings.

We have also demonstrated the use of our technique as a method for searching for substructures in protein structures, and compared it with some existing techniques, including ProSMoS. Complications in objectively assessing the performance of these methods make definite conclusions in this area difficult: we can perhaps say at most that each method has different enough properties that they are all capable of finding unique hits that others miss. A structural biologist searching for matches to a motif or substructure, then, would do well to employ several of these methods rather than relying on just one.

As noted by Li *et al.* (2007), further theoretical work to find tight sufficient conditions for the QP to have an integer solution is required, although empirically an integer solution is almost always found.

In the next chapter we describe another method for tableau-based protein substructure searching that retains the useful properties of the method described in this chapter, and, when run on commonly available graphics processing unit hardware, is one of the fastest available methods.

6

Accelerating Protein Substructure Searching

In the previous chapter, we described a technique for protein structural and substructural searching based on relaxation of the QIP for the tableau matching problem to a QP. One drawback of this computationally intensive method is that, while, unlike many existing techniques, it is capable of substructure and non-sequential searching, it is slower than many existing methods for (full) structure matching or fold classification. Using different sparse linear solvers results in only relatively modest speedups, and restricting the minimum length of SSEs considered, while resulting in greater speedups, leads to a significant drop in accuracy.

In this chapter, we describe SA Tableau Search, a new heuristic for tableau-based protein structural and substructure searching based on *simulated annealing* (Kirkpatrick *et al.*, 1983). We compare its accuracy as a fold classification method with the existing methods DaliLite, SHEBA, VAST, YAKUSA, SARF2, LOCK2, TOPS, TableauSearch, IR Tableau, and QP Tableau Search (described in Chapter 5) on three different data sets: a set of 200 queries in the ASTRAL SCOP 1.75 95% sequence identity non-redundant database (Chandonia *et al.*, 2004; Andreeva *et al.*, 2008), all-against-all queries in the Fischer data set (Fischer *et al.*, 1996), and the COPS benchmark (Frank *et al.*, 2010). Because SA Tableau Search has a parameter, the number of *restarts* of its simulated annealing schedule, that can be adjusted as a tradeoff between speed and accuracy, we perform these comparisons with a number of different values of this parameter. Furthermore, we show an implementation on modern *GPU* (graphics programming unit) hardware that achieves up to 34 times

speedup over the CPU implementation of the same algorithm, making it one of the fastest available methods.

Graphics processing units (*GPU*s) have recently been used for several bioinformatics applications, such as sequence alignment (Schatz *et al.*, 2007; Manavski and Valle, 2008; Trapnell and Schatz, 2009; Liu *et al.*, 2009), molecular dynamics (Roberts *et al.*, 2009; Friedrichs *et al.*, 2009), microarray data analysis (Buckner *et al.*, 2010), mass spectrometry data analysis (Hussong *et al.*, 2009), and phylogenetics (Suchard and Rambaut, 2009). However, despite the large variety of protein structural search methods and their computationally intensive nature, to the best of our knowledge this is the first use of GPUs to accelerate protein structural or substructural searching.

6.1 Methods

We use the well-known and relatively simple technique of simulated annealing, whereby the system has a global "temperature" that controls the probability of a state change that does not improve the current value of the objective value being accepted, in order to help prevent the system from becoming trapped in a local maximum. The temperature is decreased as the simulation progresses, so that such non-improving moves become less likely over time.

6.1.1 CUDA

Modern graphics processing units (GPUs), often referred to as general-purpose GPUs (GPGPUs) in the context of applications other than graphics rendering, are highly parallel multithreaded processors, which can greatly accelerate data-parallel applications. We use here the NVIDIA CUDA programming model (NVIDIA, 2009b), which provides extensions to the C programming language in order to make use of NVIDIA GPUs. The CUDA model allows C functions called *kernels* to run in parallel on the GPU as *threads*. The threads are arranged in a hierarchy such that a fixed number of threads makes up a *block* of threads, and in turn a fixed number of blocks makes up a *grid*. Each block has a (small) amount of shared memory (e.g. 16 KB on the CUDA devices used here), accessible by threads in that block only, which is as fast as register access (that is, in general it introduces no additional

memory latency). In contrast, the large (e.g. 1 GB on the NVIDIA GeForce GTX 285) global memory of the GPU, accessible by all threads, has very high latency (400 to 600 clock cycles). The *local* memory for each thread, not shared by any other, also has this high latency. There is also a small constant memory (e.g. 64 KB on the NVIDIA GeForce GTX 285), which all threads can read without incurring the high memory latency of global or local memory. It is therefore important to optimize the memory access pattern of kernels, and make the best use of the limited shared memory (NVIDIA, 2009b,a). This architecture is illustrated in Figure 6.1.

Since the GPU is essentially a data-parallel architecture (NVIDIA, 2009b), efficient implementations require that essentially the same code path is run simultaneously in a large number of threads, just with different data. The more threads diverge in their code path, the less efficient the parallelization will be. This is a somewhat more complex model than traditional *SIMD* (single-instruction multiple-data), or vector processing, where a single instruction operates on multiple words simultaneously. NVIDIA refers to its GPU architecture as *SIMT* (single-instruction multiple-thread) (NVIDIA, 2009b). In this architecture each thread executes independently with its own instruction address and register state. Threads are grouped together in *warps*, which start at the same address but are subsequently able to branch and execute independently. However, when threads in a warp diverge, each branch path must be serially executed until all execution paths are complete and the threads converge to the same execution path again. It is for this reason that maximum efficiency is achieved by ensuring that threads diverge in their code path as little as possible.

6.1.2 Application of simulated annealing to the problem

As in QP Tableau Search, each structure in the database to be searched, and each query, is represented by its tableau and distance matrix. These tableaux and distance matrices are pre-generated.

We now describe the application of simulated annealing to the problem of extracting maximally-similar subtableaux. We will find a (local) maximum of the objective function (Equation 5.7) subject to the same constraints (Equations 5.8–5.12) described in Chapter 5. We use the same notation here as we did in that chapter.

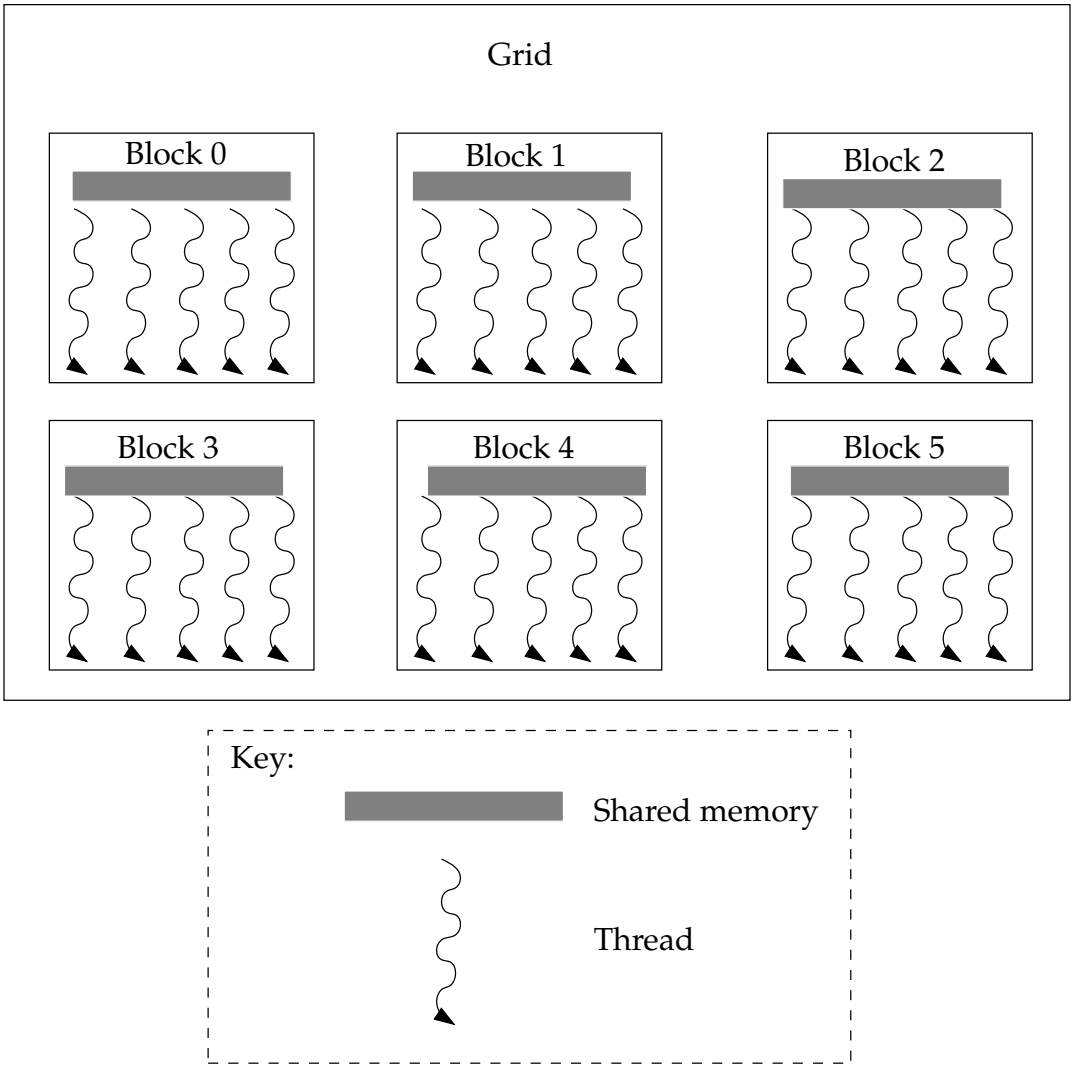


FIGURE 6.1: *The CUDA thread programming model.*

The grid consists of a set of blocks. Each block consists of a set of threads. Each block has its own shared memory, accessible by the threads in that block only. The global memory is shared by all threads in all blocks; each thread also has its own local memory. On the NVIDIA GeForce GTX 285, the shared memory size is 16 KB per block and the global memory size is 1 GB. Access to the global memory has a 400 to 600 clock cycle latency; this latency is not incurred in accesses to the shared memory.

The state (or configuration) of the system for matching structure A with N_A SSEs and structure B with N_B SSEs, represented by tableaux T_A and T_B, respectively, is represented by a vector v of dimension N_A, where $v_i = j$, $1 \leq i \leq N_A$, $0 \leq j \leq N_B$ indicates that the ith SSE in structure A is matched with the jth SSE in structure B, or, if $v_i = 0$, that the ith SSE in A is not matched with any SSE in B. The nonzero elements of v, regarded as a set, are constrained to be a subset of $\{1, \ldots, N_B\}$, so that each SSE in a structure is matched with at most one SSE in the other (achieved by constraints (5.8) and (5.9) in the QIP formulation). We can (optionally) forbid non-sequential matchings by ensuring that the nonzero elements of v considered as a sequence are strictly increasing, that is, $i < k \Rightarrow v_k = 0 \vee v_i < v_k$, $1 \leq i, k \leq N_A$.

Then, also incorporating the distance difference constraint (Equation 5.12) the objective function $g(v)$, that we seek to maximize, becomes

$$g(v) = \sum_{\substack{1 \leq i,k \leq N_A \\ 1 \leq j,l \leq N_B}} \begin{cases} \zeta(t^A_{ik}, t^B_{jl}), & \text{if } v_i = j \wedge v_k = l \quad \wedge \\ & |d^A_{ik} - d^B_{jl}| \leq \tau \\ 0, & \text{otherwise.} \end{cases} \tag{6.1}$$

Note that this can then be computed efficiently, in $O(N_A^2)$ time, by a nested iteration over the v vector, since the summand is nonzero only when $v_i = j \wedge v_k = l$, that is, we need only consider values of j and l that are actually present in the v vector.

We set a random initial state of the system by matching, with probability p_m, each SSE in structure A with the first SSE of the same type (helix or strand) in structure B, in sequence from $1 \ldots N_A$ and $1 \ldots N_B$.

At each iteration, the "move" to a neighbour state is generated by choosing uniformly at random an SSE, i, in structure A and changing its mapping to a random SSE, j, in structure B, from the set of such SSEs that satisfy the type and (optionally) ordering constraints on the mapping, giving a new state v'. If no SSE in structure B that meets the constraints can be found, the SSE that was chosen is removed from the mapping, that is we set $v'_i = 0$.

The simulated annealing algorithm proceeds by efficiently computing the new value of the objective function (Equation 6.1) and accepting the new state if the new value of the objective $g(v')$ is greater than the maximum so far found, or if $\exp\left(\frac{g(v')-g(v)}{T}\right) > p$ where p is a random number in $[0, 1]$ and T is the current

temperature of the system. The temperature is multiplied by the constant α for the next iteration. After the maximum number of iterations is reached, the state with the maximum value of the objective function found is returned as the best state.

The recomputation for a new state of the value of the objective function is computed in only $O(N_A)$ time by computing the difference in the value caused by the new SSE matching, rather than directly recomputing Equation 6.1. That is, the contribution to the score of the previous matching of the SSE i that was chosen to be changed is subtracted, and the contribution of its new mapping (or 0, if it is removed from the mapping) is added.

We optimize the simulated annealing parameters by using the eight folds in Table 5.1 as queries against the ASTRAL SCOP 1.75 95% sequence identity non-redundant database as a training set, manually adjusting them to maximize the average AUC of these queries. We find that suitable values for these parameters are initial temperature $T_0 = 10$, temperature multiplier $\alpha = 0.95$, number of iterations 100, and initial SSE matching probability $p_m = 0.5$. In addition, the entire simulated annealing process is run M times (by default, $M = 128$), and the best solution over all runs is returned. We use the same value $\tau = 4.0$Å for the distance difference threshold as in Chapter 5.

6.1.3 Parallel implementation on a GPU

As well as its simplicity, another advantage of a simulated annealing heuristic such as that just described is that it is easily parallelizable, by running the M restarts of the simulated annealing schedule in parallel rather than serially. In addition, since in a structural database search the query is compared to many structures in the database, another level of parallelization is to compare the query to many database structures simultaneously. This two-level parallelization maps well to the CUDA programming model: each thread in a block of B threads runs the simulated annealing schedule (each from an independent random initialization) for a single comparison of the query to a database structure, and the grid of G blocks therefore executes G such comparisons simultaneously (that is, the query structure is compared to G database structures in parallel). If $M > B$ then the B parallel executions of the simulated annealing process are repeated until all M restarts of the simulated annealing schedule have been run.

The entire database of tableaux and distance matrices is loaded into the (large) global memory, and the query structure (tableau and distance matrix) is loaded into the constant memory where it can be quickly read by all threads. Each block of B threads first parallel copies (that is, each thread copies one or more memory locations in parallel) the database tableau and distance matrix into shared memory for fast access during the actual simulated annealing process. The G blocks in the grid perform comparison of the query against G database structures simultaneously, and so (for databases with more than G structures) this process is repeated until the whole database has been processed. By using the CUDA occupancy calculator (NVIDIA, 2009a, Ch. 4) and some experimentation, we determined good values for G and B of 128 (for both), that is, a total of 16384 threads.

Because the size of the constant and shared memory is so small (16 KB on the graphics cards we used), structures in which the combined size of the tableau and distance matrix exceed this limit present a problem. We solve this problem by compiling two versions of the kernel, one of which uses shared memory for database structures and constant memory for query structures, and the other which leaves them both in the global memory (the shared memory is still used for the vector of maximum scores found in each thread, and the vector of SSE types (helix, strand) copied from the main diagonal of the tableau). The latter is slower than the kernel using shared and constant memory, but removes any limitation on the size of structures that can be processed. A third version of the kernel is compiled for execution on the host CPU, running the same code as the kernel but without any GPU extensions and purely single-threaded.

An alternative solution that we experimented with was to process the structures too large for the shared memory on the host CPU simultaneously with the GPU processing the smaller structures, but the GPU is so much faster than the CPU that it is faster to run the two kernels in serial on the GPU, even though of 16602 tableaux and distance matrices in the ASTRAL SCOP 1.75 95% sequence identity non-redundant database, only 668 are too large for the shared memory.

Because the CUDA library does not contain an equivalent of the standard C library function `rand()` or similar, we used the CUDA SDK implementation (Podlozhnyuk, 2007) of the Mersenne Twister (Matsumoto and Nishimura, 1998) pseudorandom number generator. We used the dynamic creation program (Matsumoto

and Nishimura, 2000) to create parameters to enable 16384 different Mersenne Twister pseudorandom number generators (one for each of the maximum number of threads we use on the GPU). Because the use of double precision floating point causes a considerable performance penalty on the GPU (NVIDIA, 2009a), we use only single precision.

An additional technique we used to increase the speedup on the GPU was to pre-sort the tableau and distance matrix database by size, so that structures of similar size are processed simultaneously, thereby ensuring that the running times of blocks in the grid are as close as possible. A similar technique was used successfully by Manavski and Valle (2008) for their CUDA acceleration of the Smith-Waterman (Smith and Waterman, 1981) algorithm.

6.1.4 Evaluation

We computed tableaux and distance matrices for all 16712 (110 are omitted as DSSP (Kabsch and Sander, 1983) finds no SSEs for them, resulting in 16602 in our database) domains in the 95% sequence identity non-redundant subset of the ASTRAL SCOP 1.75 database (Chandonia *et al.*, 2004; Andreeva *et al.*, 2008).

We define a query set of 200 queries chosen from the ASTRAL SCOP 1.75 95% sequence identity non-redundant data set. The queries were chosen so that each of the SCOP classes α, β, α/β, $\alpha + \beta$ is represented in the query set in the same ratio as it is in the database. This set was based on that defined in Chapter 5 for ASTRAL SCOP 1.73; 14 structures in that set no longer present in ASTRAL SCOP 1.75 were replaced by others in the same superfamily.

The Fischer data set was described in Section 5.2.4 of Chapter 5.

The COPS Benchmark 2009/6 data set (Frank *et al.*, 2010) consists of a database of 1056 structures and a query set of 176 structures. The queries are not present in the database, and each query has exactly six true positives, structurally similar according to the COPS classification (Suhrer *et al.*, 2009), but lacking a high degree of sequence similarity.

Matches of a query against itself (where the query structures are also present in the database, as in the ASTRAL 95% 200 query set and the Fischer data set, but not in the COPS benchmark) are excluded from the results. Comparisons for which a

method can provide no score are assigned an arbitrary value which is at least as low as the lowest provided score for that method.

In the ASTRAL 95% 200 query set, we evaluated the accuracy of structural search by counting a hit (a score above the threshold) as a true positive if the structure is in the same SCOP fold as the query structure, and a false positive otherwise. For the Fischer data set, a true positive is counted when the score is above the current cutoff and the two structures are in the same fold according to Table II of Fischer *et al.* (1996). For the COPS benchmark data set, the true positives are defined by COPS as described above.

For substructure search, true positives are defined by Table 1 of Shi *et al.* (2007) for the β-grasp query, and by counting a hit as true positive only when it is to a structure in the "Serpins" fold according to SCOP for the serpin B/C sheet query.

The Area under the ROC Curve (AUC) and the statistical significance of ΔAUC values can then be computed as described in Section 2.5 of Chapter 2, using the ROCR package (Sing *et al.*, 2005) in R (R Development Core Team, 2007). 95% confidence intervals for the AUC values are calculated by the Hanley-McNeil method (Hanley and McNeil, 1982). The statistical significance of ΔAUC values at p-value 0.05 are calculated with a nonparametric approach (DeLong *et al.*, 1988) using the StAR program (Vergara *et al.*, 2008). As in Chapter 5 (Section 5.2.4), we use the `norm2` normalization function to normalize scores from multiple queries.

6.1.5 Implementation

We used the CUDA C compiler `nvcc` version 2.3 and CUDA SDK version 2.3, and GNU C compiler `gcc` version 4.3.4. Host code was run on an AMD Quad Core Opteron (2.3GHz, 2 processors, 32 GB RAM) running Linux. We used two GPU cards: an NVIDIA Tesla C1060 (4 GB global memory, 30 multiprocessors, 240 cores, 1.30 GHz) and an NVIDIA GeForce GTX 285 (1 GB global memory, 30 multiprocessors, 240 cores, 1.48 GHz). Both were running the CUDA Driver and Runtime version 2.30. Scripts for generating tableaux and distance matrices and building the database, evaluating results against SCOP, and processing output for visualization were written in Python. We used the BioPython library (Cock *et al.*, 2009) and the Bio.PDB file parsing and structure class (Hamelryck and Manderick,

2003) to parse PDB files and the Bio.SCOP interface (Casbon *et al.*, 2006) to read SCOP and ASTRAL data.

Source code, data sets, and executable binaries are available from `http://www.csse.unimelb.edu.au/~astivala/satabsearch/`.

For comparisons with other methods, the versions described in Chapter 5 were used. In addition, we compare with DaliLite version 2.4.5, LOCK2 (Shapiro and Brutlag, 2004) from `http://motif.stanford.edu/distributions/foldminer/`, and SARF2 (Alexandrov, 1996) downloaded from `ftp://ftp.ncifcrf.gov/pub/SARF2/`. The version of QP Tableau Search which uses the MA57 sparse symmetric solver (Duff, 2004; HSL, 2007), the fastest implementation, as described in Chapter 5, was used. We re-implemented the IR Tableau (Zhang *et al.*, 2010) algorithm in Fortran 77, and compiled it with the same compiler as QP Tableau Search.

Default parameters were used for all programs, except that the `-n` option was used on YAKUSA to output a score for all database structures rather than just the top 50 hits. In computing the ROC curve, YAKUSA and DaliLite results are ranked by their Z-scores, SHEBA results by the SHEBA *m* value, LOCK2 results by native LOCK2 score, VAST results by *Pcli* value, TOPS results by native ("compression") score, and IR Tableau results by cosine similarity score. Normalization is only required for SA Tableau Search, QP Tableau Search and TableauSearch; results from these methods are ranked according to their scores normalized by the `norm2` normalization function (see Chapter 5). SARF2 does not provide a matching score as such, but rather the number of residues aligned and the RMSD of the aligned residues. To combine these into a matching score, we used the SSM Q score (Krissinel and Henrick, 2004):

$$Q = \frac{N_{align}^2}{(1 + (\mathrm{RMSD}/R_0)^2)N_1N_2} \tag{6.2}$$

where N_{align} is the number of residues aligned, RMSD is the root mean square deviation (in Ångströms) between these residues, and $R_0 = 3.0$Å is an empirical parameter taken from Krissinel and Henrick (2004).

For substructure (motif) queries with TOPS, the motif queries were constructed by editing the TOPS cartoons with the `EditTops` program and manually editing the resulting TOPS string.

Method	Platform	Restarts	Elapsed time	AUC	std. error	95% C. I. lower	95% C. I. upper
SHEBA	CPU	-	25 h 22 m	0.931	0.004	0.924	0.938
SA Tableau Search	CPU	4096	142 h 42 m	0.930	0.004	0.923	0.937
SA Tableau Search	GTX 285	4096	4 h 11 m	0.930	0.004	0.923	0.937
SA Tableau Search	Tesla C1060	4096	5 h 40 m	0.930	0.004	0.923	0.937
SA Tableau Search	Tesla C2050	4096	5 h 23 m	0.930	0.004	0.923	0.937
DaliLite	CPU	-	620 h 03 m	0.929	0.004	0.922	0.936
SA Tableau Search	CPU	128	4 h 18 m	0.921	0.004	0.913	0.929
SA Tableau Search	GTX 285	128	0 h 08 m	0.920	0.004	0.912	0.927
SA Tableau Search	Tesla C1060	128	0 h 11 m	0.920	0.004	0.912	0.927
SA Tableau Search	Tesla C2050	128	0 h 10 m	0.919	0.004	0.911	0.926
QP Tableau Search	CPU	-	157 h 51 m	0.914	0.004	0.906	0.922
LOCK2	CPU	-	208 h 09 m	0.912	0.004	0.904	0.920
TableauSearch	CPU	-	1 h 11 m	0.858	0.005	0.848	0.867
TOPS	CPU	-	1 h 11 m	0.855	0.005	0.845	0.864
VAST	CPU	-	14 h 26 m	0.855	0.005	0.846	0.865
IR Tableau	CPU	-	0 h 01 m	0.854	0.005	0.844	0.864
YAKUSA	CPU	-	4 h 14 m	0.827	0.005	0.816	0.837

Table 6.1: *AUC and elapsed time for 200 queries in the ASTRAL 95% data set.*

Area under the ROC curve (AUC) and elapsed time for different methods on the 200 query set against the ASTRAL SCOP 95% sequence identity non-redundant database. The table is sorted by AUC descending. In the Platform column, either a GPU card is described, or CPU, which is a single core of an AMD Quad Core Opteron (2.3GHz, 32 GB RAM) running Linux. SARF2 is not present in this table, as it was unable to process a data set this large, and MSVNS4MaxCMO cannot process a data set this large in reasonable time.

6.2 Results and discussion

6.2.1 Protein structure search (fold classification)

Figure 6.2 shows the AUC and Table 6.1 shows AUC and elapsed time for several methods run on a set of 200 queries in the ASTRAL SCOP 1.75 95% sequence identity non-redundant subset. Run with 128 restarts, SA Tableau Search is one of the faster methods even on the host CPU, taking approximately the same time as YAKUSA: only TableauSearch, TOPS, and (especially) IR Tableau (Zhang *et al.*, 2010) are faster. Run with 4096 restarts, SA Tableau Search is one of the most accurate methods, with AUC not statistically significantly different from that of SHEBA and DaliLite (Table 6.2). It is, however, considerably faster than DaliLite, but not as fast as SHEBA (Table 6.1).

Note that the results for SARF2 are not included for this data set, as it was unable to process a data set this large.

Considering now the GPU implementation of SA Tableau Search, we can see that on average the GTX 285 card provides a 33 times speedup (34 times for 4096

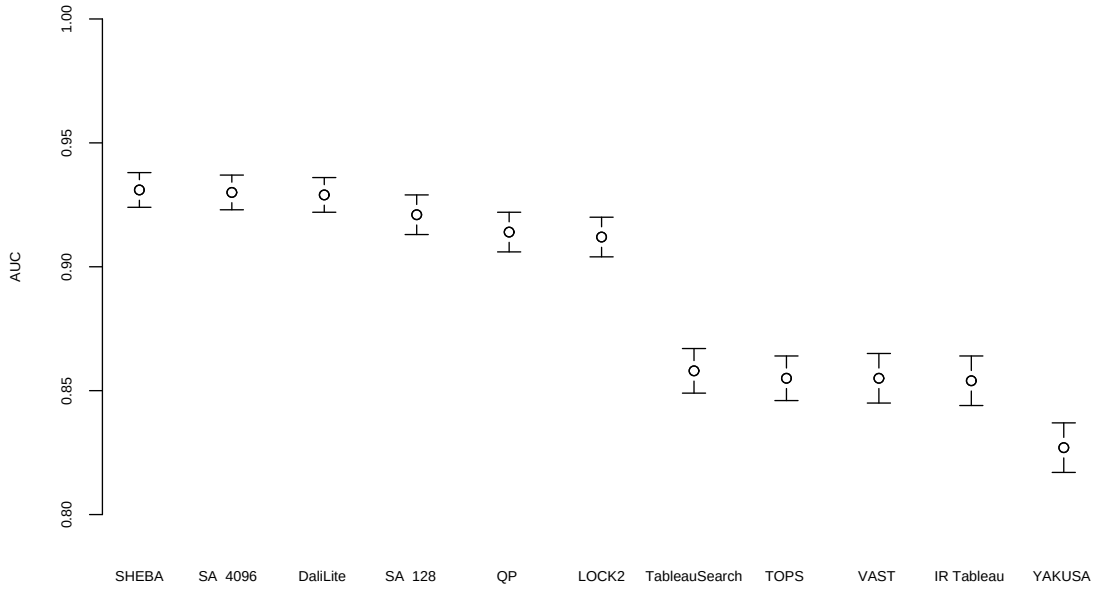


FIGURE 6.2: *AUC for different methods for 200 queries in the ASTRAL 95% data set.*

Area under the ROC curve (AUC) for different methods on the 200 query set against the ASTRAL SCOP 95% sequence identity non-redundant database, excluding comparisons of a structure against itself. Each AUC value is shown with a 95% confidence interval. SA *refers to SA Tableau Search (followed by number of restarts) and* QP *refers to QP Tableau Search. SARF2 is not present in this graph, as it was unable to process a data set this large, and MSVNS4MaxCMO cannot process a data set this large in reasonable time.*

Method(s)	ΔAUC
SA Tableau Search 4096, SHEBA, DaliLite, SA Tableau Search 8192	0.000
SA Tableau Search 2048	0.0007
SA Tableau Search 1024	0.0012
SA Tableau Seach 512	0.0023
SA Tableau Search 256	0.0039
SA Tableau Search 128	0.0090
QP Tableau Search	0.0160
LOCK2	0.0183
TableauSearch	0.0723
VAST	0.0746
TOPS	0.0756
IR Tableau	0.0761
YAKUSA	0.1034

TABLE 6.2: *ΔAUC relative to SA Tableau Search for 200 queries in the ASTRAL 95% data set.*

*Difference in AUC relative to SA Tableau Search (4096 restarts) for different methods on the 200 query set against the ASTRAL SCOP 95% sequence identity non-redundant database. The table is sorted by ΔAUC, so that methods with lower AUC than SA Tableau Search (*ΔAUC > 0*, with p-value* < 0.05*) are at the bottom of the table. Methods for which there is no statistically significant difference in AUC from SA Tableau Search (4096 restarts) at p-value* 0.05 *are shown in a single row with* ΔAUC = 0.000*.*

restarts), and the Tesla C1060 card provides on average a 24 times speedup. SA Tableau Search run with 4096 restarts on the GTX 285 takes 4 hours and 11 minutes, making it not only one of the most accurate methods considered, but also one of the fastest.

Figure 6.3 shows that, while elapsed time obviously increases linearly with the number of restarts (above 128, the number run in parallel), the accuracy as measured by the AUC figure increases much more slowly, and in fact there is no statistically significant difference between the value for 4096 and for 8192 restarts (Table 6.2). Hence, although changing the number of restarts allows a user-selectable tradeoff between accuracy and speed, one runs into rapidly diminishing returns for numbers of restarts beyond a certain point; on this evaluation data set, more than 4096 restarts achieves nothing, and 512 restarts (taking therefore approximately 1/8 the time of 4096 restarts) is enough to achieve the same accuracy (within statistical significance at p-value 0.05) as SHEBA but in just 31 minutes 25 seconds on the GTX 285. On the host CPU, this takes 17 hours 53 minutes, and SHEBA takes over 25 hours.

Figure 6.4, Table 6.3, and Table 6.4 show the AUC and elapsed time for all-against-all queries in the Fischer data set. This data set is much smaller, but was constructed specifically to benchmark fold recognition and so contains structurally

Method	Platform	Restarts	Elapsed time	AUC	std. error	95% C. I. lower	95% C. I. upper
SHEBA	CPU	-	03 m 47 s	0.877	0.016	0.845	0.909
DaliLite	CPU	-	113 m 34 s	0.876	0.016	0.845	0.908
LOCK2	CPU	-	32 m 12 s	0.874	0.016	0.842	0.907
IR Tableau	CPU	-	< 1s	0.859	0.015	0.830	0.888
QP Tableau Search	CPU	-	52 m 10 s	0.848	0.018	0.813	0.882
SA Tableau Search	CPU	4096	12 m 07 s	0.837	0.018	0.801	0.872
SA Tableau Search	Tesla C1060	4096	00 m 55 s	0.836	0.018	0.800	0.871
SA Tableau Search	Tesla C2050	4096	00 m 47 s	0.836	0.018	0.801	0.872
SA Tableau Search	GTX 285	4096	00 m 40 s	0.829	0.018	0.792	0.865
SA Tableau Search	CPU	128	00 m 25 s	0.822	0.019	0.786	0.859
SA Tableau Search	GTX 285	128	00 m 02 s	0.809	0.019	0.771	0.846
SA Tableau Search	Tesla C2050	128	00 m 04 s	0.809	0.019	0.772	0.847
SA Tableau Search	Tesla C1060	128	00 m 02 s	0.804	0.019	0.767	0.842
SARF2	CPU	-	19 m 34 s	0.797	0.020	0.759	0.835
YAKUSA	CPU	-	00 m 02 s	0.741	0.021	0.700	0.782
MSVNS4MaxCMO	CPU	-	541 m 33 s	0.730	0.021	0.689	0.772
TOPS	CPU	-	01 m 05 s	0.692	0.022	0.649	0.734
TableauSearch	CPU	-	< 1s	0.662	0.022	0.619	0.705
VAST	CPU	-	02 m 33 s	0.600	0.022	0.557	0.643

Table 6.3: *AUC and elapsed time in the Fischer data set.*

Area under the ROC curve (AUC) and elapsed time for different methods for all-against-all comparisons in the Fischer data set. The table is sorted by AUC descending. In the Platform column, either a GPU card is described, or CPU, which is a single core of an AMD Quad Core Opteron (2.3GHz, 32 GB RAM) running Linux.

Method(s)	ΔAUC
QP Tableau Search	-0.0533
SHEBA	-0.0409
DaliLite	-0.0399
LOCK2	-0.0379
SA Tableau Search 4096, IR Tableau	0.000
SA Tableau Search 1024	0.0055
SA Tableau Search 128	0.0143
SARF2	0.0398
YAKUSA	0.0954
MSVNS4MaxCMO	0.1062
TOPS	0.1449
TableauSearch	0.1744
VAST	0.2365

Table 6.4: *ΔAUC relative to SA Tableau Search in the Fischer data set.*

Difference in AUC relative to SA Tableau Search (4096 restarts) methods for all-against-all comparisons in the Fischer data set. The table is sorted by ΔAUC, so that methods with higher AUC than SA Tableau Search ($\Delta\text{AUC} < 0$*, with p-value* < 0.05*) are at the top of the table, and those with lower AUC than SA Tableau Search (*$\Delta\text{AUC} > 0$*, with p-value* < 0.05*) are at the bottom of the table. Methods for which there is no statistically significant difference in AUC from SA Tableau Search (4096 restarts) at p-value* 0.05 *are shown in a single row with* $\Delta\text{AUC} = 0.000$*.*

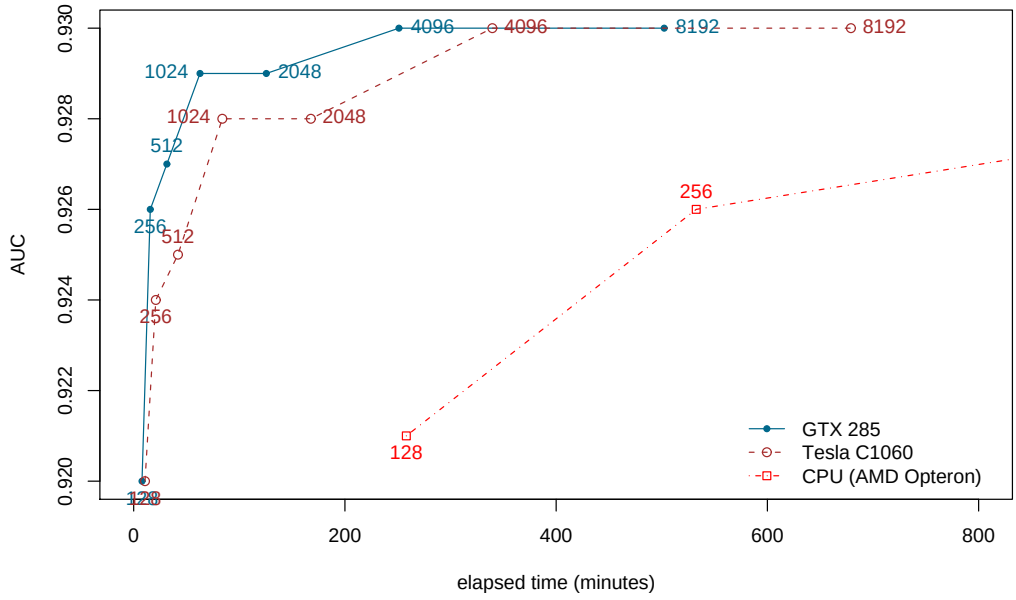


FIGURE 6.3: *AUC versus elapsed time for SA Tableau Search.*

AUC versus elapsed time for SA Tableau Search on different platforms for the 200 query set against the ASTRAL SCOP 95% sequence identity non-redundant database. Each data point is an AUC value, and is labelled with the number of restarts that SA Tableau Search was run with.

similar proteins of very low sequence similarity (Fischer *et al.*, 1996). In this data set, QP Tableau Search, SHEBA, DaliLite and LOCK2 all have statistically significantly higher AUC values than SA Tableau Search, although the latter when run on a GPU card is considerably faster than these methods. We find that IR Tableau works particularly well on this data set, with an AUC not statistically significantly different from that of SA Tableau Search, and faster (on the host CPU) than even SA Tableau Search on the GTX 285 card (the fastest available to us).

Figure 6.5, Table 6.5, and Table 6.6 show the AUC and elapsed time for the queries in the COPS benchmark (Frank *et al.*, 2010). DaliLite, SHEBA, YAKUSA, and SARF2 have the best AUC values for this data set, followed by SA Tableau Search which has an AUC value not significantly different from that of VAST and LOCK2. Although it is the best performing on AUC, DaliLite is by a considerable margin the slowest program. In this benchmark, TOPS and the other tableau based methods have significantly lower AUC than the other methods.

The Fischer and COPS data sets have quite different properties. The Fischer data set was designed to benchmark fold recognition methods, independent of the

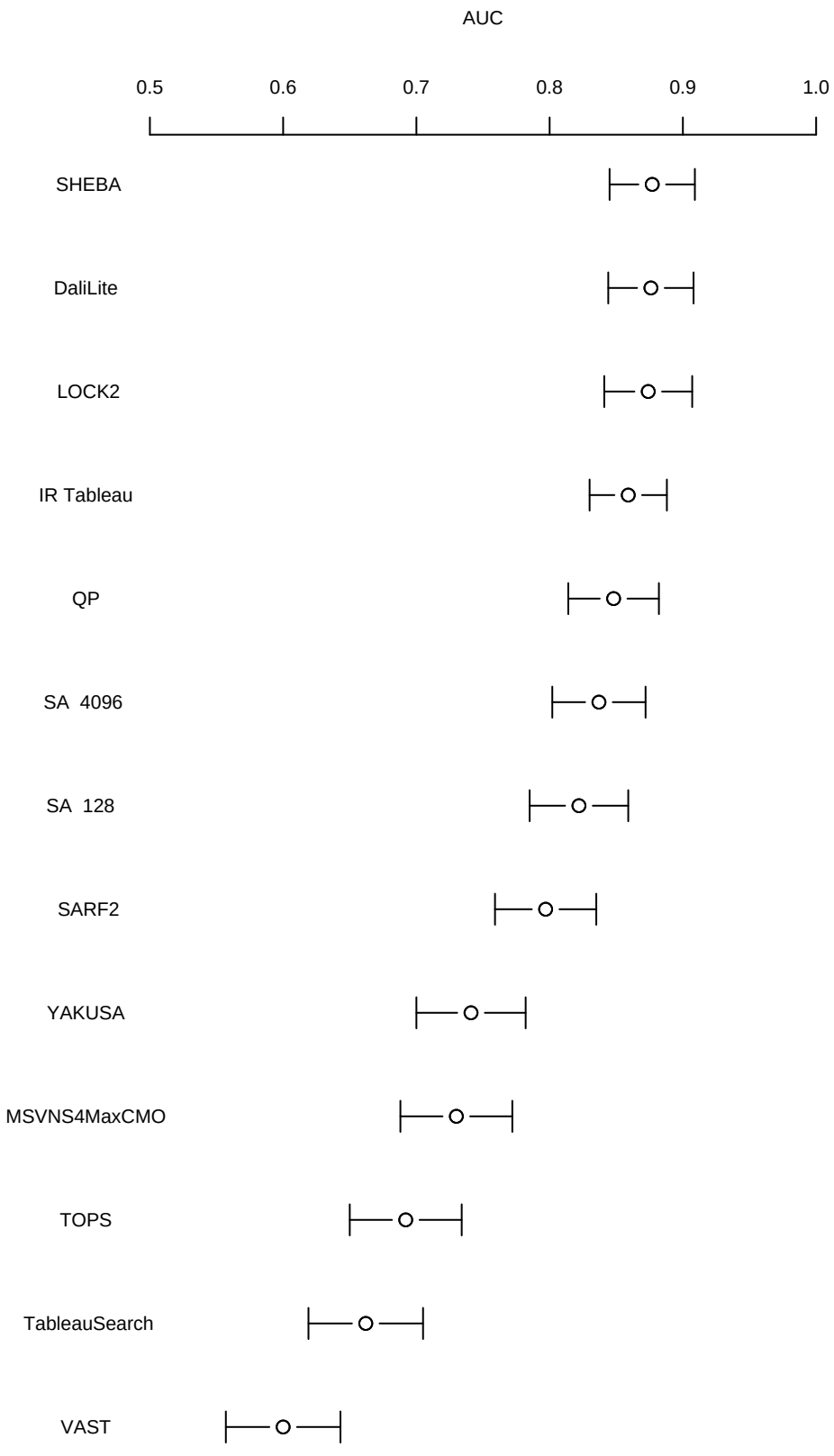


FIGURE 6.4: *AUC for different methods in the Fischer data set. Area under the ROC curve (AUC) for different methods for all-against-all comparisons in the Fischer data set, excluding comparisons of a structure against itself. Each AUC value is shown with a 95% confidence interval.* SA *refers to SA Tableau Search (followed by number of restarts) and* QP *refers to QP Tableau Search.*

FIGURE 6.5: *AUC for different methods in the COPS benchmark data set. Each AUC value is shown with a 95% confidence interval. SA refers to SA Tableau Search (followed by number of restarts) and QP refers to QP Tableau Search.*

Method	Platform	Restarts	Elapsed time	AUC	std. error	95% C. I. lower	95% C. I. upper
DaliLite	CPU	-	123 h 29 m 47 s	0.992	0.002	0.989	0.996
SHEBA	CPU	-	6 h 04 m 16 s	0.978	0.003	0.971	0.984
YAKUSA	CPU	-	0 h 02 m 25 s	0.975	0.003	0.969	0.982
SARF2	CPU	-	13 h 57 m 34 s	0.972	0.004	0.964	0.978
SA Tableau Search	CPU	4096	8 h 41 m 18 s	0.957	0.004	0.949	0.966
SA Tableau Search	GTX 285	4096	0 h 26 m 59 s	0.957	0.004	0.949	0.966
SA Tableau Search	Tesla C1060	4096	0 h 30 m 30 s	0.957	0.004	0.948	0.965
SA Tableau Search	Tesla C2050	4096	0 h 40 m 02 s	0.957	0.004	0.948	0.965
VAST	CPU	-	1 h 37 m 53 s	0.955	0.004	0.947	0.964
LOCK2	CPU	-	72 h 11 m 06 s	0.954	0.005	0.945	0.962
SA Tableau Search	CPU	128	0 h 15 m 17 s	0.951	0.005	0.942	0.960
SA Tableau Search	Tesla C1060	128	0 h 03 m 00 s	0.948	0.005	0.939	0.958
SA Tableau Search	GTX 285	128	0 h 03 m 02 s	0.947	0.005	0.938	0.957
SA Tableau Search	Tesla C2050	128	0 h 08 m 50 s	0.939	0.005	0.929	0.949
QP Tableau Search	CPU	-	74 h 47 m 43 s	0.925	0.006	0.914	0.936
TOPS	CPU	-	0 h 18 m 25 s	0.881	0.007	0.868	0.894
IR Tableau	CPU	-	< 1s	0.845	0.008	0.831	0.860
TableauSearch	CPU	-	0 h 18 m 07 s	0.840	0.008	0.825	0.855
MSVNS4MaxCMO	CPU	-	587 h 50 m 36 s	0.738	0.009	0.720	0.755

TABLE 6.5: *AUC and elapsed time in COPS benchmark data set.*

Area under the ROC curve (AUC) and elapsed time for different methods on the COPS benchmark data set. The table is sorted by AUC descending. In the Platform column, either a GPU card is described, or CPU, which is a single core of an AMD Quad Core Opteron (2.3GHz, 32 GB RAM) running Linux.

Method(s)	ΔAUC
DaliLite	-0.0349
SHEBA	-0.0201
YAKUSA	-0.0180
SARF2	-0.0140
SA Tableau Search 4096, VAST, LOCK2	0.000
SA Tableau Search 128	0.0064
QP Tableau Search	0.0325
TOPS	0.0764
IR Tableau	0.1120
TableauSearch	0.1172
MSVNS4MaxCMO	0.2200

TABLE 6.6: *ΔAUC relative to SA Tableau Search in the COPS benchmark data set.*

Difference in AUC relative to SA Tableau Search (4096 restarts) methods on the COPS benchmark data set. The table is sorted by ΔAUC, so that methods with higher AUC than SA Tableau Search ($\Delta\text{AUC} < 0$*, with p-value* < 0.05*) are at the top of the table, and those with lower AUC than SA Tableau Search (*$\Delta\text{AUC} > 0$*, with p-value* < 0.05*) are at the bottom of the table. Methods for which there is no statistically significant difference in AUC from SA Tableau Search (4096 restarts) at p-value* 0.05 *are shown in a single row with* $\Delta\text{AUC} = 0.000$*.*

structural similarity score being tested, and no two proteins in the data set have sequence similarity over 35% (Fischer *et al.*, 1996), and is quite small, consisting of only 68 structures (in which we perform all-against-all queries, resulting in 4624 pairwise comparisons). The COPS benchmark, in contrast, was designed to benchmark sequence similarity database searches, and true positives in this benchmark are defined by the COPS classification (Suhrer *et al.*, 2009; Frank *et al.*, 2010). This classification is defined by structural similarity according to the TopMatch structural alignment algorithm (Sippl and Wiederstein, 2008; Sippl, 2008), and so the COPS benchmark in fact assesses a method on its agreement with TopMatch.

We note that in these last two benchmark data sets (Fischer and COPS), in both cases, four methods have a statistically significantly better AUC than SA Tableau Search. However, it is a different set of four methods in the two cases, with SHEBA and DaliLite in common between them. Hence, given the different purposes of the Fischer and COPS benchmarks, it would seem that SHEBA and DaliLite are superior for both "difficult" (distantly related and low sequence similarity) and "easier" (more closely related) database searching tasks, while YAKUSA and SARF2 perform better only on the more closely related searches (or, to be precise, agree more closely with TopMatch), and LOCK2 and QP Tableau Search on the more distantly related searches.

In summary, over the three benchmark data sets we tested, no one method is consistently the best performing, although SHEBA is consistently in the top two methods measured by AUC, and it is considerably faster than DaliLite, the only method that has a higher accuracy on one of the benchmarks. All the other methods tested appear at different ranks in different benchmarks. SA Tableau Search ranks equal first with SHEBA in the ASTRAL 95% 200 query benchmark, the largest one, but one that also contains many similar structures and sequences. In all three benchmark data sets, SA Tableau Search has a (statistically significantly) higher AUC than TOPS and TableauSearch, and in all but COPS (where it has a not significantly different AUC), it has a higher AUC than VAST.

It is worth noting that QP Tableau Search and SA Tableau Search use exactly the same formulation of the protein substructure search problem as extraction of maximally-similar subtableaux first described by Konagurthu *et al.* (2008), enhanced by distance difference constraints introduced in Chapter 5. The difference is in the

method of approximation: QP Tableau Search relaxes the problem to a QP to find a (locally) optimal solution, while SA Tableau Search, described in this chapter, uses simulated annealing. Not only is the latter method faster and often more accurate (with sufficient restarts) on the host CPU than QP Tableau Search, but its simplicity allows parallelization on GPU cards, which are currently quite restricted in some respects, such as availability of sophisticated math libraries. For example, the complexity of QP Tableau Search in its use of an interior point solver made it impossible for us to implement a GPU version.

As described in Section 6.1.3, there are two levels of parallelization in our implementation: first, each restart of the simulated annealing schedule for a single comparison of two structures is run in parallel, and second, multiple comparisons between a single query and multiple database structures are run in parallel. Any database scanning method can be trivially parallelized in the second way, since each pairwise comparison is independent; to implement this at all on a GPU requires that the method is capable of being implemented within the limited computational and fast memory resources of the GPU. Since there is no recursion and no dynamic memory allocation in the NVIDIA CUDA programming model (NVIDIA, 2009b), this can make implementing sophisticated algorithms such as those required for pattern searching in YAKUSA (Carpentier *et al.*, 2005), for example, extremely difficult, if not impossible. Even where this is possible, since the GPU is essentially a data-parallel architecture (NVIDIA, 2009b), efficient implementations require that essentially the same code path is run simultaneously in a large number of threads, just with different data. The more threads diverge in their code path, the less efficient the parallelization will be, another reason why very simple algorithms are more suited to such parallelization. An efficient parallel implementation on a GPU requires in addition a "fine-grained" level of parallelization, in order to maximize the usage ("occupancy") of the thread multiprocessors (NVIDIA, 2009b) and the small amount of fast shared memory that this finer level of parallelization has access to. SA Tableau Search is ideally suited to this architecture since its data (tableaux and distance matrices) are shared between the threads running each restart of the simulated annealing schedule independently, and are usually small enough to fit in the fast shared memory. It may be possible to parallelize SHEBA in a similar manner. However, assuming a similar two-level parallelization, where the coarser

level is to run multiple comparisons independently in parallel, and the finer level is to parallelize each comparison, this would require a parallelization of both the dynamic programming procedure and the three-dimensional translation and rotation procedure used iteratively by SHEBA (Jung and Lee, 2000). This is a much more challenging task than simply running the multiple restarts of SA Tableau Search in parallel.

6.2.2 Substructure queries

Evaluating the accuracy of substructure (motif) queries in a quantitative and objective way such as AUC is quite challenging; there is no database such as SCOP to provide a set of all true occurrences of a motif in general. We therefore provide two examples where we can provide such an evaluation: the β-grasp motif, and the serpin (serine protease inhibitor) B/C sheet substructure (Law *et al.*, 2006). We perform these queries in the ASTRAL SCOP 1.75 95% sequence identity non-redundant database.

In evaluating the accuracy of a substructure query for the β-grasp motif, we use the data from Table 1 of Shi *et al.* (2007) as the gold standard, as described in Section 5.2.4 of Chapter 5. We demonstrate two queries for this test. First we use ubiquitin (SCOP identifier `d1ubia_`), an exemplar of the β-grasp fold, as the query. Second, we use a subset of the SSEs, namely the four largest strands and the α-helix in `d1ubia_`, chosen to represent the essential part of the β-grasp motif. This motif query, illustrated in Figure 6.6, is based on that defined by Shi *et al.* (2007) as a "meta-matrix" for ProSMoS.

The serpin B/C sheet substructure (see Figure 5.18 in Chapter 5) is such a large and specific structure that we can be confident it is truly present only in instances of the serpin fold. Therefore, in evaluating the accuracy of a substructure query for this substructure, we consider a hit a true positive if it is a member of the serpin fold in SCOP, and a false positive otherwise. We chose the B/C sheet of the canonical active serpin, α_1-antitrypsin, PDB id 1QLP (Elliott *et al.*, 2000) as the query structure.

Of the protein structure comparison methods discussed in this chapter, only QP Tableau Search, SA Tableau Search, TOPS and ProSMoS are also substructure (motif) finding methods. However, comparison using AUC with ProSMoS is not possible as ProSMoS does not rank all database structures. Instead, it returns a

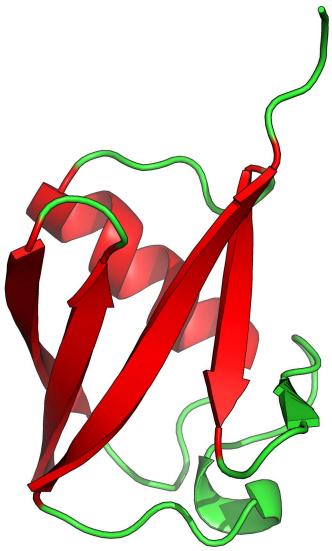


FIGURE 6.6: *The β-grasp motif query. The SSEs forming the query are in red.*

set of structures that contain the query motif. The web server version (Shi *et al.*, 2009) provides a ranking within this set only, the downloadable version provides no scores or ranking.

Table 6.7 shows the results of these substructure queries. We can see that, compared to QP Tableau Search, SA Tableau Search has similar accuracy, and is up to 6 times faster on the CPU implementation. The GPU implementation (on the GTX 285 card) is faster still, providing a speedup of up to 26 times over the CPU implementation of the same algorithm.

It should be noted that (as described in Section 6.1.2), `d1ubia_` is one of the queries used in tuning the simulated annealing parameters. For that purpose it was evaluated as a fold recognition task, rather than as a substructure query. Hence SA Tableau Search could not be expected to perform badly on the `d1ubia_` query as it has been in some part optimized for this query. This is not the case, however, for the serpin B/C sheet substructure.

Query	Method	Platform	Restarts	Elapsed time	AUC	std. error	95% C. I. lower	95% C. I. upper
d1ubia_	SA	GTX 285	128	00 m 03 s	0.918	0.011	0.896	0.940
d1ubia_	SA	CPU	128	01 m 17 s	0.912	0.011	0.890	0.935
d1ubia_	QP	CPU	-	05 m 11 s	0.902	0.012	0.879	0.926
d1ubia_	TOPS	CPU	-	00 m 10 s	0.894	0.012	0.870	0.918
β-grasp	SA	CPU	128	01 m 11 s	0.939	0.010	0.920	0.958
β-grasp	QP	CPU	-	02 m 01 s	0.938	0.010	0.918	0.957
β-grasp	SA	GTX 285	128	00 m 03 s	0.934	0.010	0.914	0.954
β-grasp	TOPS	CPU	-	00 m 09 s	0.847	0.014	0.819	0.875
serpin B/C sheet	SA	GTX 285	128	00 m 03 s	0.993	0.013	0.968	1.019
serpin B/C sheet	SA	CPU	128	01 m 19 s	0.991	0.015	0.962	1.021
serpin B/C sheet	QP	CPU	-	08 m 16 s	0.986	0.019	0.949	1.023
serpin B/C sheet	TOPS	CPU	-	00 m 24 s	0.491	0.054	0.385	0.597

TABLE 6.7: *AUC and elapsed time for some motif (substructure) queries.*

Area under the ROC curve (AUC) and elapsed time for some motif (substructure) queries on the ASTRAL SCOP 95% sequence identity non-redundant database. In the Method column, QP refers to the QP Tableau Search method, and SA refers to SA Tableau Search. In the Platform column, either a GPU card is described, or CPU, which is a single core of an AMD Quad Core Opteron (2.3GHz, 32 GB RAM) running Linux. The results for each query are sorted by AUC descending.

6.2.3 Statistical analysis of tableau matching scores

The ROC curve analysis provides an objective method of assessing the relative performance of fold classification methods. However, in actual use of a structure comparison or motif finding method, a measure of the statistical significance of a match is desirable. For example, in Chapter 5 we needed to determine a score above which a match is considered significant in order to count the number of superfamilies a motif was considered to be found in. In that chapter, this was done by a simple z-score computation based on the entire set of scores for that query in the database. However, that method implicitly assumed a normal distribution of scores, which is not actually the case; and even without examining the actual distribution of scores we would expect protein structural matching scores to have an extreme value distribution (Levitt and Gerstein, 1998). Here we show that tableau matching scores (as computed by SA Tableau Search) follow an extreme value distribution (the Gumbel distribution), and we can compute p-values for these matches with the method used by Abagyan and Batalov (1997); Ortiz *et al.* (2002); Vesterstrøm and Taylor (2006); Kolbeck *et al.* (2006).

The Gumbel distribution has the probability density function:

$$f(x) = \frac{1}{b}\zeta e^{-\zeta} \tag{6.3}$$

$$\zeta = e^{-\frac{x-a}{b}} \tag{6.4}$$

where a is the location parameter and b is the scale parameter of the Gumbel distribution. It has mean

$$\mu = a + b\gamma \tag{6.5}$$

where $\gamma \approx 0.577215665$ is the Euler-Mascheroni constant, and standard deviation

$$\sigma = \frac{\pi}{\sqrt{6}}b \tag{6.6}$$

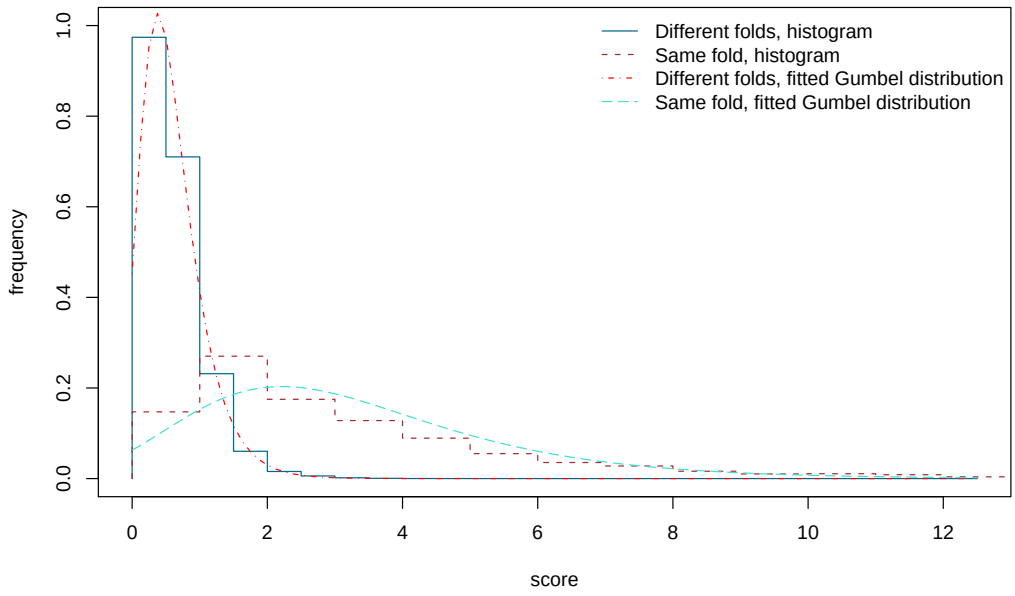


FIGURE 6.7: *Distribution of SA Tableau Search scores.*

Histograms of SA Tableau Search scores for structures in the same fold and structures in different folds, with Gumbel distributions fitted. For structures in different folds, we obtain Gumbel distribution parameters $a = 0.37803$ *and* $b = 0.35826$.

Figure 6.7 shows histograms of SA Tableau Search scores for structures in the same fold and structures in different folds, with Gumbel distributions fitted. These scores are from SA Tableau Search with 4096 restarts run on the 200 query set against

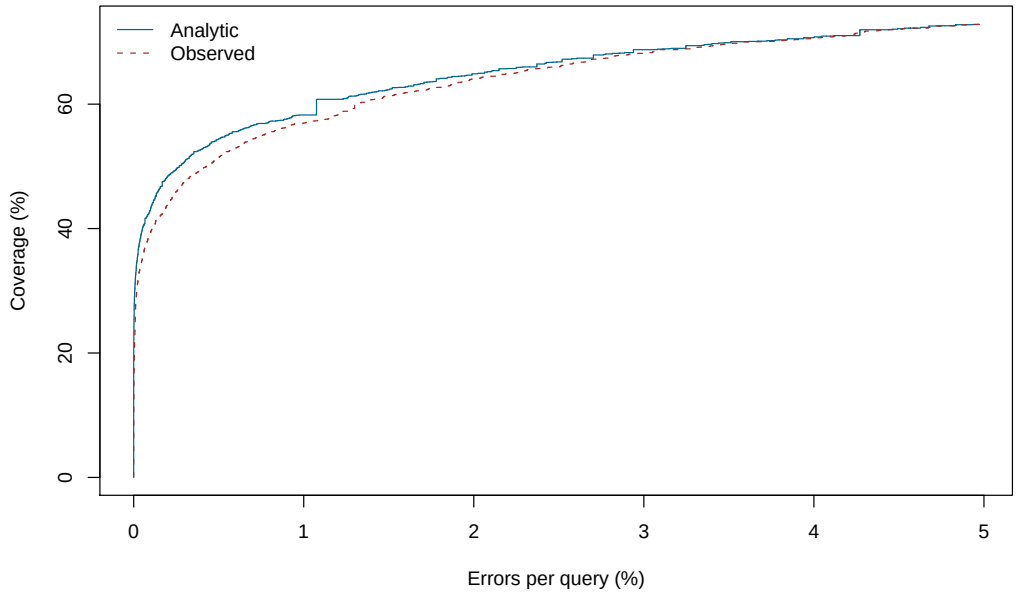


FIGURE 6.8: *Coverage versus Error plot for SA Tableau Search scores.*

the ASTRAL SCOP 95% sequence identity non-redundant database, and normalized with the `norm2` normalization function, with SCOP as the gold standard for fold classification, as described in Section 6.1.4. The Gumbel distribution is fitted using the `evir` and `evd` (Stephenson, 2002) packages in R. For a particular tableau matching score, we want to calculate the probability of obtaining a score greater than that, with the null hypothesis that the two structures are not in the same fold. Hence, following Abagyan and Batalov (1997) and Ortiz *et al.* (2002), we use the parameters (a and b) of the Gumbel distribution fitted to the histogram of scores for proteins in different folds. This probability is:

$$P(Z > z) = 1 - e^{-e^{-\left(\frac{\pi}{\sqrt{6}}z+\gamma\right)}} \tag{6.7}$$

where z is the z-score:

$$z = \frac{x - \mu}{\sigma} \tag{6.8}$$

for a particular tableau matching score x, with μ and σ defined by Equation 6.5 and Equation 6.6 respectively.

Figure 6.8 shows a coverage versus error plot (see Section 2.5 of Chapter 2) comparing the analytic (Equation 6.7) and empirical p-values, up to the 95% confidence level. At each point on these curves the coverage is the fraction of true positives that are more significant than the current p-value. The analytic p-value is that computed by Equation 6.7 and the empirical p-value is the false positive rate at that threshold (value of the analytic p-value). It can be seen that there is quite good agreement between the two p-value computations, although the analytic p-value is less than the observed p-value at very low (less than 2%) observed errors per query rates.

6.3 Conclusions

We have demonstrated an improved heuristic for tableau-based protein structure and substructure searching, that is comparable or better in speed and accuracy with some widely used existing methods when run on a standard CPU. In addition, we have provided a parallel implementation on modern GPU cards that achieves a speedup of up to 34 times over the CPU implementation, making it one of the fastest available methods. To the best of our knowledge, this is the first application of GPUs to the protein structural search problem.

There may well be scope for application of this technique to other optimization problems in bioinformatics that can be approximated by relatively simple optimization heuristics capable of being parallelized by implementation on a GPU. For example, the multistart variable neighbourhood search (VNS) heuristic for maximum contact map overlap (Pelta *et al.*, 2008), or some other heuristic such as simulated annealing for the same problem, could benefit from parallelization on the GPU. Another candidate problem of much current interest is biological network alignment, a highly computationally intensive problem that has previously been solved by quadratic programming (Li *et al.*, 2007), and has recently been approximated by local search heuristics (Chindelevitch *et al.*, 2010) amongst other methods.

In the next chapter, we describe a system for automatically generating two-dimensional protein structure cartoons, and its application as a user interface to a protein structural search system.

7

Pro-origami: Protein Structure Cartoons

In this chapter, we describe a new technique, which we call "Pro-origami", for the automatic generation of protein structure cartoons, by means of constraint-based diagram layout.

Section 7.1 reviews the history of protein structure cartoons and some existing algorithms for their automatic generation. Section 7.2 gives a brief overview of constraint-based diagram layout. Section 7.3 describes the algorithm and implementation of our technique. Section 7.4 discusses some of the resulting cartoons, and in Section 7.5 we describe a web server interface to the Pro-origami system, which is used as an interface to build and submit queries to the protein structural search programs described in Chapters 5 and 6. Finally, in Section 7.6 we conclude by summarizing the usefulness of these cartoons for protein structural analysis.

7.1 Introduction

Schematic drawings, or "cartoons" of three-dimensional protein structure, in which β-strands are drawn as arrows and α-helices as helices have been drawn both manually (Richardson, 1985), and, with α-helices rendered as cylinders, automatically by computer programs (Lesk and Hardman, 1982, 1985).

A three-dimensional schematic rendering, now usually with α-helices rendered as helices, is a widespread technique for visualizing protein structure, implemented in a large number of computer programs such as PyMOL (DeLano, 2002), VMD (Humphrey *et al.*, 1996), and many others, some of which are available as visualization options on the RCSB Protein Data Bank website (Berman *et al.*, 2000). Despite

the usefulness of three-dimensional representations, and the wide availability of tools for generating them, two-dimensional representations are a complementary technique and are attractive as they are simple to understand "at a glance" (Zhou and Shang, 2009), and are particularly useful for inclusion in (printed) publications.

Automatically generating such cartoons is a challenging problem, requiring the simplification of a complex three-dimensional object into a two-dimensional representation, with the (often competing) requirements of correctness, maximizing the amount of information conveyed, ease of comprehension, and aesthetic appeal. For this reason, two-dimensional schematic representations of protein structure or topology tend mainly to be manually drawn in the literature. The TopDraw (Bond, 2003) sketchpad program was designed to assist in this process by providing a palette of pre-defined shapes such as arrows and cylinders for drawing protein structure cartoons. Such diagrams are useful for comparing protein topology and structure visually, particularly for very distantly related folds. For example, in Rosado *et al.* (2007), hand-drawn topology diagrams (Rosado *et al.*, 2007, Suppl. Fig. S4) are used to show the core elements of secondary structure in common between perfringolysin O from *Clostridium perfringens* and the newly-solved structure of membrane attack complex/perforin from *Photorhabdus luminescens*.

Possibly the earliest published two-dimensional protein structure cartoons were those by Schulz and Schirmer (1974) and Rossmann *et al.* (1974). These were drawn by hand, and show topology by representing strands and helices as circles on the page, with arrows drawn through them indicating the sequence direction, and topology by arrows going "into" or "out of" the plane of the page (see Figure 7.1), a technique later also used by Levitt and Chothia (1976) and the TOPS program (Flores *et al.*, 1994), as discussed below. Schulz *et al.* (1974) includes a different kind of diagram, showing each residue as a circle arranged in a "string" drawn on the page to approximately indicate the fold, and residues marked to show their participation in a helix or strand (see Figure 7.2), not dissimilar to a hydrogen bonding diagram of the kind automated by HERA (Hutchinson and Thornton, 1990), described below.

Levitt and Chothia (1976) draw topology diagrams based on those just described, using circles to represent helices and rectangles for strands, with lines indicating the sequence relationship between them, using arrowheads to indicate direction.

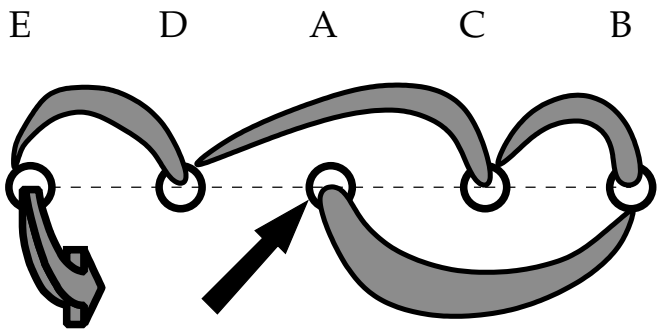


FIGURE 7.1: *Early protein topology diagram style used by Schulz and Schirmer (1974).*

Diagram showing the topology of a parallel sheet in adenyl kinase, similar to that in Schulz and Schirmer (1974, Fig. 1). The five circles are the five strands in the sheet, each of which points (from the N- to C- terminus) "out of the page" towards the viewer. The sequential order of the strands is A, B, C, D, E along the chain, in the direction of the arrows.

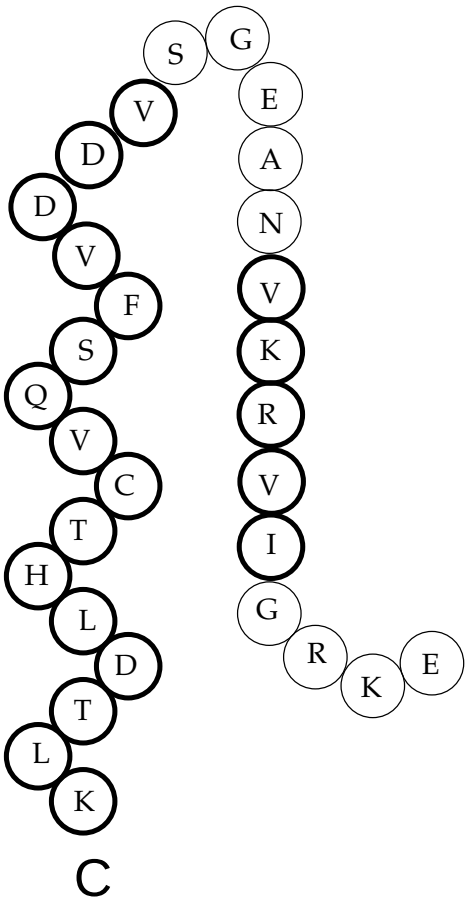


FIGURE 7.2: *Early protein sequence/topology style used by Schulz* et al. *(1974).*

Diagram showing the chain and approximate fold of a small C-terminal region of adenyl kinase, similar to a small part of Schulz et al. *(1974, Fig. 3). Each circle represents a residue (labelled with its one-character amino acid abbreviation). Residues that are part of a helix or strand are drawn with heavier circles, and the diagram roughly shows the chain fold.*

The lines go "into" or "out of" the plane of the page on the circles and rectangles to indicate SSE direction, and hence parallel or anti-parallel relationships between adjacent strands. In addition, the SSE shapes are arranged on the page to indicate (approximately) their packing, with the distance on the page between symbols representing interacting SSEs drawn proportionally to their actual distances (Levitt and Chothia, 1976). Figure 7.3 shows an example of such a diagram for an α/β protein; Levitt and Chothia (1976) used these diagrams to define the now well-known four classes all-α, all-β, $\alpha+\beta$ and α/β. Nagano (1977) uses a similar technique, but representing strands as circles and helices as hexagons or rectangles, depending on whether or not they are parallel or anti-parallel to strands. Sternberg and Thornton (1977) used similar diagrams but with triangles for strands and circles for helices; an advantage of the use of triangles is that now the parallel or anti-parallel relationship of strands in a sheet can be easily seen by drawing the triangles pointing up or down on the page (Figure 7.4). One of the important contributions of TOPS (described below) is to make a consistent set of rules for drawing these diagrams (and provide a program for generating such diagrams automatically), and as TOPS adopted the circle and triangle style of Sternberg and Thornton (1977), this style would now appear familiar to those involved in protein topology and structural biology. A TOPS cartoon for the same fold as that shown in Figure 7.4 is shown in Figure 7.5, clearly showing that TOPS automates the process of drawing topology cartoons in the Sternberg and Thornton (1977) style.

A simpler style of diagram, used to show β-sheet topology, was introduced by Richardson (1977). In this style, strands are simply shown as arrows pointing up or down the page, arranged across the page according to their order in the sheet, and connected to show the sequence. An example of such a diagram is shown in Figure 7.6. The relationships represented by such diagrams have been used as the basis of a system of representing β-sheet topologies (Flower, 1994b, 1995). Flower (1994b) also uses the Sternberg and Thornton (1977) style of diagram to illustrate different topologies, converted to graph representations for use in automatic secondary structure motif searching (Mitchell *et al.*, 1990).

The HERA (Hutchinson and Thornton, 1990) program, now part of PROMOTIF (Hutchinson and Thornton, 1996), draws a hydrogen bonding diagram showing each residue in the strands and helices and the hydrogen bonds between them.

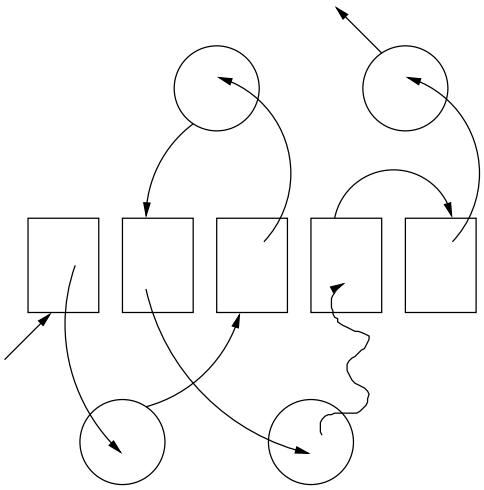


FIGURE 7.3: *Topology/packing diagram used by Levitt and Chothia (1976).*

Topology packing diagram of the α/β protein thioredoxin, similar to that in Levitt and Chothia (1976, Fig. 4). Rectangles represent β-strands and circles α-helices.

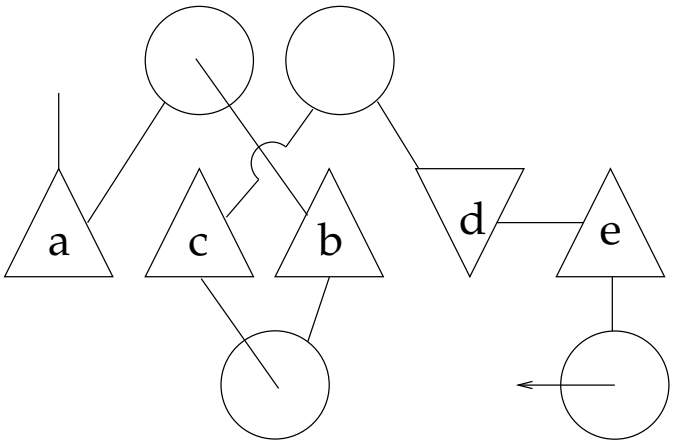


FIGURE 7.4: *Topology diagram used by Sternberg and Thornton (1977).*

Topology diagram of the α/β protein thioredoxin, similar to that in Sternberg and Thornton (1977, Fig. 3). Triangles represent β-strands and circles α-helices. Note that the use of triangles makes the parallel and anti-parallel relationships between the strands in the sheet much easier to see than those in the Levitt and Chothia (1976) style diagrams (Figure 7.3).

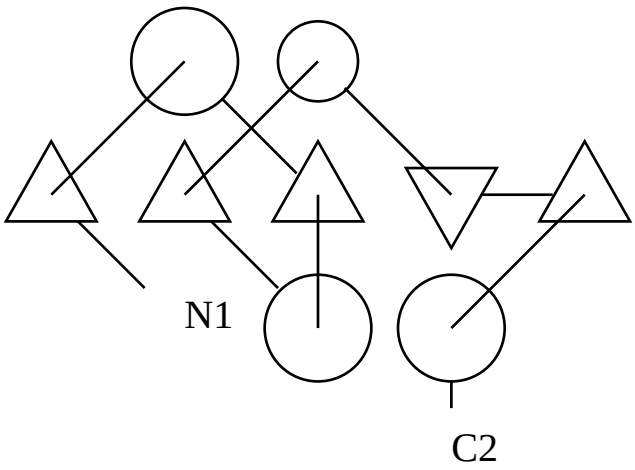


FIGURE 7.5: *TOPS cartoon for thioredoxin.*

Topology diagram of the α/β protein thioredoxin, SCOP identifier `d1xoba_`*, rotated* 180° *about the y axis and reflected in the xy plane with the* `EditTops` *program so that it has the same orientation as the Sternberg and Thornton (1977) topology diagram in Figure 7.4. The latter step is necessary to obtain the same direction of triangles as in Figure 7.4 as Sternberg and Thornton (1977) uses △ for a strand viewed from the N-terminus and ▽ for a strand viewed from the C-terminus, while TOPS by default uses ▽ for a strand viewed from the N-terminal end ("into" the paper) and △ for a strand viewed from the C-terminal end ("out of" the paper) (Flores* et al.*, 1994).*

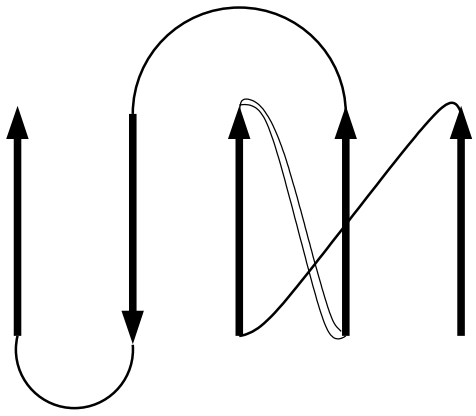


FIGURE 7.6: *β-sheet topology diagram used by Richardson (1977).*

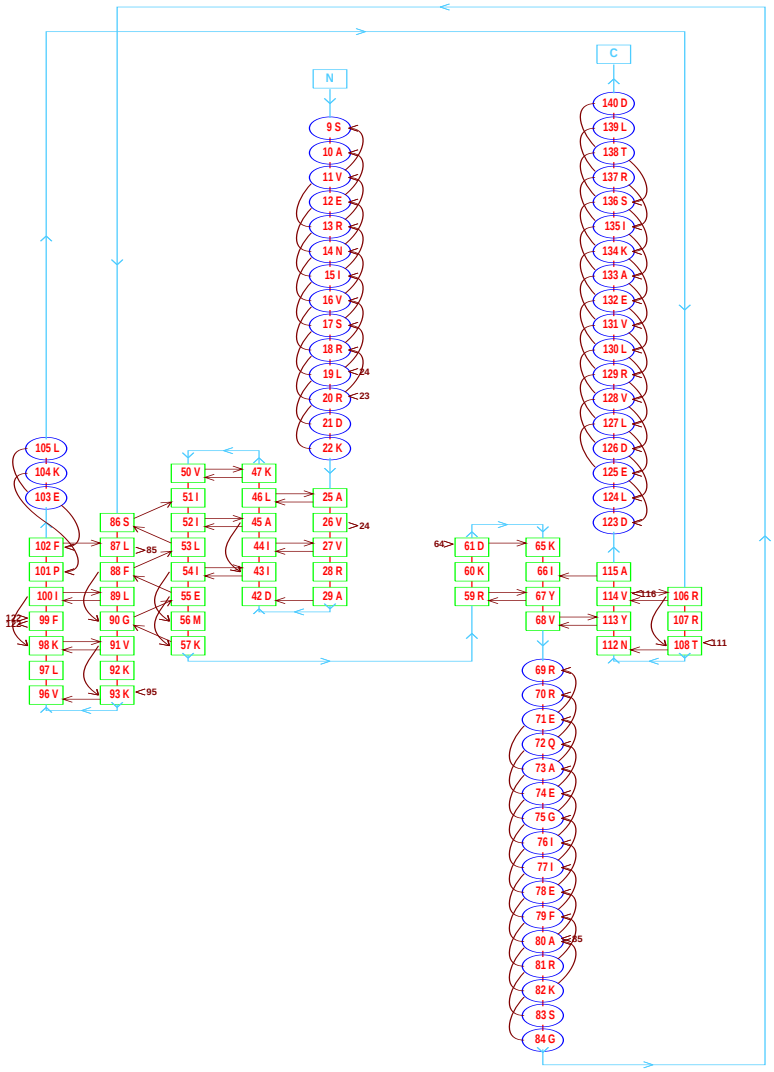
β-sheet topology diagram for the thioredoxin fold similar to that in Richardson (1977, Fig. 1). Only the topological connectivity of β-strands is represented.

FIGURE 7.7: *Holliday junction resolvase, PDB id 1HH1 (Bond* et al.*, 2001). Image generated with PyMOL (DeLano, 2002).*

Residues in strands are drawn as rectangles, and those in helices as ellipses. The orientation and position of strands in a sheet is represented by drawing residues in two strands that have a hydrogen bond between them side-by-side. Helices are positioned in columns above or below strands with nearby residues positioned near each other where possible, resulting in an orthogonal drawing style. Loops are drawn as orthogonal lines connecting secondary structural elements, and drawn so that loops with larger endpoint distances are drawn further from the centre of the diagram, in order to try to minimize line crossings (Hutchinson and Thornton, 1990). Figure 7.8 shows an example of a HERA hydrogen bonding diagram for Holliday junction resolvase, PDB identifier 1HH1 (Bond *et al.*, 2001). The three-dimensional structure of this protein is illustrated in Figure 7.7. We will use this structure as a running example throughout the rest of this chapter to illustrate the Pro-origami algorithm, although we will also use additional structures to show features not present in 1HH1.

The PDBsum protein atlas website[1] (Laskowski, 2001; Laskowski *et al.*, 2005) recently added a feature to draw protein topology diagrams derived from HERA in a simple "side-on" style (Laskowski, 2009). These figures are essentially the same as HERA diagrams, however they are much simpler as all the low-level detail of individual residues and hydrogen bonds is removed, each helix and strand being

[1] `http://www.ebi.ac.uk/pdbsum`



1HH1

FIGURE 7.8: *HERA Hydrogen bonding diagram of Holliday junction resolvase Hjc (Bond* et al.*, 2001), PDB code 1HH1, drawn with PROMOTIF version 2.*

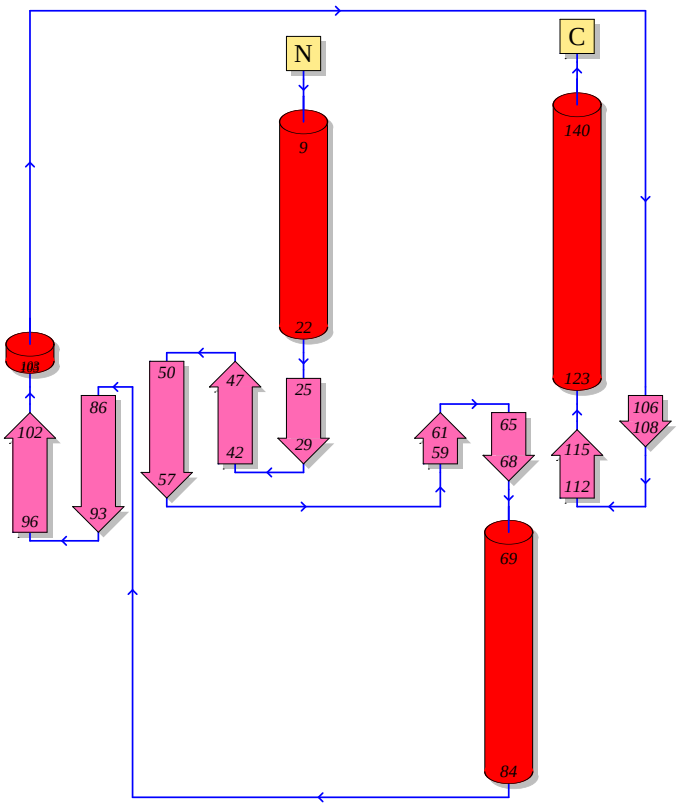


FIGURE 7.9: *PDBsum topology diagram of Holliday junction resolvase Hjc (Bond* et al.*, 2001), PDB code 1HH1, from the PDBsum website.*

represented as a single cylinder or arrow respectively. Figure 7.9 shows an example of such a diagram for 1HH1 (the HERA diagram for which is shown in Figure 7.8).

The TOPS program (Flores *et al.*, 1994; Westhead *et al.*, 1999) draws topology diagrams ("TOPS cartoons") with triangles representing β-strands and circles representing α-helices. TOPS cartoons are much less detailed than HERA diagrams, as they are designed to show only topological relationships, omitting the lengths and precise orientations of secondary structural elements (Westhead *et al.*, 1999). Orientations are shown by the direction of secondary structure elements (SSEs) being "into" or "out of" the page. An example of a TOPS cartoon is given in Figure 7.10. The TOPS algorithm derives secondary structure definitions and hydrogen bond information from DSSP (Kabsch and Sander, 1983) or STRIDE (Frishman and

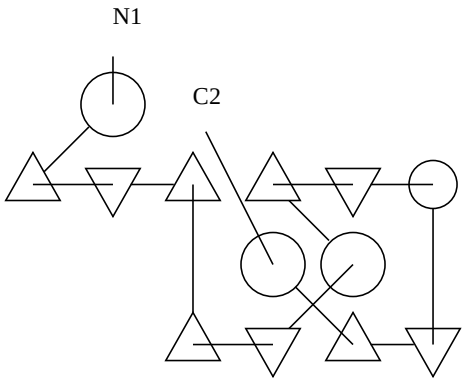


FIGURE 7.10: *TOPS cartoon of 1HH1.*

Argos, 1995) then detects bridge partner relationships between β-strands and identifies β-sheets and β-barrels, and calculates the relative position and orientation of the strands in them. β-barrels are drawn by arranging the triangles representing the constituent strands in a circle. Sheet curvature, β-sandwiches and chirality are also taken into account, and the cartoon is then laid out by using simulated annealing to minimize an energy function which penalizes various undesirable features such as nearby SSEs not being drawn close together, symbols being drawn too close together, long connecting lines, crossing lines, symbols drawn within barrels and incorrect chiralities (Westhead *et al.*, 1999). TOPS includes the `EditTops` topology cartoon editor which can be used to manually edit TOPS cartoons. More recent work on TOPS has been directed towards using a formalization of the TOPS cartoon, known as a TOPS diagram (Gilbert *et al.*, 1999) for motif-based searching of topology databases[2] (Gilbert *et al.*, 1999; Michalopoulos *et al.*, 2004) and multiple structural alignment (Williams *et al.*, 2003).

As described by Bond (2003), HERA hydrogen bonding diagrams tend to be too complex for clear presentation in a paper, and the style of TOPS cartoons is not always well suited to the purposes of structural biologists. The PDBsum diagrams are much simpler than HERA (having removed the detail of individual residues and hydrogen bonds), and have a similar style to those that might be drawn manually with TopDraw. However they are not easily editable and do not show the relative orientation of sheets and helices, or other additional information such as clusters of helices. In order to overcome these shortcomings of existing automatic cartoon

[2]This is discussed in Chapter 5.

drawing programs, we have designed and implemented a system to automatically draw protein structure cartoons in a style similar to those drawn manually using TopDraw.

As has been mentioned above, two-dimensional protein structure cartoons in the "side-on" (rather than TOPS) style, tend to be hand-drawn (often with the aid of TopDraw) in the literature. Zhou and Shang (2009, p. 250) cite three examples of published two-dimensional topology diagrams (Jayaram *et al.*, 2002; Bewley *et al.*, 2006; He *et al.*, 2008), all of which appear to be hand-drawn (possibly using TopDraw) and are in the TopDraw style. Just as was the case with the "top-down" (TOPS style) topology diagrams until the first TOPS publication (Flores *et al.*, 1994), there is no defined standard for drawing the "side-on" (TopDraw style) diagrams, and different authors have used different techniques for representing structures in this style. TopDraw, when it is used, implicitly defines some rules, simply by the fact of the shapes (cylinders, arrows) it provides for helices and strands, but because it just provides these shapes for the user to manually create diagrams with, rather than implementing any drawing algorithms itself, the way in which structures are represented can still vary greatly between different authors.

We choose to use the same shapes (cylinders for helices, arrows for strands) as TopDraw, and now describe our choices for the rules we use to govern their use in representing protein structure, based on examination of published diagrams in similar styles (usually those whose authors have cited the use of TopDraw), and our experience in developing the Pro-origami algorithm.

β-sheets are to be represented by strands arranged across the page (with a fixed distance between each strand), representing their positions in the sheet, and pointing up or down the page to represent their parallel or anti-parallel relationship. These may also be rotated 90 degrees so that they are arranged down the page and point across the page. This is the style that TopDraw assumes and which all of the published diagrams we examined use. TOPS style diagrams show this strand relationship information by the orientation of the triangles representing the strands.

The lengths of the arrows representing the strands should be proportional to the number of residues in the strand, and hence visually represent the length of the strand. TopDraw allows this by letting the user assemble strands from a variable number of "blocks", and most published diagrams in this style make some attempt

to have the arrow lengths roughly proportional their corresponding strand length. The same applies to helices and the cylinders representing them. HERA (and PDBsum) diagrams do this exactly since they represent each residue. TOPS style diagrams can also represent SSE size to some extent by varying the relative sizes of the triangles and circles representing strands and helices respectively, but this is a much less direct representation and not nearly as easy to understand at a glance.

The positions of neighbouring strands in a sheet relative to each other should reflect their actual positions, i.e. the parts of the two neighbouring (vertical) arrows that represent residues that are hydrogen-bonded between the two strands should be on the same horizontal axis. The same applies for horizontal strands (neighbouring hydrogen-bonded residues should be on the same vertical axis). HERA (and hence PDBsum) diagrams do this since they are essentially hydrogen bonding diagrams, but only some TopDraw style diagrams are drawn with this additional information, quite possibly since it is quite difficult to do this correctly manually (which is what a user of TopDraw must do), and it is not always considered the most important information in a topology cartoon. TOPS, by its "top-down" nature cannot represent the hydrogen bonding positions between strands at all, and since TOPS is probably the most widely used automatic topology cartoon generating tool, hand-drawn TopDraw cartoons are sometimes based on the TOPS generated cartoons (e.g. Spraggon *et al.* (2004); Bitto *et al.* (2005); Ramasubbu *et al.* (2005); Ricagno *et al.* (2006)).

Although some authors (e.g. Blair and van Aalten (2004, Fig. 2), Blair *et al.* (2005, Fig. 1), Blair *et al.* (2006, Fig. 1), Ramasubbu *et al.* (2005, Fig. 1(a))) have represented certain structures, such as β/α-barrels, by drawing the strand arrows on top of the cylinders representing the helices they are packed against (Figure 7.11), the use of overlapping shapes is usually avoided, and automatic cartoon generation systems such as HERA (PDBsum) and TOPS never draw shapes on top of each other, and we also believe that this is the best option. Packing of helices against β-sheets can also represented by placing the helices between strands (e.g. Yu *et al.* (2006, Fig. 2D) (see Figure 7.12), Ferreira *et al.* (2006, Fig. 1), Brzezinski *et al.* (2007, Fig. 3), Porter *et al.* (2007, Fig. 2)) or above or below the strands they are sequentially adjacent to (e.g. Bond *et al.* (2001, Fig. 2), Bitto *et al.* (2005, Fig. 1)). We choose the latter style

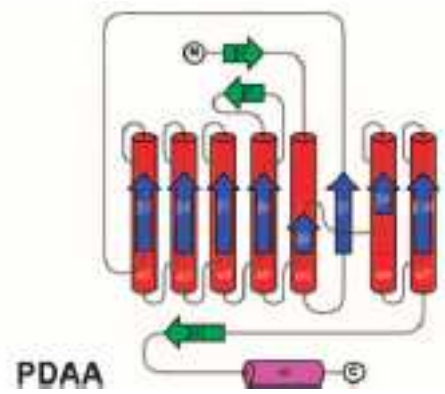


FIGURE 7.11: *Example of a cartoon with overlapping symbols.*

Topology of Bacillus subtilis PdaA (a peptidoglycan deacetylase), PDB identifier 1W1A (Blair and van Aalten, 2004), drawn by Blair et al. *(2005) with TopDraw, with the β/α-barrel structure represented by drawing the strands on top of the helices. This figure is reproduced from Blair* et al. *(2005, Fig. 1) Copyright (2005) National Academy of Sciences, USA.*

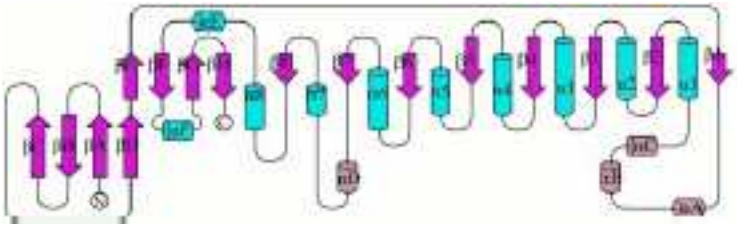


FIGURE 7.12: *Example of a cartoon of a β/α-barrel structure drawn with helices between strands in a sheet.*

Topology diagram of imidazolonepropionase from Bacillus subtilis, PDB identifier 2G3F, drawn by Yu et al. *(2006) with TopDraw, with the β/α-barrel structure represented by drawing the helices between the strands of the sheet. This figure is reproduced from Yu* et al. *(2006, Fig. 2D) ©The American Society for Biochemistry and Molecular Biology.*

(Figure 7.13), making a rule that β-sheets are drawn containing only the strands in the sheet.

There are two major alternatives for the positioning of helices. The first is to position them (approximately) according to their packing relative to strands and other helices, as is done in TOPS style diagrams. This maximizes the amount of 3D spatial information that is conveyed in the 2D cartoon, but can result in confusing cartoons due largely to the large number of connector crossings that this style can create (see, for example, Figure 7.23(c) and Figure 7.23(d)). The second is to place helices on the same axes as the strands to which they are connected in sequence, which results in a neater and less confusing orthogonal drawing style,

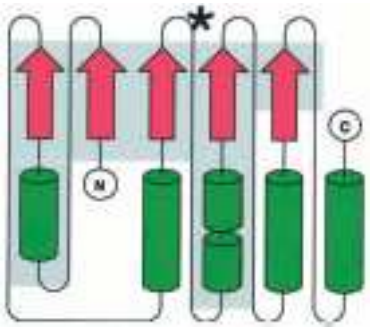


Figure 7.13: *Example of a parallel sheet structure with the helices drawn below the strands of the sheet.*

Topology diagram of the bacterial chemotaxis protein CheY from Thermotoga maritima, PDB identifier 1TMY (Usher et al.*, 1998), drawn by Bond* et al. *(2001) using TopDraw, based on information from PROMOTIF. Rather than drawing the strands and helices overlapping (as in Figure 7.11) or the helices between strands of the sheet (as in Figure 7.12), the helices are drawn below strands they are connected in sequence with, so that the sheet is drawn containing only the strands that make it up. This figure is reproduced from Bond* et al. *(2001, Fig. 2) Copyright (2001) National Academy of Sciences, USA.*

as is done by HERA (and hence PDBsum). We implement both styles, leaving it to the user to determine whether it is more important to show the helix packing, or to favour neatness and clarity of the cartoon at the expense of no longer showing this information. In addition, we provide a compromise by the use of "helix clustering" (see Section 7.3.6) so that a sequence of helices that are in contact in 3D space are drawn according to their packing, but other helices are drawn aligned on strand axes. This allows the packing of such helix "clusters" to be shown, while not making the cartoon unnecessarily confusing by having other helices not neatly aligned on axes.

The sequence relationship between SSEs is shown by connectors (lines) between them. It is important that the sequence can be easily followed by eye. To do this we use *orthogonal connector routing* (Wybrow *et al.*, 2010), that is, connectors that consist of horizontal and vertical line segments (with 90 degree "bends" connecting them). Connectors must also not "collide" with (i.e. cross over) the cylinders and arrows representing SSEs. In order to best draw connectors, we need to (as far as possible) simultaneously minimize connector length, number of connector crossings, number of "bends" in connectors, and number of "overlaps". These "overlaps" are cases where connectors run too close to each other, make it difficult to follow the sequence by eye.

As has been previously described, HERA (and PDBsum) draw connectors so that loops with larger endpoint distances are drawn further from the centre of the diagram, as a heuristic to try to minimize crossings (Hutchinson and Thornton, 1990), while TOPS uses simulated annealing to optimize a function that penalizes these undesirable connector features (Flores *et al.*, 1994; Westhead *et al.*, 1999). By using constraint-based diagram layout (Dwyer *et al.*, 2006, 2007; Wybrow *et al.*, 2006; Wybrow, 2008; Dwyer *et al.*, 2009; Wybrow *et al.*, 2010) the requirements just described can be satisfied in a principled manner.

7.2 Constraint-based diagram layout

Constraint-based diagram layout and editing tools use *constraints* that specify relationships between elements of the diagram. Examples of such constraints are that shapes must not overlap, or that shapes must be aligned in a certain way. In interactive use, the user can edit the diagram, while the constraint-based diagram editing software maintains the constraints. A thorough history and explanation of constraint-based diagram editing can be found in Wybrow (2008), which also contains a short description of an early version of Pro-origami as a case study.

We make use of the following kinds of constraints: non-overlap constraints, alignment constraints, and distribution constraints.

Non-overlap constraints specify that the shapes in the diagram cannot overlap. Not only are these constraints enforced in initially laying out the diagram, but they are maintained during interactive use; if the user moves a shape onto another shape, the first shape "pushes" the other shape so they do not overlap.

Alignment constraints between two or more objects specify that those objects are aligned on vertical or horizontal axes. Again, these are maintained during interactive diagram editing so that if two shapes are aligned on a vertical axis, moving one of them in a horizontal direction moves the other so that they remain aligned vertically.

Distribution constraints between multiple objects specify a separation distance between those objects (in the horizontal or vertical direction) that must be maintained. We use these constraints to preserve the spacing between strands in a sheet. When the distribution constraint is between only two objects, we refer to it as an

exact spacing constraint. These are used to maintain strand offsets as described in Section 7.3.4 and in particular Figure 7.18. As with all constraints, these distribution and exact spacing constraints are maintained during interactive diagram editing.

In addition, we make use of *clusters*, whereby several objects are grouped together and treated as a single object. This can be indicated on the diagram by shading the background inside the bounding box of the cluster of shapes. In interactive use, the user can move the entire cluster as a single object. Pro-origami uses such clusters to group together strands in a sheet (Section 7.3.4) and helices in a "helix cluster" (Section 7.3.6).

Another diagram layout and editing feature we use is *automatic connector routing*. As has already been described in Section 7.1, we use orthogonal poly-line connectors. That is, the connectors between shapes consist of horizontal and vertical line segments. These connectors are specified only by their two endpoints, and the constraint based diagram editor routes them automatically so as not to collide with shapes, and (as far as possible) simultaneously minimizes connector length, number of connector crossings, number of bends , and number of connector "overlaps". In interactive use, the user can move shapes, and the connectors are automatically re-routed so as to maintain these requirements.

We use a constraint-based diagram editor, Dunnart, that implements constraint-based diagram layout and connector routing, and is fast enough that the constraints are satisfied in real time, allowing interactive editing of the diagram while maintaining the constraints (Dwyer *et al.*, 2009; Wybrow *et al.*, 2010).

7.3 Methods

SSEs and the hydrogen-bonding and spatial relationships between them are represented as graph and matrix data structures. This information is then used to specify sizes, orientations, and initial positions of graphical elements representing the SSEs in the cartoon, and connectors between them, together with a set of constraints on their relative positions for use in an interactive constraint-based layout system.

The top-level description of the algorithm is as follows. Detailed descriptions are given in subsequent sections below.

1. Read atomic co-ordinates (and optionally HELIX and SHEET definitions) from PDB file.

2. Run STRIDE or DSSP to assign secondary structure and find all H-bonds.

3. Optionally, run DDOMAIN (Zhou *et al.*, 2007) or parse a CATH (Pearl *et al.*, 2005) Domall file (CDF) to decompose into domains, in order to process one domain at a time.

4. Build a graph with one kind of edge for sequence and another kind (*bridge edges*) for bridges between strands.

5. Calculate distance matrices (Section 7.3.3), containing the distance between each pair of secondary structure elements in the domain.

6. Calculate the tableau (Section 5.2.1 of Chapter 5) for the domain. Tableaux describe the relative orientation of each pair of secondary structure elements in a domain.

7. Group strands into sheets by connected components (by bridge edges) in the graph built in step 4.

8. Detect barrels by cycle in sheet graph, and "flatten" by breaking a bridge edge.

9. Annotate bridge edges as parallel or anti-parallel using information from STRIDE or DSSP.

10. Annotate bridge edges with same/other side of shared strand using H-bond information and angle between two planes intersecting at common strand (similar to TOPS (Westhead *et al.*, 1999)).

11. Position and orient sheets and helices according to distance matrix and tableau information and/or heuristics.

12. Write specifications of appropriate shapes (sized proportional to their length in residues), connectors and constraints for the Dunnart constraint-based diagram editor:

    - Draw the strands as arrows (as per TopDraw) to show parallel/anti-parallel relationships in sheet.

- Make the strands in a sheet a cluster.
- Align the strands beside each other in correct order, and positioned beside each other, with offsets according to the bonds between them.

7.3.1 Graph representation of secondary structure

Graphs have been used for some time to represent the topology of β-sheets, for example by Koch *et al.* (1992) and Flower (1994b), complementing the original notation of Richardson (1977). Graphs have also been used to represent general protein secondary structure, with nodes representing helices and strands and edges being undirected and labelled with distances and angles, in order to implement motif searching by subgraph isomorphism (Mitchell *et al.*, 1990; Grindley *et al.*, 1993; Harrison *et al.*, 2003; Artymiuk *et al.*, 2005). TOPS uses graphs to determine strand ordering in β-sheets (Westhead *et al.*, 1999). Pro-origami represents protein secondary structure as a graph in which the nodes are SSEs. These may be one of two kinds, namely β-strands or helices (which include α-helices and, optionally π and 3_{10} helices). Note that this is not dissimilar to the protein graph representation used in the DomainICA topological algorithm for protein domain identification (Emmert-Streib and Mushegian, 2007), which however also includes loops as SSEs represented by nodes in the "protein graph", which is an unweighted, undirected, unlabelled graph, with edges connecting SSEs whose spatial distance is below a threshold (Emmert-Streib and Mushegian, 2007).

Conceptually, two kinds of edges exist in our protein structure graph, sequence edges and bridge partner edges. Sequence edges simply denote the sequential order of the SSEs from N- to C-terminus and hence these need not be represented explicitly, but left implicit in the ordering of nodes. Bridge partner edges exist between β-strands that are bridge partners in a sheet, and are labelled "P" for parallel or "N" for anti-parallel as defined by information from the secondary structure definitions generated by DSSP or STRIDE. These edges are directed but symmetric, any two strands that are bridge partners having two edges between them, one in each direction. The edges are later given an additional label in order to denote which side of a given strand each of its neighbours are on, using an algorithm used in TOPS (Westhead *et al.*, 1999) and described in Section 7.3.4. Figure 7.14 shows an example of such a graph.

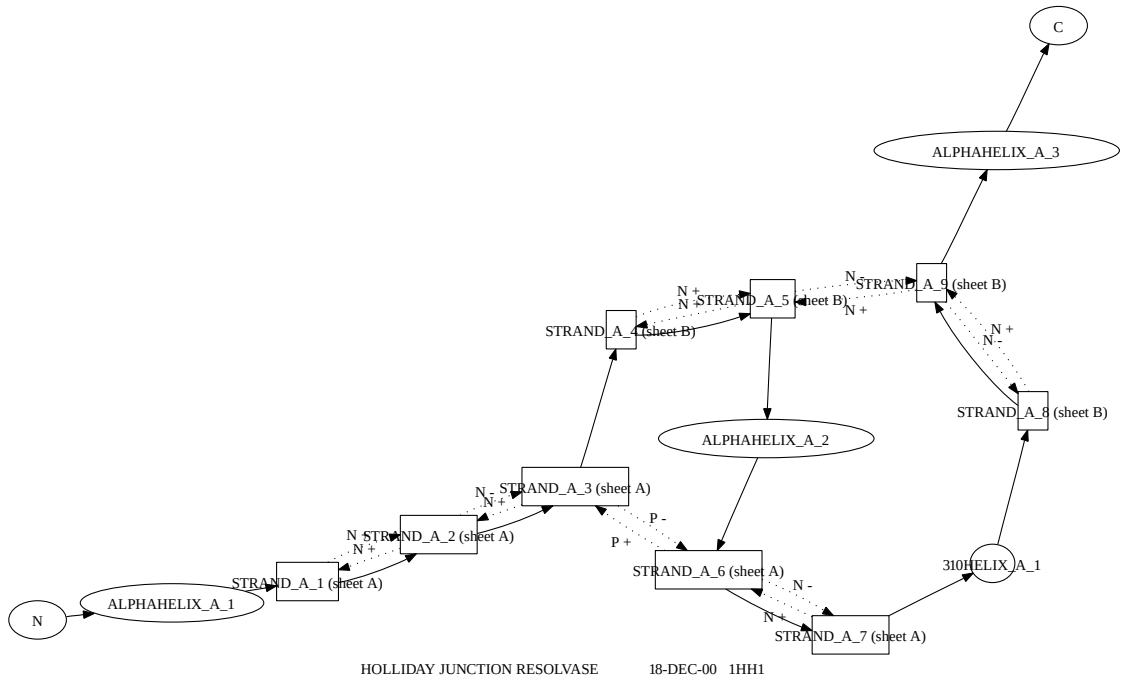


FIGURE 7.14: *The graph representation of 1HH1 used internally and drawn with the Graphviz* `neato` *program (Gansner and North, 2000).*

Solid lines are sequence edges and dotted lines are bridge partner edges. Bridge partner edges are labelled P for parallel and N for anti-parallel. The + or − label on an edge from a node A to a node B indicates which of the two sides of strand A strand B lies on, as per Westhead et al. *(1999). For example strand 6 to strand 3 is labelled + and strand 6 to strand 7 is labelled − so strands 3 and 7 are on opposite sides of strand 6 (in fact no two strands are on the same side of any strand in this figure). The width of nodes (rectangles for strands, ellipses for helices) is proportional to the length of the strand or helix.*

7.3.2 SSE and sheet orientation and tableaux

Tableaux are used to represent the orientation of SSEs, and are described in detail in Section 5.2.1 of Chapter 5.

We choose to define the orientation of sheets relative to one another by the orientation of their longest strands.

7.3.3 Distance matrices

As we treat sheets as indivisible elements in the cartoon, and want to place other objects relative to them, we need methods to determine the distances between sheets and other elements. We do this with a three-level hierarchy of distance matrices, in which the distances at each level are between residues, SSEs, and sheets (or helices), respectively.

We start with a conventional distance map, a square symmetric matrix where each element is the distance between the C_α atoms of two residues. We then build an SSE distance matrix, in which each element is the distance between two SSEs, defined as the distance between the two closest residues, one in each SSE. Note that this matrix has the same dimensions as the tableau — for any two SSEs, the tableau gives their relative orientation, and the (SSE) distance matrix gives their distance. The third level of distance matrix in the hierarchy is then built, in which the elements are either helices or entire sheets. The distance between two sheets is defined as the distance between the two closest strands, one in each sheet, and the distance between a helix and a sheet is defined as the distance between the helix and the strand in the sheet closest to it.

7.3.4 Sheet and barrel detection

Sheets and barrels are detected by finding connected components and cycles in the β-graph (Flower, 1994a, 1995; Westhead *et al.*, 1999), i.e. the graph where nodes are β-strands and edges denote bridge partner relationships between strands. This is done by a depth-first search (DFS) traversal of the graph. This results in every node being given a label that is the same for all nodes in a connected component, and distinct for any two nodes which are not in the same connected component.

A sheet is detected as a barrel if there is a cycle in the connected component corresponding to the sheet. We use the concept of a *fundamental cycle* or *basic cycle*. Such a cycle is created by adding a single edge to a spanning tree of the graph. In a connected graph, the number of fundamental cycles with respect to a spanning tree, N_f, equals $|E| - |V| + 1$, where E is set the of edges and V the set of vertices in the graph. Here, E is the set of bridge partner edges (treating each symmetric pair of edges as a single undirected edge) and V is the set of nodes representing strands in the connected component representing the sheet. Therefore, if $N_f > 0$ then the sheet forms a barrel.

There are a large number of algorithms for finding cycles in a graph (Downs *et al.*, 1989), but following Flower (1994a, 1995), we use a variation of Paton's algorithm (Paton, 1969) as a relatively simple and efficient method to find the fundamental cycles in the graph.

Barrels are "flattened" into a sheet by removing a bridge partner edge. The two strands between which the bridge relationship was broken then form the two ends of the sheet and are marked (with an asterisk) in the cartoon to indicate that the sheet is actually a barrel.

A slight modification of the algorithm used in TOPS as described by Westhead *et al.* (1999) is used to determine the relative positions of strands within sheets. The bridge partner strands of each strand are divided into two groups labelled + and −, all strands in a group being on the same side of the reference strand, which is the strand that they share in common as a bridge partner. The labels are put on the bridge edge pointing from the reference strand to each of its bridge partner strands (see Figure 7.14 for an example). As in TOPS, first a hydrogen bond overlap criterion and then (if the first criterion is inconclusive) a geometric criterion is used. However, unlike the overlap criterion described by Westhead *et al.* (1999), rather than test for the more restrictive criterion of bonds from two strands sharing a residue in the reference strand (meaning the two strands must be on opposite sides of the reference strand), we employ a different test. Our algorithm concludes that two strands that are bridge partners of a reference strand must be on opposite sides if there is any interleaving of hydrogen bonds from the partner strands to the reference strand. That is, the two bridge partner strands of the reference strand A, which we shall label B and C, must be on opposite sides of strand A if $B_{max} \geq C_{min} \wedge C_{max} \geq B_{min}$ where

B_{max} and B_{min} are, respectively, the highest and lowest residue sequence numbers in strand A that have hydrogen bonds to residues in strand B, and similarly for C_{max} and C_{min}. This is illustrated in Figure 7.15.

If this test does not conclude that they must be on opposite sides, a dihedral angle calculation similar to that described by Westhead *et al.* (1999) is used. This calculation is illustrated in Figure 7.16. Once all the bridge edges are a labelled + or − in this way, the relative positions of strands in the sheet are determined by the method described in Westhead *et al.* (1999). That is, a depth-first traversal of the graph is performed, and the label of the edge from which the current node was reached together with that of the edge from which the previous node was reached are used to determine whether to place the current strand on the same or opposite side of the previous node as the node from which the previous node was reached.

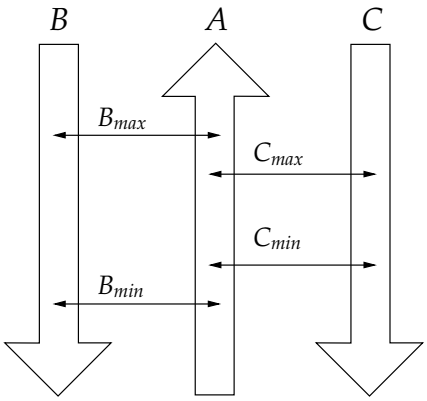
The relative positions of strands to their neighbours is then calculated by using the offset of the first residue in a strand with a hydrogen bond to its neighbour from the top of the strand, and using that offset as a distance between the top of the strand and the top of the neighbour that needs to be enforced by a constraint on the position of the strand on its axis (on which it is already constrained to lie). These offsets are summed along all strands starting from the first in the sheet, so that they are all relative to the top of the first strand. That is, the constraint on each strand specifies the exact distance between the top of that strand and the top of the first strand. This is illustrated in Figures 7.17 and 7.18.

In some rare cases, the offsets calculated in this way result in strands overlapping each other. This can occur due to irregularities in sheet structure such as extreme bending of strands or β-bulges. When this situation occurs, the constraint on strand offset is removed and the strands drawn with a minimum distance between them so that they do not overlap.

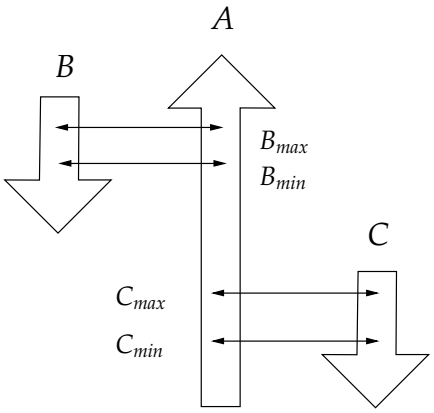
This defines the precise positioning of all strands in a sheet, and sheets can now be treated as single elements.

7.3.5 Secondary structure element placement and orientation

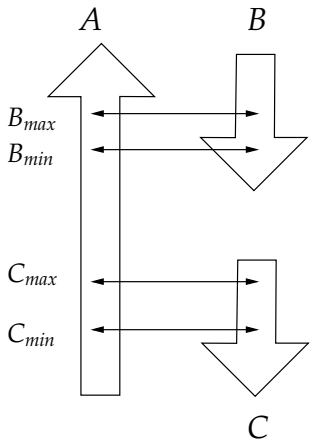
Two algorithms are implemented for placing and orienting sheets and helices: a greedy algorithm that uses distance matrix and tableau information for all sheets



(a) Hydrogen bond overlap criterion indicates strands must be on opposite sides.



(b) Hydrogen bond overlap criterion indicates strands need not be on opposite sides; the geometric test is needed to distinguish between this situation and that in Figure 7.15(c).



(c) Hydrogen bond overlap criterion indicates strands need not be on opposite sides.

FIGURE 7.15: *Hydrogen bond overlap criterion for determining if two strands are on the same or opposite sides of a reference strand.*

Strand A is the reference strand, relative to which we must determine whether strands B and C are on the same, or opposite, sides. The residues B_{min}, B_{max}, C_{min}, and C_{max} are residues on strand A.

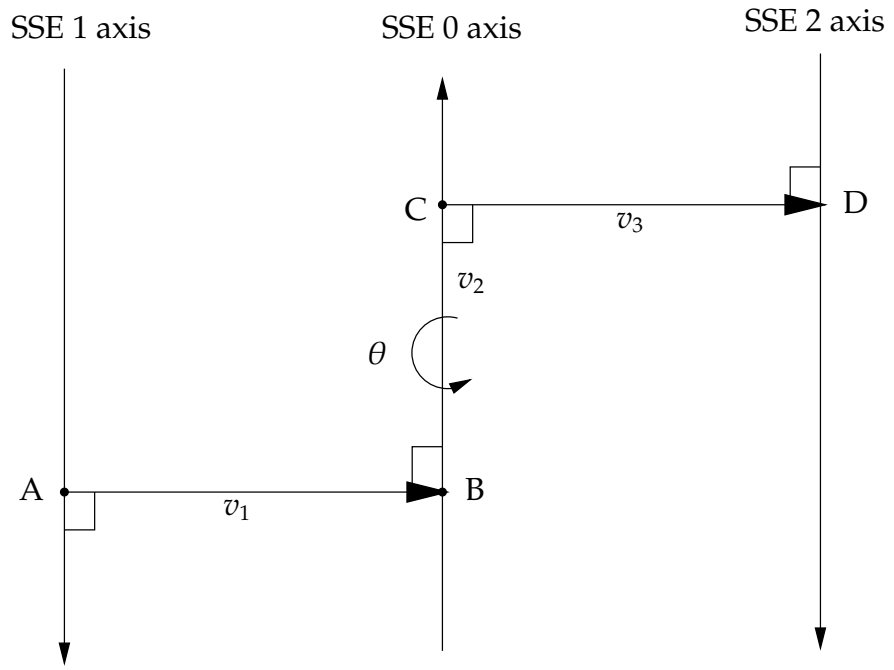


FIGURE 7.16: *Using dihedral angle between strand axes to determine if two strands are on the same or opposite sides of a common neighbour strand.*

AB is the shortest line segment between SSE 1 and SSE 0, defining vector v_1. CD is the shortest line segment between SSE 0 and SSE 2, defining vector v_3. v_2 is the vector defined by the segment BC. Then the dihedral angle θ between the planes ABC and BCD is given by $\cos(\theta) = \frac{v_1 \times v_2}{|v_1 \times v_2|} \cdot \frac{v_2 \times v_3}{|v_2 \times v_3|}$. If the dihedral angle $|\theta| > \frac{\pi}{2}$ then they are on opposite sides of the common strand, otherwise they are on the same side.

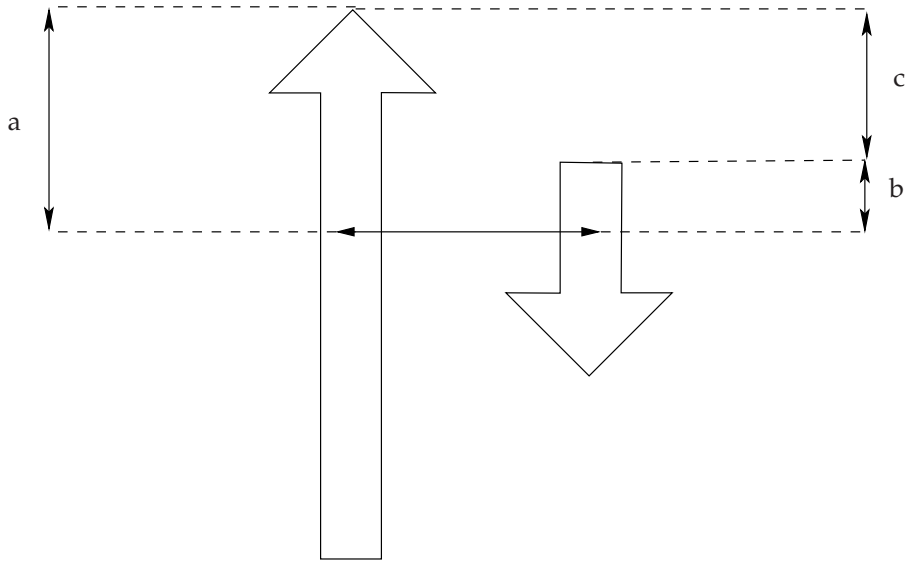


FIGURE 7.17: *The strand offset value is c where $c = a - b$.*

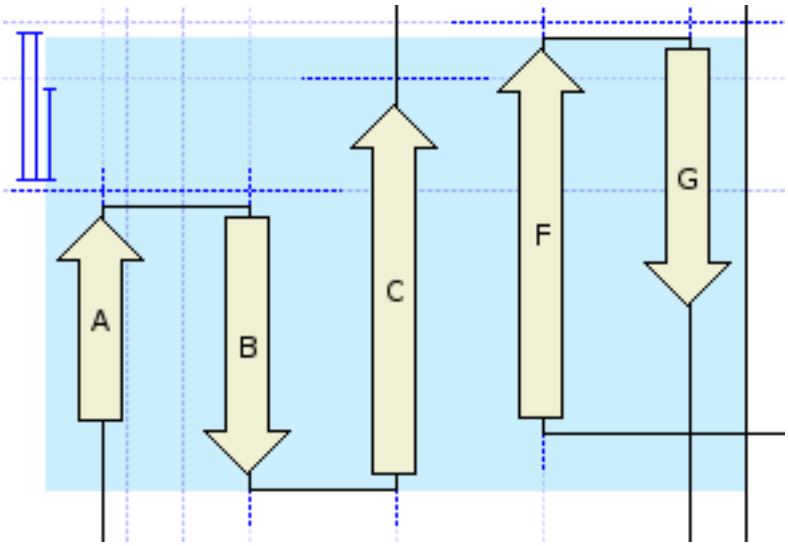


FIGURE 7.18: *Strand offsets used as exact spacing constraints.*

The strand offsets calculated as shown in Figure 7.17 are used as exact spacing constraints for the top of each strand in the sheet relative to the first strand in the sheet. This is a screen capture from the Dunnart diagram editor showing the indicator guides and handles for the exact spacing constraints on the larger sheet in the 1HH1 cartoon.

and helices, and a heuristic that only uses this information for sheets, but tries to place helices in such a way as to make the cartoon easy to read.

Greedy algorithm

The greedy algorithm is as follows:

1. Place the largest element somewhere near the middle of canvas.
2. Use the distance matrix (see Section 7.3.3) to find the closest element to some already placed element.
3. Position that closest element relative to the chosen already placed one, using distance matrix and tableaux to determine relative position and orientation.
4. Repeat from step 2 until all elements placed.

The largest element is the sheet containing the largest number of residues. If there is no sheet in the domain, the longest helix is used as the largest element.

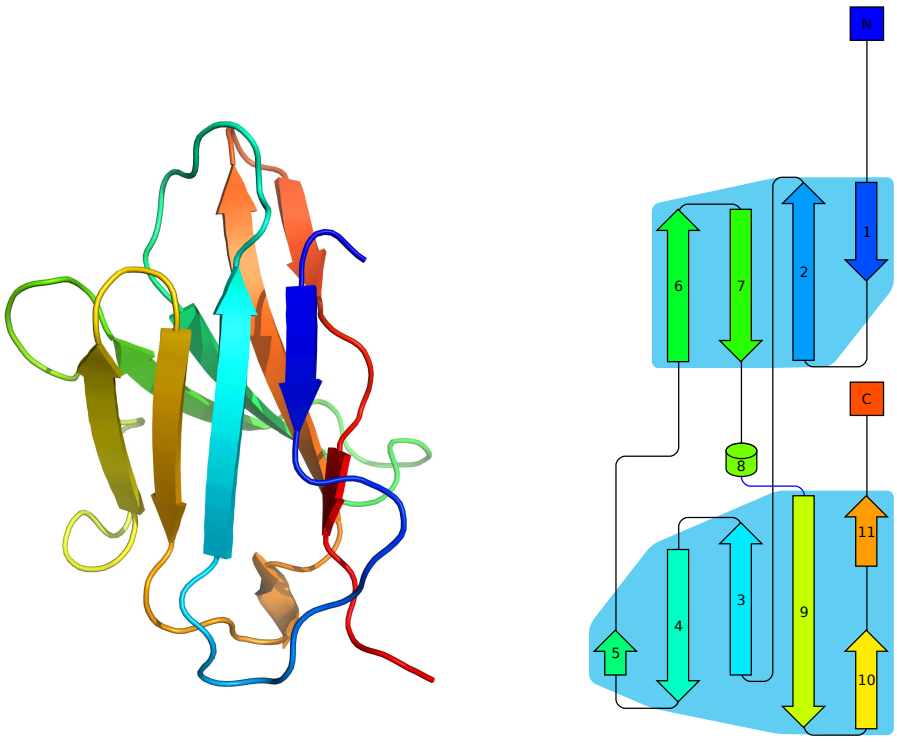
Sheets that are placed near each other are placed so that their closest strands are nearby, and aligned if they have parallel or anti-parallel orientation.

The relative orientation of two sheets is determined by the orientation of their longest strands, according to the tableau for the domain (see Section 5.2.1 of Chapter 5). This results in a reasonable two-dimensional representation of the relative orientation of sheets in the case that the sheets are approximately co-planar or that the planes in which the sheets lie are orthogonal in three-dimensional space. However, when the planes in which the two sheets lie are approximately parallel, as for example in the "face-to-face" packing of sheets in β-sandwich motifs (Cohen *et al.*, 1981), a more easily understood representation is obtained by reversing the orientation (e.g. from anti-parallel to parallel). To see this, consider "unfolding" the two sheets so that they are co-planar using the coil regions between them as a "hinge": two strands that are anti-parallel in the original configuration become parallel when a sheet is rotated so that the two sheets lie in the same plane. This is illustrated in Figure 7.19.

A heuristic is used to determine if this reversal of relative (parallel or anti-parallel) orientation is a more reasonable representation than the tableau orientation. If at least two strands in one of the sheets are immediately adjacent in sequence to strands in the other sheet, then it is considered a better representation to "unfold" the sheets around the coil regions between these strands as a "hinge", and the orientation of the sheets is therefore reversed.

This heuristic is a weaker test than a test explicitly for the β-sandwich structures, such as that implemented in TOPS, which uses a minimum number intersheet strand-strand contacts to classify two sheets as a β-sandwich (Westhead *et al.*, 1999). We want to "unfold" not only β-sandwiches this way, but any structure which is more likely to be interpreted correctly from a cartoon in which packed sheets have been unfolded in this manner. An example of a structure that is not a β-sandwich but for which this "unfolding" heuristic is also used is shown in Figure 7.20.

Helices are placed near the strand to which they are closest. Connectors are drawn so that they reflect the orientation of the strands or helices they connect to. For strands, this information is already visible as they are drawn as an arrow pointing towards the C-terminal direction, but for helices, which are drawn as cylinders, the connector, which goes into the N-terminal end and out of the C-terminal end, is a visible indicator of the orientation of the helix.

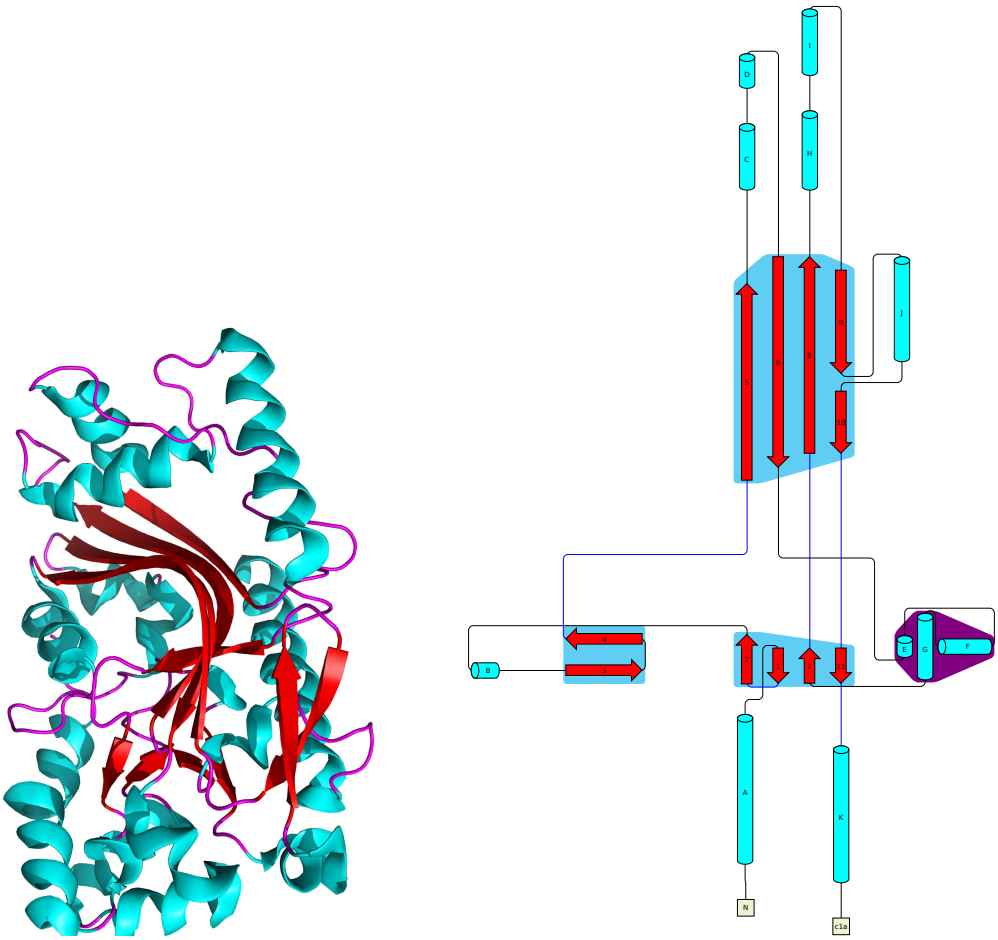


(a) A β-sandwich structure rendered by PyMOL, positioned so that the N-terminal sheet is in front of the C-terminal sheet.

(b) A Pro-origami cartoon of the β-sandwich. The C-terminal sheet is positioned below the N-terminal sheet, and oriented as if the sandwich had been "unfolded" with the coil regions at the bottom of Figure 7.19(a) as a "hinge"

FIGURE 7.19: *Side-by-side comparison of a 3D cartoon and a Pro-origami cartoon of β-sandwich structure (the human T cell coreceptor CD8 (Leahy* et al.*, 1992), PDB identifier 1CD8).*

The structure has been classified as one in which a better representation is obtained by reversing the orientation of the strands, so that, for instance, the most C-terminal strand (strand 11) is drawn as anti-parallel to the most N-terminal strand (strand 1) in the Pro-origami cartoon, although the relative orientation (tableau orientation) of these strands is parallel in the actual structure, as can be seen in the 3D cartoon.



(a) The MACPF domain rendered by PyMOL, positioned to show the large B sheet (with bend) and the C sheet immediately below it.

(b) A Pro-origami cartoon of the MACPF domain. The orientation of the C sheet (strands 2, 1, 7, and 11) relative to the B sheet (strands 5, 6, 8, 9, and 10) has been chosen so that the strands immediately following each other in sequence (7 and 8, and 10 and 11) are oriented in the same direction, despite this orientation not necessarily reflecting their real orientation in 3D space (as shown in Figure 7.20(a)).

FIGURE 7.20: *A 3D cartoon and Pro-origami cartoon of the N-terminal domain of Plu-MACPF (Rosado* et al.*, 2007), PDB identifier 2QP2.*

Although this structure is not a β-sandwich, the same "unfolding" heuristic is used in order to obtain a cartoon that is easier to interpret.

Heuristic helix placement

In the heuristic algorithm for SSE placement, only sheets are placed using the greedy algorithm previously described. Helices are then positioned on the axes of strands to which they are adjacent in sequence, and when a choice can be made between two strands (i.e. when a helix or sequence of helices are between two strands, whether in the same sheet or not), they are placed on the axis that has fewest items already aligned on it. An exception is made for helices that interrupt strands on a common axis in a sheet (i.e. a bifurcation) — these helices are drawn beside the sheet and not aligned on the strand axis. An example of this is shown in Figure 7.20, in which helix J has been placed beside the two strands (9 and 10) that it is between in sequence.

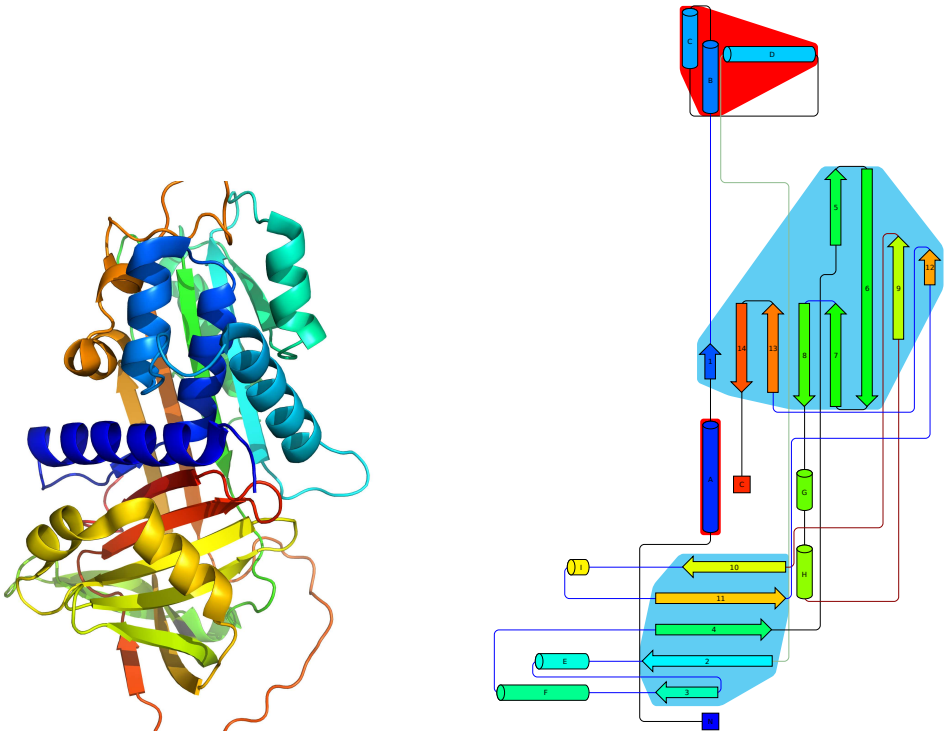
Optionally, a further rule may be employed to prevent helices from being placed between neighbouring sheets. When this rule is used, helices that are not in immediate sequence between a strand in one sheet and a strand in another sheet that has been placed next to the first one are placed in another position where they do not interfere with the relative placement of sheets. This rule has been enabled in generating the cartoon shown in Figure 7.20, thereby preventing the long N-terminal helix (helix A) from being aligned above the strand immediately following it in sequence (strand 2), which would place it between the two sheets.

When using the heuristic algorithm, connector placement on helices is no longer indicative of helix orientation, and is chosen purely to try to minimize crossings in order to make the cartoon easy to read.

7.3.6 Helix clustering

When using the heuristic algorithm for helix placement described in Section 7.3.5, helices may optionally be "clustered" so that a series of helices between strands (or between a strand and a terminus) are placed together in a group, or cluster, rather than aligned on strand axes. In such a cluster, the greedy algorithm described in Section 7.3.5 is used to place helices relative to each other in the cluster.

The helix in the cluster that is immediately adjacent in sequence to a strand is aligned on the axis of that strand, and used as the starting element for the greedy placement algorithm. The sheet containing this strand is used as the reference for



(a) 3D rendering of 1QLP, rotated to show helix A (blue, positioned horizontally across centre of diagram) and the cluster of helices B,C,D (also blue, above helix A)

(b) Helices B,C, and D form a cluster, and the single-helix cluster consisting of helix A has the same shading colour as the three-helix cluster, indicating that it is close to that cluster.

FIGURE 7.21: *A 3D PyMOL cartoon side-by-side with a Pro-origami cartoon for α_1-antitrypsin (Elliott* et al.*, 2000), PDB id 1QLP, with helix clustering and helix proximity colouring enabled.*

deciding on which side of an already-placed helix the next helix is to be placed. This is done by deciding if the new helix is on the same, or opposite, side of the strand with which the already placed helix is aligned as a neighbour (in the sheet) of that strand. This decision is made using the dihedral angle calculation described in Figure 7.16, with the central axis in that figure as the axis of the strand that the already placed helix is aligned on, and the other two axes are those of the new helix and a strand adjacent to the first strand.

As with strands in a sheet, helices in a cluster are grouped together as a single unit in the diagram, and given a shaded background to indicate that they form a single cluster. By default, this background shading is the same colour for all helix clusters. Optionally, however, they may be coloured so that only nearby helix clusters are the same colour as each other. Nearby clusters are defined as those in which the distance between the two closest helices (determined from the distance

matrices described in Section 7.3.3), one from each cluster, is less than a threshold, currently set to 7.0 Å.

In order to be given the same shading colour, all helix clusters must be pairwise close (where "close" is defined above). That is, every helix cluster must be close to every other helix cluster of the same shading colour. This prevents the helix clusters having the same colour in the situation where for example helix cluster A is close to helix cluster B, and helix cluster B is close to helix cluster C, and they are all shaded the same colour, although helix cluster C may not be close to helix cluster A.

This can itself lead to problems in that the membership of the group of nearby clusters can now depend on the order in which the clusters are processed. This is resolved by using another greedy technique: the list of candidate clusters is sorted by distance and processed starting at the closest.

For the purposes of this helix cluster proximity colouring, we allow a helix cluster to consist of a single helix in order that it may be coloured to show its proximity to one or more multi-helix clusters to which it is close.

7.3.7 Domain placement

Domains are always processed independently (unless domain decomposition is disabled, so the entire protein is considered as one domain), and may be shown as separate cartoons, one for each domain, or all placed on one cartoon.

Note that domains, not chains, are processed independently. Multiple chains may exist in a single cartoon, even as parts of a single sheet or helix cluster. Chains are only processed independently if they are declared to be separate domains according to the domain decomposition method being used.

When domains are drawn on separate cartoons, pseudo-terminus symbols are drawn to show that the chain continues in a separate domain. The same system may be selected for multi-domain cartoons, or alternatively the connectors may be drawn on the multi-domain cartoon showing coil regions between domains.

Domains are placed relative to each other using the same greedy algorithm as that described in Section 7.3.5 for SSE placement. The largest domain is placed first, then the nearest domain is placed, and so on.

The largest domain is defined as the domain containing the largest sheet (or the longest helix if there are no sheets). There are several alternative definitions for the

largest domain that also make sense, such as the one with the most SSEs, or with the most residues. We choose the one with the largest sheet as we believe that is the element of both a three-dimensional protein picture and the cartoon that is seen as central, and around which viewers tend to orient other structures. If we choose the domain with the most SSEs as the largest, we may end up with a domain that has many small helices as the largest domain, and similarly if we use the domain with the most residues we may find as largest a domain with large helices. We wish to avoid such situations as we would prefer a sheet to be the central element relative to which others are placed.

The distance between two domains is defined as the distance between their two closest SSEs, using the distance matrices described in Section 7.3.3. The relative orientation of two domains is the tableau orientation (as described in Section 5.2.1 of Chapter 5) between the longest strands in the largest sheets of each domain. Domains are placed so that their closest SSEs are nearby, in the same manner as for sheet placement within a domain.

7.3.8 Shape colouring and labelling

The symbols representing helices and strands may either be labelled sequentially starting from 1 and continuing in an N- to C-terminal direction along chains (continuing the numbering between chains), or with separate numbering for helices and strands. In the latter case, helices are labelled A, B, C, and so on, from the N- to C-terminal direction, restarting for each chain, and strands are labelled in a similar fashion, but numerically as 1, 2, 3, and so on. In the latter case, a (lowercase) chain identifier is also appended to the label for cartoons that contain multiple chains.

In both cases, the N- and C- terminal ends of a chain are labelled with N and C symbols respectively, with a chain identifier appended in the case of multi-chain cartoons. Figure 7.23(b) is an example of a protein containing two chains, showing the labelling of the N- and C-terminus symbols with a chain identifier. As described in Section 7.3.7, "pseudo-terminus" symbols may be introduced to show where a chain crosses a domain boundary. These symbols are labelled with lowercase "n" or "c" (at the N- and C-terminal sides of the boundary respectively), with a domain identifier (1, 2, 3, and so on) and, for multi-chain proteins, a chain identifier appended. An example of this may be seen in Figure 7.20(b), which shows the N-

terminal domain of a two-domain protein. The pseudo-terminus node is labelled "c1a" to indicate that this termination of chain A in domain 1 at the C-terminal end is an artifact of domain decomposition, and in domain 2 (not shown), it continues at a pseudo-terminus labelled "n2a".

The SSE symbols may be coloured according to several schemes. The simplest, suitable where colour printing is not available, is to simply colour everything the same default colour. Another is to colour all strands in sheets one colour, helices in a cluster another colour, and other helices a third colour. Optionally, each sheet may be shaded a different colour.

Another colouring scheme is to colour helices and strands in a colour gradient from the N- to C- terminus to allow easy visual interpretation of the approximate position in the sequence of a particular SSE. In order to match the scheme used in PyMOL (DeLano, 2002), the colour gradient runs from blue at the N-terminus to red at the C-terminus. Using this scheme, it is easy to see the correspondence between an SSE in the PyMOL three-dimensional picture and the Pro-origami cartoon by their similar colour.

The colour gradient is obtained by moving the H (hue) value in the HSV (hue, saturation, value) colour space from 240° (blue) to 0° (red) in the negative direction, and converting each point in the HSV colour space to the RGB colour space using the algorithm described in Foley and van Dam (1982, p. 616).

A rather different colouring scheme is the "folding" colouring scheme. In this colouring scheme, strands in a sheet that are contiguous, that is, connected only by a single turn or coil region (connector in the Pro-origami cartoon), are all coloured the same colour. Each such set of contiguous strands in a sheet is given its own colour. In this way, it is easy to see immediately the sets of strands in a sheet are connected directly in sequence. In order to make these sets visibly distinct, we wish to make the colours used for each set in a sheet maximally distinct; for this purpose we use the set of 32 colours from Table II of Glasbey *et al.* (2007).

7.3.9 Connector colouring

The coil regions between SSEs are represented in the cartoon as lines between them. In terms of the constraint-based diagram editor these lines are connectors, specified by their two endpoints. These connectors are represented as orthogonal poly-lines

and routed so as to avoid objects and try to minimize length and reduce crossings by the Dunnart constraint-based diagram editor (Dwyer *et al.*, 2007; Wybrow *et al.*, 2006; Wybrow, 2008; Dwyer *et al.*, 2009).

Connectors are by default all drawn in black, but can also be drawn so that each chain has different coloured connectors, or so that connectors between domains are a different colour than connectors inside domains. Colours for connectors are chosen from a list of dark colours (brown, dark blue, and violet for instance) so that they are easily visible against the (white) background.

Despite the Dunnart connector routing algorithms, and the helix placement heuristics described in Section 7.3.5 that tend to reduce crossings by aligning helices on axes rather than orienting them according to their tableau orientation, crossings of connectors are inevitable in the cartoons for proteins of reasonable size or complexity.

When connectors cross, or are routed so that two or more connectors share a path and therefore are drawn closely beside each other for some distance, it can become difficult to follow the path of the chain by eye. The latter problem is reduced to some degree by the "nudging" feature of Dunnart, which finds such connectors and moves them further apart (Wybrow *et al.*, 2010). However, when a larger number of connectors are involved, and for crossings, and connector "overlaps", it may still be difficult to follow the path of connectors when reading the cartoon.

In order to solve this difficulty, another connector colouring scheme is provided, in which connectors are coloured so that no two connectors which cross or share a path have the same colour. This could be achieved trivially by giving every connector a different colour, but we would prefer to minimize the number of colours used in order to make the diagram easier to interpret.

In order to do this, an *interference graph* is built, in which the nodes correspond to connectors, and an edge between two nodes represents those connectors interfering. Two connectors are said to interfere if they cross, or share a path for some part of their routing. Assigning colours so that no two interfering connectors have the same colour corresponds to the graph colouring problem, which is in general NP-hard (Garey and Johnson, 1979).

As we do not require the absolute minimum number of colours, the use of a heuristic which quickly finds a valid colouring (with no guarantee that the minimum

number of colours is used) is satisfactory. For this purpose we use the Welsh-Powell algorithm (Welsh and Powell, 1967), a greedy algorithm that assigns colours to nodes in decreasing order of degree, resulting in a valid graph colouring using $\alpha(G)$ colours where

$$\alpha(G) \leq \max_i \min \begin{cases} d_i + 1, \\ i \end{cases} \tag{7.1}$$

in which d_i is the degree of node i and the nodes are ordered by decreasing degree, that is $d_1 \geq d_2 \geq \ldots \geq d_n$ (Welsh and Powell, 1967).

7.3.10 Diagram layout

The Pro-origami algorithm generates a set of cartoon elements (helices and strands), constraints on their layout relative to each other, and a set of connections between them (a connector between two elements means they are adjacent in the chain). The constraints consist of groupings of strands into sheets and the relative positions of strands in a sheet, separation between adjacent strands in a sheet, and the alignment of strands and helices on horizontal or vertical axes.

The descriptions and initial positions of the cartoon elements and the set of constraints and connectors is then represented in scalable vector graphics (SVG)[3] format as input to the Dunnart constraint-based diagram editor. Dunnart then uses the `IPSep-CoLa` algorithm (Dwyer *et al.*, 2006) to lay out the diagram according to the constraints, ensuring that elements do not overlap. Orthogonal connectors between the elements are routed so as to not collide with them, and to try to reduce unnecessary crossings (Dwyer *et al.*, 2007; Wybrow *et al.*, 2006, 2010).

Dunnart can be run in a "batch" mode, so that, given a PDB file, a script can be run that outputs a Pro-origami cartoon for it in SVG format, which can then be rendered for viewing. If manual editing of the cartoon is desired, this can be done with Dunnart, which is primarily an interactive constraint-based diagram editor (Dwyer *et al.*, 2009).

7.3.11 Implementation

Pro-origami has been implemented in Python with the BioPython (Cock *et al.*, 2009) and Numeric libraries, and uses the `Bio.PDB` PDB file parsing and structure class

[3] `http://www.w3.org/Graphics/SVG`

(Hamelryck and Manderick, 2003) to parse PDB files. The cartoon is represented in the form of SVG for input into the Dunnart constraint-based diagram editor[4] (Wybrow *et al.*, 2006; Wybrow, 2008; Dwyer *et al.*, 2009; Wybrow *et al.*, 2010), which lays it out according to the initial positions and constraints specified, saving the resulting diagram in SVG, and allows interactive editing of the diagram. Connector interference colouring is implemented as part of the Dunnart code, in C++.

7.4 Results and discussion

We ran Pro-origami on all 16712 domains in the ASTRAL SCOP 1.75 95% sequence identity non-redundant database (Chandonia *et al.*, 2004; Andreeva *et al.*, 2008). This results in a total of 16602 cartoons, since 110 have no secondary structure according to DSSP. Pro-origami produces a cartoon with at least one connector "overlap" for only 2.29% of these domains. On our server, a Dell PowerEdge R200 (Intel Xeon 3.2 GHz dual core, 8 GB memory, running Linux) the average time taken to generate a cartoon in this set is 2.3 seconds.

Figure 7.22 shows the cartoon of 1HH1 from Pro-origami (with heuristic helix placement) side-by-side with one manually drawn by the authors of the paper describing its structure (Bond *et al.*, 2001). It can be seen that the style of the Pro-origami cartoon is very similar to the manually-drawn one, and contains information on the lengths and relative positions along their axes of strands in a sheet, which is not depicted in a TOPS cartoon (Figure 7.10). The smaller sheet (strands D, E, H, and I) is drawn in a different orientation to that in the manually-drawn cartoon. This is because strands C and E (the longest strands in each of the two sheets) are closer to orthogonal than parallel or anti-parallel to each other (according to the relative angle between them around their mutual perpendicular in 3D space — see Section 5.2.1), and the Pro-origami algorithm therefore draws them as orthogonal in the 2D cartoon. A different orientation, however, has been chosen in the manually-drawn cartoon, one which has the beneficial effect of requiring no connector crossings at all. Figure 7.22(c) shows the 1HH1 Pro-origami cartoon with the greedy distance matrix based helix placement algorithm used rather than the heuristic helix placement. The position and orientation of the helices now better

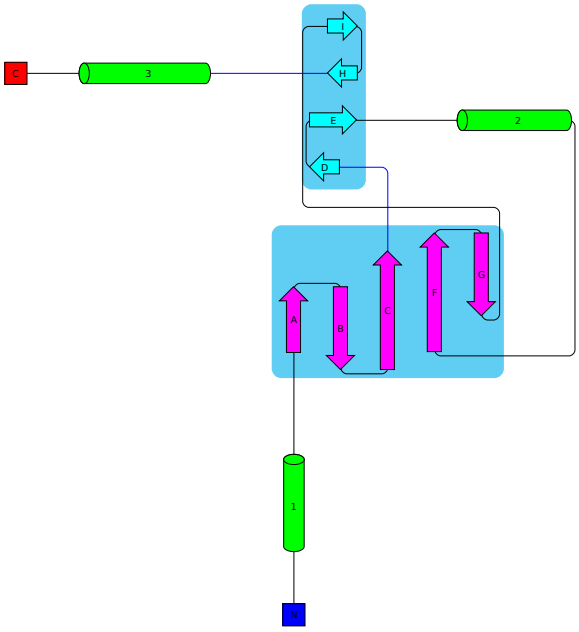
[4] `http://www.csse.monash.edu.au/~mwybrow/dunnart`

reflects their actual orientation and position in three-dimensional space (compare Figure 7.7). Note that this has also affected the placement of the smaller sheet, since the order of placement of sheets and helices is different when using this algorithm for this structure. Also, this algorithm results in a much "messier" diagram than the heuristic placement (Figure 7.22(a)): more connector crossings and more bends in the connectors are required. This is generally true for most structures, and hence the heuristic algorithm is the default, as it usually produces easier to interpret cartoons.

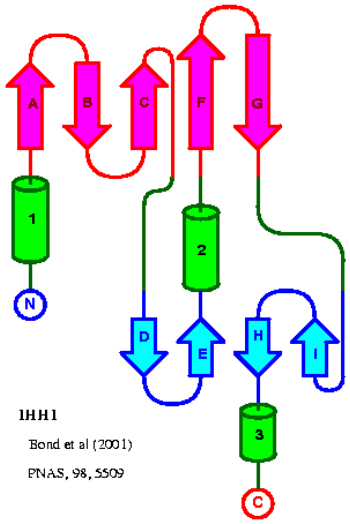
Figures 7.23(a) and 7.23(c) are a comparison of a Pro-origami cartoon and TOPS cartoon, respectively, for α_1-antitrypsin (Elliott *et al.*, 2000), PDB id 1QLP. Note that the Pro-origami cartoon clearly makes visible the bifurcation at the longest strand (strand 6) resulting in strands 5, 9 and 12 being clearly visible as offset towards the N-terminal end of strand 6 relative to strands 1, 14, 13, 8 and 7. This information is not visible in the TOPS cartoon.

To illustrate the fact that Pro-origami draws cartoons that are similar to those drawn manually with TopDraw, we show another example comparing an automatically generated Pro-origami cartoon with a manually drawn topology cartoon from the literature. Figure 7.24 compares the manually drawn topology cartoon, the TOPS cartoon, and the Pro-origami cartoon for *E. coli* MobB (McLuskey *et al.*, 2003). The Pro-origami and manually drawn cartoons are very similar, differing mainly in the placement of helices. The strands and helices in the manually drawn cartoon (Figure 7.24(c)) have been coloured so that the two subunits (chains) are red and black, with blue and green used to indicate some structural homologues and a particular motif (McLuskey *et al.*, 2003). This latter information cannot be automatically generated by Pro-origami, and so is an example where manual editing could be used to add this (or other) information that cannot be automatically generated. The connectors in the Pro-origami cartoon (Figure 7.24(d)) are coloured red and black to indicate the two different chains, similar to the use of red and black strands and helices in the manually drawn cartoon.

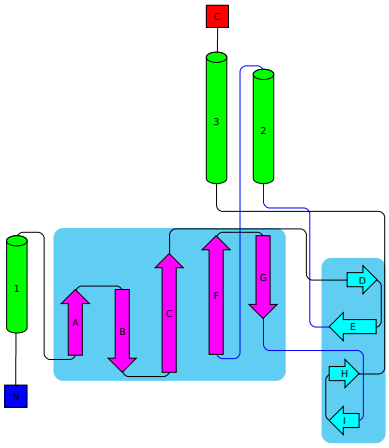
As has just been demonstrated in the previous example (Figure 7.24(d)), Pro-origami does not treat chains as separate entities in its processing; multiple chains may exist in a single cartoon, even as parts of a single sheet. This can be of some importance in showing similarity between structures that may consist of multiple chains in one PDB entry but not another. Figure 7.23 shows an example of this,



(a) 1HH1 cartoon automatically drawn by Pro-origami and Dunnart, with heuristic helix placement.

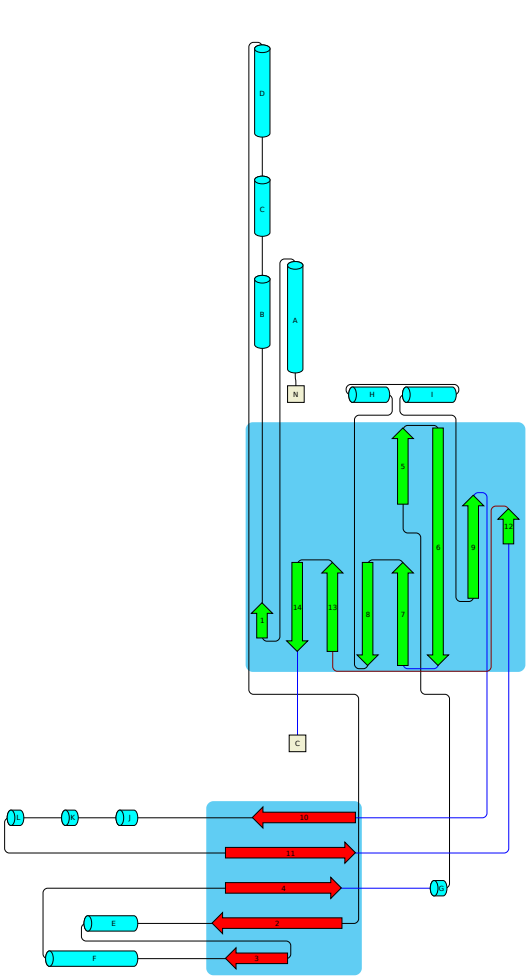


(b) 1HH1 cartoon manually drawn by Bond *et al.* (2001) using TopDraw (Bond, 2003), as distributed as the example in TopDraw.

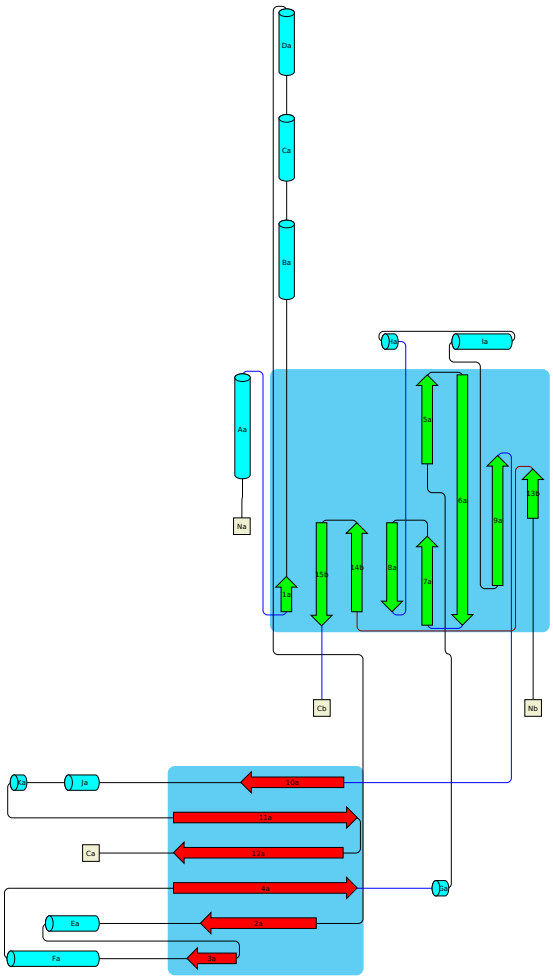


(c) 1HH1 cartoon automatically drawn by Pro-origami and Dunnart, with distance matrix (greedy) helix placement.

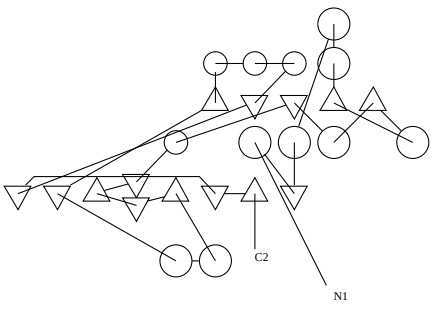
FIGURE 7.22: *Comparison of a manually drawn cartoon of 1HH1 with those automatically generated by Pro-origami and Dunnart (Wybrow, 2008; Dwyer* et al.*, 2009).*



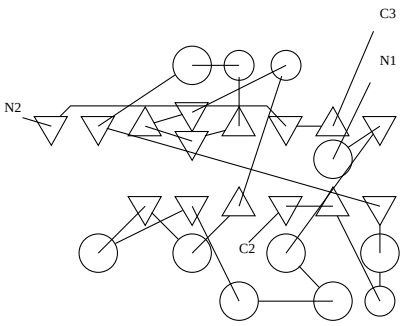
(a) A Pro-origami cartoon for the canonical active serpin, α_1-antitrypsin, PDB id 1QLP (Elliott *et al.*, 2000).



(b) A Pro-origami cartoon for the prokaryote serpin thermopin, PDB id 1MTP (Irving *et al.*, 2003).

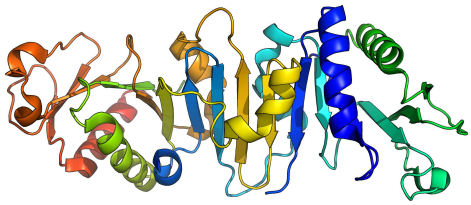


(c) 1QLP TOPS cartoon.

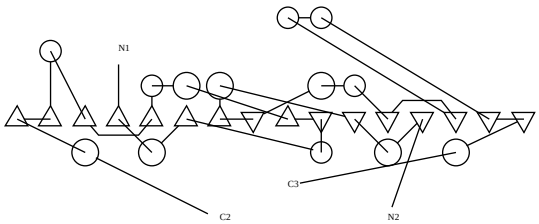


(d) TOPS cartoon for 1MTP.

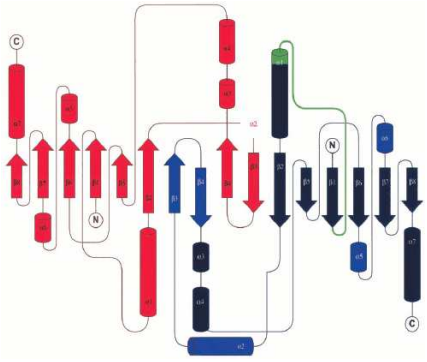
FIGURE 7.23: *Pro-origami and TOPS cartoons for two serpins, one of which has two chains in the PDB structure, showing that the structural similarity between them is still strongly conveyed in these cartoons.*



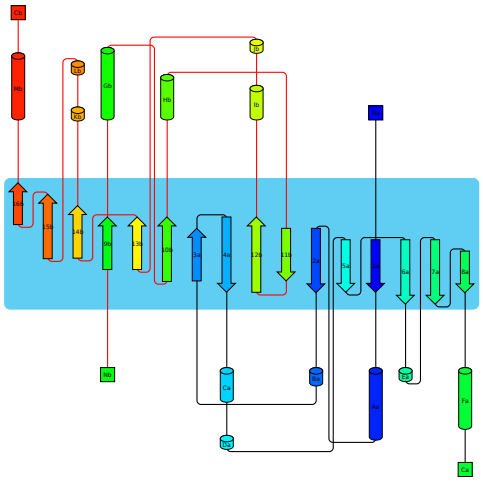
(a) 3D rendering generated with PyMOL.



(b) TOPS cartoon.



(c) Topology cartoon drawn by McLuskey *et al.* (2003) with TopDraw. This figure is reproduced from Fig. 2B of McLuskey *et al.* (2003) ©The American Society for Biochemistry and Molecular Biology.



(d) Pro-origami cartoon.

FIGURE 7.24: *Comparison of different cartoon styles for E. coli MobB.*

3D structure, TOPS cartoon, manually drawn cartoon, and Pro-origami cartoon for E. coli MobB, PDB identifier 1NP6 (McLuskey et al.*, 2003). Strands and helices in the Pro-origami cartoon are coloured in a gradient from the N- to C- terminus (continuing from chain A to chain B) in order to match the colours in the PyMOL cartoon.*

in which Pro-origami clearly shows the similarity between the two serpins, one of which has two chains. TOPS can also include multiple chains in one cartoon, as it has in the case of Figure 7.23(d), but the structural similarity is not as clearly conveyed in the TOPS cartoons (Figures 7.23(c) and 7.23(d)) as it is in the Pro-origami cartoons (Figures 7.23(a) and 7.23(b)), even allowing for the fact that TOPS has drawn the two serpins in different orientations.

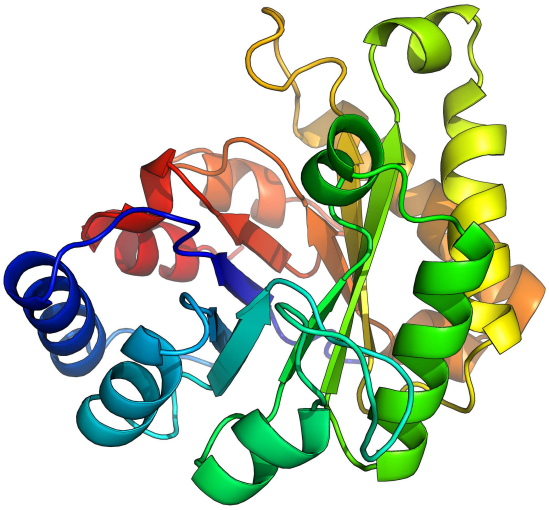
The relative orientation of sheets in a cartoon is usually determined by the relative orientation of strands in those sheets in three-dimensional space, as described in Section 5.2.1. In some cases, however, a better representation is obtained by reversing the orientation, as if the two sheets were "unfolded" along a "hinge" formed by coil regions between the sheets. This is particularly the case for β-sandwich motifs, such as that shown in Figure 7.19, but it is also a more suitable orientation for some other structures involving two sheets that are not a sandwich motif, but which nevertheless are quite far from co-planar. The membrane attack complex/perforin (MACPF) domain of the Plu-MACPF protein from *Photorhabdus luminescens* (Rosado *et al.*, 2007), shown in Figure 7.20, is such a structure. The heuristic used for determining when such a representation is suitable is described in Section 7.3.5.

As discussed in Section 7.1, Pro-origami draws β-sheets with only the strands in the sheet, never with helices between, or superimposed on, the strands. In addition, since the Pro-origami cartoon style is "side-on" rather than the TOPS "top-down" style, barrel structures are "flattened" as described in Section 7.3.4. This leads to quite different representations of such barrel structures between different cartoon methods, illustrated in Figure 7.25. The TOPS cartoon (Figure 7.25(b)) arranges the parallel strands that make up the barrel in a circle, with the helices on the outside. The PDBsum (Figure 7.25(c)) and Pro-origami (Figure 7.25(d)) cartoons must "flatten" the barrel out due to their "side-on" style, with the Pro-origami cartoon marking the two strands that have had their bridge partner relationship broken in the "flattening" process marked with asterisks. Although the parallel relationship of the strands is clear in all three 2D cartoon styles, the TOPS "top-down" style shows the barrel structure most clearly. The PDBsum and Pro-origami cartoons are quite similar, although the Pro-origami cartoon is "neater" and we would argue, easier to interpret visually, since it is more consistent in its placement

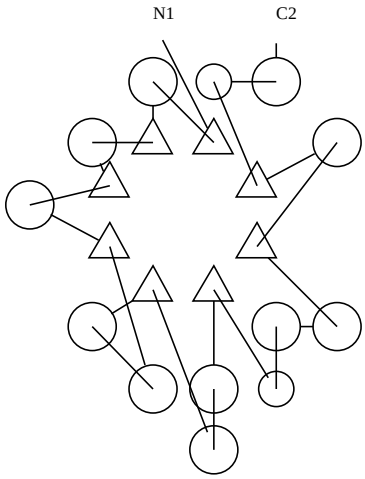
of helices, and has fewer connector crossings. The use of asterisks on the labels to mark the two strands on the ends of the sheet indicating that it is actually a barrel are also an improvement on the PDBsum cartoon, although this feature could be trivially added to PDBsum also.

By default, Pro-origami shares with TOPS the shortcoming that the secondary structure is defined by DSSP or STRIDE and therefore may not agree with what a human expert would assign (Westhead *et al.*, 1999). For example, in 1QLP (Figure 7.23(a)), the large sheet shown as a single sheet in the cartoons is generally considered to be two sheets, with the long strand (strand 6) shared, as that strand has such a degree of curvature that the sheet consisting of part of that strand together with strands 5, 9 and 12 is roughly orthogonal to the larger part. The visibility of the offsets of the strands along the axis on which they lie in the Pro-origami cartoon, however, at least allows some visibility of the distinction between these two parts of the sheet compared to the TOPS cartoon. To help overcome this problem, an option is provided to use the HELIX and SHEET records from the PDB file to define the secondary structure: depending on the author of the PDB file, these may be manually defined, or adapted from the output of STRIDE or DSSP. 1QLP, for example, has quite different definitions of the sheets in its PDB file than those assigned by DSSP or STRIDE (Figure 7.26 and Figure 7.27).

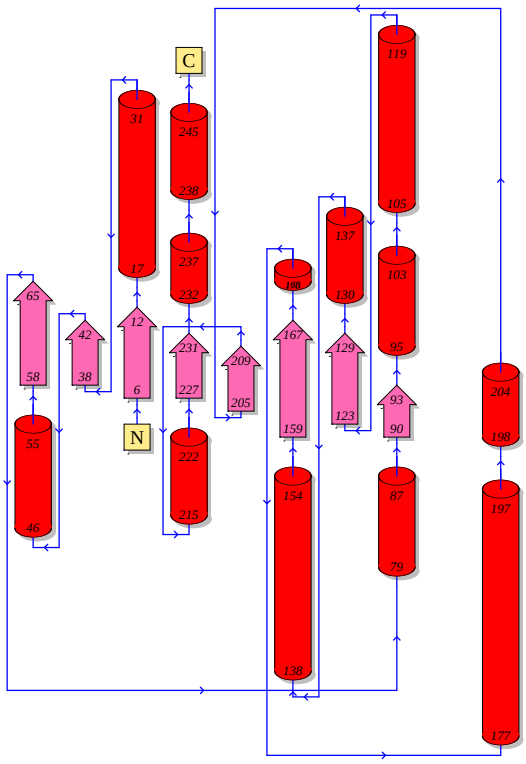
Figure 7.28 compares TOPS, PDBsum, and Pro-origami cartoons for subtilisin. Subtilisin has an interesting structure in that it contains a left-handed β-α-β motif (Sternberg and Thornton, 1977; Branden and Tooze, 1999, p. 217). This is unusual as almost all β-α-β motifs are right-handed (Sternberg and Thornton, 1977; Branden and Tooze, 1999, pp. 28,48). That this structure contains a left-handed β-α-β motif is evident from the 3D rendering in Figure 7.28(a): if all three β-α-β motifs had the same chirality, they would all be on the same side of the parallel β-sheet. It is evident, however, that one of the α-helices (coloured cyan) is on the other side of the sheet from the other two (coloured green), so the β-α-β motif to which it belongs is therefore of opposite chirality. This information is clearly conveyed in the TOPS cartoon (Figure 7.28(b)), which shows the helix from the left-handed β-α-β motif on the other side of the parallel β-sheet from the helices in the right-handed β-α-β motifs. Neither the PDBsum nor Pro-origami cartoons convey this information. It may appear that the PDBsum cartoon (Figure 7.28(c)) does so, since the helix



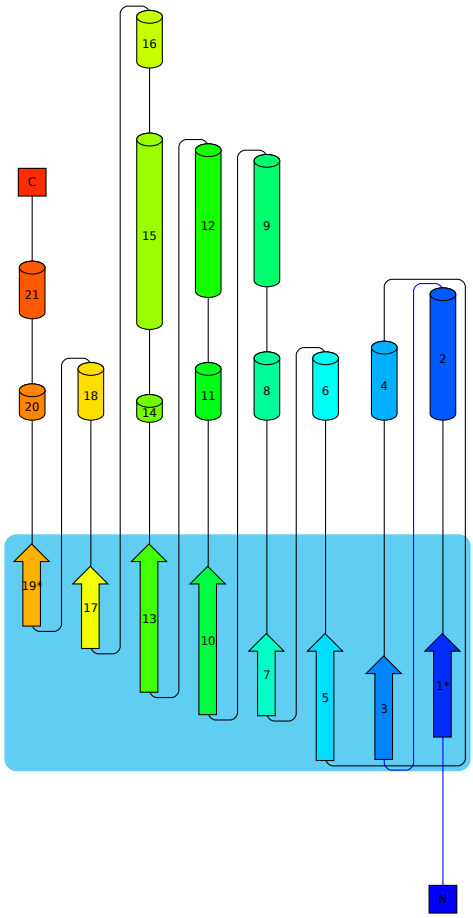
(a) 3D rendering generated with PyMOL.



(b) TOPS cartoon.



(c) PDBsum topology diagram.



(d) Pro-origami cartoon.

FIGURE 7.25: *Comparison of different cartoon styles for a TIM-barrel structure.*

3D structure, TOPS cartoon, PDBsum cartoon and Pro-origami cartoon for chain A of PDB id 1TIM, triose phosphate isomerase from chicken muscle.

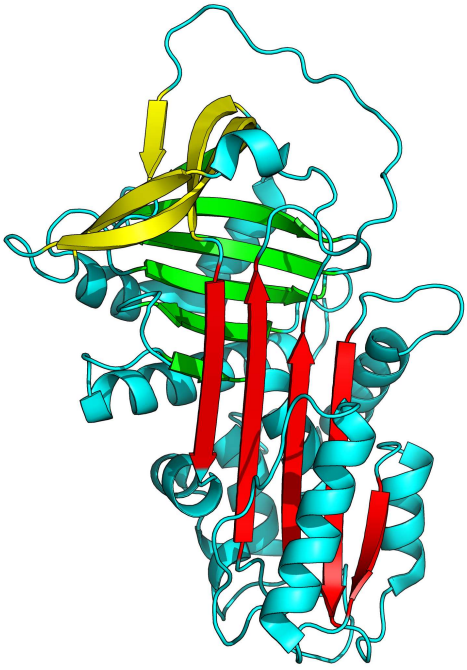
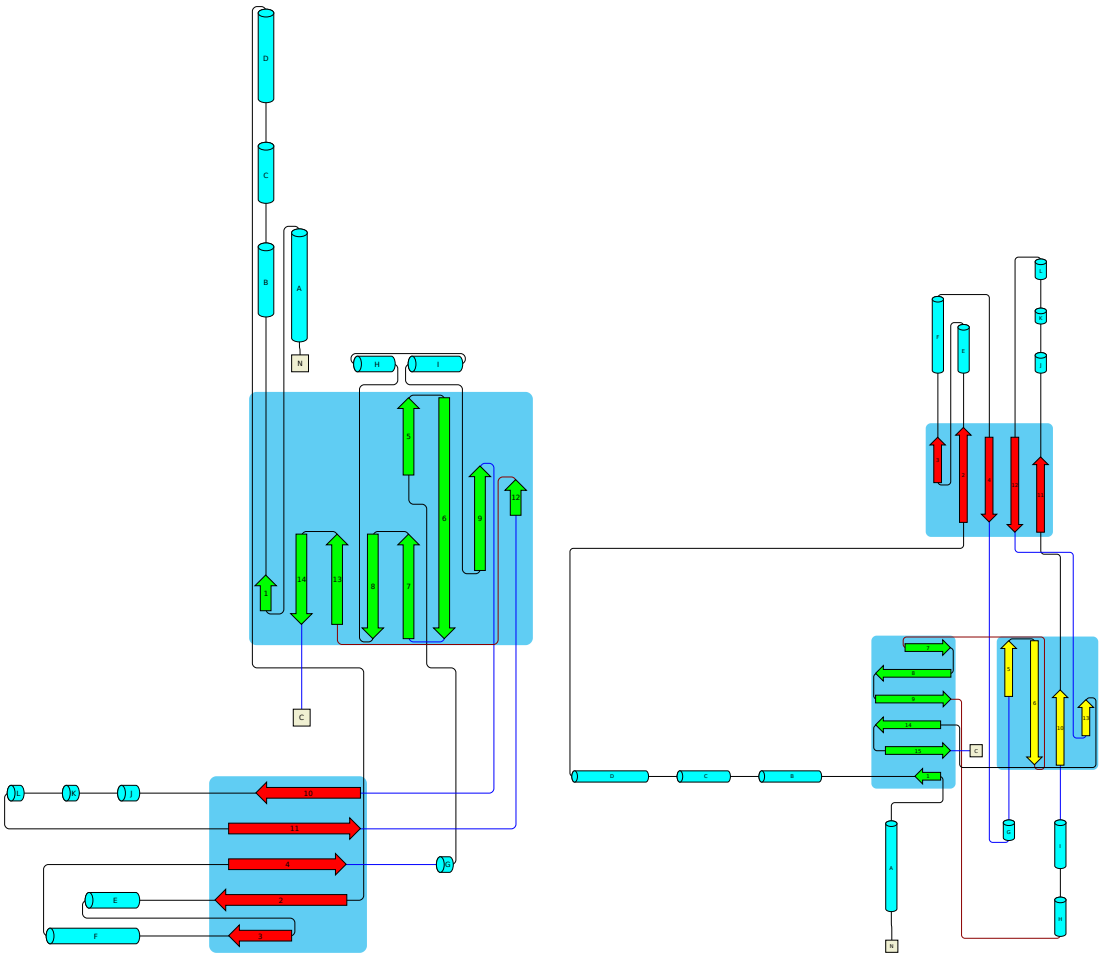


FIGURE 7.26: *The structure of α_1-antitrypsin, PDB identifier 1QLP.*

The A-sheet is red, the B-sheet is green, and the C-sheet is yellow. This cartoon was generated with PyMOL.

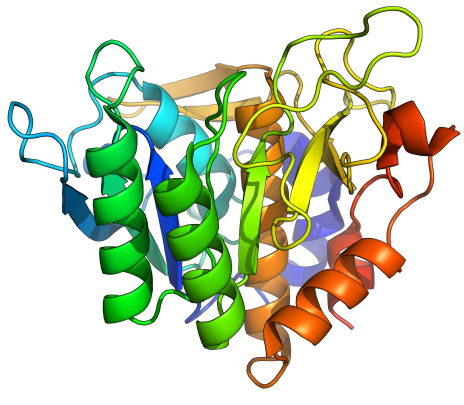


(a) A Pro-origami cartoon for 1QLP with SSEs defined by DSSP.

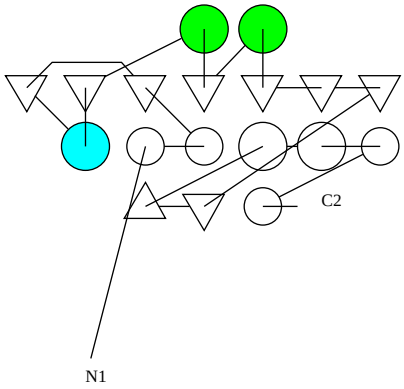
(b) A Pro-origami cartoon for 1QLP with SSEs and sheets defined by the HELIX and SHEET records in the PDB file.

FIGURE 7.27: *Comparison of Pro-origami cartoons for 1QLP when the sheets and helices are defined by DSSP and by records in the PDB file.*

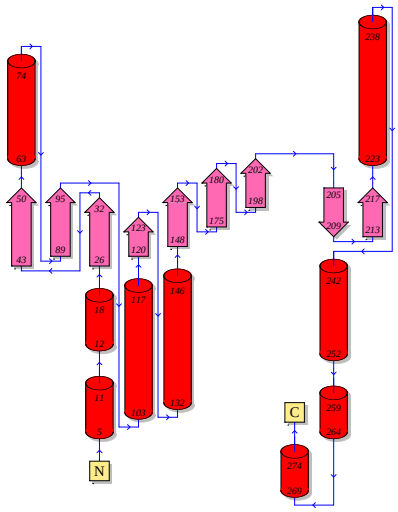
The five-stranded A-sheet (red) is recognisably the same in both Pro-origami figures (although oriented differently), but the single B/C-sheet defined by DSSP is considered as separate B- and C-sheets by the PDB SHEET records, with the single strand 6 in the figure on the left now split into two strands: 6 in the C-sheet (yellow) and 7 in the B-sheet (green). This shows the orthogonal orientation of the C-sheet (yellow) to the B-sheet (green), not visible in the figure on the left where they are shown as a single sheet.



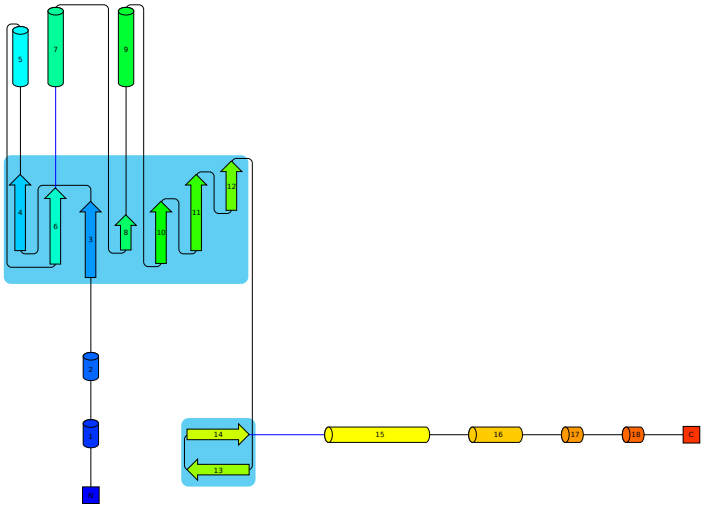
(a) PyMOL 3D rendering.



(b) TOPS cartoon.



(c) PDBsum topology diagram.



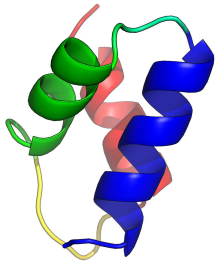
(d) Pro-origami cartoon.

FIGURE 7.28: *Comparison of different cartoon styles for subtilisin.*

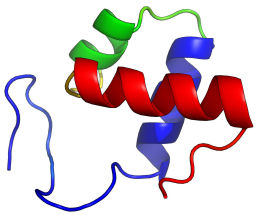
3D structure, TOPS cartoon, PDBsum cartoon and Pro-origami cartoon for Bacillus amyloliquefaciens subtilisin, PDB identifier 1ST2 (Bott et al.*, 1988). The TOPS cartoon conveys chirality information that the other 2D cartoon styles do not. There are three β-α-β motifs, one of which is a different chirality from the other two. In the PyMOL, TOPS and Pro-origami cartoons, the helix in the left-handed β-α-β motif is coloured cyan, while the other two are coloured green (colours are added to TOPS cartoons manually using the* `EditTops` *program).*

from the left-handed β-α-β motif is drawn above the parallel β-sheet and the other two are drawn below it. However this is actually just an artifact of the underlying algorithm from HERA that positions helices on a vertical axis so that they are adjacent on that axis to the strand or helix with the closest residue to the first or last residue of the helix (Hutchinson and Thornton, 1990). To see that this is the case, consider the PDBsum cartoon for a TIM-barrel (Figure 7.25(c)): if the helix positions in the PDBsum cartoon were to show the chirality of the β-α-β motifs to which they belong, all helices in this cartoon would be on the same side of the parallel β-sheet, which is clearly not the case. Note that for this structure, the Pro-origami cartoon (Figure 7.25(d)) may appear to convey this chirality information, since all the helices are positioned above the parallel β-sheet, however this is just an artifact of the heuristic placement of helices chosen to make a "neat" cartoon. It is perhaps possible to design a method of conveying this kind of chirality information in the "side-on" style of diagram, by, for example positioning helices above or below a sheet according to the chirality of the β-α-β motif to which they belong (as happens by chance in Figure 7.28(c)). However, we feel that this tries to convey "too much" information in this style of cartoon, at the expense of other requirements such as ease of comprehension of the cartoon, and the TOPS style is more appropriate for conveying this type of chirality information. In this respect, TOPS and Pro-origami (or PDBsum) are complementary; showing both styles of cartoon is a better way of conveying more information than trying to include it all in one style.

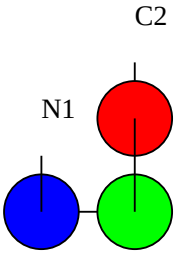
Some types of chirality information are not conveyed by any of the 2D cartoon styles. Consider the two three-helix bundle motifs in Figure 7.29. The two motifs are of different chiralities, which can be seen in the 3D renderings (Figures 7.29(a) and 7.29(b)). Neither the TOPS nor the Pro-origami cartoons convey this chirality information, and nor does PDBsum (not shown). Both the TOPS and the Pro-origami cartoons convey the anti-parallel relationship between helix 1 (blue) and helix 2 (green). The Pro-origami cartoon, in addition, shows the orthogonal orientation of helix 3 (red) with respect to helices 1 and 2. This example does, however, demonstrate a shortcoming of Pro-origami with respect to α-only domains: the heuristic helix placement algorithm cannot be used, since it depends on β-sheets to define the axes for aligning helices. Therefore the distance matrix based greedy algorithm is used. This algorithm can result in untidy cartoons that can vary significantly



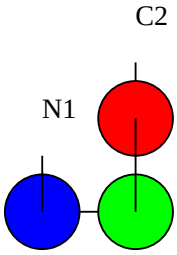
(a) PyMOL 3D rendering of RuvA.



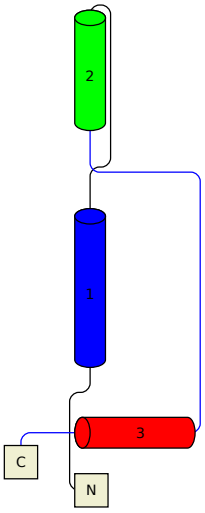
(b) PyMOL 3D rendering of Pax-5.



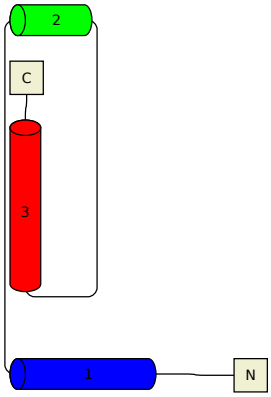
(c) TOPS cartoon for RuvA.



(d) TOPS cartoon for Pax-5.



(e) Pro-origami cartoon for RuvA.



(f) Pro-origami cartoon for Pax-5.

FIGURE 7.29: *Some chirality information is conveyed neither by TOPS nor by Pro-origami cartoons.*

3D structure, TOPS and Pro-origami cartoons of the 3rd domain of RuvA (Rafferty et al.*, 1996), PDB identifier 1CUK, residues 156–203), and of the 1st domain of human Pax-5 (Garvie* et al.*, 2002), PDB identifier 1MDM, chain A residues 19–81, a member of the "paired domain" family. The corresponding helices in each cartoon have the same colour. These two motifs can only be distinguished by chirality: the first (blue) and third (red) helices in RuvA are left-handed, while they are right-handed in Pax-5, a paired domain motif (Harrison* et al.*, 2003). This can be seen in the 3D PyMOL cartoons, however both the TOPS cartoons and the Pro-origami cartoons, while they show the anti-parallel relationship between the first (blue) and second (green) helices, do not show the different chiralities of the two motifs.*

between similar structures that differ in helix size since it uses the largest helix as the initial element relative to which others are placed, and the placement depends on details of which SSEs are closest to each other. The results of this can be seen in Figures 7.29(e) and 7.29(f), which appear quite different, despite the fact that the two motifs differ only in chirality. TOPS, in contrast, shows the same cartoon for the two motifs, which, although it does not show their differing chiralities, probably more usefully conveys the similarity of these two folds.

7.5 Pro-origami web server interface

A web server is a convenient method for users of a bioinformatics application, who are quite possibly molecular biologists (or other life scientists) and may not have the inclination to compile and install applications themselves, to use the application without having to do so. Therefore, we provide a web server that runs the Pro-origami software, allowing users to either retrieve a cartoon from a pre-built database, or create a cartoon for any structure. In addition, this server allows the user to specify queries to use in a protein structural database search, using the methods described in Chapter 5 and Chapter 6.

Most structural search servers perform only whole structure queries (rather than sub-structure or motif queries) and so the query input consists usually of entering a PDB (or SCOP, or CATH) identifier, or uploading a PDB file. Examples include the web servers for VAST[5], DaliLite[6] (Holm *et al.*, 2008), SSM[7], TOPOFIT[8] (Ilyin *et al.*, 2004; Leslin *et al.*, 2007), GANGSTA+[9], SABERTOOTH[10] (Teichert *et al.*, 2007), MISTRAL[11] (Micheletti and Orland, 2009), and TableauSearch[12].

Two exceptions to this (which have already been discussed in Chapter 5) are TOPS and ProSMoS. The ProSMoS web server[13] (Shi *et al.*, 2009) allows pattern (motif) search by query matrix or query structure. The "query matrix" option requires the user to enter a query meta-matrix, as discussed in Section 5.2.4 when

[5] http://www.ncbi.nih.gov/Structure/VAST/vastsearch.html
[6] http://ekhidna.biocenter.helsinki.fi/dali_server
[7] http://www.ebi.ac.uk/msd-srv/ssm
[8] http://topofit.ilyinlab.org/topofit
[9] http://gangsta.chemie.fu-berlin.de
[10] http://www.fkp.tu-darmstadt.de/sabertooth
[11] http://ipht.cea.fr/protein.php
[12] http://hollywood.bx.psu.edu/TabSearch
[13] http://prodata.swmed.edu/ProSMoS

the comparison of our method with ProSMoS is discussed. The "query structure" option generates the meta-matrix from a supplied structure, but it generally requires manually editing, as discussed in the same section. The TOPS server at Glasgow University[14] (Torrance *et al.*, 2005) allows motif queries of either one of the eight predefined "classic" motifs (TIM-barrel, Ubiquitin, Greek key, Rossmann, Immunoglobulin, Plait, Jelly-roll, Key-barrel), or by composing a topology cartoon for a user-defined query.[15] User-defined topological queries by means of a notation to represent TOPS diagrams (Gilbert *et al.*, 1999) were also able to be defined on the old TOPS server (Michalopoulos *et al.*, 2004), but this has been unavailable since April 2008.[16]

By using the Pro-origami web server as user interface, we are now able to build a system where substructure queries are created by the user interactively using a structure cartoon.

7.5.1 The Pro-origami web server

The web server contains a copy of the entire PDB, as well as the ASTRAL SCOP 1.75 95% sequence identity non-redundant data set, so the user can enter a PDB or SCOP identifier and generate a cartoon for that structure. Alternatively, an atomic co-ordinate file in PDB format may be uploaded, and a cartoon generated for that structure. A large number of options for controlling the Pro-origami algorithm and cartoon appearance (heuristic versus distance-matrix based helix placement, different colouring and labelling styles, etc.) are provided. By default, only a simplified subset of options are shown as the full set is quite large and possibly confusing for non-expert users. In addition, a pre-built database of cartoons for every domain in the ASTRAL SCOP 1.75 95% sequence identity non-redundant data set, and every structure in the PDB, is provided. The local copy of the PDB is updated weekly from the RCSB PDB server,[17] and any new or changed structures have their cartoons automatically created or updated.

[14] `http://balabio.dcs.gla.ac.uk/tops/index.html`

[15] I have been never been able to get the latter service to work. On Firefox/Linux, the browser seems to go into an infinite loop, on Internet Explorer (Windows), the Java applet gives a "class not found" error. This capability is not mentioned in the paper (Torrance *et al.*, 2005) and so perhaps was never functional.

[16] `http://www.tops.leeds.ac.uk` on October 29, 2009, still says "TOPS is currently undergoing essential maintenance and will be offline for a short while ... James Bradford, 24/04/2008".

[17] `rsync.wwpdb.org`

Cartoons are shown by default in SVG format, with interactive features such as "hover text", which shows the residue at the position in the cartoon over which the mouse pointer is placed, and allows selection of SSEs to build and submit a structure query, as described in the next section. The user is also able to view the three-dimensional structure with Jmol.[18] An example screen shot is shown in Figure 7.30.

Two additional features[19] are useful for complex cartoons with lots of connectors in similar paths, or in the rare cases where connector "overlap" occurs. While the pointer is placed over a connector, not only is the sequences of residues in that coil region shown as "hover text", but also the connector is shown highlighted by being drawn as a thicker line. In addition, in those rare cases where a connector "overlap" occurs and cannot be automatically resolved, Dunnart highlights the overlap region with purple shading, so that the user can manually edit the cartoon to correct the problem. A screen shot illustrating this feature is shown in Figure 7.31.

Alternatively, the cartoon can be downloaded in SVG format for further editing, or viewed or downloaded in bitmap (PNG) format or PDF format, for browsers that do not support SVG or for inclusion in publications or other documents.

7.5.2 Building substructure queries

Substructure queries can be built by selecting SSEs in the cartoon. This is done by clicking on an SSE to select (or deselect, if it is already selected) it. An option is also provided to select an entire sheet (or helix cluster) by selecting a strand (or helix) within it. Selecting an N- or C- terminus of a chain selects that entire chain. An example is shown in Figure 7.32, where the serpin B/C sheet of α_1-antitrypsin has been selected; this is the same query as that shown in Figure 5.17 and discussed in Section 5.3.2

7.5.3 Job scheduling and results presentation

When the "`QP Tableau Search`" or "`SA Tableau Search`" button is pressed, a query is built for the selected SSEs and submitted via a simple batch scheduling

[18] Jmol: an open-source Java viewer for chemical structures in 3D. `http://www.jmol.org`
[19] These features were created by Dr Michael Wybrow.

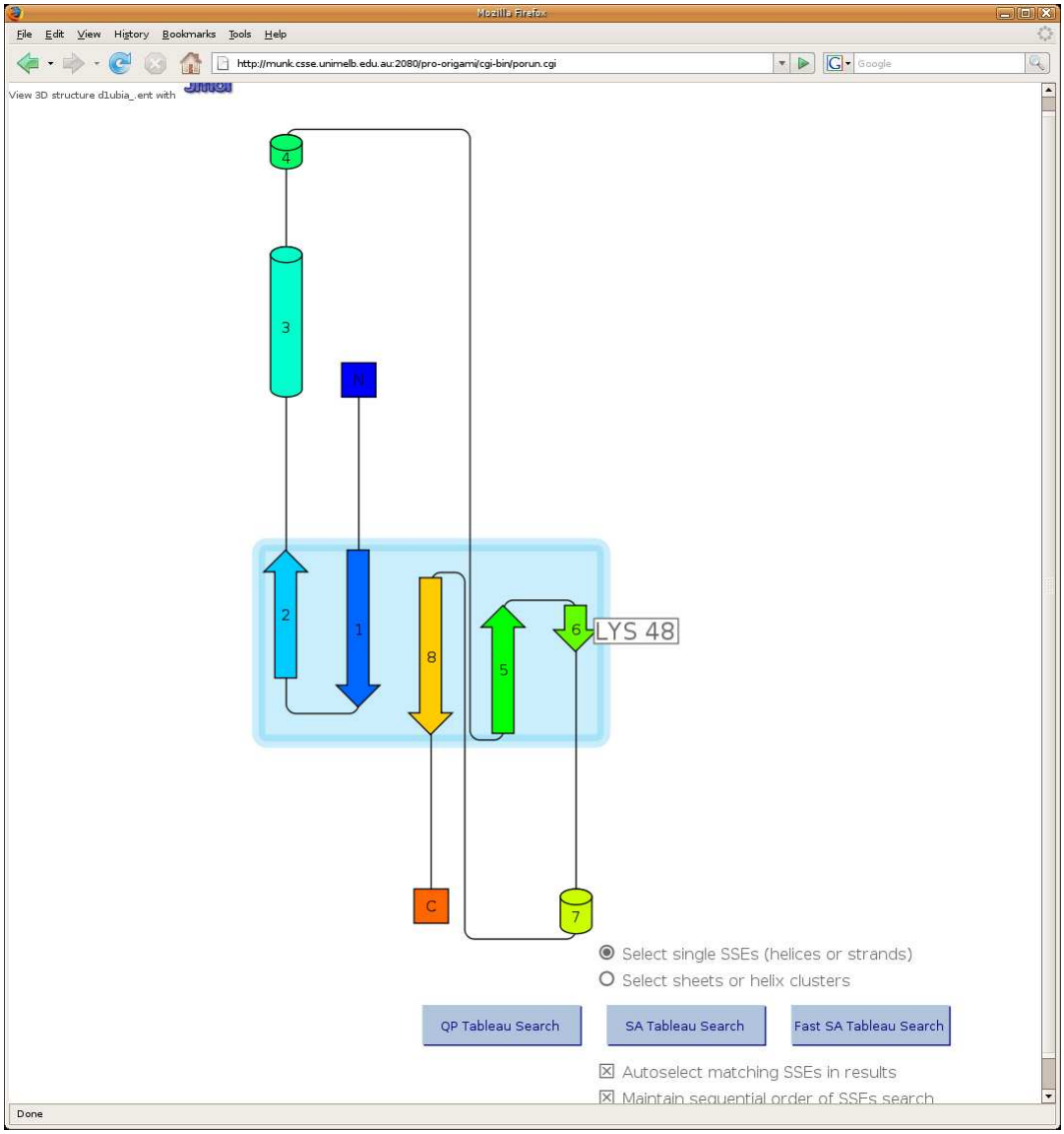


FIGURE 7.30: *Web server hovertext example.*

Screen shot of a browser connected to the Pro-origami web server, with a cartoon for `d1ubia_` *showing the hover text for the lysine 48 residue where the pointer is positioned on the strand labelled 6.*

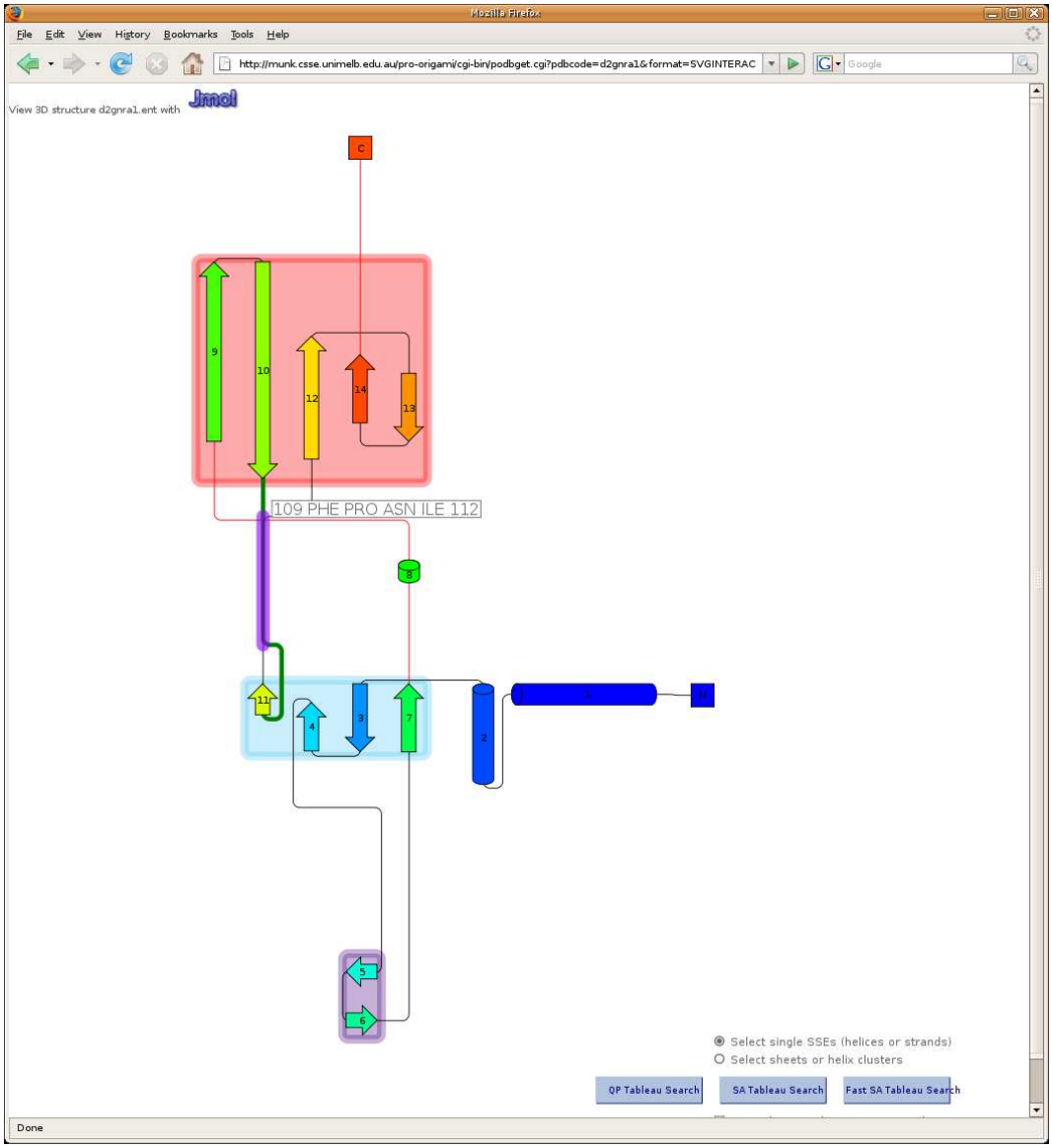


FIGURE 7.31: *Web server connector overlap example.*

Screen shot of a browser connected to the Pro-origami web server, showing a connector "overlap" highlighted in purple shading, and hover text showing the residues and highlighting the connector over which the pointer is positioned.

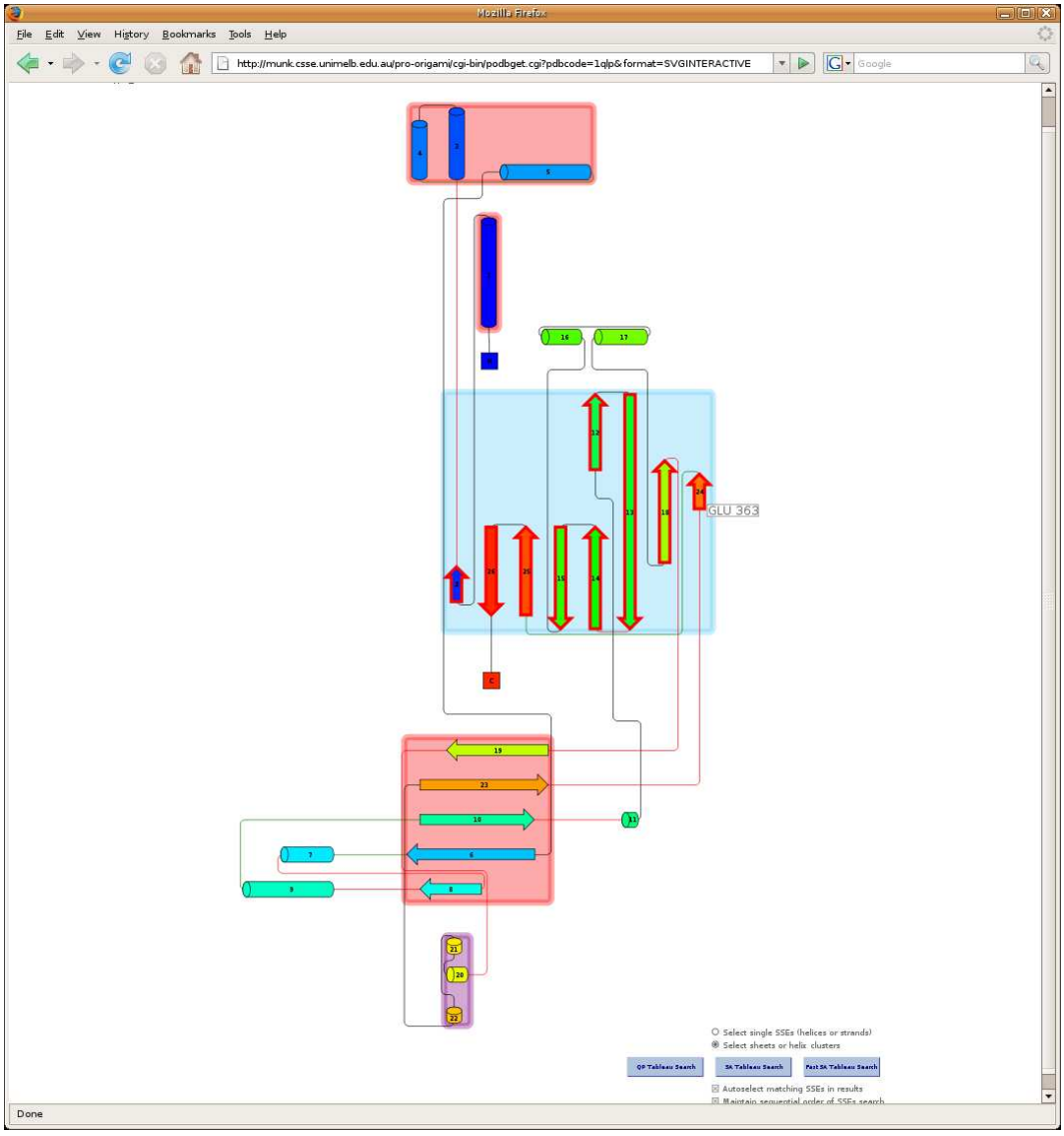


FIGURE 7.32: *Web server substructure query building example.*

Screen shot of a browser connected to the Pro-origami web server, with a cartoon for α_1-antitrypsin, PDB id 1QLP, showing, by the thick red outlines, that the serpin B/C sheet has been selected. This is the same query structure as the example shown in Figure 5.17.

system to run the query with the appropriate program, and the user is presented with a link to the location where the results will be placed when the job is finished.

The "`SA Tableau Search`" button runs SA Tableau Search (Chapter 6) with 1024 restarts. The "`Fast SA Tableau Search`" button runs SA Tableau Search with 128 restarts. This is fast enough that, rather than submitting a batch job, the results are returned directly to the browser.

Currently, the queries run against the database of tableaux for the ASTRAL SCOP 1.75 95% sequence identity non-redundant data set, as described in Section 5.2.

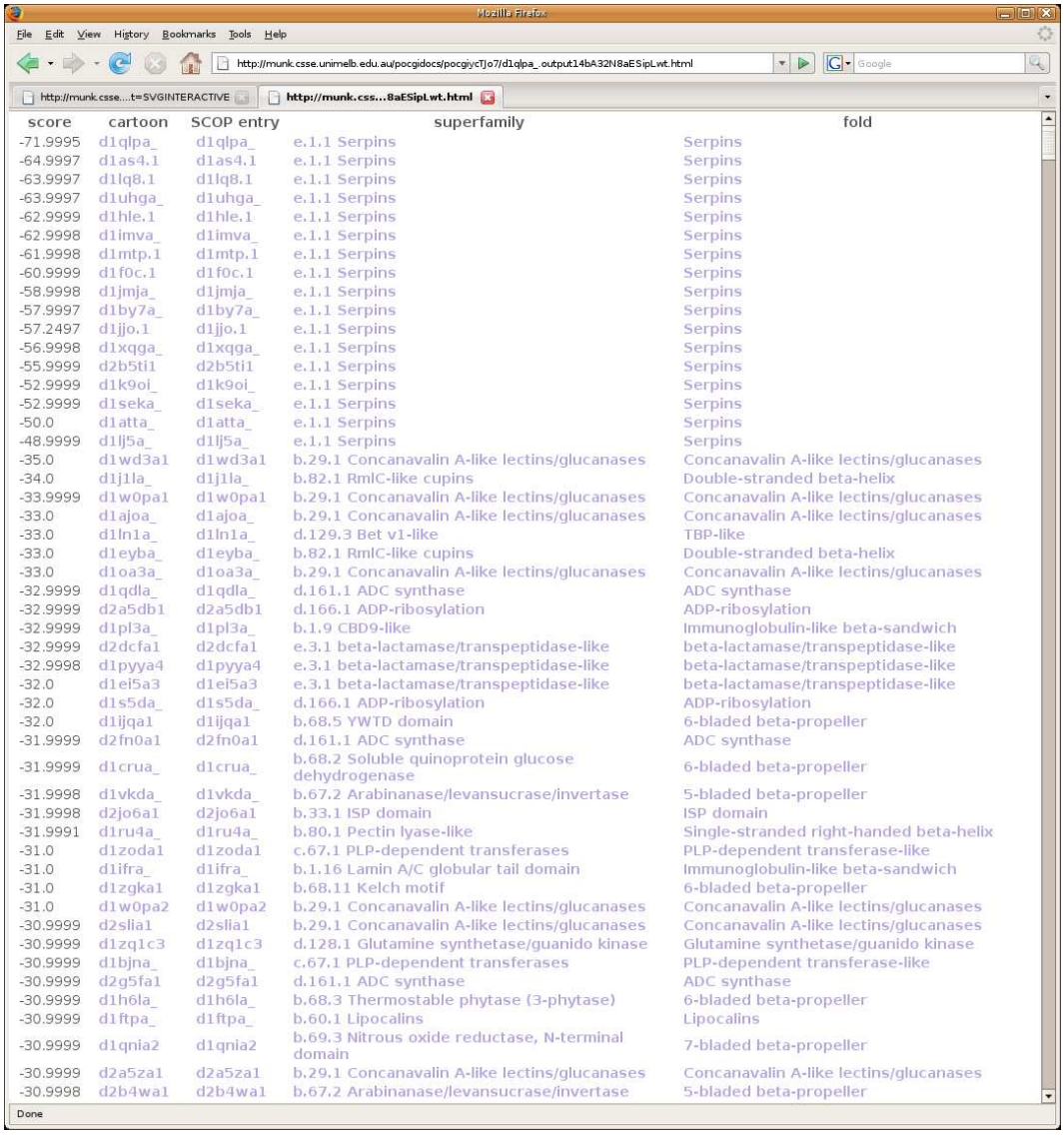
The results are presented as a simple list showing the score for each structure in the database, sorted from best hit to worst hit. The identifier for each entry in the list is a link to the pre-built Pro-origami cartoon for that structure, and there is also a link to the SCOP entry for the structure. The results from the serpin B/C sheet are shown in Figure 7.33, showing serpins in the top ten positions, as discussed in Section 5.3.2, with their alignment shown in Figure 5.19.

7.5.4 Implementation

The web server needs to run the Pro-origami software, which is written in Python as described in Section 7.3.11 as well as Dunnart, written in C++, the QP Tableau Search software written in Python and Fortran 77 as described in Section 5.2.5 of Chapter 5, and the SA Tableau Search software which is written in C with CUDA extensions as described in Chapter 6. The server is implemented using CGI (*Common Gateway Interface*) scripts written in Python, with JavaScript used to implement the "active" parts of the web pages and the interactive SVG. The `carto.net mapApp` library[20] and related SVG utilities are used in the interactive SVG. The Inkscape[21] vector graphics editor is used to render SVG into PNG or PDF formats for browsers that do not support SVG. Apache HTTP Server[22] running under Linux is used to run the web server.

The Pro-origami web server is available at `http://munk.csse.unimelb.edu.au/pro-origami`.

[20] `http://www.carto.net`
[21] `http://inkscape.org`
[22] `http://httpd.apache.org`



score	cartoon	SCOP entry	superfamily	fold
-71.9995	d1qlpa_	d1qlpa_	e.1.1 Serpins	Serpins
-64.9997	d1as4.1	d1as4.1	e.1.1 Serpins	Serpins
-63.9997	d1lq8.1	d1lq8.1	e.1.1 Serpins	Serpins
-63.9997	d1uhga_	d1uhga_	e.1.1 Serpins	Serpins
-62.9999	d1hle.1	d1hle.1	e.1.1 Serpins	Serpins
-62.9998	d1imva_	d1imva_	e.1.1 Serpins	Serpins
-61.9998	d1mtp.1	d1mtp.1	e.1.1 Serpins	Serpins
-60.9999	d1f0c.1	d1f0c.1	e.1.1 Serpins	Serpins
-58.9998	d1jmja_	d1jmja_	e.1.1 Serpins	Serpins
-57.9997	d1by7a_	d1by7a_	e.1.1 Serpins	Serpins
-57.2497	d1jjo.1	d1jjo.1	e.1.1 Serpins	Serpins
-56.9998	d1xqga_	d1xqga_	e.1.1 Serpins	Serpins
-55.9999	d2b5ti1	d2b5ti1	e.1.1 Serpins	Serpins
-52.9999	d1k9oi_	d1k9oi_	e.1.1 Serpins	Serpins
-52.9999	d1seka_	d1seka_	e.1.1 Serpins	Serpins
-50.0	d1atta_	d1atta_	e.1.1 Serpins	Serpins
-48.9999	d1lj5a_	d1lj5a_	e.1.1 Serpins	Serpins
-35.0	d1wd3a1	d1wd3a1	b.29.1 Concanavalin A-like lectins/glucanases	Concanavalin A-like lectins/glucanases
-34.0	d1j1la_	d1j1la_	b.82.1 RmlC-like cupins	Double-stranded beta-helix
-33.9999	d1w0pa1	d1w0pa1	b.29.1 Concanavalin A-like lectins/glucanases	Concanavalin A-like lectins/glucanases
-33.0	d1ajoa_	d1ajoa_	b.29.1 Concanavalin A-like lectins/glucanases	Concanavalin A-like lectins/glucanases
-33.0	d1ln1a_	d1ln1a_	d.129.3 Bet v1-like	TBP-like
-33.0	d1eyba_	d1eyba_	b.82.1 RmlC-like cupins	Double-stranded beta-helix
-33.0	d1oa3a_	d1oa3a_	b.29.1 Concanavalin A-like lectins/glucanases	Concanavalin A-like lectins/glucanases
-32.9999	d1qdla_	d1qdla_	d.161.1 ADC synthase	ADC synthase
-32.9999	d2a5db1	d2a5db1	d.166.1 ADP-ribosylation	ADP-ribosylation
-32.9999	d1pl3a_	d1pl3a_	b.1.9 CBD9-like	Immunoglobulin-like beta-sandwich
-32.9999	d2dcfa1	d2dcfa1	e.3.1 beta-lactamase/transpeptidase-like	beta-lactamase/transpeptidase-like
-32.9998	d1pyya4	d1pyya4	e.3.1 beta-lactamase/transpeptidase-like	beta-lactamase/transpeptidase-like
-32.0	d1ei5a3	d1ei5a3	e.3.1 beta-lactamase/transpeptidase-like	beta-lactamase/transpeptidase-like
-32.0	d1s5da_	d1s5da_	d.166.1 ADP-ribosylation	ADP-ribosylation
-32.0	d1ijqa1	d1ijqa1	b.68.5 YWTD domain	6-bladed beta-propeller
-31.9999	d2fn0a1	d2fn0a1	d.161.1 ADC synthase	ADC synthase
-31.9999	d1crua_	d1crua_	b.68.2 Soluble quinoprotein glucose dehydrogenase	6-bladed beta-propeller
-31.9998	d1vkda_	d1vkda_	b.67.2 Arabinanase/levansucrase/invertase	5-bladed beta-propeller
-31.9998	d2jo6a1	d2jo6a1	b.33.1 ISP domain	ISP domain
-31.9991	d1ru4a_	d1ru4a_	b.80.1 Pectin lyase-like	Single-stranded right-handed beta-helix
-31.0	d1zoda1	d1zoda1	c.67.1 PLP-dependent transferases	PLP-dependent transferase-like
-31.0	d1ifra_	d1ifra_	b.1.16 Lamin A/C globular tail domain	Immunoglobulin-like beta-sandwich
-31.0	d1zgka1	d1zgka1	b.68.11 Kelch motif	6-bladed beta-propeller
-31.0	d1w0pa2	d1w0pa2	b.29.1 Concanavalin A-like lectins/glucanases	Concanavalin A-like lectins/glucanases
-30.9999	d2slia1	d2slia1	b.29.1 Concanavalin A-like lectins/glucanases	Concanavalin A-like lectins/glucanases
-30.9999	d1zq1c3	d1zq1c3	d.128.1 Glutamine synthetase/guanido kinase	Glutamine synthetase/guanido kinase
-30.9999	d1bjna_	d1bjna_	c.67.1 PLP-dependent transferases	PLP-dependent transferase-like
-30.9999	d2g5fa1	d2g5fa1	d.161.1 ADC synthase	ADC synthase
-30.9999	d1h6la_	d1h6la_	b.68.3 Thermostable phytase (3-phytase)	6-bladed beta-propeller
-30.9999	d1ftpa_	d1ftpa_	b.60.1 Lipocalins	Lipocalins
-30.9999	d1qnia2	d1qnia2	b.69.3 Nitrous oxide reductase, N-terminal domain	7-bladed beta-propeller
-30.9999	d2a5za1	d2a5za1	b.29.1 Concanavalin A-like lectins/glucanases	Concanavalin A-like lectins/glucanases
-30.9998	d2b4wa1	d2b4wa1	b.67.2 Arabinanase/levansucrase/invertase	5-bladed beta-propeller

FIGURE 7.33: *Web server search results example.*

Screen shot of a browser connected to the Pro-origami web server, showing the search results for the serpin B/C sheet query. The top ten hits are shown aligned in Figure 5.19.

7.6 Conclusions

Pro-origami automates the process of generating protein topology diagrams, showing the relative size of strands and helices, the alignment of strands in a sheet and the general positioning of sheets and helices with respect to one another. Such information is not completely conveyed by the topological style of TOPS cartoons or HERA (PDBsum) diagrams. Our approach also reduces unnecessary connector crossings and generates diagrams that are interactively editable with an editor (Dunnart). The latter approach permits preservation of meaningful constraints such as the positions of strands in a sheet, and the connectors between SSEs, while the user edits the diagram.

To the best of our knowledge, the Pro-origami web server is the only system apart from TOPS that allows the user to interactively graphically specify a substructure query. Even if the TOPS user-defined query motif search were currently functional, the results discussed in Section 5.3 of Chapter 5 and Section 6.2 of Chapter 6, and the comparison of TOPS and Pro-origami cartoon properties discussed in this chapter mean that Pro-origami is a valuable alternative method for specifying protein substructure queries. Currently, the query motif must be created by selecting a subset of SSEs in some existing structure. In this respect ProSMoS (and possibly TOPS) have a capability that the Pro-origami structural search interface lacks: the ability to create a motif query "from scratch". This (as is pointed out by Shi *et al.* (2007, 2009)) is a key feature of ProSMoS. Nevertheless, we believe the interface provided by Pro-origami is advantageous, and an interesting future direction may be to create some means of graphically (interactively) creating arbitrary structural motifs from scratch with a similar system based on constraint-based diagram layout.

8

Conclusions

Bioinformatics and computational molecular biology are research areas that contain many computationally challenging problems. Some of these problems relate to the structural comparison of RNA and protein molecules, tasks that are important for the study of functional genomics, structural biology and evolution. In this thesis we have addressed some problems in RNA structural alignment, protein structural database searching, and protein structure visualization.

The work described in Chapter 3, where bounding techniques were applied to the dynamic program for RNA base pairing matrix alignment, was largely unsuccessful. That chapter, however, contains a review of many existing algorithms for RNA structural alignment, and a benchmarking exercise comparing some recent methods in a consistent manner. From this benchmarking we have shown that an existing implementation of an algorithm for alignment the RNA base pairing probability matrices is consistently the most accurate (on the measure used in the benchmark).

We have described a completely general technique for parallelizing dynamic programs, and applied it to the RNA base pairing matrix alignment dynamic program, resulting in a significant speedup. Although other existing RNA structural alignment algorithms may have more efficient implementations, without requiring parallelization, this contribution is a more general contribution to computer science, rather than bioinformatics. The technique of parallelizing dynamic programs on shared memory multiprocessors by means of a shared lock-free hash table and

randomization of subproblem ordering can be applied to any dynamic program, without the need to analyse data dependencies. The generality of this technique is exemplified by the diverse problems to which we have successfully applied it: well-known "textbook" dynamic programs such as knapsack and shortest paths, as well as RNA structural alignment and a sophisticated dynamic program for "open stacks", a manufacturing logistics problem.

We described QP Tableau Search, an algorithm for protein structure and substructure searching, based on the relaxation to a quadratic program (QP) of a previously described formulation as a quadratic integer program (QIP). We have shown in addition that a new constraint (the distance difference constraint) improves the specificity of this method. Although slower than some existing techniques, QP Tableau Search is, unlike most existing algorithms, capable of substructure (motif) searching as well as finding non-sequential matchings. These are matchings in which structurally equivalent secondary structure elements (SSEs) are not in the same sequential order in the two structures. In comparing the performance of QP Tableau Search as a motif finding technique with existing methods, we have shown that each method is capable of finding unique hits that the others do not. From this we conclude that relying on one particular method for motif searching would be perhaps unwise, and a thorough motif search should make use of several different methods.

In trying to improve the speed of QP Tableau Search, we find that we need to go beyond minor improvements such as using a faster solver or limiting the number of SSEs considered, and use an entirely different approximation technique. By using simulated annealing to find a (local) optimum of the same function as QP Tableau Search, we create SA Tableau Search. Our implementation of this algorithm is shown to be comparable or better in speed and accuracy with some widely used existing methods when run on a standard CPU. Moreover, we demonstrate a parallel implementation for modern graphics programming units (GPUs) that achieves a significant speedup, making SA Tableau Search one of the fastest available methods. To the best of our knowledge, this is the first application of a GPU to the protein structural search problem. Future work in applying this parallelization technique for GPUs to other protein structural alignment techniques such as maximum con-

tact map overlap (MAX-CMO) or other bioinformatics problems such as biological network alignment could well be rewarding.

Finally, we described Pro-origami, a system for the automatic generation of two-dimensional protein structure cartoons by means of constraint-based diagram layout. We reviewed the history of such cartoons (from both manually drawn and automated techniques) and compared Pro-origami cartoons with hand-drawn cartoons as well as those from the existing automatic methods. In many cases we find that Pro-origami cartoons better convey desirable protein structure information, although for certain types of structures TOPS cartoons are superior, or at least complementary. In these cases, the best choice may be to show both a TOPS and a Pro-origami cartoon, much as an architectural drawing may show both a floor plan and a front elevation of a building. We also described a system that combines Pro-origami with QP and SA Tableau Search, allowing interactive graphical specification of structure or substructure queries. To the best of our knowledge, this is the only system apart from TOPS to provide such a facility.

This thesis has contributed to the fields of bioinformatics and computational molecular biology by describing two new algorithms for protein structure and substructure searching, and a new method of automatically generating two-dimensional protein structure cartoons ("Pro-origami"). By integrating these contributions into a system whereby protein substructure queries can be graphically created, and the results likewise displayed using the two-dimensional Pro-origami cartoons, we have gone a step further than most existing protein structure search methods.

Bibliography

Abagyan, RA and Batalov, S (1997). Do aligned sequences share the same fold? *Journal of Molecular Biology*, **273**:355–368.

Alberts, B, Johnson, A, Lewis, J, Raff, M, Roberts, K, and Walter, P (2008). *Molecular Biology of the Cell*. Garland Science, 5th edition.

Alexandrov, NN (1996). SARFing the PDB. *Protein Engineering*, **9**(9):727–732.

Altschul, SF, Gish, W, Miller, W, Myers, EW, and Lipman, DJ (1990). Basic local alignment search tool. *Journal of Molecular Biology*, **215**:403–410.

Altschul, SF, Madden, TL, Schäffer, AA, Zhang, J, Zhang, Z, Miller, W, and Lipman, DJ (1997). Gapped BLAST and PSI-BLAST: a new generation of protein database search programs. *Nucleic Acids Research*, **25**(17):3389–3402.

Anderson, E, Bai, Z, Bischof, C, Demmel, J, Dongarra, J, Du Croz, J, Greenbaum, A, Hammarling, S, McKenney, A, Ostrouchov, S, and Sorensen, D (1992). *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia.

Andreeva, A, Howorth, D, Brenner, SE, Hubbard, TJP, Chothia, C, and Murzin, AG (2004). SCOP database in 2004: refinements integrate structure and sequence family data. *Nucleic Acids Research*, **32**(Database issue):D226–D229.

Andreeva, A, Howorth, D, Chandonia, JM, Brenner, SE, Hubbard, TJP, Chothia, C, and Murzin, AG (2008). Data growth and its impact on the SCOP database: new developments. *Nucleic Acids Research*, **36**(Database issue):D419–D425.

Anfinsen, CB (1973). Principles that govern the folding of protein chains. *Science*, **181**(4096):223–230.

Artymiuk, PJ, Spriggs, RV, and Willett, P (2005). Graph theoretic methods for the analysis of structural relationships in biological macromolecules. *Journal of the American Society for Information Science and Technology*, **56**(5):518–528.

Askitis, N (2009). Fast and compact hash tables for integer keys. In *Proceedings of the Thirty-Second Australasian Computer Science Conference (ACSC 2009)*, pages 101–110. Australian Computer Society.

Bauer, M and Klau, GW (2005). Structural alignment of two RNA sequences with Lagrangian relaxation. volume 3341 of *Lecture Notes in Computer Science*, pages 113–123. Springer.

Bauer, M, Klau, GW, and Reinert, K (2005a). Fast and accurate structural RNA alignment by progressive Lagrangian relaxation. volume 3695 of *Lecture Notes in Bioinformatics*, pages 217–228. Springer, Konstanz, Germany.

Bauer, M, Klau, GW, and Reinert, K (2005b). Multiple structural RNA alignment with Lagrangian relaxation. volume 3692 of *Lecture Notes in Bioinformatics*, pages 303–314. Springer, Mallorca, Spain. Extended Abstract.

Bauer, M, Klau, GW, and Reinert, K (2007). Accurate multiple sequence-structure alignment of RNA sequences using combinatorial optimization. *BMC Bioinformatics*, **8**:271.

Bellman, R (1957). *Dynamic Programming*. Princeton University Press, Princeton, New Jersey.

Berman, HM, Olson, WK, Beveridge, DL, Westbrook, J, Gelbin, A, Demeny, T, Hsieh, SH, Srinivasan, AR, and Schneider, B (1992). The nucleic acid database. a comprehensive relational database of three-dimensional structures of nucleic acids. *Biophysical Journal*, **63**:751–759.

Berman, HM, Westbrook, J, Feng, Z, Gilliland, G, Bhat, TN, Weissig, H, Shindyalov, IN, and Bourne, PE (2000). The protein data bank. *Nucleic Acids Research*, **28**(1):235–242.

Bernhart, SH, Hofacker, IL, and Stadler, PF (2006). Local RNA base pairing probabilities in large sequences. *Bioinformatics*, **22**(5):614–615.

Bewley, MC, Graziano, V, Jiang, J, Matz, E, Studier, FW, Pegg, AE, Coleman, CS, and Flanagan, JM (2006). Structures of wild-type and mutant human spermidine/spermine N^1-acetyletransferase, a potential therapeutic drug target. *Proceedings of the National Academy of Sciences of the USA*, **103**(7):2063–2068.

Bille, P (2005). A survey on tree edit distance and related problems. *Theoretical Computer Science*, **337**:217–239.

Bitto, E, Bingman, CA, McCoy, JG, Allard, STM, Wesenberg, GE, and Phillips Jr., GN (2005). The structure at 1.6 Å resolution of the protein product of the At4g34215 gene from *Arabidopsis thaliana*. *Acta Crystallographica*, **D61**:1655–1661.

Blair, DE, Hekmat, O, Schüttelkopf, AW, Shrestha, B, Tokuyasu, K, Withers, SG, and van Aalten, DMF (2006). Structure and mechanism of chitin deacetylase from the fungal pathogen *Colletotrichum lindemuthianum*. *Biochemistry*, **45**:9416–9426.

Blair, DE, Schüttelkopf, AW, MacRae, JI, and van Aalten, DMF (2005). Structure and metal-dependent mechanism of peptidoglycan deacetylase, a streptococcal virulence factor. *Proceedings of the National Academy of Sciences of the USA*, **102**(43):15429–15434.

Blair, DE and van Aalten, DMF (2004). Structures of *Bacillus subtilis* PdaA, a family 4 carbohydrate esterase, and a complex with *N*-acetyl-glucosasmine. *FEBS Letters*, **570**:13–19.

Bond, CS (2003). TopDraw: a sketchpad for protein structure topology cartoons. *Bioinformatics*, **19**(2):311–312.

Bond, CS, Kvaratskhelia, M, Richard, D, White, MF, and Hunter, WN (2001). Structure of Hjc, a Holliday junction resolvase, from *Sulfolobus solfataricus*. *Proceedings of the National Academy of Sciences of the USA*, **98**(10):5509–5514.

Bonhoeffer, S, McCaskill, JS, Stadler, PF, and Schuster, P (1993). RNA multi-structure landscapes. a study based on temperature dependent partition functions. *European Biophysics Journal*, **22**:13–24.

Bonwick, J (1994). The slab allocator: An object-caching kernel memory allocator. In *Proceedings of the Summer 1994 USENIX Conference*, pages 87–98.

Bonwick, J (2001). Magazines and vmem: Extending the slab allocator to many CPUs and arbitrary resources. In *Proceedings of the 2001 USENIX Annual Technical Conference*, pages 15–34.

Bott, R, Ultsch, M, Kossiakoff, A, Graycar, T, Katz, B, and Power, S (1988). The three-dimensional structure of *Bacillus amyloliquefaciens* subtilisin at 1.8 Å and an analysis of the structural consequences of peroxide inactivation. *Journal of Biological Chemistry*, **263**(16):7895–7906.

Branden, C and Tooze, J (1999). *Introduction to Protein Structure*. Garland Science, 2nd edition.

Brenner, SE, Chothia, C, and Hubbard, TJP (1998). Assessing sequence comparison methods with reliable structurally identified distant evolutionary relationships. *Proceedings of the National Academy of Sciences of the USA*, **95**:6073–6078.

Breu, H and Kirkpatrick, DG (1998). Unit disk graph recognition is NP-hard. *Computational Geometry*, **9**:3–24.

Bryant, D and Moulton, V (2004). Neighbor-net: An agglomerative method for the construction of phylogenetic networks. *Molecular Biology and Evolution*, **21**(2):255–265.

Brzezinski, K, Stepkowski, T, Panjikar, S, Bujacz, G, and Jaskolski, M (2007). High-resolution structure of NodZ fucosyltransferase involved in the biosynthesis of the nodulation factor. *Acta Biochimica Polonica*, **54**(3):537–549.

Buckner, J, Wilson, J, Seligman, M, Athey, B, Watson, S, and Meng, F (2010). The gputools package enbales GPU computing in R. *Bioinformatics*, **26**(1):134–135.

Byun, Y and Han, K (2009). PseudoViewer3: generating planar drawings of large-scale RNA structures with pseudoknots. *Bioinformatics*, **25**(11):1435–1437.

Caprara, A, Carr, R, Istrail, S, Lancia, G, and Walenz, B (2004). 1001 optimal PDB structure alignments: Integer programming methods for finding the maximum contact map overlap. *Journal of Computatinal Biology*, **11**(1):27–52.

Caprara, A and Lancia, G (2002). Structural alignment of large-size proteins via lagrangian relaxation. In *Proceedings of the Sixth Annual International Conference on Computational Molecular Biology (RECOMB '02)*, pages 100–108. ACM Press.

Carpentier, M, Brouillet, S, and Pothier, J (2005). YAKUSA: A fast structural database scanning method. *Proteins*, **61**:137–151.

Carraway, RL and Schmidt, RL (1991). An improved discrete dynamic programming algorithm for allocating resources among interdependent projects. *Management Science*, **37**(9):1195–1200.

Carroll, HD, Kann, MG, Sheetlin, SL, and Spouge, JL (2010). Threshold average precision (TAP-*k*): a measure of retrieval designed for bioinformatics. *Bioinformatics*, **26**(14):1708–1713.

Carugo, O and Pongor, S (2002). Protein fold similarity estimated by a probabilistic approach based on C^{α}- C^{α} distance comparison. *Journal of Molecular Biology*, **315**:887–898.

Casbon, JA, Crooks, GE, and Saqi, MAS (2006). A high level interface to SCOP and ASTRAL implemented in python. *BMC Bioinformatics*, **7**:10.

Chandonia, JM, Hon, G, Walker, NS, Lo Conte, L, Koehl, P, Levitt, M, and Brenner, SE (2004). The ASTRAL compendium in 2004. *Nucleic Acids Research*, **32**(Database issue):D189–D192.

Chen, VB, Arendall III, WB, Headd, JJ, Keedy, DA, Immormino, RM, Kapral, GJ, Murray, LW, Richardson, JS, and Richardson, DC (2010). MolProbity: all-atom structure validation for macromolecular crystallography. *Acta Crystallographica*, **D66**:12–21.

Chindelevitch, L, Liao, CS, and Berger, B (2010). Local optimization for global alignment of protein interaction networks. In *Pacific Symposium on Biocomputing*, volume 15, pages 123–132.

Chothia, C, Levitt, M, and Richardson, D (1981). Helix to helix packing in proteins. *Journal of Molecular Biology*, **145**:215–250.

Chowdhury, RA and Ramachandran, V (2008). Cache-efficient dynamic programming algorithms for multicores. In *Proceedings of the twentieth annual ACM Symposium on Parallel Algorithms and Architectures*, pages 207–216. ACM.

Chu, G and Stuckey, PJ (2009). Minimizing the maximum number of open stacks by customer search. In IP Gent, editor, *Principles and Practice of Constraint Programming - CP 2009*, volume 5732 of *Lecture Notes in Computer Science*, pages 242–257.

Clote, P and Backofen, R (2000). *Computational Molecular Biology: An Introduction*. Wiley Series in Mathematical and Computational Biolgy. Wiley.

Cock, PJ, Antao, T, Chang, JT, Chapman, BA, Cox, CJ, Dalke, A, Friedberg, I, Hamelryck, T, Kauff, F, Wilczynski, B, and de Hoon, MJ (2009). Biopython: freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, **25**(11):1422–1423.

Cohen, FE, Sternberg, MJE, and Taylor, WR (1981). Analysis of the tertiary structure of protein β-sheet sandwiches. *Journal of Molecular Biology*, **148**(3):253–272.

Crick, F (1970). Central dogma of molecular biology. *Nature*, **227**:561–563.

Dalli, D, Wilm, A, Mainz, I, and Steger, G (2006). StrAl: progressive alignment of non-coding RNA using base pairing probability vectors in quadratic time. *Bioinformatics*, **22**(13):1593–1599.

Dalton, JAR, Michalopoulos, I, and Westhead, DR (2003). Calculation of helix packing angles in protein structures. *Bioinformatics*, **19**(10):1298–1299.

Dantzig, GB (1957). Discrete-variable extremum problems. *Operations Research*, **5**(2):266–277.

Darty, K, Denise, A, and Ponty, Y (2009). VARNA: Interactive drawing and editing of the RNA secondary structure. *Bioinformatics*, **25**(15):1974–1975.

Davis, TA (2004a). Algorithm 832: UMFPACK V4.3 — an unsymmetric-pattern multifrontal method. *ACM Transactions on Mathematical Software*, **30**(2):196–199.

Davis, TA (2004b). A column pre-ordering strategy for the unsymmetric-pattern multifrontal method. *ACM Transactions on Mathematical Software*, **30**(2):165–195.

Davis, TA and Duff, IS (1997). An unsymmetric-pattern multifrontal method for sparse LU factorization. *SIAM Journal on Matrix Analysis and Applications*, **18**(1):140–158.

Davis, TA and Duff, IS (1999). A combined unifrontal/multifrontal method for unsymmetric sparse matrices. *ACM Transactions on Mathematical Software*, **25**(1):1–20.

DeLano, WL (2002). The PyMOL molecular graphics system. DeLano Scientific `http://www.pymol.org`.

DeLong, ER, DeLong, DM, and Clarke-Pearson, DL (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach. *Biometrics*, **44**(3):837–845.

Dill, KA, Ozkan, SB, Weikl, TR, Chodera, JD, and Voelz, VA (2007). The protein folding problem: when will it be solved? *Current Opinion in Structural Biology*, **17**:342–346.

Dongarra, JJ, Du Croz, J, Hammarling, S, and Hanson, RJ (1988). An extended set of FORTRAN basic linear algebra subprograms. *ACM Transactions on Mathematical Software*, **14**(1):1–17.

Dowell, RD and Eddy, SR (2006). Efficient pairwise RNA structure prediction and alignment using sequence alignment constraints. *BMC Bioinformatics*, **7**:400.

Downs, GM, Gillet, VJ, Holliday, JD, and Lynch, MF (1989). Review of ring perception algorithms for chemical graphs. *Journal of Chemical Information and Computer Sciences*, **29**(3):172–187.

Duff, IS (2004). MA57 — a code for the solution of sparse symmetric definite and indefinite systems. *ACM Transactions on Mathematical Software*, **30**(2):118–144.

Durbin, R, Eddy, S, Krogh, A, and Mitchison, G (1998). *Biological sequence analysis*. Cambridge University Press. 11th printing (2006).

Dwyer, T, Koren, Y, and Marriott, K (2006). IPSep-CoLa: An incremental procedure for separation constraint layout of graphs. *IEEE Transactions on Visualization and Computer Graphics*, **12**(5):821–828.

Dwyer, T, Marriott, K, and Wybrow, M (2007). Integrating edge routing into force-directed layout. In *14th International Symposium on Graph Drawing (GD'06)*, volume 4372 of *Lecture Notes in Computer Science*, pages 8–19. Springer.

Dwyer, T, Marriott, K, and Wybrow, M (2009). Dunnart: A constraint-based network diagram authoring tool. In *Graph Drawing 2008*, volume 5417 of *Lecture Notes in Computer Science*, pages 420–431. Springer.

Eddy, SR (2002). Computational genomics of noncoding RNA genes. *Cell*, **109**:137–140.

Eddy, SR and Durbin, R (1994). RNA sequence analysis using covariance models. *Nucleic Acids Research*, **22**(11):2079–2088.

Elliott, PR, Pei, XY, Dafforn, TR, and Lomas, DA (2000). Topography of a 2.0 Å structure of α_1-antitrypsin reveals targets for rational drug design to prevent conformational disease. *Protein Science*, **9**:1274–1281.

Emmert-Streib, F and Mushegian, A (2007). A topological algorithm for identification of structural domains of proteins. *BMC Bioinformatics*, **8**:237.

Fang, X, Luo, Z, Yuan, B, and Wang, Z (2007). Detecting and assessing conserved stems for accurate structural alignment of RNA sequences. *Proceedings of the 7th IEEE International Conference on Bioinformatics and Bioengineering (BIBE 2007)*, pages 301–306.

Farrar, M (2007). Striped Smith-Waterman speeds database searches six times over other SIMD implementations. *Bioinformatics*, **23**(2):156–161.

Feng, DF and Doolittle, RF (1987). Progressive sequence alignment as a prerequisite to correct phylogenetic trees. *Journal of Molecular Evolution*, **25**:351–360.

Ferreira, FM, Mendoza-Hernandez, G, Castañeda-Bueno, M, Aparicio, R, Fischer, H, Calcagno, ML, and Oliva, G (2006). Structural analysis of *N*-acetylglucosamine-6-phosphate deacetylase apoenzyme from *Escherichia coli*. *Journal of Molecular Biology*, **359**:308–321.

Fischer, D, Elofsson, A, Rice, D, and Eisenberg, D (1996). Assessing the performance of fold recognition methods by means of a comprehensive benchmark. In *Pacific Symposium on Biocomputing*, pages 300–318.

Flores, TP, Moss, DS, and Thornton, JM (1994). An algorithm for automatically generating protein topology cartoons. *Protein Engineering*, **7**(1):31–37.

Flower, DR (1994a). Automating the identification and analysis of protein β-barrels. *Protein Engineering*, **7**(11):1305–1310.

Flower, DR (1994b). β-sheet topology: A new system of nomenclature. *FEBS Letters*, **344**:247–250.

Flower, DR (1995). FOLD: Integrated analysis and display of protein secondary structure. *Journal of Molecular Graphics*, **13**:377–384.

Floyd, RW (1962). Algorithm 97: Shortest path. *Communications of the Association for Computing Machinery*, **5**(6):345.

Foley, JD and van Dam, A (1982). *Fundamentals of Interactive Computer Graphics*. Addison-Wesley Systems Programming Series. Addison-Wesley.

Fontana, W, Konings, DAM, Stadler, PF, and Schuster, P (1993). Statistics of RNA secondary structures. *Biopolymers*, **33**:1389–1404.

Frank, K, Gruber, M, and Sippl, MJ (2010). COPS benchmark: interactive analysis of database search methods. *Bioinformatics*, **26**(4):574–575.

Friedrichs, MS, Eastman, P, Vaidyanathan, V, Houston, M, Legrand, S, Beberg, AL, Ensign, DL, Bruns, CM, and Pande, VS (2009). Accelerating molecular dynamic simulation on graphics processing units. *Journal of Computational Chemistry*, **30**(6):864–872.

Frishman, D and Argos, P (1995). Knowledge-based protein secondary structure assignment. *Proteins*, **23**:566–579.

Gansner, ER and North, SC (2000). An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, **30**(11):1203–1233.

Gao, H, Groote, JF, and Hesselink, WH (2005). Lock-free dynamic hash tables with open addressing. *Distributed Computing*, **18**(1):21–42.

Garcia de la Banda, M and Stuckey, PJ (2007). Dynamic programming to minimize the maximum number of open stacks. *INFORMS Journal on Computing*, **19**(4):606–617.

Gardner, PP, Wilm, A, and Washietl, S (2005). A benchmark of multiple sequence alignment programs upon structural RNAs. *Nucleic Acids Research*, **33**(8):2433–2439.

Garey, MR and Johnson, DS (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.

Garvie, CW, Pufall, MA, Graves, BJ, and Wolberger, C (2002). Structural analysis of the autoinhibition of Ets-1 and its role in protein partnerships. *Journal of Biological Chemistry*, **277**(47):45529–45536.

Gáspári, Z, Vlahovicek, K, and Pongor, S (2005). Efficient recognition of folds in protein 3D structures by the improved PRIDE algorithm. *Bioinformatics*, **21**(15):3322–3323.

Gibrat, JF, Madej, T, and Bryant, SH (1996). Surprising similarities in structure comparison. *Current Opinion in Structural Biology*, **6**(3):377–385.

Gilbert, D, Westhead, D, Nagano, N, and Thornton, J (1999). Motif-based searching in TOPS protein topology databases. *Bioinformatics*, **15**(4):317–326.

Glasbey, C, van der Heijden, G, Toh, VFK, and Gray, A (2007). Colour displays for categorical images. *Color Research and Application*, **32**(4):304–309.

Gould, NIM, Scott, JA, and Hu, Y (2007). A numerical evaluation of sparse direct solvers for the solution of large sparse symmetric linear systems of equations. *ACM Transactions on Mathematical Software*, **33**(2):10.

Gramm, J, Guo, J, and Niedermeier, R (2006). Pattern matching for arc-annotated sequences. *ACM Transactions on Algorithms*, **2**(1):44–65.

Gribskov, M and Robinson, NL (1996). Use of receiver operating characteristic (ROC) analysis to evaluate sequence matching. *Computers & Chemistry*, **20**(1):25–33.

Griffiths-Jones, S, Bateman, A, Marshall, M, Khanna, A, and Eddy, SR (2003). Rfam: an RNA family database. *Nucleic Acids Research*, **31**(1):439–441.

Grindley, HM, Artymiuk, PJ, Rice, DW, and Willett, P (1993). Identification of tertiary structure resemblance in proteins using a maximal common subgraph isomorphism algorithm. *Journal of Molecular Biology*, **229**:707–721.

Guerler, A and Knapp, EW (2008). Novel protein folds and their nonsequential structural analogs. *Protein Science*, **17**:1374–1382.

Hamelryck, T and Manderick, B (2003). PDB file parser and structure class implemented in python. *Bioinformatics*, **19**(17):2308–2310.

Hanley, JA and McNeil, BJ (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, **143**(1):29–36.

Harrison, A, Pearl, F, Mott, R, Thornton, J, and Orengo, C (2002). Quantifying the similarities within fold space. *Journal of Molecular Biology*, **323**:909–926.

Harrison, A, Pearl, F, Sillitoe, I, Slidel, T, Mott, R, Thornton, J, and Orengo, C (2003). Recognizing the fold of a protein structure. *Bioinformatics*, **19**(14):1748–1759.

Havgaard, JH, Lyngsø, RB, Stormo, GD, and Gorodkin, J (2005). Pairwise local structural alignment of RNA sequences with sequence similarity less than 40%. *Bioinformatics*, **21**(9):1815–1824.

Havgaard, JH, Torarinsson, E, and Gorodkin, J (2007). Fast pairwise structural RNA alignments by pruning of the dynamical programming matrix. *PLoS Computational Biology*, **3**(10):e193.

He, X, Zhou, J, Bartlam, M, Zhang, R, Ma, J, Lou, Z, Li, X, Li, J, Joachimiak, A, Zeng, Z, Ge, R, Rao, Z, and Liu, Y (2008). Crystal structure of the polymerase PA_C-$PB1_N$ complex from an avian influenza H5N1 virus. *Nature*, **454**:1123–1127.

Herlihy, M and Shavit, N (2008). *The Art of Multiprocessor Programming*. Morgan Kaufmann, Burlington MA, USA.

Herlihy, M, Shavit, N, and Tzafrir, M (2008). Hopscotch hashing. In *Lecture Notes in Computer Science*, volume 5218, pages 350–364. Proceedings of the 22nd international symposium on distributed computing.

Herrbach, C, Denise, A, and Dulucq, S (2010). Average complexity of the Jiang-Wang-Zhang pairwise tree alignment algorithm and of a RNA secondary structure alignment algorithm. *Theoretical Computer Science*, **411**:2423–2432.

Ho, HK, Kuiper, MJ, and Kotagiri, R (2008). PConPy - a python module for generating 2D protein maps. *Bioinformatics*, **24**(24):2934–2935.

Hofacker, IL (2003). Vienna RNA secondary structure server. *Nucleic Acids Research*, **31**(13):3429–3431.

Hofacker, IL, Bernhart, SHF, and Stadler, PF (2004). Alignment of RNA base pairing probability matrices. *Bioinformatics*, **20**(14):2222–2227.

Hofacker, IL, Fekete, M, and Stadler, PF (2002). Secondary structure prediction for aligned RNA sequences. *Journal of Molecular Biology*, **319**:1059–1066.

Hofacker, IL, Fontana, W, Stadler, PF, Bonhoeffer, LS, Tacker, M, and Schuster, P (1994). Fast folding and comparison of RNA secondary structures. *Monatshefte für Chemie*, **125**:167–188.

Holm, L, Kääriäinen, S, Rosenström, P, and Schenkel, A (2008). Searching protein structure databases with DaliLite v.3. *Bioinformatics*, **24**(23):2780–2781.

Holm, L and Park, J (2000). DaliLite workbench for protein structure comparison. *Bioinformatics*, **16**(6):566–567.

Holm, L and Sander, C (1996). Mapping the protein universe. *Science*, **273**:595–602.

Holmes, I (2005). Accelerated probabilistic inference of RNA structure evolution. *BMC Bioinformatics*, **6**:73.

HSL (2007). A collection of Fortran codes for large-scale scientific computation. `http://www.hsl.rl.ac.uk`.

Huang, X and Ali, H (2007). High sensitivity RNA pseudoknot prediction. *Nucleic Acids Research*, **35**(2):656–663.

Hulsen, T, de Vlieg, J, Leunissen, JAM, and Groenen, PMA (2006). Testing statistical significance scores of sequence comparison methods with structure similarity. *BMC Bioinformatics*, **7**:444.

Humphrey, W, Dalke, A, and Schulten, K (1996). VMD – Visual Molecular Dynamics. *Journal of Molecular Graphics*, **14**:33–38.

Hussong, R, Gregorius, B, Tholey, A, and Hildebrandt, A (2009). Highly accelerated feature detection in proteomics data sets using modern graphics processing units. *Bioinformatics*, **25**(15):1937–1943.

Hutchinson, EG and Thornton, JM (1990). HERA — a program to draw schematic diagrams of protein secondary structures. *Proteins*, **8**:203–212.

Hutchinson, EG and Thornton, JM (1996). PROMOTIF — a program to identify and analyze structural motifs in proteins. *Protein Science*, **5**:212–220.

Ilyin, VA, Abyzov, A, and Leslin, CM (2004). Structural alignment of proteins by a novel TOPOFIT method, as a superimposition of common volumes at a topomax point. *Protein Science*, **13**:1865–1874.

Irving, JA, Cabrita, LD, Rossjohn, J, Pike, RN, Bottomley, SR, and Whisstock, JC (2003). The 1.5 Å Crystal Structure of a Prokaryote Serpin Controlling Conformational Change in a Heated Environment. *Structure*, **11**(4):387–397.

Jayaram, H, Taraporewala, Z, Patton, JT, and Prasad, BBV (2002). Rotavirus protein involved in genome replication and packaging exhibits a HIT-like fold. *Nature*, **417**:311–315.

Ji, Y, Xu, X, and Stormo, GD (2004). A graph theoretical approach for predicting common RNA secondary structure motifs including pseudoknots in unaligned sequences. *Bioinformatics*, **20**(10):1591–1602.

Jiang, T, Wang, L, and Zhang, K (1995). Alignment of trees — an alternative to tree edit. *Theoretical Computer Science*, **143**:137–148.

Jossinet, F, Ludwig, TE, and Westhof, E (2007). RNA structure: bioinformatic analysis. *Current Opinion in Microbiology*, **10**:279–285.

Jung, J and Lee, B (2000). Protein structure alignment using environmental profiles. *Protein Engineering*, **13**(8):535–543.

Kabsch, W and Sander, C (1983). Dictionary of protein secondary structure: Pattern recognition of hydrogen-bonded and geometrical features. *Biopolymers*, **22**:2577–2637.

Kamat, AP and Lesk, AM (2007). Contact patterns between helices and strands of sheet define protein folding patterns. *Proteins*, **66**:869–876.

Karypis, G and Kumar, V (1998). A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, **20**(1):359–392.

Katoh, K, Kuma, Ki, Toh, H, and Miyata, T (2005). MAFFT version 5: improvement in accuracy of multiple sequence alignment. *Nucleic Acids Research*, **33**(2):511–518.

Kirillova, S and Carugo, O (2008). Progress in the PRIDE technique for rapidly comparing protein three-dimensional structures. *BMC Research Notes*, **1**:44.

Kirkpatrick, S, Gelatt, CD, and Vecchi, MP (1983). Optimization by simulated annealing. *Science*, **220**(4598):671–680.

Kiryu, H, Tabei, Y, Kin, T, and Asai, K (2007). Murlet: a practical multiple alignment tool for structural RNA sequences. *Bioinformatics*, **23**(13):1588–1598.

Knipscheer, P, van Dijk, WJ, Olsen, JV, Mann, M, and Sixma, TK (2007). Noncovalent interaction between Ubc9 and SUMO promotes SUMO chain formation. *The EMBO Journal*, **26**(11):2797–2807.

Koch, I, Kaden, F, and Selbig, J (1992). Analysis of protein sheet topologies by graph theoretical methods. *Proteins*, **12**:314–323.

Kolbeck, B, May, P, Schmidt-Goenner, T, Steinke, T, and Knapp, EW (2006). Connectivity independent protein-structure alignment: a hierarchical approach. *BMC Bioinformatics*, **7**:510.

Kolk, M, van der Graaf, M, Wijmenga, SS, Pleij, CWA, Heus, HA, and Hilbers, CW (1998). NMR structure of a classical pseudoknot: Interplay of single- and double-stranded RNA. *Science*, **280**:434–438.

Konagurthu, AS (2006). *Multiple Sequence and Structural Alignment Problems in Biology*. Ph.D. thesis, The University of Melbourne.

Konagurthu, AS, Stuckey, PJ, and Lesk, AM (2008). Structural search and retrieval using a tableau representation of protein folding patterns. *Bioinformatics*, **24**(5):645–651.

Konagurthu, AS, Whisstock, J, and Stuckey, PJ (2004). Progressive multiple alignment using sequence triplet optimizations and three-residue exchange costs. *Journal of Bioinformatics and Computational Biology*, **2**(4):719–745.

Konagurthu, AS, Whisstock, JC, Stuckey, PJ, and Lesk, AM (2006). MUSTANG: A multiple structural alignment algorithm. *Proteins*, **64**:559–574.

Koo, BK, Jung, J, Jung, H, Nam, HW, Kim, YS, Yee, A, and Lee, W (2007). Solution structure of the hypothetical novel-fold protein TA0956 from *Thermoplasma acidophilum*. *Proteins*, **69**(2):444–447.

Krishna, SS and Grishin, NV (2005). Structural drift: a possible path to protein fold change. *Bioinformatics*, **21**(8):1308–1310.

Krissinel, E (2007). On the relationship between sequence and structure similarities in proteomics. *Bioinformatics*, **23**(6):717–723.

Krissinel, E and Henrick, K (2004). Secondary-structure matching (SSM), a new tool for fast protein structure alignment in three dimensions. *Acta Crystallographica*, **D60**:2256–2268.

Kruspe, M and Stadler, PF (2007). Progressive multiple sequence alignments from triplets. *BMC Bioinformatics*, **8**:254.

Kurtz, S, Phillippy, A, Delcher, AL, Smoot, M, Shumway, M, Antonescu, C, and Salzberg, SL (2004). Versatile and open software for comparing large genomes. *Genome Biology*, **5**(2):R12.

Laederach, A (2007). Informatics challenges in structured RNA. *Briefings in Bioinformatics*, **8**(5):294–303.

Lancia, G, Carr, R, Walenz, B, and Istrail, S (2001). 101 optimal PDB structure alignments: a branch-and-cut algorithm for the maximum contact map overlap problem. In *Proceedings of the Fifth Annual International Conference on Computational Molecular Biology (RECOMB '01)*, pages 193–202. ACM Press.

Laskowski, RA (2001). PDBsum: summaries and analyses of PDB structures. *Nucleic Acids Research*, **29**(1):221–222.

Laskowski, RA (2009). PDBsum new things. *Nucleic Acids Research*, **37**(Database issue):D335–D359.

Laskowski, RA, Chistyakov, VV, and Thornton, JM (2005). PDBsum more: new summaries and analyses of the known 3D structures of proteins and nucleic acids. *Nucleic Acids Research*, **33**(Database issue):D266–D268.

Law, RHP, Zhang, Q, McGowan, S, Buckle, AM, Silverman, GA, Wong, W, Rosado, CJ, Langendorf, CG, Pike, RN, Bird, PI, and Whisstock, JC (2006). An overview of the serpin superfamily. *Genome Biology*, **7**:216.

Leahy, DJ, Axel, R, and Hendrickson, WA (1992). Crystal structure of a soluble form of the human T cell coreceptor CD8 at 2.6 A resolution. *Cell*, **68**(6):1145–1162.

Lesk, AM (1995). Systematic representation of folding patterns. *Journal of Molecular Graphics*, **13**:159–164.

Lesk, AM (2003). From electrons to proteins and back again. *International Journal of Quantum Chemistry*, **95**:678–682.

Lesk, AM and Hardman, KD (1982). Computer-generated schematic diagrams of protein structures. *Science*, **216**(4545):539–540.

Lesk, AM and Hardman, KD (1985). Computer-generated pictures of proteins. *Methods in Enzymology*, **115**:381–390.

Leslin, CM, Abyzov, A, and Ilyin, VA (2007). TOPOFIT-DB, a database of protein structural alignments based on the TOPOFIT method. *Nucleic Acids Research*, **35**(Database issue):D317–D321.

Levitt, M and Chothia, C (1976). Structural patterns in globular proteins. *Nature*, **261**:552–558.

Levitt, M and Gerstein, M (1998). A unified statistical framework for sequence comparison and structure comparison. *Proceedings of the National Academy of Sciences of the USA*, **95**:5913–5920.

Li, Z, Zhang, S, Wang, Y, Zhang, XS, and Chen, L (2007). Alignment of molecular networks by integer quadratic programming. *Bioinformatics*, **23**(13):1631–1639.

Lindgreen, S, Gardner, PP, and Krogh, A (2007). MASTR: multiple alignment and structure prediction of non-coding RNAs using simulated annealing. *Bioinformatics*, **23**(24):3304–3311.

Liu, J, Wang, JTL, Hu, J, and Tian, B (2005). A method for aligning RNA secondary structures and its application to RNA motif detection. *BMC Bioinformatics*, **6**:89.

Liu, N and Wang, T (2006). A method for rapid similarity analysis of RNA secondary structures. *BMC Bioinformatics*, **7**:493.

Liu, W, Schmidt, B, Voss, G, and Müller-Wittig, W (2007). Streaming algorithms for biological sequence alignment on GPUs. *IEEE Transactions on Parallel and Distributed Systems*, **18**(9):1270–1281.

Liu, Y, Maskell, DL, and Schmidt, B (2009). CUDASW++: optimizing Smith-Waterman sequence database searches for CUDA-enabled graphics processing units. *BMC Research Notes*, **2**:73.

Lyngsø, RB, Zuker, M, and Pedersen, CNS (1999). Fast evaluation of internal loops in RNA secondary structure prediction. *Bioinformatics*, **15**(6):440–445.

Madej, T, Gibrat, JF, and Bryant, SH (1995). Threading a database of protein cores. *Proteins*, **23**:356–369.

Majumdar, I, Krishna, SS, and Grishin, NV (2005). PALSSE: A program to delineate linear secondary structural elements from protein structures. *BMC Bioinformatics*, **6**:202.

Manavski, SA and Valle, G (2008). CUDA compatible GPU cards as efficient hardware accelerators for Smith-Waterman sequence alignment. *BMC Bioinformatics*, **9**(Suppl 2):S10.

Mann, HB and Whitney, DR (1947). On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, **18**(1):50–60.

Martello, S, Pisinger, D, and Toth, P (1999). Dyanamic programming and strong bounds for the 0-1 knapsack problem. *Management Science*, **45**(3):414–424.

Martin, J, Letellier, G, Marin, A, Taly, JF, de Brevern, AG, and Gibrat, JF (2005). Protein secondary structure assignment revisited: a detailed analysis of different assignment methods. *BMC Structural Biology*, **5**:17.

Mathews, DH, Sabina, J, Zuker, M, and Turner, DH (1999). Expanded sequence dependence of thermodynamic parameters improves prediction of RNA secondary structure. *Journal of Molecular Biology*, **288**:911–940.

MathWorks (2006). MATLAB® r2006a. The MathWorks, Inc. `http://www.mathworks.com`.

Matsumoto, M and Nishimura, T (1998). Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, **8**(1):3–30.

Matsumoto, M and Nishimura, T (2000). Dynamic creation of pseudorandom number generators. In *Monte Carlo and Quasi-Monte Carlo Methods*, pages 56–69. Proceedings of a conference held at the Claremont Graduate University, Claremont, California, USA, June 22-26, 1998.

Mattick, JS (2003). Challenging the dogma: the hidden layer of non-protein-coding RNAs in complex organisms. *BioEssays*, **25**:930–939.

Mattick, JS (2005). The functional genomics of noncoding RNA. *Science*, **309**:1527–1528.

Mattick, JS and Makunin, IV (2006). Non-coding RNA. *Human Molecular Genetics*, **15**(Review Issue 1):R17–R29.

McCaskill, JS (1990). The equilibrium partition function and base pair binding probabilities for RNA secondary structure. *Biopolymers*, **29**:1105–1119.

McLuskey, K, Harrison, JA, Schüttelkopf, AW, Boxer, DH, and Hunter, WN (2003). Insight into the role of *Escherichia coli* MobB in molybdenum cofactor biosynthesis based on the high resolution crystal structure. *Journal of Biological Chemistry*, **278**(26):23706–23713.

Meyer, IM and Miklós, I (2007). SimulFold: Simultaneously inferring RNA structures including psuedoknots, alignments, and trees using a Bayesian MCMC framework. *PLoS Computational Biology*, **3**(8):e149.

Michael, MM (2002). High performance dynamic lock-free hash tables and list-based sets. In *Proceedings of the fourteenth annual ACM Symposium on Parallel Algorithms and Architectures*, pages 73–82. ACM.

Michael, MM (2004). Scalable lock-free dynamic memory allocation. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation*, pages 35–46. ACM.

Michalopoulos, I, Torrance, GM, Gilbert, DR, and Westhead, DR (2004). TOPS: an enhanced database of protein structural topology. *Nucleic Acids Research*, **32**(Database issue):D251–D254.

Micheletti, C and Orland, H (2009). MISTRAL: a tool for energy-based multiple structural alignment of proteins. *Bioinformatics*, **25**(20):2663–2669.

Michie, D (1968). "memo" functions and machine learning. *Nature*, **218**:19–22.

Mitchell, EM, Artymiuk, PJ, Rice, DW, and Willett, P (1990). Use of techniques dervived from graph theory to compare secondary structure motifs in proteins. *Journal of Molecular Biology*, **212**:151–166.

Moffat, A and Zobel, J (2008). Rank-biased precision for measurement of retrieval effectiveness. *ACM Transactions on Information Systems*, **27**(1):2.

Möhl, M, Salari, R, Will, S, Backofen, R, and Sahinalp, SC (2010). Sparsification of RNA structure prediction including pseudoknots. volume 6293 of *Lecture Notes in Bioinformatics*, pages 40–51. Springer, Liverpool, UK.

Morin, TL and Marsten, E (1976). Branch-and-bound strategies for dynamic programming. *Operations Research*, **24**(4):611–627.

Murzin, AG, Brenner, SE, Hubbard, T, and Chothia, C (1995). SCOP: A structural classification of proteins database for the investigation of sequences and structures. *Journal of Molecular Biology*, **247**:536–540.

Nagano, K (1977). Logical analysis of the mechanism of protein folding iv. supersecondary structures. *Journal of Molecular Biology*, **109**:235–250.

Needleman, SB and Wunsch, CD (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, **48**:443–453.

Notredame, C (2007). Recent evolutions of multiple sequence alignment algorithms. *PLoS Computational Biology*, **3**(8):e123.

Notredame, C, Higgins, DG, and Heringa, J (2000). T-Coffee: A novel method for fast and accurate multiple sequence alignment. *Journal of Molecular Biology*, **302**:205–217.

Nussinov, R, Pieczenik, G, Griggs, JR, and Kleitman, DJ (1978). Algorithms for loop matchings. *SIAM Journal on Applied Mathematics*, **35**(1):68–82.

NVIDIA (2009a). *NVIDIA CUDA C Programming Best Practices Guide*. CUDA Toolkit 2.3.

NVIDIA (2009b). *NVIDIA CUDA Programming Guide*. Version 2.3.

Oliver, TF, Schmidt, B, and Maskell, DL (2005). Reconfigurable architectures for bio-sequence database scanning on FPGAs. *IEEE Transactions on Circuits and Systems II*, **52**(12):851–855.

Ortiz, AR, Strauss, CEM, and Olmea, O (2002). MAMMOTH (matching molecular models obtained from theory): An automated method for model comparison. *Protein Science*, **11**:2606–2621.

Pagh, R and Rodler, FF (2004). Cuckoo hashing. *Journal of Algorithms*, **51**(2):122–144.

Palù, AD, Möhl, M, and Will, S (2010). Alignment of RNA with structures of unlimited complexity. In *Proceedings of WCB 2010: Workshop on Constraint Based Methods for Bioinformatics*, pages 31–35. Edinburgh.

Pang, KC, Stephen, S, Dinger, ME, Engström, PG, Lenhard, B, and Mattick, JS (2007). RNAdb 2.0 — an expanded database of mammalian non-coding RNAs. *Nucleic Acids Research*, **35**(Database issue):D178–D182.

Paton, K (1969). An algorithm for finding a fundamental set of cycles of a graph. *Communications of the Association for Computing Machinery*, **12**(9):514–518.

Pauling, L and Corey, RB (1951). Configurations of polypeptide chains with favored orientations around single bonds: two new pleated sheets. *Proceedings of the National Academy of Sciences of the USA*, **37**:729–740.

Pauling, L, Corey, RB, and Branson, HR (1951). The structure of proteins: two hydrogen-bonded helical configurations of the polypeptide chain. *Proceedings of the National Academy of Sciences of the USA*, **37**:205–211.

Pearl, F, Todd, A, Sillitoe, I, Dibley, M, Redfern, O, Lewis, T, Bennett, C, Marsden, R, Grant, A, Lee, D, Akpor, A, Maibaum, M, Harrison, A, Dallman, T, Reeves, G, Diboun, I, Addou, S, Lise, S, Johnhston, C, Sillero, A, Thornton, J, and Orengo, C

(2005). The CATH domain structure database and related resources Gene3D and DHS provide comprehensive domain family information for genome analysis. *Nucleic Acids Research*, **33**(Database issue):D247–D251.

Pearson, WR and Lipman, DJ (1988). Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences of the USA*, **85**(8):2444–2448.

Pelta, DA, González, JR, and Moreno Vega, M (2008). A simple and fast heuristic for protein structure comparison. *BMC Bioinformatics*, **9**:161.

Perriquet, O, Touzet, H, and Dauchet, M (2003). Finding the common structure shared by two homologous RNAs. *Bioinformatics*, **19**(1):108–116.

Podlozhnyuk, V (2007). *Parallel Mersenne Twister*. NVIDIA. CUDA SDK 2.3.

Porter, CJ, Schuch, R, Pelzek, AJ, Buckle, AM, McGowan, S, Wilce, MCJ, Rossjohn, J, Russell, R, Neslon, D, Fischetti, VA, and Whisstock, JC (2007). The 1.6 Å Crystal structure of the catalytic domain of PlyB, a bacteriophage lysin active against *Bacillus anthracis*. *Journal of Molecular Biology*, **366**:540–550.

Puchinger, J and Stuckey, PJ (2008). Automating branch-and-bound for dynamic programs. In *Proceedings of the 2008 ACM SIGPLAN symposium on partial evaluation and semantics-based program manipulation*, pages 81–89. ACM.

Purcell, C and Harris, T (2005). Non-blocking hashtables with open addressing. In *Proceedings of the 19th International Conference on Distributed Computing (DISC 2005)*, volume 3724 of *Lecture Notes in Computer Science*, pages 108–121.

R Development Core Team (2007). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL `http://www.R-project.org`.

Rafferty, JB, Sedelnikova, SE, Hargreaves, D, Artymiuk, PJ, Baker, PJ, Sharples, GJ, Mahdi, AA, Lloyd, RG, and Rice, DW (1996). Crystal structure of DNA recombination protein RuvA and a model for its binding to the Holliday junction. *Science*, **274**(5286):415–421.

Ramage, R, Green, J, Muir, TW, Ogunjobi, OM, Love, S, and Shaw, K (1994). Synthetic, structural and biological studies of the ubiquitin system: the total chemical synthesis of ubiquitin. *Biochemical Journal*, **299**(Pt 1):151–158.

Ramasubbu, N, Thomas, LM, Ragunath, C, and Kaplan, JB (2005). Structural analysis of Dispersin B, a biofilm-releasing glycoside hydrolase from the periodontopathogen *Actinobacillus actinomycetemcomitans*. *Journal of Molecular Biology*, **349**:475–486.

Ricagno, S, Egloff, MP, Ulferts, R, Coutard, B, Nurizzo, D, Campanacci, V, Cambillau, C, Ziebuhr, J, and Canard, B (2006). Crystal structure and mechanistic determinants of SARS coronavirus nonstructural protein 15 define an endoribonuclease family. *Proceedings of the National Academy of Sciences of the USA*, **103**(32):11892–11897.

Richards, FM and Kundrot, CE (1988). Identification of structural motifs from protein coordinate data: Secondary structure and first-level supersecondary structure. *Proteins*, **3**:71–84.

Richardson, JS (1977). β-sheet topology and the relatedness of proteins. *Nature*, **268**:495–500.

Richardson, JS (1981). The anatomy and taxonomy of protein structure. *Advances in Protein Chemistry*, **34**:167–339.

Richardson, JS (1985). Schematic drawings of protein structures. *Methods in Enzymology*, **115**:359–380.

Roberts, E, Stone, JE, Sepúlveda, L, Hwu, WMW, and Luthey-Schulten, Z (2009). Long time-scale simulations of *in vivo* diffusion using GPU hardware. In *IPDPS 2009: Proceedings of the 2009 IEEE International Symposium on Parallel & Distributed Processing*, pages 1–8.

Rognes, T (2001). ParAlign: a parallel sequence alignment algorithm for rapid and sensitive database searches. *Nucleic Acids Research*, **29**(7):1647–1652.

Rognes, T and Seeberg, E (2000). Six-fold speed-up of Smith-Waterman sequence database searches using parallel processing on common microprocessors. *Bioinformatics*, **16**(8):699–706.

Rosado, CJ, Buckle, AM, Law, RHP, Butcher, RE, Kan, WT, Bird, CH, Ung, K, Browne, KA, Baran, K, Bashtannyk-Puhalovich, TA, Faux, NG, Wong, W, Porter,

CJ, Pike, RN, Ellisdon, AM, Pearce, MC, Bottomley, SP, Emsley, J, Smith, AI, Rossjohn, J, Hartland, EL, Voskoboinik, I, Trapani, JA, Bird, PI, Dunstone, MA, and Whisstock, JC (2007). A common fold mediates vertebrate defense and bacterial attack. *Science*, **317**:1548–1551.

Rossmann, MG, Moras, D, and Olsen, KW (1974). Chemical and biological evolution of a nucleotide-binding protein. *Nature*, **250**:194–199.

Saenger, W (1984). *Principles of Nucleic Acid Structure*. Springer Advanced Texts in Chemistry. Springer-Verlag.

Sam, V, Tai, CH, Garnier, J, Gibrat, JF, Lee, B, and Munson, PJ (2006). ROC and confusion analysis of structure comparison methods identify the main causes of divergence from manual protein classification. *BMC Bioinformatics*, **7**:206.

Sankoff, D (1985). Simultaneous solution of the RNA folding, alignment and protosequence problems. *SIAM Journal on Applied Mathematics*, **45**(5):810–825.

Schatz, MC, Trapnell, C, Delcher, AL, and Varshney, A (2007). High-throughput sequence alignment using graphics processing units. *BMC Bioinformatics*, **8**:474.

Schenk, O and Gärtner, K (2004). Solving unsymmetric sparse systems of linear equations with PARDISO. *Future Generation Computer Systems*, **20**:475–487.

Schenk, O and Gärtner, K (2006). On fast factorization and pivoting methods for sparse symmetric indefinite systems. *Electronic Transactions on Numerical Analysis*, **23**:158–179.

Schneider, S, Antonopoulos, CD, and Nikolopoulos, DS (2006). Scalable locality-conscious multithreaded memory allocation. In *Proceedings of the 5th International Symposium on Memory Management (ISMM '06)*, pages 84–94. ACM, Ottawa, Canada.

Schulz, GE, Elzinga, M, Marx, F, and Schirmer, RH (1974). Three-dimensional structure of adenyl kinase. *Nature*, **250**:120–123.

Schulz, GE and Schirmer, RH (1974). Topological comparison of adenyl kinase with other proteins. *Nature*, **250**:142–144.

Shalev, O and Shavit, N (2006). Split-ordered lists: Lock-free extensible hash tables. *Journal of the Association for Computing Machinery*, **53**(3):379–405.

Shapiro, BA (1988). An algorithm for comparing multiple RNA structures. *Computer Applications in the Biosciences (CABIOS)*, **4**(3):387–393.

Shapiro, BA and Zhang, K (1990). Comparing multiple RNA secondary structures using tree comparisons. *Computer Applications in the Biosciences (CABIOS)*, **6**(4):309–318.

Shapiro, J and Brutlag, D (2004). FoldMiner: Structural motif discovery using an improved superposition algorithm. *Protein Science*, **13**:278–294.

Sheahan, D (2007). *Developing and Tuning Applications On UltraSPARC T1 Chip Multithreading Systems*. Sun Microsystems, 1.2 edition. `http://www.sun.com/blueprints/1205/819-5144.pdf`.

Shi, S, Chitturi, B, and Grishin, NV (2009). ProSMoS server: a pattern-based search using interaction matrix representation of protein structures. *Nucleic Acids Research*, **37**(Web Server issue):W526–W531.

Shi, S, Zhong, Y, Majumdar, I, Krishna, SS, and Grishin, NV (2007). Searching for three-dimensional secondary structural patterns in proteins with ProSMoS. *Bioinformatics*, **23**(11):1331–1338.

Siebert, S and Backofen, R (2005). MARNA: multiple alignment and consensus structure prediction of RNAs based on sequence structure comparisons. *Bioinformatics*, **21**(16):3352–3359.

Sing, T, Sander, O, Beerenwinkel, N, and Lengauer, T (2005). ROCR: visualizing classifier performance in R. *Bioinformatics*, **21**(20):3940–3941.

Singh, AP and Brutlag, DL (1997). Hierarchical protein structure superposition using both secondary structure and atomic representations. In *Proceedings of the Fifth International Conference on Intelligent Systems for Molecular Biology (ISMB 1997)*, pages 284–293.

Sippl, MJ (2008). On distance and similarity in fold space. *Bioinformatics*, **24**(6):872–873.

Sippl, MJ and Wiederstein, M (2008). A note on difficult structure alignment problems. *Bioinformatics*, **24**(3):426–427.

Smith, TF and Waterman, MS (1981). Identification of common molecular subsequences. *Journal of Molecular Biology*, **147**:195–197.

Spraggon, G, Pantazatos, D, Klock, HE, Wilson, IA, Woods Jr., VL, and Lesley, SA (2004). On the use of DXMS to produce more crystallizable proteins: Structures of the *T. maritima* proteins TM0160 and TM1171. *Protein Science*, **13**:3187–3199.

Stephenson, AG (2002). evd: Extreme value distributions. *R News*, **2**(2):31–32. URL `http://CRAN.R-project.org/doc/Rnews/Rnews_2002-2.pdf`.

Sternberg, MJE and Thornton, JM (1977). On the conformation of proteins: The handedness of the connection between parallel β-strands. *Journal of Molecular Biology*, **110**:269–283.

Suchard, MA and Rambaut, A (2009). Many-core algorithms for statistical phylogenetics. *Bioinformatics*, **25**(11):1370–1376.

Suhrer, SJ, Wiederstein, M, Gruber, M, and Sippl, MJ (2009). COPS — a novel workbench for explorations in fold space. *Nucleic Acids Research*, **37**(Web Server issue):W539–W544.

Tabei, Y, Tsuda, K, Kin, T, and Asai, K (2006). SCARNA: fast and accurate structural alignment of RNA sequences by matching fixed-length stem fragments. *Bioinformatics*, **22**(14):1723–1729.

Tai, KC (1979). The tree-to-tree correction problem. *Journal of the Association for Computing Machinery*, **26**(3):422–433.

Tan, G, Sun, N, and Gao, GR (2007). A parallel dynamic programming algorithm on a multi-core architecture. In *Proceedings of the nineteenth annual ACM Symposium on Parallel Algorithms and Architectures*, pages 135–144. ACM.

Taylor, WR (2001). Defining linear segments in protein structure. *Journal of Molecular Biology*, **310**:1135–1150.

Taylor, WR, Thornton, JM, and Turnell, WG (1983). An ellipsoidal approximation of protein shape. *Journal of Molecular Graphics*, **1**:30–38.

Teichert, F, Bastolla, U, and Porto, M (2007). SABERTOOTH: protein structural alignment based on a vectorial signature representation. *BMC Bioinformatics*, **8**:425.

Thiruv, B, Quon, G, Saldanha, SA, and Steipe, B (2005). Nh3D: A reference dataset of non-homologous protein structures. *BMC Structural Biology*, **5**:12.

Thompson, JD, Higgins, DG, and Gibson, TJ (1994). CLUSTAL W: improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice. *Nucleic Acids Research*, **22**(22):4673–4680.

Torarinsson, E, Havgaard, JH, and Gorodkin, J (2007). Multiple structural alignment and clustering of RNA sequences. *Bioinformatics*, **23**(8):926–932.

Torrance, GM, Gilbert, DR, Michalopoulos, I, and Westhead, DW (2005). Protein structure topological comparison, discovery and matching service. *Bioinformatics*, **21**(10):2537–2538.

Trapnell, C and Schatz, MC (2009). Optimizing data intensive GPGPU computations for DNA sequence alignment. *Parallel Computing*, **35**:429–440.

Usher, KC, de la Cruz, AFA, Dahlquist, FW, Swanson, RV, Simon, MI, and Remington, SJ (1998). Crystal structures of CheY from *Thermotoga maritima* do not support conventional explanations for the structural basis of enhanced thermostability. *Protein Science*, **7**:403–412.

Vassura, M, Margara, L, Di Lena, P, Medri, F, Fariselli, P, and Casadio, R (2008). Reconstruction of 3D structures from protein contact maps. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, **5**(3):357–367.

Veretnik, S, Bourne, PE, Alexandrov, NN, and Shindyalov, IN (2004). Toward consistent assignment of structural domains in proteins. *Journal of Molecular Biology*, **339**(3):647–678.

Vergara, IA, Norambuena, T, Ferrada, E, Slater, AW, and Melo, F (2008). StAR: a simple tool for the statistical comparison of ROC curves. *BMC Bioinformatics*, **9**:265.

Vesterstrøm, J and Taylor, WR (2006). Flexible secondary structure based protein structure comparison applied to the detection of circular permutation. *Journal of Computatinal Biology*, **13**(1):43–62.

Wang, S, Gutell, RR, and Miranker, DP (2007). Biclustering as a method for RNA local multiple sequence alignment. *Bioinformatics*, **23**(24):3289–3296.

Warshall, S (1962). A theorem on boolean matrices. *Journal of the Association for Computing Machinery*, **9**(1):11–12.

Washietl, S, Hofacker, IL, and Stadler, PF (2005). Fast and reliable prediction of noncoding RNAs. *Proceedings of the National Academy of Sciences of the USA*, **102**(7):2454–2459.

Weinberg, Z and Ruzzo, WL (2004). Exploiting conserved structure for faster annotation of non-coding RNAs without loss of accuracy. *Bioinformatics*, **20**(Suppl. 1):i334–i341.

Welsh, DJA and Powell, MB (1967). An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, **10**(1):85–86.

Westhead, DR, Slidel, TWF, Flores, TPJ, and Thornton, JM (1999). Protein structural topology: Automated analysis and diagrammatic representation. *Protein Science*, **8**:897–904.

Will, S, Reiche, K, Hofacker, IL, Stadler, PF, and Backofen, R (2007). Inferring noncoding RNA families and classes by means of genome-scale structure-based clustering. *PLoS Computational Biology*, **3**(4):e65.

Williams, A, Gilbert, DR, and Westhead, DR (2003). Multiple structural alignment for distantly related all β structures using TOPS pattern discovery and simulated annealing. *Protein Engineering*, **16**(12):913–923.

Wilm, A, Higgins, DG, and Notredame, C (2008). R-Coffee: a method for multiple alignment of non-coding RNA. *Nucleic Acids Research*, **36**(9):e52.

Wilm, A, Mainz, I, and Steger, G (2006). An enhanced RNA alignment benchmark for sequence alignment programs. *Algorithms for Molecular Biology*, **1**:19.

Wirawan, A, Kwoh, CK, Hieu, NT, and Schmidt, B (2008). CBESW: Sequence alignment on the PlayStation 3. *BMC Bioinformatics*, **9**:377.

Wozniak, A (1997). Using video-oriented instructions to speed up sequence comparison. *Computer Applications in the Biosciences (CABIOS)*, **13**(2):145–150.

Wybrow, M (2008). *Using semi-automatic layout to improve the usability of diagramming software*. Ph.D. thesis, Monash University.

Wybrow, M, Marriott, K, and Stuckey, PJ (2006). Incremental connector routing. In *Proceedings of the 13th International Symposium on Graph Drawing (GD 2005)*, volume 3843 of *Lecture Notes in Computer Science*, pages 446–457. Springer.

Wybrow, M, Marriott, K, and Stuckey, PJ (2010). Orthogonal connector routing. In *Graph Drawing 2009*, volume 5849 of *Lecture Notes in Computer Science*, pages 219–231. Springer.

Xia, F, Dou, Y, Zhou, X, Yang, X, Xu, J, and Zhang, Y (2009). Fine-grained parallel RNAalifold algorithm for RNA secondary structure prediction on FPGA. *BMC Bioinformatics*, **10**(Suppl. 1):S37.

Xie, W and Sahinidis, NV (2006). A branch-and-reduce algorithm for the contact map overlap problem. In *Research in Computational Molecular Biology (RECOMB 2006)*, volume 3909 of *Lecture Notes in Bioinformatics*, pages 516–529. Springer.

Xu, X, Ji, Y, and Stormo, GD (2007). RNA Sampler: a new sampling based algorithm for common RNA secondary structure prediction and structural alignment. *Bioinformatics*, **23**(15):1883–1891.

Yao, Z, Weinberg, Z, and Ruzzo, WL (2005). CMfinder — a covariance model based RNA motif finding algorithm. *Bioinformatics*, **22**(4):445–452.

Ye, Y (1992). On affine scaling algorithms for nonconvex quadratic programming. *Mathematical Programming*, **56**:285–300.

Ye, Y (1997). *Interior Point Algorithms: Theory and Analysis*. Wiley-Interscience Series in Discrete Mathematics and Optimization. Wiley, New York.

Ye, Y and Tse, E (1989). An extension of Karmarkar's projective algorithm for convex quadratic programming. *Mathematical Programming*, **44**:157–179.

Yu, Y, Liang, YH, Brostromer, E, Quan, JM, Panjikar, S, Dong, YH, and Su, XD (2006). A catalytic mechanism revealed by the crystal structures of the imidazolonepropionase from *Bacillus subtilis*. *Journal of Biological Chemistry*, **281**(48):36929–36936.

Yuen, BJ (1991). Heuristics for sequencing cutting patterns. *European Journal of Operational Research*, **55**:183–190.

Yuen, BJ and Richardson, KV (1995). Establishing the optimality of sequencing heuristics for cutting stock problems. *European Journal of Operational Research*, **84**:590–598.

Zhang, K and Shasha, D (1989). Simple fast algorithms for the editing distance between trees and related problems. *SIAM Journal on Computing*, **18**(6):1245–1262.

Zhang, L, Bailey, J, Konagurthu, AS, and Ramamohanarao, K (2010). A fast indexing approach for protein structure comparison. *BMC Bioinformatics*, **11**(Suppl 1):S46. The Eighth Asia Pacific Bioinformatics Conference (APBC 2010).

Zhou, H, Xue, B, and Zhou, Y (2007). DDOMAIN: Dividing structures into domains using a normalized domain-domain interaction profile. *Protein Science*, **16**:947–955.

Zhou, P and Shang, Z (2009). 2D molecular graphics: a flattened world of chemistry and biology. *Briefings in Bioinformatics*, **10**:247–258.

Zuker, M and Stiegler, P (1981). Optimal computer folding of large RNA sequences using thermodynamics and auxilliary information. *Nucleic Acids Research*, **9**(1):133–148.